A HAND-HELD GUIDE AND REFERENCE

Thinking in Common Lisp

From the REPL to a shipped executable: expressions, lists, functions, recursion, macros, conditions, CLOS and packages, then interop, generators, testing and a program you build one chapter at a time. Every example is code you run as you read it.

No forward references. Nothing skipped.

Prepared for Shaul Khayyat · August 2026

Contents

How to Use This Guide 1
1. The REPL and the Edit-Evaluate Loop 6
2. Expressions, Atoms and Lists 10
3. Numbers, Characters, Strings and Symbols 15
4. Variables 23
5. Functions 30
6. Conditionals 37
Part I review 45
7. Cons Cells and List Structure 48
8. Recursion 57
9. Functions as Values 68
10. Working with Sequences 77
Part II review 87
11. Loops 90
12. format 100
13. Structures, Hash Tables and Arrays 110
14. Macros 119
15. Conditions and the Debugger 129
16. Packages, Files and ASDF 138
Part III review 146
17. Lexical and Dynamic Variables 150
18. The Reader 158
19. Conditions II — handler-bind and the Condition Hierarchy 166
20. Compilation, Loading and eval-when 175
21. Type Declarations and Performance 181
22. The REPL and Debugging Workflow 187
23. Symbols and the Property List 192
24. Multiple Values 197
25. Closures as Objects 206
26. CLOS 212
27. CLOS Deep Dive 222
28. Packages and Symbols Internals 229
Part IV review 236
29. Interoperability 240
30. Iterators and Generators 249
31. Reading Other People's Lisp 258
32. Building Executables 264
33. Testing 271
34. The Bigger Picture 282
Part V review 288
A. Installing SBCL 289
B. Setting Up Emacs for Common Lisp 291
C. Emacs Survival Guide 295

D. Answers to Exercises 299
E. The Game's Redefinitions 426
F. SBCL Error Messages 429
G. The Complete Game 431
H. Glossary 441
I. Concept Index 442
Colophon: How the Guide Grew 444

How to Use This Guide

What this guide is

This guide takes you from zero to reading, writing and thinking in Common Lisp. It is long on purpose. Read it in order the first time. Afterwards, use it as a reference.

Two promises:

1. **No forward references.** You will never see "don't worry about this yet". Every piece of code uses only what has already been explained.
2. **You run everything.** Every example is a live code block. You run it inside this file and see the result right below it.

You know Python and Java. Now and then a short note will say "this is roughly like X in Java". Treat those as landmarks, not translations. Lisp works best when you meet it on its own terms.

First: set up your machine

Before anything else, do these in order:

1. Appendix A: install SBCL (the Lisp itself).
2. Appendix B: make Emacs talk to SBCL and run code blocks.
3. Skim Appendix C: the Emacs keys you need.

Come back here when the test block in Appendix B prints `3`.

How each chapter is built

Every chapter has the same parts:

- **Explanation** with code blocks you run as you read.
- **Common mistakes:** the errors beginners actually make, what the error message looks like, and how to fix it.
- **Project step:** one new piece of the running project (see below).
- **Exercises:** answers are in Appendix D.

Running code blocks (Org Babel)

A code block looks like this:

```
(+ 1 2)
```

To run it:

1. Make sure Lisp is running. Type `M-x slime` and press Enter once per Emacs session. A REPL window opens. You can leave it alone.
2. Put the cursor anywhere inside the block.
3. Press `C-c C-c`.

The result appears right under the block:

```
#+RESULTS:
: 3
```

Run the block again and the result is replaced, not duplicated.

Two kinds of results

By default you get the **value** of the last expression in the block, which is the thing the code "returns".

If the code **prints** text, add `:results output` to the first line to capture what was printed:

```
(print "hello")
(print "world")
```

Without `:results output`, the printed text goes to the REPL window and the block shows only the final value. When a block's result looks "wrong", this is usually why.

Blocks share one Lisp

Every block runs in the same Lisp session. If you define something in one block, later blocks can use it. This means:

- Run blocks in order, from top to bottom.
- If you restart Emacs or Lisp, re-run the earlier blocks that define things before running later ones.

Editing a block comfortably

Press `C-c '` (Control-c, then apostrophe) inside a block. It opens in its own window with full Lisp editing support. Press `C-c '` again to go back.

Writing and checking your exercise answers

Under each exercise, make your own block and type your answer:

```
*** Exercise 1.1
Write a function =twice= that returns twice its argument.

#+begin_src lisp
;; my answer
(defun twice (n)
  (* n 2))
#+end_src
```

Then check it. Each answer in Appendix D comes with **test lines**. Paste them into a second block under yours and run both:

```
#+begin_src lisp
(list (twice 4) (twice 0) (twice -3))
#+end_src

#+RESULTS:
: (8 0 -6)
```

If your result matches the one in Appendix D, you're done. Your version doesn't need to look like the answer. There are many correct ways to write most things.

Habits that help:

- **Type the examples, don't paste them.** Your fingers learn the shape of Lisp.
- **Break things on purpose.** Delete a parenthesis, misspell a name, and read the error. Every chapter's "Common mistakes" section shows how.
- **If an error window (the debugger) pops up,** press `q` to close it. A later chapter teaches you how to use it properly.

The running project

Across the chapters you will build a small **text adventure**. You'll walk between rooms, pick up items and type commands. It starts as a few lines of data and grows a little with each chapter. Each project step uses only that chapter's ideas and the ones before it.

PART I

First Steps

> Part I gives you the core of the language: how Lisp reads and runs code, the basic kinds of data, variables, functions and decisions. By the end you will write small programs and read most beginner Lisp code without trouble.

0.1. A note on reading results

Most blocks in this guide show their expected result as a comment:

```
(+ 1 2)   ; => 3
```

A semicolon starts a comment. Lisp ignores everything from `;` to the end of the line. The `; =>` comment means "this evaluates to". When you run the block with `C-c C-c`, the `#+RESULTS:` line should match.

CHAPTER 1.

The REPL and the Edit-Evaluate Loop

1.1. What Lisp does with your code

Lisp works in a loop of four steps:

1. **Read** the text you typed and turn it into data.
2. **Evaluate** that data, which means work out its value.
3. **Print** the value.
4. **Loop** back and wait for more.

That is the REPL. Every time you press `C-c C-c` on a block, the same thing happens: the block is read, evaluated, and the value is shown.

In Java you write a whole program, compile it, and run `main`. In Lisp you talk to a running program, one expression at a time, and it answers. The program grows while it runs. This is the single biggest change in how you will work.

1.2. Your first expressions

Run each block.

```
42
```

A number evaluates to itself. You get `42`.

```
(+ 2 3)   ; => 5
```

This is a *call*. The parentheses mean "do something". The first thing inside is the operator (`+`). The rest are the arguments (`2` and `3`). This style, operator first, is called *prefix notation*.

```
(+ 1 2 3 4 5)   ; => 15
```

`+` accepts any number of arguments. So do `-`, `*` and many others.

```
(* 2 (+ 3 4))   ; => 14
```

Expressions nest. Lisp works out the inner one first, `(+ 3 4)` is `7`, then the outer one, `(* 2 7)` is `14`.

There is no operator precedence to remember. In Python, `2 + 3 * 4` depends on rules about `*` binding tighter than `+`. In Lisp you write exactly what you mean:

```
(+ 2 (* 3 4))   ; => 14
(* (+ 2 3) 4)   ; => 20
```

When a block contains several expressions, they all run, top to bottom, and the block shows the value of the **last** one. The block above shows `20`.

1.3. Using the REPL directly

Switch to the REPL window (`C-x o`, or `C-x b` and type `repl`). At the `CL-USER>` prompt, type:

```
CL-USER> (* 6 7)
42
```

Press `RET` after the closing parenthesis. If you press `RET` before the parentheses are balanced, the REPL waits for the rest. That isn't an error. Finish typing and press `RET` again.

Use `M-p` to bring back what you typed before. Use the REPL for quick experiments. Use code blocks for anything you want to keep.

1.4. Printing

`print` writes a value to the output:

```
(print 42)
(print (+ 1 1))
```

Text in double quotes is a *string*. Chapter 3 covers strings fully. For now, you can print one:

```
(print "Hello, Lisp")
```

`print` starts a new line before the value. A string prints with its quotes, because `print` shows values the way Lisp would read them back.

`princ` prints for humans, without quotes and without the new line:

```
(princ "Hello, ")
(princ "Lisp")
```

`print` and `princ` also *return* the value they printed. So in a REPL you see the value twice: once printed, once as the result. That's normal.

Remember the difference between the two kinds of block results:

- Without `:results output`: you see the **value** of the last expression.
- With `:results output`: you see what was **printed**.

1.5. Comments

```
;;; Three semicolons: a heading for a section of code.

;; Two semicolons: a comment on its own line,
;; about the code below it.
(+ 1 2)   ; One semicolon: a short note at the end of a line.
```

Lisp treats all three the same. The number of semicolons is a convention that tells human readers what kind of comment it is.

1.6. Your first error

Run this. It has a mistake on purpose:

```
(plus 1 2)
```

A debugger window opens. The first line says something like:

```
The function COMMON-LISP-USER::PLUS is undefined.
```

Read it carefully:

- `PLUS` is the name you used. Lisp shows names in capitals. Chapter 3 explains why.
- `COMMON-LISP-USER::` is the area where your names live. You can ignore it for now. It's always there for your own code.
- "undefined" means there is no function with that name.

Press `q` to close the window. Nothing is broken. The Lisp is still running and everything you defined is still there.

Errors in Lisp don't crash your program. The program pauses and asks you what to do. Closing the window with `q` means "give up on this expression and go back to the prompt".

1.7. Reading parentheses

New Lispers worry about parentheses. Two facts make them easy:

1. **Emacs counts them for you.** Put the cursor right after a closing parenthesis and Emacs highlights the opening one.
2. **Experienced Lispers read indentation, not parentheses.** The shape of the code on the page shows its structure. Emacs indents it for you: select a region and press `TAB`, or inside a code block press `C-c '` and then `TAB` on each line.

Closing parentheses go together at the end of the last line:

```
;; Good Lisp style
(* 2
   (+ 3
      4))

;; Not Lisp style: closing parentheses on their own lines
(* 2
   (+ 3
      4
   )
)
```

Both work. Only the first looks like Lisp. Use it.

1.8. Common mistakes

- **Putting the operator in the middle:** `(1 + 2)`. The error is "Illegal function call" or "1 is not a function name". The operator always comes first: `(+ 1 2)`.
- **Leaving out spaces:** `(+1 2)` is not "plus, one, two". Lisp reads `+1` as the number positive one, then complains that `+1` is not a function. Always put a space after the operator.
- **Using commas between arguments:** `(+ 1, 2)`. Lisp separates arguments with spaces only. A comma here gives "comma not inside a backquote", which is a feature you will meet in Part IV.
- **Expecting printed text in the block result:** you used `print` but the block shows only the final value. Add `:results output`.

- **Unbalanced parentheses in a block:** the block gives an “end of file” error. One closing parenthesis is missing. Put the cursor after each closing parenthesis to find the one that has no partner.

1.9. Project step: the adventure begins

The project is a text adventure. The player is in a small house and moves between rooms. In this chapter, print the opening line of the game.

```
(princ "You wake up in a small, quiet house.")
```

That’s the whole step. In the next chapter, the rooms become data.

1.10. Exercises

1.1 Write an expression that calculates the number of minutes in a week (7 days, 24 hours per day, 60 minutes per hour).

1.2 Write this calculation in Lisp: `(10 - 4) * (3 + 2)`.

1.3 Write this one: `100 / (2 * 5) - 3`.

1.4 Print the line `Welcome, traveller.` without quotes. Which block header do you need to see it in the result?

1.5 Each line below has a mistake. Say what’s wrong and write the fixed version.

```
(5 * 3)
(+2 3)
(+ 1 (* 2 3)
(* 2, 4)
```

CHAPTER 2.

Expressions, Atoms and Lists

2.1. Everything is an expression

In Java, there are *statements* (like `if` or `for`) and *expressions* (like `a + b`). Statements do things. Expressions have values.

In Lisp, everything is an expression. Everything has a value. You will see in Chapter 6 that even `if` gives back a value.

Lisp code is built from just two kinds of thing.

2.2. Atoms

An *atom* is anything that isn't a list. Some atoms:

```
42          ; a number
3.14        ; another number
"hello"     ; a string
```

Numbers and strings evaluate to themselves.

There is a third important kind of atom: the *symbol*. A symbol is a name, like `x`, `total`, `+` or `print`. When Lisp evaluates a symbol, it looks up the value that name stands for. Chapter 3 covers symbols in depth. Chapter 4 shows how to give them values.

2.3. Lists

A *list* is some things inside parentheses, separated by spaces:

```
(1 2 3)
(+ 1 2)
(a b c)
(1 (2 3) 4)
()
```

The last one is the empty list. A list can hold other lists.

Here is the key idea of Lisp: **code is written as lists.** `(+ 1 2)` is a list of three things: the symbol `+`, the number `1` and the number `2`. When Lisp evaluates a list, it treats the first element as the name of something to do, and the rest as the arguments.

2.4. The evaluation rule

When Lisp evaluates `(f a b)`, it does this:

1. Look at the first element, `f`. It names an operator.
2. Evaluate each argument in order, left to right.
3. Call the operator with those values.

Trace it on a nested example:

```
(+ (* 2 3) (- 10 4))
1. The operator is +.
2. Evaluate (* 2 3)  → 6
   Evaluate (- 10 4) → 6
3. Call + with 6 and 6 → 12
```

Each argument is itself evaluated by the same rule. That's why nesting works to any depth.

This is the *normal* rule, for *function calls*. A few special operators follow different rules. You'll meet them one at a time, and each time the guide will say exactly how it differs. `quote`, below, is the first.

2.5. Quote: "don't evaluate this"

Try to make a list of three numbers:

```
(1 2 3)
```

Error: "illegal function call". Lisp tried to *call* `1` as an operator. You want the list as data, not as code.

`quote` stops evaluation. It gives back its argument untouched:

```
(quote (1 2 3))   ; => (1 2 3)
```

You will write `quote` so often that it has a short form, the apostrophe:

```
'(1 2 3)          ; => (1 2 3)
'(+ 1 2)          ; => (+ 1 2)
```

The second one is the *list* `(+ 1 2)`, not the number `3`. Quote keeps code from running and gives it to you as data.

`quote` breaks the normal evaluation rule on purpose: it does **not** evaluate its argument. That is what makes it special.

2.6. Building lists with `list`

`list` is an ordinary function. It evaluates its arguments and makes a list of the results:

```
(list 1 2 3)          ; => (1 2 3)
(list 1 (+ 1 1) 3)    ; => (1 2 3)
'(1 (+ 1 1) 3)        ; => (1 (+ 1 1) 3)
```

Compare the last two. `list` computed `(+ 1 1)`. The quote didn't.

Use quote when you know the exact contents ahead of time. Use `list` when some parts must be computed.

2.7. Taking lists apart

```
(first '(a b c))    ; => A
(second '(a b c))   ; => B
(third '(a b c))    ; => C
(rest '(a b c))     ; => (B C)
```

- `first` gives the first element.
- `rest` gives a list of everything after the first element.
- `second` through `tenth` also exist.

Lisp shows `a` as `A`. Chapter 3 explains why.

You'll also see `car` and `cdr` in older code. They mean the same as `first` and `rest`. Part II explains where those names come from.

```
(first (rest '(a b c)))   ; => B
```

Read nested calls from the inside out: "the rest of `(a b c)` is `(b c)`, and the first of that is `b`".

2.8. How Org shows results

The line `#+PROPERTY: header-args:lisp :results verbatim` at the top of this file tells Org to show every result exactly as the REPL would print it. Without it, Org tries to be helpful. A list like `(1 2 3)` turns into a table `| 1 | 2 | 3 |`, and a string loses its quotes.

If you ever see a table where you expected a list, check that the `#+PROPERTY` line is there. Then press `C-c C-c` on it to make Org notice it.

2.9. Counting and indexing

```
(length '(a b c d))    ; => 4
(nth 0 '(a b c d))     ; => A
(nth 2 '(a b c d))     ; => C
```

`nth` counts from zero, like array indexes in Java. `(nth 0 ...)` is the same as `first`. The argument order is index first, then the list.

2.10. The empty list and NIL

The empty list has a second name, `nil`:

```
'()              ; => NIL
(rest '(a))      ; => NIL
(length nil)     ; => 0
```

`()` and `nil` are the same thing. Lisp always prints it as `NIL`.

Taking apart an empty list gives back `nil` instead of an error:

```
(first '())      ; => NIL
(rest '())       ; => NIL
```

`nil` also means *false*. The symbol `t` means *true*. Lisp has no separate true/false type like Java's `boolean`. Any value that isn't `nil` counts as true. Chapter 6 relies on this.

2.11. Asking questions about data

A function that answers yes or no is called a *predicate*. It returns `t` or `nil`. Many predicate names end in `p` (for "predicate"):

```
(listp '(1 2))   ; => T     is it a list?
(listp 5)        ; => NIL
(atom 5)         ; => T     is it an atom?
(atom '(1 2))    ; => NIL
(null '())       ; => T     is it the empty list?
(null '(a))      ; => NIL
```

One surprise: `nil` is both an atom and a list.

```
(atom nil)       ; => T
(listp nil)      ; => T
```

It's an atom because it isn't built from parts. It's a list because it is the empty list. Part II shows how lists are built, and this will then look natural.

2.12. Combining and reversing lists

```
(append '(a b) '(c d))       ; => (A B C D)
(append '(1) '(2 3) '(4))    ; => (1 2 3 4)
(reverse '(1 2 3))           ; => (3 2 1)
(last '(1 2 3))              ; => (3)
```

`append` joins any number of lists into one. `reverse` gives a new list in the opposite order. Neither changes the lists you gave it.

Look closely at `last`. It returns a list holding the last element, not the element itself. To get the element, take its `first`:

```
(first (last '(1 2 3)))      ; => 3
```

Part II explains why `last` works this way.

2.13. Common mistakes

- **Forgetting the quote:** `(first (1 2 3))` gives "illegal function call". Lisp tried to call `1`. Write `(first '(1 2 3))`.
- **Quoting something that should be computed:** `'(1 (+ 1 1))` gives `(1 (+ 1 1))`, not `(1 2)`. If a part must be worked out, use `list`: `(list 1 (+ 1 1))`.
- **Swapping the arguments of `nth`:** `(nth '(a b c) 1)` gives an error saying the list "is not of type UNSIGNED-BYTE". The index comes first: `(nth 1 '(a b c))`.
- **Counting `nth` from one:** `(nth 1 '(a b c))` is `B`, not `A`. Indexes start at zero.
- **Expecting `last` to return the element:** it returns a one-element list. Use `(first (last ...))`.
- **Thinking `()` and `nil` are different:** they're the same object. Both print as `NIL`.

2.14. Project step: rooms as data

The house has four rooms. A quoted list holds their names:

```
'(bedroom hallway kitchen garden)
;; => (BEDROOM HALLWAY KITCHEN GARDEN)
```

The player starts in the first room:

```
(first '(bedroom hallway kitchen garden))   ; => BEDROOM
```

A room also needs to know where you can go from it. Put the room's name first and a list of its neighbours second:

```
'(kitchen (hallway garden))
```

Now take it apart:

```
(first '(kitchen (hallway garden)))    ; => KITCHEN
(second '(kitchen (hallway garden)))   ; => (HALLWAY GARDEN)
```

The first block shows `KITCHEN`. The second shows the list of exits. You've just designed a data format with nothing but lists. In Java you'd write a `Room` class first. In Lisp you often start with plain lists and add structure later, when you know what you need.

2.15. Exercises

2.1 Predict the result of each line, then run them one at a time to check:

```
'(+ 2 3)
(list '+ 2 3)
(+ 2 3)
```

2.2 Get the symbol `C` out of `'(a b c d)` in three different ways.

2.3 Get the symbol `BLUE` (not the list `(BLUE)`) out of `'(red green blue)`.

2.4 Using `'(1 2 3)` and `'(4 5 6)`, build `(6 5 4 3 2 1)` in one expression.

2.5 Predict the result of each line:

```
(listp nil)
(null '(nil))
(length '((a b) (c d e)))
(rest '(only))
```

2.6 The room data `'(garden (kitchen shed))` says the garden connects to the kitchen and the shed. Write an expression that returns `SHED`.

CHAPTER 3.

Numbers, Characters, Strings and Symbols

This chapter covers the atoms you'll use every day. Each section is a small tour. Run every block, and come back to this chapter as a reference.

3.1. Integers

Integers in Lisp have no size limit. Java's `int` overflows past about two billion. Lisp just keeps going:

```
(expt 2 10)     ; => 1024
(expt 2 100)    ; => 1267650600228229401496703205376
```

`expt` raises its first argument to the power of the second. You never need to think about `int` versus `long` versus `BigInteger`. Lisp switches to big numbers by itself when needed.

3.2. Ratios: exact fractions

Here is the first real surprise:

```
(/ 6 3)     ; => 2
(/ 7 2)     ; => 7/2
(/ 1 3)     ; => 1/3
```

Dividing two integers gives an exact answer. If it isn't a whole number, Lisp gives a *ratio*. Java would give `3` for `7 / 2` and throw away the remainder. Lisp keeps the exact value.

You can write ratios yourself, and they reduce automatically:

```
(+ 1/3 1/6)    ; => 1/2
(* 2/3 3)      ; => 2
(/ 4 6)        ; => 2/3
```

3.3. Floats

A number with a decimal point is a *float*:

```
(/ 7 2.0)        ; => 3.5
(+ 1.5 2)        ; => 3.5
(float 7/2)      ; => 3.5
(float 1/3)      ; => 0.33333334
```

If any argument is a float, the result is a float. `float` converts a number to a float.

By default, `1.5` is a *single-float*, with about 7 digits of precision, like Java's `float`. For more precision, like Java's `double`, write `d0` at the end:

```
(float 1/3 1d0)  ; => 0.3333333333333333d0
(* 1.5d0 2)      ; => 3.0d0
```

The second argument to `float` is an example of the precision you want. The `d0` means "times ten to the power zero, in double precision". For most work in this guide, integers and ratios are enough.

3.4. Arithmetic you'll use

Expression	Result	Meaning
`(+ 1 2 3)`	`6`	add
`(- 10 3)`	`7`	subtract
`(- 5)`	`-5`	negate
`(* 2 3 4)`	`24`	multiply
`(/ 12 4)`	`3`	divide
`(1+ 5)`	`6`	add one
`(1- 5)`	`4`	subtract one
`(mod 17 5)`	`2`	remainder (like Java's `%`)
`(expt 2 8)`	`256`	power
`(sqrt 16)`	`4.0`	square root (gives a float)
`(isqrt 17)`	`4`	whole-number square root
`(abs -7)`	`7`	absolute value
`(max 3 9 2)`	`9`	largest
`(min 3 9 2)`	`2`	smallest

`1+` is a single name, not "one plus". It's a function called `1+`. Lisp names can contain characters like `+`, `-` and `*`.

3.5. Comparing numbers

```
(= 3 3)          ; => T
(= 3 3.0)        ; => T
(/= 3 4)         ; => T    "not equal"
(< 1 2)          ; => T
(< 1 2 3 4)      ; => T    each one smaller than the next
(< 1 3 2)        ; => NIL
(>= 5 5)         ; => T
```

`(< 1 2 3 4)` checks a whole chain at once. In Java you'd write `1 < 2 && 2 < 3 && 3 < 4`.

`/=` is "not equal". There is no `!=` in Lisp.

`=` compares numbers only. Using it on anything else is an error.

3.6. Number predicates

```
(numberp 5)      ; => T     is it a number?
(integerp 5)     ; => T     is it an integer?
(integerp 5.0)   ; => NIL
(zerop 0)        ; => T     is it zero?
(plusp 3)        ; => T     is it positive?
(minusp -3)      ; => T     is it negative?
```

```
(evenp 4)        ; => T
(oddp 4)         ; => NIL
```

3.7. Characters

A single character is written with `#\` in front:

```
#\a              ; => #\a
#\Z              ; => #\Z
#\Space          ; => #\Space
#\Newline        ; => #\Newline
```

A character and a one-letter string are different things. `#\a` is a character. `"a"` is a string that holds one character.

```
(characterp #\a)        ; => T
(characterp "a")        ; => NIL
(char-upcase #\a)       ; => #\A
(char-downcase #\Q)     ; => #\q
(char-code #\A)         ; => 65
(code-char 97)          ; => #\a
(char= #\a #\a)         ; => T
(alpha-char-p #\a)      ; => T
(alpha-char-p #\7)      ; => NIL
(digit-char-p #\7)      ; => 7
(upper-case-p #\A)      ; => T
(char< #\a #\b)         ; => T
(char-equal #\a #\A)    ; => T
```

What each one does:

- `char-upcase` and `char-downcase` change the case of a character.
- `char-code` gives the character's number. `code-char` goes back the other way.
- `char=` tests whether two characters are the same. Case matters: `#\a` and `#\A` are different.
- `char-equal` is the same test, but ignores case.
- `alpha-char-p` asks "is it a letter?"
- `digit-char-p` returns the digit's value, or `nil` if the character isn't a digit. A value that isn't `nil` counts as true, so it works as a yes/no question too.
- `char<` compares characters by alphabetical order.

Use `char=` for characters, not `=`. Remember that `=` is only for numbers.

3.8. Strings

A string is text inside double quotes:

```
"Hello"              ; => "Hello"
(stringp "Hello")    ; => T
(length "Hello")     ; => 5
```

To put a double quote inside a string, write `\"`. To put in a backslash, write `\\`:

```
(princ "She said \"hi\" and left.")
```

`princ` shows the text the way a human would read it. `print` would show the backslashes, because it shows strings the way you'd type them into Lisp.

A string can span several lines. The line breaks become part of the text:

```
(princ "Line one
Line two")
```

3.8.1. Getting characters out of a string

```
(char "lisp" 0)      ; => #\l
(char "lisp" 3)      ; => #\p
```

`char` takes the string first, then the index. Indexes start at zero, like `nth`. Notice the order is the opposite of `nth`, which takes the index first. This is a common source of mistakes.

3.8.2. Pieces of strings

```
(subseq "hello world" 0 5)   ; => "hello"
(subseq "hello world" 6)     ; => "world"
```

`subseq` ("sub-sequence") takes a start index and an optional end index. The end index is *not* included, like Java's `substring`. With no end, it goes to the end of the string.

```
(search "lo" "hello")        ; => 3
(search "xyz" "hello")       ; => NIL
```

`search` finds where the first string appears inside the second. It returns the starting index, or `nil` if it isn't there.

3.8.3. Joining strings

```
(concatenate 'string "Hello" ", " "world")   ; => "Hello, world"
```

The first argument, `'string`, is a quoted symbol. It tells `concatenate` what kind of result to build. `concatenate` can also join lists:

```
(concatenate 'list '(1 2) '(3 4))   ; => (1 2 3 4)
```

For strings you'll always write `'string`. Forgetting it is one of the most common beginner errors.

3.8.4. Changing case and trimming

```
(string-upcase "hello")            ; => "HELLO"
(string-downcase "HELLO")          ; => "hello"
(string-capitalize "hello world")  ; => "Hello World"
(string-trim " " "   hi   ")       ; => "hi"
(reverse "abc")                    ; => "cba"
```

`string-trim` removes the characters in its first argument from both ends of the second. All of these return a *new* string. The original is unchanged.

`reverse` and `length` work on strings as well as lists. Lists and strings are both *sequences*, and many functions accept either. Part III covers sequences fully.

3.8.5. Comparing strings

```
(string= "abc" "abc")       ; => T
(string= "abc" "ABC")       ; => NIL
(string-equal "abc" "ABC")  ; => T
```

`string=` cares about case. `string-equal` ignores it. This matches the `char=` and `char-equal` pair.

Alphabetical order:

```
(string< "apple" "banana")  ; => 0
(string< "banana" "apple")  ; => NIL
```

`string<` does not return `T` when the answer is yes. It returns the index where the two strings first differ. That's `0` here, because `a` and `b` differ at position zero. Since `0` isn't `nil`, it counts as true. If you only need yes or no, you can treat any non-`nil` result as yes.

3.8.6. Turning other things into strings

```
(princ-to-string 42)       ; => "42"
(princ-to-string 'kitchen) ; => "KITCHEN"
(string #\a)               ; => "a"
```

`princ-to-string` gives the text that `princ` would have printed, as a string. `string` turns a character into a one-character string.

3.9. Symbols

You've used symbols since Chapter 1: `+`, `first`, `kitchen`. A symbol is a name. It's a real object that Lisp creates the first time it reads that name, and it's the same object every time you write that name again.

3.9.1. Two jobs of a symbol

1. **As code**, a symbol names something: a function (`+`), or a value (Chapter 4 shows how).
2. **As data**, after a quote, a symbol is just itself:

```
'kitchen          ; => KITCHEN
(symbolp 'kitchen)  ; => T
(symbolp "kitchen") ; => NIL
```

Use symbols as data for fixed labels: room names, directions, colours, states. In Java you'd use an `enum`. In Lisp you just write the name.

3.9.2. Why everything is in capitals

When Lisp reads your code, it converts symbol names to upper case. `kitchen`, `Kitchen` and `KITCHEN` all become the same symbol, `KITCHEN`:

```
'Kitchen               ; => KITCHEN
(symbol-name 'kitchen) ; => "KITCHEN"
```

`symbol-name` gives the name as a string. You still write code in lower case. Lisp displays symbols in capitals. That's all there is to it.

If you really need lower-case letters in a symbol, put the name between vertical bars: `'|Kitchen|`. You'll rarely need this. When you see vertical bars in a result, it usually means you made a symbol from a lower-case string by mistake.

Strings keep their case. `"Kitchen"` stays `"Kitchen"`. Use strings for text a human reads, and symbols for labels the program uses.

3.9.3. Is it the same symbol?

`eq` tells you whether two symbols are the same one:

```
(eq 'north 'north)   ; => T
(eq 'north 'NORTH)   ; => T
(eq 'north 'south)   ; => NIL
```

For symbols, `eq` is the right test. For the other atoms you know:

To compare	Use
numbers	`=`
characters	`char=`
strings	`string=`
symbols	`eq`

Part IV covers equality fully, including how to compare lists.

3.9.4. Keywords

A symbol that starts with a colon is a *keyword*:

```
:north              ; => :NORTH
(keywordp :north)   ; => T
```

A keyword evaluates to itself, so it never needs a quote. You'll use keywords a lot from Chapter 5 onwards, as labels for function arguments.

3.9.5. T and NIL are symbols

`t` and `nil` are symbols too, and like keywords they evaluate to themselves:

```
t      ; => T
nil    ; => NIL
(symbolp t)   ; => T
```

3.10. Common mistakes

- **Integer division surprise:** expecting `(/ 7 2)` to give `3` or `3.5`. It gives `7/2`. Use `(float (/ 7 2))` for a decimal, or `(floor 7 2)` for the whole-number part. (`floor` shows a second number in the REPL, the remainder. The first number is the answer.)
- **Using `=` on non-numbers:** `(= "a" "a")` gives an error saying the value "is not a NUMBER". Use `string=` for strings, `char=` for characters, `eq` for symbols.

- **Forgetting `'string` in `concatenate`:** `(concatenate "a" "b")` gives an error about an unknown type or "is not a valid sequence type specifier". Write `(concatenate 'string "a" "b")`.
- **Mixing up `char` and `nth` argument order:** `char` takes the string first, `(char "abc" 0)`. `nth` takes the index first, `(nth 0 '(a b c))`.
- **Confusing `"a"` and `#\a`:** functions for characters won't accept a string. `(char-upcase "a")` is an error. Use `(char-upcase #\a)` or `(string-upcase "a")`.
- **Unescaped quote in a string:** `"He said "hi""` ends the string early and confuses Lisp. Write `"He said \"hi\""`.
- **Expecting `string<` to return `T`:** it returns an index. Any number means yes.
- **Expecting symbols to keep their case:** `'Kitchen` becomes `KITCHEN`. If you need to show it nicely, use `string-capitalize` on its name.

3.11. Project step: room descriptions

A room now has three parts: its name, a description for the player, and its exits.

```
'(kitchen "A warm kitchen that smells of bread." (hallway garden))
```

The name is a symbol, because the program uses it as a label. The description is a string, because the player reads it.

Build the line the game shows when you enter the kitchen:

```
(concatenate 'string
             "You are in the "
             (string-capitalize
              (symbol-name
               (first '(kitchen "A warm kitchen that smells of bread."
                        (hallway garden)))))
             ". "
             (second '(kitchen "A warm kitchen that smells of bread."
                       (hallway garden))))
;; => "You are in the Kitchen. A warm kitchen that smells of bread."
```

Read it from the inside out:

1. `(first '(kitchen ...))` gives the symbol `KITCHEN`.
2. `symbol-name` turns it into the string `"KITCHEN"`.
3. `string-capitalize` turns that into `"Kitchen"`.
4. `(second '(kitchen ...))` gives the description.
5. `concatenate` joins all four strings.

You had to type the room list twice. That's clumsy, and it's exactly the problem Chapter 4 solves: you'll give the list a name and use the name instead.

To show the line to the player, wrap the same expression in `princ` and add `:results output`:

```
(princ (concatenate 'string
                    "You are in the "
                    (string-capitalize (symbol-name 'kitchen))
```

```
                    ". "
                    "A warm kitchen that smells of bread."))
```

You should see the text without quotes:

```
You are in the Kitchen. A warm kitchen that smells of bread.
```

3.12. Exercises

3.1 Add 2/3 and 1/4 and get the exact answer. Then write a second expression that gives the same answer as a decimal.

3.2 Predict the result of each line, then check:

```
(/ 10 4)
(/ 10 4.0)
(mod 10 4)
(isqrt 50)
```

3.3 Use `subseq` to get `"Lisp"` out of `"I love Lisp"`.

3.4 Starting from the strings `"hello"` and `"world"`, build `"HELLO, WORLD!"` in one expression.

3.5 Write two expressions: 1. One that gives the numeric value of the character `#\7`. 2. One that turns `#\b` into upper case and then gives its character code.

3.6 Predict the result of each line:

```
(eq 'Apple 'APPLE)
(string= "Apple" "APPLE")
(string-equal "Apple" "APPLE")
(symbol-name :north)
```

3.7 project Given this room:

```
'(garden "An overgrown garden." (kitchen shed))
```

Write an expression that returns the string “You are in the Garden. An overgrown garden.”. Take the name and the description out of the list. Don’t type them in by hand.

CHAPTER 4.

Variables

So far, every value had to be typed out in full, sometimes twice. A *variable* gives a value a name, so you can use the name instead.

Lisp has two kinds of variables:

- **Global variables** exist everywhere, for as long as Lisp runs.
- **Local variables** exist only inside one piece of code.

4.1. Global variables with `defparameter`

```
(defparameter *price* 25)
```

The result is `*PRICE*`, the name of the variable you just made. `defparameter` returns the name, not the value.

Now use it:

```
*price*                 ; => 25
(* *price* 4)           ; => 100
```

This is the evaluation rule for symbols from Chapter 2: when Lisp evaluates a symbol, it looks up the value the name stands for. Before this chapter, the only symbols you evaluated were `t` and `nil`, which stand for themselves. Now you can make your own.

Compare with the quoted symbol:

```
'*price*                ; => *PRICE*
```

Quoted, it's just the name. Unquoted, it's the value.

4.1.1. The stars

The stars around `*price*` are part of the name. They're called *earmuffs*. Every Lisp programmer writes global variables this way, so that a reader can tell at a glance which names are global. Lisp doesn't force you, but always do it. A later chapter shows a real reason behind the habit, and you'll already have it.

4.1.2. Variables live in the running Lisp

The variable now exists in your Lisp session. Every later block can use it. If you restart Lisp (or Emacs), it's gone. Run the `defparameter` block again to bring it back.

4.2. Changing a variable with `setf`

```
(setf *price* 30)       ; => 30
*price*                 ; => 30
```

`setf` ("set field") stores a new value in an existing variable and returns that value. It's Lisp's version of Java's `=`.

`setf` can set several variables at once, in pairs:

```
(defparameter *width* 0)
(defparameter *height* 0)
(setf *width* 7
      *height* 3)
(* *width* *height*)    ; => 21
```

4.2.1. Adding and subtracting: `incf` and `decf`

Changing a number by a little is common enough to have its own operators:

```
(defparameter *score* 0)
(incf *score*)          ; => 1
(incf *score*)          ; => 2
(incf *score* 10)       ; => 12
(decf *score* 5)        ; => 7
```

`incf` adds (1 by default), `decf` subtracts. Both change the variable and return its new value. They're like Java's `score++` and `score + 10`.

Run that block twice and the result is different: the `defparameter` resets `*score*` to `0`, then the rest counts up again to `7`. Keep this in mind for the next section.

4.3. `defvar`: define once, keep the value

`defvar` also makes a global variable, with one difference. If the variable already exists, `defvar` leaves it alone.

Run each block once, in order:

```
(defvar *lives* 3)
*lives*                 ; => 3
```

```
(setf *lives* 1)
```

```
(defvar *lives* 3)
*lives*                 ; => 1
```

The second `defvar` did nothing, because `*lives*` already had a value. With `defparameter`, it would have gone back to `3`.

When to use which:

Use	When
`defparameter`	Settings and data you want reset each time you re-run the code
`defvar`	State you build up while the program runs, and don't want lost if you re-run the file

When you're not sure, use `defparameter`. Its behaviour is easier to predict.

4.4. Constants with `defconstant`

```
(defconstant +days-per-week+ 7)
(* +days-per-week+ 52)  ; => 364
```

A constant can never be changed. `(setf +days-per-week+ 8)` is an error. The plus signs around the name are the convention for constants, like the stars for global variables.

Use constants only for values that truly never change. For anything else, `defparameter` is more convenient.

4.5. Local variables with `let`

Global variables are handy, but most names in a program are only needed for a moment. `let` makes variables that exist only inside it:

```
(let ((width 7)
      (height 3))
  (* width height))     ; => 21
```

The shape of a `let`:

```
(let ((name1 value1)        ← a list of bindings
      (name2 value2))         each binding is (name value)
  expression1               ← the body: one or more expressions
  expression2)              ← the value of the last one is the result
```

Watch the parentheses. There's one pair around the whole list of bindings, and one pair around each binding. That's why a `let` starts with ((.

Linking a name to a value is called *binding* the name. The names `width` and `height` are *bound* only inside the body. Outside, they don't exist:

```
width
```

The debugger opens with "The variable WIDTH is unbound." "Unbound" means "this name has no value here". Press `q`.

Local variables don't get earmuffs. The stars are only for globals.

4.5.1. The body can have several expressions

```
(let ((name "Ada")
      (age 36))
  (print name)
  (print age))
```

The body runs top to bottom. Without `:results output`, the block would show only `36`, the value of the last expression.

4.5.2. Values can be computed

The value in a binding is evaluated first, like any argument:

```
(let ((total (* 25 4))
      (items (list 'pen 'ink)))
  (list total (length items)))   ; => (100 2)
```

4.5.3. Changing a local variable

`setf` and `incf` work on local variables too:

```
(let ((count 0))
  (incf count)
  (incf count)
  count)                ; => 2
```

4.5.4. Using globals inside `let`

The body can use global variables as well as its own:

```
(defparameter *tax-rate* 1/5)

(let ((price 50))
  (+ price (* price *tax-rate*)))   ; => 60
```

4.6. `let*`: bindings that use earlier bindings

In a `let`, every value is worked out *before* any name is bound. So a binding can't use a name from the same `let`:

```
(let ((a 2)
      (b (* a 10)))
  b)
```

Error: "The variable A is unbound." When Lisp computed `(* a 10)`, `a` didn't exist yet. Press `q`.

`let*` binds the names one at a time, top to bottom. Each value can use the names above it:

```
(let* ((a 2)
       (b (* a 10)))
  b)                    ; => 20
```

A good habit: use `let` by default and switch to `let*` when one binding needs another. Readers then know that a `let*` has bindings that depend on each other.

4.7. Nesting and shadowing

A `let` can go inside another one:

```
(let ((x 1))
  (let ((y 2))
    (+ x y)))           ; => 3
```

The inner body sees both `x` and `y`.

If the inner `let` uses the same name, the inner one wins while you're inside it. This is called *shadowing*:

```
(let ((x 1))
  (list x
        (let ((x 100))
          x)
        x))             ; => (1 100 1)
```

Read it slowly. The first `x` is `1`. Inside the inner `let`, `x` is `100`. After the inner `let` ends, `x` is `1` again. The inner binding never changed the outer one. It only hid it for a while.

4.8. How to choose

- Need a value in one calculation? Use `let`.
- Need it to stay around between blocks, or for the whole program? Use `defparameter` or `defvar`.
- Never changes? Use `defconstant`.

Most Lisp code uses many local variables and only a few globals.

4.9. Common mistakes

- **`setf` on a name that was never defined:** `(setf *gold* 10)` with no `defparameter` first. SBCL warns "undefined variable: **GOLD**". It may seem to work, but the behaviour isn't reliable. Always create globals with `defparameter` or `defvar` first.
- **Missing parentheses in `let`:** `(let (x 5) x)` is wrong. Lisp reads `(x 5)` as the list of bindings, and complains that the binding `5` is malformed. Write `(let ((x 5)) x)`.
- **Using a name from the same `let`:** `(let ((a 1) (b (+ a 1))) b)` gives "The variable A is unbound". Use `let*`.
- **Using a local name outside its `let`:** the name is unbound there. Move the code inside the body.
- **Expecting `defvar` to reset a value:** re-running `(defvar *x* 0)` keeps the old value. Use `setf` to change it, or use `defparameter`.
- **Quoting a variable:** `(* '*price* 2)` gives "**PRICE** is not a NUMBER". The quote gives you the name, not the value. Leave the quote off.
- **Running blocks out of order:** after restarting Lisp, a block that uses `*price*` fails with "unbound". Run the `defparameter` block first.
- **Forgetting the earmuffs when using a global:** if you defined `*price*`, then `price` is a different name and is unbound.

4.10. Project step: naming the rooms

Give each room a name, so you never type it twice again:

```
(defparameter *bedroom*
  '(bedroom "A small bedroom with an unmade bed." (hallway)))

(defparameter *hallway*
  '(hallway "A narrow hallway with creaky floorboards." (bedroom kitchen)))
```

```
(defparameter *kitchen*
  '(kitchen "A warm kitchen that smells of bread." (hallway garden)))

(defparameter *garden*
  '(garden "An overgrown garden." (kitchen)))
```

Now keep track of where the player is. A global variable holds the current room. The player starts in the bedroom:

```
(defparameter *current-room* *bedroom*)
```

`*current-room*` now holds the same list as `*bedroom*`.

Build the description with `let`, so each part has a clear name:

```
(let ((name (first *current-room*))
      (description (second *current-room*)))
  (concatenate 'string
               "You are in the "
               (string-capitalize (symbol-name name))
               ". "
               description))
;; => "You are in the Bedroom. A small bedroom with an unmade bed."
```

Compare this with the Chapter 3 version. The room list appears nowhere in the code. The `let` names its parts, and the last expression puts them together.

Move the player to the hallway, then run the description block again:

```
(setf *current-room* *hallway*)
```

The same description code now says "You are in the Hallway." The code didn't change. Only the data did.

You still have to copy the description block each time you want to use it. Chapter 5 turns it into a function you can call by name.

4.11. Exercises

4.1 Make two global variables: `*apples*` with the value 12 and `*pears*` with the value 8. Write an expression for the total number of fruit. Then change `*pears*` to 20 with `setf` and run the total again.

4.2 Predict the final value of `*mode*`, then check by running the lines in order:

```
(defvar *mode* 'easy)
(setf *mode* 'hard)
(defvar *mode* 'easy)
(defparameter *mode* 'normal)
(defvar *mode* 'easy)
```

4.3 Using `let`, bind `width` to 8 and `height` to 5. Return a list of the area and the perimeter.

4.4 Using `let*`, bind `celsius` to 30, then `fahrenheit` computed from it with the formula F = C × 9/5 + 32. Return a list of both values.

4.5 Make a global `*steps*` starting at 0. Write one block that adds 1 three times and then adds 10, and shows the final value.

4.6 Each line has a mistake. Say what's wrong and fix it.

```
(let (total 10) total)
(let ((a 3) (b (* a a))) b)
(* '*apples* 2)
```

4.7 Predict the result:

```
(let ((x 5))
  (let ((x (* x 2)))
    (setf x (+ x 1))
    x))
```

Then predict what changes if you replace the last line with `(list x)` placed *after* the inner `let` instead of inside it.

4.8 project Set `*current-room*` to `*kitchen*`. Using `let`, write an expression that returns a list of two things: the room's name and the number of exits it has.

CHAPTER 5.

Functions

In Chapter 4 you still had to copy the description code each time you wanted it. A *function* gives a piece of code a name, so you can run it again with different inputs.

5.1. Defining a function with `defun`

```
(defun twice (n)
  (* n 2))
```

The result is `TWICE`, the function's name. Now call it like any built-in function:

```
(twice 21)            ; => 42
(twice (twice 5))     ; => 20
(+ (twice 3) 1)       ; => 7
```

The shape of a `defun`:

```
(defun name (parameter1 parameter2)   ← name, then the parameter list
  expression1                         ← the body
  expression2)                        ← its last value is returned
```

When you call `(twice 21)`:

1. Lisp evaluates the argument, `21`.
2. It binds the parameter `n` to `21`, just like a `let`.
3. It runs the body, `(* n 2)`.
4. The value of the last expression, `42`, is the result.

There is no `return` statement. A function returns the value of the last expression in its body. In Java you write `return n * 2;`. In Lisp, the last expression *is* the return value.

5.2. Parameters are local variables

A parameter works exactly like a variable made by `let`. It exists only inside the function body:

```
(defun area (width height)
  (* width height))

(area 7 3)             ; => 21
```

Outside the function, `width` and `height` are unbound. Each call gets its own fresh bindings, so calls never interfere with each other.

You can use `let` and `let*` inside a body, as you would anywhere else:

```
(defun rectangle-info (width height)
  (let ((area (* width height))
        (perimeter (* 2 (+ width height))))
    (list area perimeter)))
```

```
(rectangle-info 8 5)   ; => (40 26)
```

5.3. Functions with no parameters

The parameter list can be empty, but it must be there:

```
(defun dice-faces ()
  6)

(dice-faces)           ; => 6
```

To call it, put the name in parentheses with nothing after it. Writing just `dice-faces` without parentheses asks for a *variable* called `dice-faces`, which doesn't exist.

5.4. A body with several expressions

```
(defun shout (text)
  (princ (string-upcase text))
  (princ "!"))

(shout "hello")
```

The body runs top to bottom. The value of the last expression is returned. Here that's the value of the second `princ`, which is `"!"`. With `:results output` you see only what was printed: `HELLO!`.

Run the same call without `:results output`:

```
(shout "hello")        ; => "!"
```

The text was still printed, but it went to the REPL window. The block shows only the return value.

Keep two ideas apart:

- **Printing** shows text to a human.
- **Returning** gives a value back to the code that called the function.

Most functions should return values and not print. A returned value can be used by other code. Printed text can only be read. Print only in the few functions whose job is to talk to the user.

5.5. Documentation strings

If the first thing in the body is a string, and more expressions follow it, that string is the function's *documentation string*, or *docstring*:

```
(defun twice (n)
  "Return N multiplied by two."
  (* n 2))
```

Lisp keeps it, and you can ask for it:

```
(documentation 'twice 'function)   ; => "Return N multiplied by two."
```

In Emacs, put the cursor on a function name and press `C-c C-d d` to see its documentation. It works on built-in functions too. Try it on `concatenate`.

By convention, parameter names in a docstring are written in capitals, like `N` above.

5.6. Redefining a function

You just defined `twice` a second time. Lisp replaced the old definition with the new one. SBCL may print a note like:

```
WARNING: redefining COMMON-LISP-USER::TWICE in DEFUN
```

That's harmless. It's telling you what you already know.

This is how you work in Lisp. You don't restart the program to fix a function. You change the `defun`, run it again with `C-c C-c`, and the running program uses the new version immediately.

5.7. Functions calling functions

Functions you define can call each other, and the built-in ones:

```
(defun square (n)
  (* n n))

(defun sum-of-squares (a b)
  (+ (square a) (square b)))

(sum-of-squares 3 4)   ; => 25
```

You can define them in any order, as long as each one exists by the time it is *called*. If you define `sum-of-squares` first, SBCL may warn "undefined function: SQUARE". That's only a reminder. Once `square` is defined, the call works.

5.8. Optional parameters: `&optional`

Parameters after `&optional` can be left out when calling:

```
(defun greeting (name &optional (punctuation "."))
  (concatenate 'string "Hello, " name punctuation))

(greeting "Ada")       ; => "Hello, Ada."
(greeting "Ada" "!")   ; => "Hello, Ada!"
```

`(punctuation ".")` means "a parameter called `punctuation`, with the default `"."` if the caller leaves it out". The shape is the same as a `let` binding: a name and a value.

If you write a plain name with no default, it becomes `nil` when left out:

```
(defun show-optional (a &optional b)
  (list a b))

(show-optional 1)      ; => (1 NIL)
(show-optional 1 2)    ; => (1 2)
```

Optional arguments are filled in order. To give the second one, you must also give the first.

5.9. Keyword parameters: &key

Keyword parameters are given by name, in any order:

```
(defun make-label (text &key (prefix "[") (suffix "]"))
  (concatenate 'string prefix text suffix))

(make-label "ok")                       ; => "[ok]"
(make-label "ok" :prefix "<")           ; => "<ok]"
(make-label "ok" :suffix ">" :prefix "<")  ; => "<ok>"
```

When calling, you write the keyword (Chapter 3), then its value. The keyword `:prefix` matches the parameter `prefix`.

Use `&key` when a function has several settings that callers will rarely change. Many built-in functions work this way. You'll see keyword arguments throughout the rest of this guide.

Use `&optional` for one or two arguments in a natural order. Use `&key` for anything more. Don't use both in the same function: the combination behaves in confusing ways.

5.10. Any number of arguments: &rest

`&rest` collects all remaining arguments into a list:

```
(defun collect (&rest things)
  things)

(collect)              ; => NIL
(collect 1 2 3)        ; => (1 2 3)
```

You can put normal parameters before it:

```
(defun tag (label &rest items)
  (list label (length items) items))

(tag 'fruit 'apple 'pear)   ; => (FRUIT 2 (APPLE PEAR))
```

This is how `+` and `list` accept any number of arguments.

5.11. The parameter list, in full

```
(defun name (required1 required2
             &optional (opt1 default1)
             &rest more
             &key (key1 default1))
  body)
```

You will rarely need more than one of `&optional`, `&rest` and `&key` at a time. The required parameters always come first.

5.12. Naming functions

- Use lower case with hyphens: `sum-of-squares`, not `sumOfSquares`. Lisp would turn `sumOfSquares` into `SUMOFSQUARES` anyway.
- A function that answers yes or no ends in `p`: `emptyp`, `room-dark-p`. Use `-p` when the name already has a hyphen, and plain `p` when it's a single word.
- Don't reuse the names of built-in functions such as `list`, `first` or `room`. SBCL stops you with "Lock on package COMMON-LISP violated". Pick another name. (If it happens, choose the restart that says to ignore the lock only if you know what it means; otherwise press `q`.)

5.13. Common mistakes

- **Wrong number of arguments:** `(area 7)` when `area` needs two. The debugger says "invalid number of arguments: 1". Check the parameter list. Press `C-c C-d d` on the name if you forget it.
- **Forgetting the empty parameter list:** `(defun dice-faces 6)`. Lisp reads `6` as the parameter list and complains. Write `(defun dice-faces () 6)`.
- **Calling without parentheses:** writing `dice-faces` instead of `(dice-faces)`. The error is "The variable DICE-FACES is unbound".
- **Quoting the parameter:** `(defun twice (n) (* 'n 2))` gives "N is not a NUMBER". Inside the body, use `n` without a quote.
- **A typo in the body:** `(defun area (width height) (* widht height))`. SBCL warns "undefined variable: WIDHT" when you define it. Read those warnings: they catch the bug before you call the function.
- **Printing instead of returning:** a function that ends with `princ` returns the printed value, which is usually not what the caller needs. Let the function return the value, and print it at the place that talks to the user.
- **A docstring that is the whole body:** `(defun name () "Ada")` returns `"Ada"`. A string counts as a docstring only when more expressions follow it.
- **Forgetting the keyword when calling:** `(make-label "ok" "<")` gives an error about an "odd number of &KEY arguments". Write `(make-label "ok" :prefix "<")`.

5.14. Project step: room functions

Run the Chapter 4 project blocks first, so the room variables exist.

Each room is a list of name, description and exits. Instead of remembering which position holds what, give each part a function:

```
(defun room-name (place)
  "Return the name symbol of PLACE."
  (first place))

(defun room-description (place)
  "Return the description string of PLACE."
  (second place))

(defun room-exits (place)
  "Return the list of room names PLACE connects to."
  (third place))
```

The parameter is called `place` because `room` is the name of a built-in function.

```
(room-exits *hallway*)     ; => (BEDROOM KITCHEN)
```

These small functions are called *accessors*. If you later change how a room is stored, you only change these three functions. The rest of the game keeps working.

Now the description, written once:

```
(defun describe-room (place)
  "Return the text shown when the player is in PLACE."
  (concatenate 'string
               "You are in the "
               (string-capitalize (symbol-name (room-name place)))
               ". "
               (room-description place)))
```

```
(describe-room *kitchen*)
;; => "You are in the Kitchen. A warm kitchen that smells of bread."
```

The player mostly wants to know about the room they're standing in. A function with no parameters handles that:

```
(defun look ()
  "Describe the room the player is in."
  (describe-room *current-room*))
```

```
(setf *current-room* *bedroom*)
(look)
;; => "You are in the Bedroom. A small bedroom with an unmade bed."
```

`look` reads the global `*current-room*` each time it runs. Change the variable and `look` follows:

```
(setf *current-room* *garden*)
(look)
;; => "You are in the Garden. An overgrown garden."
```

The player still moves by you typing `setf`, and nothing stops them from walking from the bedroom straight into the garden. Chapter 6 adds the decisions a real `go` command needs.

5.15. Exercises

5.1 Write a function `square` that returns its argument multiplied by itself.

5.2 Write a function `average3` that takes three numbers and returns their average. Give it a docstring.

5.3 Write a function `greet` that takes a name and returns `"Hello, NAME!"` with the name in capitals. `(greet "ada")` should give `"Hello, ADA!"`.

5.4 Write a function `power-of` that takes a base and an optional exponent, which defaults to 2. `(power-of 5)` gives `25`, `(power-of 2 10)` gives `1024`.

5.5 Write a function `make-greeting` that takes a name and two keyword parameters: `greeting` (default `"Hello"`) and `punctuation` (default `"!"`). `(make-greeting "Ada" :greeting "Welcome")` should give `"Welcome, Ada!"`.

5.6 Write a function `count-args` that accepts any number of arguments and returns how many it got.

5.7 Each line has a mistake. Say what's wrong and fix it.

```
(defun add (a b) (+ a b c))
(defun (twice n) (* 2 n))
(add 1)
```

5.8 project Write two functions using the room accessors:

1. `exit-count`, which takes a room and returns how many exits it has.
2. `room-title`, which takes a room and returns its name as a capitalised string, such as `"Kitchen"`.

CHAPTER 6.

Conditionals

Programs need to make choices: is the exit open, is the score high enough, did the player type a real command. This chapter covers every way Lisp makes a choice.

6.1. True and false, again

From Chapter 2:

- `nil` means false.
- Every other value means true. `t` is the usual way to write "true", but `0`, `""` and `'(a)` are all true as well.

This is different from Python, where `0` and `""` are false. In Lisp, only `nil` (the empty list) is false.

6.2. `if`

```
(if (> 5 3)
    "bigger"
    "not bigger")      ; => "bigger"
```

The shape:

```
(if test
    then-expression     ← evaluated when test is true
    else-expression)    ← evaluated when test is nil
```

`if` is a special operator. It does **not** evaluate all its arguments like a function would. It evaluates the test first, then **only one** of the other two. You can see this:

```
(if t
    (print "then")
    (print "else"))
```

Only `"then"` is printed. The else part never runs.

`if` returns the value of whichever branch ran. That makes it an expression, like Java's `test ? a : b`, and you can use it anywhere a value goes:

```
(defun describe-number (n)
  (concatenate 'string
               "The number is "
               (if (evenp n) "even" "odd")))

(describe-number 7)    ; => "The number is odd"
```

The else part can be left out. Then `if` returns `nil` when the test fails:

```
(if (> 1 5) "yes")     ; => NIL
```

6.3. `progn`: several expressions where one is expected

Each branch of `if` is a single expression. To do two things in a branch, wrap them in `progn`:

```
(if (> 5 3)
    (progn
      (print "5 is bigger")
      (print "doing a second thing"))
    (print "not bigger"))
```

`progn` runs its expressions in order and returns the value of the last one. You've seen this behaviour before: the body of `let` and `defun` works the same way. `progn` gives you that behaviour anywhere.

6.4. `when` and `unless`

When there is no else part, `when` is clearer than `if`, and its body can hold several expressions without `progn`:

```
(when (> 5 3)
  (print "5 is bigger")
  (print "no progn needed"))
```

`unless` is the opposite: it runs its body when the test is `nil`.

```
(unless (> 1 5)
  "1 is not bigger than 5")   ; => "1 is not bigger than 5"
```

Both return `nil` when the body doesn't run.

How to choose:

Use	When
`if`	You need both a "then" and an "else"
`when`	Only a "then", especially with several steps
`unless`	Only something to do when the test fails

6.5. `not`, `and`, `or`

```
(not nil)              ; => T
(not 5)                ; => NIL
(and (> 5 3) (< 1 2))  ; => T
(or (> 1 5) (< 1 2))   ; => T
```

`not` turns true into `nil` and `nil` into `t`. `and` and `or` take any number of arguments.

Two things make them more useful than Java's `&&` and `||`.

6.5.1. They stop early

`and` stops at the first `nil`. `or` stops at the first true value. The rest isn't evaluated:

```
(or t (print "never printed"))
```

Nothing is printed. This lets you guard a test:

```
(defun safe-divide-check (a b)
  (and (/= b 0) (> (/ a b) 1)))

(safe-divide-check 10 0)   ; => NIL  (no division by zero)
(safe-divide-check 10 2)   ; => T
```

When `b` is `0`, `and` stops before dividing.

6.5.2. They return real values

`or` returns the first true value itself, not just `t`. `and` returns the last value if all are true:

```
(or nil 5 10)          ; => 5
(or nil nil)           ; => NIL
(and 1 2 3)            ; => 3
(and 1 nil 3)          ; => NIL
```

A common use of `or` is a fallback value:

```
(defun display-name (name)
  (or name "stranger"))

(display-name "Ada")   ; => "Ada"
(display-name nil)     ; => "stranger"
```

6.6. `cond`: many choices

When there are more than two choices, nested `if` gets hard to read. `cond` lists the choices one after another:

```
(defun temperature-word (degrees)
  (cond ((< degrees 0)  "freezing")
        ((< degrees 15) "cold")
        ((< degrees 25) "mild")
        (t              "hot")))

(temperature-word -5)   ; => "freezing"
(temperature-word 20)   ; => "mild"
(temperature-word 31)   ; => "hot"
```

The shape:

```
(cond (test1 expression ...)   ← a clause: a test, then a body
      (test2 expression ...)
      (t     expression ...))  ← t is always true: the "otherwise"
```

`cond` tries each test from the top. At the first test that is true, it runs that clause's body and returns its last value. It ignores every clause after it. That's why the order matters: `20` is also less than `25`, but the `"cold"` test failed first and the `"mild"` test matched.

The final `(t ...)` clause catches everything else. If no clause matches and there's no `t` clause, `cond` returns `nil`.

A clause body can hold several expressions, like `when`:

```
(let ((n 7))
  (cond ((evenp n)
         (print "even")
         (print "divisible by two"))
        (t
         (print "odd"))))
```

Note the doubled parentheses at the start of each clause: one pair opens the clause, the next opens the test.

6.7. `case`: choosing by exact value

When every test is "is this value equal to X?", `case` is shorter:

```
(defun day-number (day)
  (case day
    (monday 1)
    (tuesday 2)
    (wednesday 3)
    (otherwise 0)))

(day-number 'tuesday)  ; => 2
(day-number 'sunday)   ; => 0
```

`case` evaluates `day` once, then compares it to the key at the start of each clause. The keys are **not evaluated**, so you don't quote them: write `monday`, not `'monday`. `otherwise` catches everything else. (`t` works too, but `otherwise` is clearer.)

A clause can match several keys if you put them in a list:

```
(defun weekend-p (day)
  (case day
    ((saturday sunday) t)
    (otherwise nil)))

(weekend-p 'sunday)    ; => T
(weekend-p 'monday)    ; => NIL
```

`case` compares with the same test as `eq` for symbols, and it also works for numbers and characters:

```
(defun vowel-p (letter)
  (case (char-downcase letter)
    ((#\a #\e #\i #\o #\u) t)
    (otherwise nil)))

(vowel-p #\E)          ; => T
```

`case` does **not** work with strings. `"yes"` and `"yes"` are two separate string objects, and `case` doesn't look inside them. For strings, use `cond` with `string=`.

6.8. `member`: is it in the list?

One more function belongs here, because it's the most common test in the project:

```
(member 'b '(a b c))   ; => (B C)
(member 'z '(a b c))   ; => NIL
```

`member` looks for an element in a list. If it finds it, it returns the rest of the list starting from that element. If not, it returns `nil`. Since any non-`nil` value is true, you can use it as a yes/no test:

```
(if (member 'garden '(hallway garden))
    "you can go there"
    "no way through")  ; => "you can go there"
```

Like `case`, `member` works for symbols, numbers and characters, but not for strings. Part IV shows how to make it work with strings.

6.9. Choosing the right tool

Situation	Use
Two choices	`if`
Do something only when true	`when`
Do something only when false	`unless`
Three or more choices with different tests	`cond`
Three or more choices by exact value	`case`
Combine tests	`and`, `or`, `not`
A default value	`or`

6.10. Common mistakes

- **Several expressions in an `if` branch:** `(if test (print "a") (print "b") (print "c"))` is an error: "too many elements". `if` takes exactly a test and up to two branches. Use `progn`, or `when` if there is no else part.
- **Treating `0` or `""` as false:** `(if 0 "yes" "no")` gives `"yes"`. Use `(zerop n)` or `(string= s "")` to test for them.
- **Forgetting the extra parentheses in `cond`:** `(cond (< x 0) "negative")` reads `<` as the test, which is a variable that doesn't exist. Each clause needs its own pair: `(cond ((< x 0) "negative"))`.
- **Putting a general test before a specific one in `cond`:** the first matching clause wins. Test `(< n 0)` before `(< n 15)`, not after.
- **Quoting `case` keys:** `(case day ('monday 1))`. The key becomes the list `(quote monday)`, and SBCL warns about a duplicate key. Write `(case day (monday 1))`.
- **Using `case` with strings:** `(case "yes" ("yes" 1))` never matches. Use `cond` with `string=`.
- **Forgetting the default:** a `cond` with no `t` clause, or a `case` with no `otherwise`, quietly returns `nil` when nothing matches. Add a default that returns something meaningful.
- **Expecting `member` to return `T`:** it returns part of the list. That's still true. Don't compare its result with `t`.

6.11. Project step: moving between rooms

The player should only walk through real exits. To move, the game needs two things: a way to get from a room's *name* to its *list*, and a check that the name is one of the current room's exits.

First, the lookup. `case` matches the name to the right variable:

```
(defun find-room (name)
  "Return the room whose name is NAME, or NIL if there is none."
  (case name
    (bedroom *bedroom*)
    (hallway *hallway*)
    (kitchen *kitchen*)
    (garden *garden*)
    (otherwise nil)))
```

```
(room-description (find-room 'hallway))
;; => "A narrow hallway with creaky floorboards."
```

Now the move. `cond` handles the three possible situations:

```
(defun go-to (name)
  "Move the player to the room called NAME, if it is an exit."
  (cond ((eq name (room-name *current-room*))
         "You are already there.")
        ((member name (room-exits *current-room*))
         (setf *current-room* (find-room name))
         (look))
        (t
         "You can't go there from here.")))
```

The second clause has two expressions: it changes the room, then returns the new description.

Play a few moves:

```
(setf *current-room* *bedroom*)
(list (go-to 'garden)
      (go-to 'hallway)
      (go-to 'kitchen)
      (go-to 'kitchen))
```

The result is a list of four answers:

```
("You can't go there from here."
 "You are in the Hallway. A narrow hallway with creaky floorboards."
 "You are in the Kitchen. A warm kitchen that smells of bread."
 "You are already there.")
```

The bedroom has no exit to the garden, so the first move fails. Each later move starts from where the last one left the player.

The game now has rules. In Part II you'll learn how lists are really built, and the rooms will move out of separate variables into one list the game can search.

6.12. Exercises

6.1 Write a function `sign` that returns the symbol `negative`, `zero` or `positive` for a number.

6.2 Predict the result of each line, then check:

```
(if nil 1 2)
(if 0 1 2)
(when (> 1 2) 'yes)
(unless (> 1 2) 'yes)
(if (member 3 '(1 2 3)) 'found 'missing)
```

6.3 Write `my-abs`, which returns the absolute value of a number, using `if` and without using `abs`.

6.4 Write `grade`, which takes a score from 0 to 100 and returns the symbol `a` for 90 and above, `b` for 80 to 89, `c` for 70 to 79 and `f` for anything lower.

6.5 Write `day-type` using `case`. It returns `weekend` for `saturday` and `sunday`, `weekday` for `monday` to `friday`, and `unknown` for anything else.

6.6 Each block has a mistake. Run it, read the error or the wrong result, and fix it.

```
(let ((x 5))
  (if (> x 0)
      (print "positive")
      (print "big")
      (print "negative")))

(cond (> 5 0) 'yes)

(let ((answer "yes"))
  (case answer
    ("yes" 1)
    (otherwise 0)))
```

In the first block, the intent is: when `x` is positive, print both `"positive"` and `"big"`. Otherwise print `"negative"`.

In the third block, the intent is to return `1` when `answer` is `"yes"`.

6.7 Predict the result of each line:

```
(or nil "" 5)
(and 1 0 nil)
(or (member 'x '(a b)) 'none)
(and 'a 'b)
(not 0)
```

6.8 Write `fizzbuzz-word`, which takes a number and returns:

- `"FizzBuzz"` if it divides by both 3 and 5,
- `"Fizz"` if it divides by 3,
- `"Buzz"` if it divides by 5,
- otherwise, the number as a string, such as `"7"`.

Hint: a number divides by 3 when `(mod n 3)` is zero.

6.9 project Write `can-go-p`, which takes a room name and returns exactly `T` if it is an exit of the current room, and `NIL` if not. It must not return a piece of the exit list the way `member` does.

6.10 project The garden door is locked. Make a global variable `*garden-unlocked*` that starts as `nil`. Then write a new `go-to` that answers `"The garden door is locked."` when the player tries to enter the garden while it is still locked. Every other move must work as before.

Part I review

You now know the core of Lisp:

- Code is made of atoms and lists. A list is evaluated by calling its first element with the values of the rest. `quote` stops evaluation.
- The everyday atoms are numbers (with exact ratios), characters, strings and symbols.
- `defparameter` and `defvar` make global variables. `let` and `let*` make local ones. `setf`, `incf` and `decf` change them.
- `defun` makes functions. The last expression is the return value. `&optional`, `&key` and `&rest` make parameter lists flexible.
- `if`, `when`, `unless`, `cond` and `case` make choices. Only `nil` is false.

The adventure has rooms, descriptions, a `look` command and moves that follow the exits. Before moving on, scroll back through the project steps from Chapter 1 to Chapter 6 and read the code without running it. If you can say what each line does, you're ready for Part II.

PART II

Thinking in Lists

> Part I used lists as boxes of things. Part II opens the box. You'll see what a list is made of, then learn the two ways Lisp programmers work through lists: recursion, and functions that take other functions. By the end of Part II, lists will feel like the natural shape for almost any data.

CHAPTER 7.

Cons Cells and List Structure

7.1. `cons`: adding to the front

```
(cons 1 '(2 3))        ; => (1 2 3)
(cons 'a '(b c))       ; => (A B C)
(cons 'a nil)          ; => (A)
```

`cons` takes a value and a list, and returns a new list with the value added at the front. The original list is not changed.

That sounds like a small convenience. It's actually the whole story. Every list is built from `cons`, and only from `cons`.

7.2. The cons cell

`cons` makes one small object called a *cons cell*, or just a *cons*. A cons has exactly two parts:

- the **car**, its first part,
- the **cdr** (say "could-er"), its second part.

Draw a cons as a box with two halves:

```
(cons 'a 'b)

+---+---+
| A | B |
+---+---+
 car cdr
```

```
(car (cons 'a 'b))     ; => A
(cdr (cons 'a 'b))     ; => B
```

The names come from the IBM 704 computer that the first Lisp ran on, in 1958: "Contents of the Address part of the Register" and "Contents of the Decrement part of the Register". They mean nothing today, but you'll see them everywhere in Lisp code, so learn to read them.

7.3. A list is a chain of conses

Here is how `(1 2 3)` is really stored:

```
+---+---+   +---+---+   +---+---+
| 1 | *-+-->| 2 | *-+-->| 3 |NIL|
+---+---+   +---+---+   +---+---+
```

Three conses. Each car holds an element. Each cdr points to the next cons. The last cdr is `nil`, which marks the end.

You can build it yourself:

```
(cons 1 (cons 2 (cons 3 nil)))   ; => (1 2 3)
```

Read it from the inside out: `(cons 3 nil)` is `(3)`, consing `2` on the front gives `(2 3)`, and consing `1` gives `(1 2 3)`.

If you know Java's `LinkedList`: a cons is a node, the car is the value, and the cdr is the `next` pointer. Lisp just shows you the nodes directly.

This explains several things from Part I:

- **`first` is `car` and `rest` is `cdr`.** `first` takes the car of the first cons. `rest` takes its cdr, which is the rest of the chain. The two pairs of names do exactly the same thing. `first` and `rest` read better when you think of a list. `car` and `cdr` read better when you think of a single cons.
- **`nil` is both an atom and a list.** `nil` isn't a cons, so it's an atom. It's what every chain ends with, so it's the empty list.
- **`last` returns a list.** `last` gives you the last *cons* of the chain. A cons whose cdr is `nil` prints as a one-element list, so `(last '(1 2 3))` shows `(3)`.

7.4. Lists inside lists

A car can point to another list:

```
((A B) C)

+---+---+   +---+---+
| * | *-+-->| C |NIL|
+-+-+---+   +---+---+
  |
  v
+---+---+   +---+---+
| A | *-+-->| B |NIL|
+---+---+   +---+---+
```

The top row is the outer list, with two elements. The first element is itself a chain of two conses.

```
(cons (list 'a 'b) '(c))   ; => ((A B) C)
```

7.5. Dotted pairs

What if the cdr of a cons isn't a list?

```
(cons 1 2)             ; => (1 . 2)
```

Lisp prints a dot between the car and the cdr. This is called a *dotted pair*. It's just one cons with two values in it.

Compare these carefully:

```
(list (cons 1 2)       ; one cons: car 1, cdr 2
      (list 1 2)       ; two conses: car 1, then car 2, then NIL
      (cons 1 '(2)))   ; same as (list 1 2)
;; => ((1 . 2) (1 2) (1 2))
```

A chain that ends in something other than `nil` is called an *improper list*:

```
(cons 1 (cons 2 3))    ; => (1 2 . 3)
```

A chain that ends in `nil` is a *proper list*. Every list you've used so far was proper. Most list functions expect proper lists. `(length '(1 2 . 3))` is an error.

Dotted pairs are useful on their own, as a cheap way to keep two values together. You'll use them later in this chapter.

You can also *type* a dotted pair. Put spaces around the dot:

```
'(key . bedroom)       ; => (KEY . BEDROOM)
(cdr '(key . bedroom)) ; => BEDROOM
```

7.6. Testing for conses

```
(consp '(1 2))         ; => T
(consp '(1 . 2))       ; => T
(consp nil)            ; => NIL
(listp nil)            ; => T
```

`consp` asks "is it a cons?". `listp` asks "is it a cons or `nil`?". The only difference is `nil`: the empty list is a list, but it isn't a cons. Chapter 8 depends on this difference.

7.7. Shortcuts: `cadr`, `cddr` and friends

Taking the car of the cdr is so common that it has its own name:

```
(car (cdr '(a b c)))   ; => B
(cadr '(a b c))        ; => B
```

The letters between `c` and `r` say what to do, read from **right to left**. `cadr` is "cdr, then car". Some more:

Name	Means	On `(a b c d)`	Same as
`cadr`	car of cdr	`B`	`second`
`cddr`	cdr of cdr	`(C D)`	
`caddr`	car of cdr of cdr	`C`	`third`
`caar`	car of car	(error here)	
`cdar`	cdr of car	(error here)	

`caar` and `cdar` need a list whose first element is itself a cons, such as `((a . 1) (b . 2))`: `caar` gives `A`, `cdar` gives `1`.

Use `second` and `third` when you mean list positions. Use the `c...r` names when you're working with conses, or when reading older code. Combinations go up to four letters, such as `cadddr`, but anything beyond three is hard to read.

`nthcdr` skips a number of conses:

```
(nthcdr 2 '(a b c d))  ; => (C D)
```

7.8. Sharing structure

```
(defparameter *tail* '(2 3))
(defparameter *whole* (cons 1 *tail*))
*whole*                ; => (1 2 3)
```

`cons` didn't copy `*tail*`. The new cons points straight at it:

```
*whole*      *tail*
   |            |
   v            v
+---+---+   +---+---+   +---+---+
| 1 | *-+-->| 2 | *-+-->| 3 |NIL|
+---+---+   +---+---+   +---+---+
```

That's why adding to the front of a list is fast, no matter how long the list is. It takes one new cons.

Adding to the **end** is different. `append` has to copy every cons of the lists before the last one, because their final cdrs must change to point onward. On a long list that's slow. Lisp programmers build lists from the front and, when they need the other order, call `reverse` once at the end.

Sharing is safe as long as nothing changes the shared conses. Everything in this guide so far makes new conses instead of changing old ones, so you can share freely.

7.9. `push` and `pop`: a list in a variable

`push` conses a value onto the front of the list stored in a variable, and stores the new list back in the variable:

```
(defparameter *stack* nil)
(push 'a *stack*)      ; => (A)
(push 'b *stack*)      ; => (B A)
(push 'c *stack*)      ; => (C B A)
*stack*                ; => (C B A)
```

`(push 'a *stack*)` does the same as `(setf *stack* (cons 'a *stack*))`. It's shorter and says what you mean.

`pop` does the opposite. It returns the first element and stores the rest back in the variable:

```
(pop *stack*)          ; => C
*stack*                ; => (B A)
```

Popping an empty list returns `nil` and leaves the variable as `nil`.

The names come from a *stack* of plates: you add to the top and take from the top. Java's `Deque` has `push` and `pop` methods that work the same way.

7.9.1. Building a list in order

`push` adds to the front, so the list comes out backwards. Build it with `push`, then `reverse` it once at the end:

```
(let ((result nil))
  (push 1 result)
  (push 2 result)
  (push 3 result)
  (reverse result))    ; => (1 2 3)
```

`push` works on local variables too. This "push, then reverse" pattern is one you'll use often from Chapter 8 onwards.

7.9.2. `push` changes the variable, not the list

`push` never changes an existing cons. It makes a new one and points the variable at it. Other variables that held the old list still hold the old list:

```
(defparameter *original* '(1 2))
(defparameter *copy* *original*)
(push 0 *original*)
(list *original* *copy*)   ; => ((0 1 2) (1 2))
```

`*copy*` didn't change, because nothing touched the conses it points to.

7.10. Association lists

A list of dotted pairs makes a simple lookup table. Each pair holds a *key* in its car and a *value* in its cdr:

```
(defparameter *colours*
  '((apple . red)
    (banana . yellow)
    (grape . purple)))
```

This is called an *association list*, or *alist*. `assoc` finds the pair for a key:

```
(assoc 'banana *colours*)        ; => (BANANA . YELLOW)
(cdr (assoc 'banana *colours*))  ; => YELLOW
(assoc 'kiwi *colours*)          ; => NIL
```

`assoc` returns the whole pair, not the value. Take its `cdr` to get the value.

When the key is missing, `assoc` returns `nil`, and `(cdr nil)` is also `nil`. So `(cdr (assoc 'kiwi *colours*))` is `nil`, not an error. Combine it with `or` for a default:

```
(or (cdr (assoc 'kiwi *colours*)) 'unknown)   ; => UNKNOWN
```

In Java you'd use a `HashMap` for this. For a few dozen entries an alist is simpler, and you can read it at a glance. Part III covers hash tables, for when you have many entries.

7.10.1. Newer entries hide older ones

`assoc` returns the **first** pair whose key matches. So pushing a new pair onto the front changes the answer, without removing the old pair:

```
(push '(apple . green) *colours*)
(cdr (assoc 'apple *colours*))   ; => GREEN
*colours*
;; => ((APPLE . GREEN) (APPLE . RED) (BANANA . YELLOW) (GRAPE . PURPLE))
```

The old `(APPLE . RED)` is still there, hidden behind the new one. This is a cheap way to update an alist, and the project uses it below.

7.10.2. The value can be anything

The cdr of a pair can be a list. Look at what happens then:

```
(cons 'kitchen '("A warm kitchen." (hallway garden)))
;; => (KITCHEN "A warm kitchen." (HALLWAY GARDEN))
```

A pair whose cdr is a list prints as an ordinary list. The other way round: any list is a pair whose car is its first element. So a list of lists is also an alist, with each list's first element as its key:

```
(assoc 'garden
       '((kitchen "A warm kitchen." (hallway garden))
         (garden "An overgrown garden." (kitchen))))
;; => (GARDEN "An overgrown garden." (KITCHEN))
```

Your rooms already have this shape. The project step uses it.

7.10.3. What `assoc` can compare

`assoc` compares keys the same way `member` does. It works for symbols, numbers and characters, but not for strings. Use symbols as keys. Part IV shows how to make it work with strings.

7.11. Common mistakes

- **Using `cons` to join two lists:** `(cons '(1 2) '(3 4))` gives `((1 2) 3 4)`. `cons` adds *one* element to the front, and here the element is the list `(1 2)`. To join lists, use `append`.
- **Putting the list first:** `(cons '(2 3) 1)` gives `((2 3) . 1)`. The element comes first, the list second: `(cons 1 '(2 3))`.
- **Making a dotted pair by accident:** `(cons 1 2)` is `(1 . 2)`, not `(1 2)`. For a two-element list, write `(list 1 2)`.
- **Expecting `cons` or `push` to add at the end:** both add at the front. Build from the front and `reverse` at the end.
- **Using `push` without a variable:** `(push 'x '(a b))` is an error, because `push` needs somewhere to store the new list. Use `cons`, or put the list in a variable first.
- **Reading `cadr` left to right:** the letters run right to left. `cadr` is "cdr first, then car", which is `second`.
- **Expecting `assoc` to return the value:** it returns the pair. Use `(cdr (assoc key alist))`.
- **String keys in an alist:** `(assoc "b" '(("a" . 1) ("b" . 2)))` returns `nil`. Use symbols as keys.

- **Calling `length` on an improper list:** `(length '(1 2 . 3))` gives an error saying a value is not of type `LIST`. Check how the list was built: somewhere a `cons` got a non-list as its second argument.

7.12. Project step: all rooms in one list

Your rooms are lists whose first element is the room name. Put them all in one list and you have an alist of rooms. This also adds a new room, the cellar, below the kitchen:

```
(defparameter *rooms*
  '((bedroom "A small bedroom with an unmade bed." (hallway))
    (hallway "A narrow hallway with creaky floorboards." (bedroom kitchen))
    (kitchen "A warm kitchen that smells of bread." (hallway garden cellar))
    (garden "An overgrown garden." (kitchen))
    (cellar "A damp cellar. Something drips in the dark." (kitchen))))
```

Now `find-room` becomes one line. Run this to replace the Chapter 6 version:

```
(defun find-room (name)
  "Return the room whose name is NAME, or NIL if there is none."
  (assoc name *rooms*))
```

Your `go-to` from Chapter 6 calls `find-room`, so it now uses the new list without any change. This is what the accessors and small functions from Chapter 5 were for: you changed how rooms are stored, and only one function had to change.

There's one catch. `*current-room*` still holds the list from the old `*bedroom*` variable, whose exits don't know about the cellar. Point it at the room from the new list:

```
(setf *current-room* (find-room 'bedroom))
(list (go-to 'hallway)
      (go-to 'kitchen)
      (go-to 'cellar))
```

```
("You are in the Hallway. A narrow hallway with creaky floorboards."
 "You are in the Kitchen. A warm kitchen that smells of bread."
 "You are in the Cellar. A damp cellar. Something drips in the dark.")
```

If you kept the locked-garden `go-to` from Exercise 6.10, moves into the garden still need `*garden-unlocked*` to be `t`. Everything else works the same.

The old variables `*bedroom*`, `*hallway*` and so on still exist, but the game no longer uses them. From now on, rooms live in `*rooms*`.

7.12.1. Items

A game needs things to pick up. A second alist says where each item is. The key is the item, the value is the name of the room it's in:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
```

```
    (letter . bedroom)))

(defun item-location (item)
  "Return the name of the room ITEM is in, or PLAYER if it is carried."
  (cdr (assoc item *item-locations*)))

(defun item-name (item)
  "Return ITEM as a lower-case string, for messages."
  (string-downcase (symbol-name item)))
```

```
(item-location 'key)   ; => KITCHEN
(item-name 'key)       ; => "key"
```

When the player takes an item, its location becomes the symbol `player`. Instead of changing the old pair, push a new one on the front. `assoc` finds the newest pair first, so the old one is hidden:

```
(defun take (item)
  "Pick up ITEM if it is in the current room."
  (cond ((eq (item-location item) 'player)
         "You already have that.")
        ((eq (item-location item) (room-name *current-room*))
         (push (cons item 'player) *item-locations*)
         (concatenate 'string "You take the " (item-name item) "."))
        (t
         "You don't see that here.")))
```

Try it in the kitchen:

```
(setf *current-room* (find-room 'kitchen))
(list (take 'lamp)
      (take 'key)
      (take 'key)
      (item-location 'key))
```

```
("You don't see that here." "You take the key." "You already have that." PLAYER)
```

The lamp is in the cellar, so the first `take` fails. The key is in the kitchen, so the second works. After that, the key is carried.

Run the block a second time and the first `take 'key` says “You already have that.”, because the key is still marked as carried. To start over, re-run the `defparameter` for `*item-locations*`.

The game can’t yet list the items in a room, because that means looking at every pair in the alist. Chapter 8 teaches you how to walk through a whole list.

7.13. Exercises

7.1 Build each list using only `cons` and `nil`:

1. `(1 2 3)`
2. `(A (B C))`

Then draw the box diagram for `(A (B C))`, in the style of this chapter.

7.2 Predict the result of each line, then check:

```
(cons 'a 'b)
(cons '(a) '(b))
(cons nil nil)
(cdr '(a))
(car '((x y) z))
```

7.3 Write three expressions:

1. Get `C` from `'(a b c d)` with `caddr`.
2. Get `(C D)` from `'(a b c d)` with `cddr`.
3. Get `1` from `'((a . 1) (b . 2))` with `cdar`.

7.4 Write a function `swap-pair` that takes a dotted pair and returns a new pair with the car and cdr swapped. `(swap-pair '(1 . 2))` gives `(2 . 1)`.

7.5 Predict the value `pop` returns and the final value of `*s*`:

```
(defparameter *s* '(x))
(push 'y *s*)
(push 'z *s*)
(pop *s*)
```

7.6 Make an alist `*capitals*` that maps `tunisia` to `tunis`, `france` to `paris` and `japan` to `tokyo`. Write a function `capital-of` that returns the capital of a country, or `unknown` if the country isn't in the list.

7.7 Each line has a mistake. Say what's wrong and fix it.

```
(cons '(1 2) '(3 4))                        ; wanted (1 2 3 4)
(cdr (assoc "b" '(("a" . 1) ("b" . 2))))    ; wanted 2
(push 'x '(a b))                            ; wanted (X A B)
```

7.8 project Write `drop`, which takes an item. If the player carries it, the item moves to the current room and `drop` returns a message such as `"You drop the key."`. If not, it returns `"You don't have that."`.

7.9 project Add a shed reached from the garden. Without retyping `*rooms*`, push two new rooms onto it:

- a new garden, the same as the old one but with exits `(kitchen shed)`,
- a shed with the description `"A wooden shed full of old tools."` and the exit `(garden)`.

Then put the player in the garden and walk into the shed. Why does the new garden replace the old one?

CHAPTER 8.

Recursion

Most of the work a program does is walk through a collection of things and do something to each one. In Java you write a `for` loop. In Lisp, the natural first tool is a function that calls itself.

8.1. The idea in one sentence

A recursive function solves a big case by calling itself on a smaller case, until the case is so small that the answer is obvious.

8.2. A list is either empty or it isn't

Look again at how a list is built (Chapter 7). Every list is one of two things:

- the empty list, `nil`, or
- a cons: one element, with a shorter list behind it.

The rule gives you a recipe. A function that walks a list asks one question: is this list empty?

- **If it is,** return the answer for the empty case. This is the *base case*, and it stops the recursion.
- **If it isn't,** do something with `(first lst)`, and call yourself on `(rest lst)`, which is one cons shorter.

The recursion always moves toward the empty list, so it always stops.

```
(defun all-positive-p (numbers)
  "Return T if every number in NUMBERS is greater than zero."
  (cond ((null numbers) t)
        ((plusp (first numbers)) (all-positive-p (rest numbers)))
        (t nil)))
```

```
(list (all-positive-p '(1 2 3))
      (all-positive-p '(1 -2 3))
      (all-positive-p '()))
;; => (T NIL T)
```

Read the three clauses with the two questions in mind:

- `(null numbers)`: the list is empty. Every number in an empty list is positive, so the answer is `t`. This is the base case.
- `(plusp (first numbers))`: the first number is fine, so the answer depends on the rest. Call yourself on `(rest numbers)`.
- otherwise: the first number isn't positive, so the answer is `nil`, without looking further.

If you forget the first clause, the function never stops. That's the mistake to watch for, and it's in the "Common mistakes" section below.

8.3. Building an answer as you come back

The previous example returned an answer found somewhere in the list. Often you want to build a *new list*. Use `cons` to put something on the front of the answer for the rest:

```
(defun double-all (numbers)
  "Return a list of each number in NUMBERS, doubled."
  (if (null numbers)
      nil
      (cons (* 2 (first numbers))
            (double-all (rest numbers)))))
```

```
(double-all '(1 2 3))     ; => (2 4 6)
(double-all '())          ; => NIL
```

The shape is worth naming: **handle the first element, then trust the recursive call for the rest.**

For `(double-all '(1 2 3))`, Lisp can't finish the `cons` until it knows `(double-all '(2 3))`. So the calls pile up:

```
(double-all (1 2 3))  needs (double-all (2 3))
(double-all (2 3))    needs (double-all (3))
(double-all (3))      needs (double-all ())
(double-all ())       → NIL, and now each call can finish
(double-all (3))      → (6)
(double-all (2 3))    → (4 6)
(double-all (1 2 3))  → (2 4 6)
```

The base case answers first, and the answers come back up the chain. The `cons` calls happen on the way *out*.

8.4. Watching it happen with `trace`

When recursion is hard to follow, ask Lisp to show you every call:

```
(trace double-all)
(double-all '(1 2 3))
(untrace double-all)
```

The block shows only the final value. The trace lines appear in the **REPL window**, so switch to it (`C-x o`) to read them. They look like this:

```
0: (DOUBLE-ALL (1 2 3))
  1: (DOUBLE-ALL (2 3))
    2: (DOUBLE-ALL (3))
      3: (DOUBLE-ALL NIL)
      3: DOUBLE-ALL returned NIL
    2: DOUBLE-ALL returned (6)
  1: DOUBLE-ALL returned (4 6)
0: DOUBLE-ALL returned (2 4 6)
```

The number at the start of a line is the depth. Every call indents one step further and returns one step earlier. You can see the base case at the bottom, and the answers coming back up.

`untrace` turns tracing off, and so does restarting Lisp. Trace one function at a time; if you trace something that calls another traced function, the output becomes a wall of lines.

This is the single most useful tool for learning recursion. Whenever a recursive function surprises you, trace it.

8.5. The accumulator pattern

Building the answer on the way out is elegant, but the calls pile up, and each one adds a step of work on the way back. There's a second way: carry the answer *with* you as an extra parameter, and add to it on the way in.

```
(defun double-all-into (numbers result)
  "Cons each number in NUMBERS, doubled, onto RESULT."
  (if (null numbers)
      result
      (double-all-into (rest numbers)
                       (cons (* 2 (first numbers)) result))))
```

```
(double-all-into '(1 2 3) nil)   ; => (6 4 2)
```

`(6 4 2)` is backwards. Consing onto the front while walking forwards does that: 1 goes on first, then 2 lands in front of it, then 3. Fix it with one `reverse` at the end:

```
(defun double-all (numbers)
  "Return a list of each number in NUMBERS, doubled."
  (reverse (double-all-into numbers nil)))
```

```
(double-all '(1 2 3))            ; => (2 4 6)
```

The parameter `result` is called an *accumulator*. It holds the answer so far. Each call adds one thing to it and hands it on. The base case doesn't build anything: it just returns what has been accumulated.

Because the `cons` happens on the way in, nothing is left waiting on the way out. Nothing piles up. A call like this, where the recursive call is the very last thing the function does, is in *tail position*, and Lisp can turn it into a plain jump. The section "How deep can you go" comes back to this.

Which pattern to use:

Pattern	Use when
Build with `cons` on the way out	The result has the same order as the input, or the order doesn't matter
Accumulator, then `reverse`	You are walking a long list, or the result is naturally built backwards

When in doubt, use the first. Reach for the accumulator when a list is long or when the recursion would otherwise go deep.

8.6. Recursion on numbers

Lists aren't the only thing that can be smaller. Any number can be one smaller than itself, and the base case is whatever number stops the process.

```
(defun countdown (n)
  "Return a list of the numbers from N down to 1."
  (if (zerop n)
      nil
      (cons n (countdown (1- n)))))
```

```
(countdown 5)      ; => (5 4 3 2 1)
(countdown 0)      ; => NIL
```

```
(defun factorial (n)
  "Return N!, the product of the numbers from 1 to N."
  (if (zerop n)
      1
      (* n (factorial (1- n)))))
```

```
(list (factorial 0) (factorial 5) (factorial 30))
```

```
(1 120 265252859812191058636308480000000)
```

The last value is exact, because Lisp integers have no size limit (Chapter 3). In Java you'd need `BigInteger` for that.

Notice the base cases: `nil` for building a list, `1` for a product. A sum would use `0`, and a yes/no question would use `t` or `nil`. **The base case returns the answer for "nothing", and that answer must have the right type.** Writing `nil` where a number is expected is one of the four mistakes at the end of this chapter.

8.7. Recursion on nested lists

A list whose elements are themselves lists is a *tree*. Chapter 7 drew one. Trees are the natural home of recursion, because a tree is either an atom, or a cons with two smaller trees inside it.

Here is a function that counts every atom in a tree:

```
(defun count-atoms (tree)
  "Return the number of atoms in TREE, at any depth."
  (cond ((null tree) 0)
        ((consp tree) (+ (count-atoms (car tree))
                         (count-atoms (cdr tree))))
        (t 1)))
```

```
(list (count-atoms '(a (b (c)) d))
      (count-atoms 5)
      (count-atoms nil))
;; => (4 1 0)
```

Three cases, and this time the recursion appears twice:

- `nil` is empty, so it holds no atoms. (It's a list, so it counts as zero, not one.)
- A cons is a tree of two halves, its car and its cdr, so add the counts of both. This is where a list and a tree differ: for a plain list, you only recurse on the `rest`.

- Anything else is an atom, so it counts as one.

Recursing on both halves ends, because each half contains strictly fewer conses than the whole.

8.8. Mutual recursion

A function can call another, which calls the first one back. Two functions that take turns are *mutually recursive*:

```
(defun my-even-p (n)
  "Return T when N is even, by counting down."
  (if (zerop n)
      t
      (my-odd-p (1- n))))

(defun my-odd-p (n)
  "Return T when N is odd, by counting down."
  (if (zerop n)
      nil
      (my-even-p (1- n))))
```

```
(list (my-even-p 4) (my-odd-p 4) (my-even-p 0))
;; => (T NIL T)
```

SBCL may warn "undefined function: MY-ODD-P" while you run the first `defun`, because `my-odd-p` doesn't exist yet. As Chapter 5 says, that's a note, not a problem. Once both are defined, the calls work.

They take turns right down to zero: `my-even-p 4` calls `my-odd-p 3`, which calls `my-even-p 2`, and so on. Whoever reaches zero decides.

8.9. How deep can you go

When a call is in tail position, Lisp can throw away the current frame and jump. On a stock SBCL:

```
(defun count-down-to-zero (n)
  "Return the symbol DONE after counting from N down to zero."
  (if (zerop n)
      'done
      (count-down-to-zero (1- n))))

(count-down-to-zero 1000000)   ; => DONE
```

A million calls, and no memory used up. The recursive call is the last thing `count-down-to-zero` does, so no frame is kept.

Now the other side. When the call is **not** in tail position, every frame stays alive until the answer comes back. `(countdown 100000)` builds a hundred thousand conses and needs a hundred thousand frames at once. SBCL has a fixed control stack, and when it runs out you get:

```
Control stack exhausted (no more space for function call frames).
```

The debugger opens. Press `q` to close it, or use the "abort" restart if the REPL asks, and everything else keeps working. Nothing is corrupted.

Two rules of thumb:

- A recursion that builds a result with `cons` (or `+`, or `append`) needs one frame per element. Fine for hundreds and thousands, not for millions.
- A tail recursion with an accumulator needs no frames at all. Use it for the long walks.

SBCL does this tail-call optimisation by default. Other Lisp implementations may not, and compiler settings can switch it off. Treat it as a useful property of your current setup, not as something to design around. When you meet `loop` in Part III, you'll see how Lisp programmers write the long walks without recursion at all.

8.10. How to write a recursive function

A recipe that works almost every time:

1. **Say what the function does in one sentence,** including what it returns for the empty list, or for zero, or for the smallest input. That sentence is your base case.
2. **Write the base case first.** Test it immediately, before writing anything else.
3. **Shrink the input.** For a list, that's `(rest lst)`. For a number, `(1- n)`. For a tree, the car and the cdr.
4. **Decide what to do with the smaller answer:** return it as is, wrap it with `cons`, add something to it, or pick between it and the current element.
5. **Test with an empty input, one element, then two.**
6. **If it surprises you, trace it.**

Step 3 is where the thinking happens. The function is correct when the smaller input is always genuinely smaller, and when the base case is reached for every input the function will ever get.

8.11. Common mistakes

- **No base case:** `(defun first-of (lst) (first-of lst))` never stops. SBCL prints "Control stack exhausted" and opens the debugger. Press `q`. Check that your `cond` or `if` has a clause for the empty list, and that it comes first.
- **A base case that is never reached:** counting down with `(1- n)` and testing `(zerop n)` works for `0` and positive numbers. `(my-even-p -4)` counts down forever. Check the base case against negative arguments, or add a clause that turns them around first.
- **Forgetting to use the recursive result:** `(defun my-length (lst) (if (null lst) 0 (my-length (rest lst))))` always returns `0`. The call was made, its answer was thrown away. It needs `(+ 1 (my-length (rest lst)))`.
- **Recursing on the input instead of the smaller input:** `(sum-list lst)` calling `(sum-list lst)` again. The "Control stack exhausted" message catches it. Look at the argument of the recursive call: it must be `(rest lst)`, `(1- n)`, or another smaller thing.
- **A base case of the wrong type:** `(defun sum-list (lst) (if (null lst) nil (+ (first lst) (sum-list (rest lst)))))` fails with "The value NIL is not a NUMBER" as soon as the recursion reaches the end. The base case for a sum is `0`.

- **Rebuilding a list in the wrong order, over and over:** using an accumulator and forgetting the `reverse`, then adding a second `reverse` inside the loop. Build backwards once, then reverse once.
- **Tracing everything:** `(trace)` on a function that's called in a tight loop floods the REPL. Trace one function, and `untrace` it when you're done.
- **Recursing where a built-in already does the job:** after Part III you'll reach for `loop` or `mapcar` for the common walks. Recursion earns its place on trees, on divide-and-conquer problems, and on anything where the shape of the data is itself recursive.

8.12. Project step: what's lying around

The game can move the player and take items, but the player can't see what's there. Write that now: walk the item alist and collect the items whose location matches.

```
(defun items-in (pairs place)
  "Return the items in the alist PAIRS whose place is PLACE."
  (cond ((null pairs) nil)
        ((eq (cdar pairs) place)
         (cons (caar pairs) (items-in (cdr pairs) place)))
        (t
         (items-in (cdr pairs) place))))
```

With the chapter's idea in mind, read the three clauses:

- empty alist: nothing found, so `nil`.
- the first pair's value is this place: keep its item, and look through the rest.
- otherwise: skip this pair, and look through the rest.

Every call passes `(cdr pairs)`, one pair shorter, so it stops at `nil`.

```
(defun items-here (place)
  "Return the items lying in PLACE."
  (items-in *item-locations* place))
```

Reset the items so this chapter starts from a known state:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
```

```
(setf *current-room* (find-room 'kitchen))
(list (items-here 'kitchen)
      (items-here 'cellar)
      (items-here 'player)
      (items-here 'garden))
;; => ((KEY) (LAMP) NIL NIL)
```

The third one is empty because the player is carrying nothing — and that's the same function doing the work. Carried items are marked with the symbol `player`, so asking for the items in `player` asks for the inventory.

Now turn that list of symbols into words a player can read:

```
(defun item-names (items)
  "Return a string listing ITEMS by name, separated by commas."
  (cond ((null items) "")
        ((null (cdr items)) (item-name (first items)))
        (t
         (concatenate 'string
                      (item-name (first items))
                      ", "
                      (item-names (rest items))))))
```

```
(list (item-names nil)
      (item-names '(key))
      (item-names '(key lamp)))
;; => ("" "key" "key, lamp")
```

The three clauses do three different jobs: nothing, exactly one, or the first one plus the rest. The middle clause exists so you don't get a stray `", "` at the end or a missing one in the middle. Exercise 8.11 makes it read "key and lamp".

Now let `look` mention what's there:

```
(defun look ()
  "Describe the room the player is in, and what is lying there."
  (let ((here (items-here (room-name *current-room*))))
    (if (null here)
        (describe-room *current-room*)
        (concatenate 'string
                     (describe-room *current-room*)
                     " You can see: "
                     (item-names here)
                     "."))))
```

```
(princ (look))
```

```
You are in the Kitchen. A warm kitchen that smells of bread. You can see: key.
```

And an inventory command, built from the same pieces:

```
(defun inventory ()
  "Describe what the player is carrying."
  (let ((carried (items-here 'player)))
    (if (null carried)
        "You are carrying nothing."
        (concatenate 'string
                     "You are carrying: "
                     (item-names carried)
                     "."))))
```

Check it against `take`:

```
(list (take 'key) (inventory))
;; => ("You take the key." "You are carrying: key.")
```

Both functions work with no extra bookkeeping. `take` marks an item with the symbol `player`, and `items-here` finds everything marked that way. The inventory is just another search through the same alist, using the same recursive walk.

That's the shape of a Lisp program: a few small functions that each walk a list or pick it apart, composed until they do something that looks like a game.

8.12.1. A bug: pairs that are hidden but not gone

Reset the items, take the key, and ask what's in the kitchen:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))
(list (take 'key) (items-here 'kitchen))
;; => ("You take the key." (KEY))
```

The key is in the player's hands, yet the kitchen still has it. The alist now holds two pairs for the key:

```
((KEY . PLAYER) (LAMP . CELLAR) (KEY . KITCHEN) (LETTER . BEDROOM))
```

`assoc` stops at the first pair, so `item-location` gives the right answer. `items-in` walks *every* pair, so it finds the old one too. The trick from Chapter 7 is safe only for code that looks things up with `assoc`. Code that walks the whole list sees everything.

The fix: keep exactly one pair per item. A recursive function builds a new alist in which the item's pair is replaced:

```
(defun set-location (pairs item place)
  "Return a copy of the alist PAIRS with ITEM moved to PLACE."
  (cond ((null pairs) nil)
        ((eq (caar pairs) item)
         (cons (cons item place)
               (set-location (cdr pairs) item place)))
        (t
         (cons (car pairs)
               (set-location (cdr pairs) item place)))))
```

```
(set-location '((lamp . cellar) (key . kitchen)) 'key 'player)
;; => ((LAMP . CELLAR) (KEY . PLAYER))
```

It's the same three-clause walk as `items-in`: stop at the end, replace the matching pair, copy every other pair. The old list is untouched.

Now `take` stores the new list instead of pushing a pair:

```
(defun take (item)
  "Pick up ITEM if it is in the current room."
  (cond ((eq (item-location item) 'player)
         "You already have that.")
        ((eq (item-location item) (room-name *current-room*))
         (setf *item-locations*
               (set-location *item-locations* item 'player))
         (concatenate 'string "You take the " (item-name item) "."))
        (t
         "You don't see that here.")))
```

Run the test again:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))
(list (take 'key) (items-here 'kitchen) (inventory))
;; => ("You take the key." NIL "You are carrying: key.")
```

If you wrote `drop` in Exercise 7.8, redefine it the same way:

```
(defun drop (item)
  "Put down ITEM in the current room, if the player carries it."
  (if (eq (item-location item) 'player)
      (progn
        (setf *item-locations*
              (set-location *item-locations* item
                            (room-name *current-room*)))
        (concatenate 'string "You drop the " (item-name item) "."))
      "You don't have that."))
```

The lesson is worth keeping. A shortcut that relies on how one function behaves, here "`assoc` finds the first pair", breaks as soon as other code reads the same data in a different way.

8.13. Exercises

8.1 Write `my-length`, which returns the number of elements in a list, without using `length`.

8.2 Write `my-sum`, which adds up the numbers in a list, without using `+` on the whole list at once. It should return `0` for an empty list.

8.3 Predict the result of each line, then check:

```
(count-atoms '(a (b) c))
(countdown 3)
(factorial 4)
(items-in '((lamp . cellar) (key . kitchen)) 'kitchen)
(double-all-into '(1 2) nil)
```

8.4 Write `my-reverse` using the accumulator pattern: a helper that takes the list and the answer so far, and a one-line `my-reverse` that calls it.

8.5 Write `my-find`, which takes an item and a list and returns the item when it is in the list, and `nil` when it isn't. The item can be anywhere in the list.

8.6 Write `count-symbol`, which takes a symbol and a tree and returns how many times the symbol appears in the tree, at any depth.

8.7 Write `flatten`, which turns a nested list into a flat one: `(flatten '(a (b (c)) d))` gives `(a b c d)`. It should also handle an empty list, giving `nil`.

8.8 Each function below is wrong. Say what's wrong without running it, then fix it and check.

```
(defun bad-length (lst)
  (if (null lst)
      0
      (bad-length (rest lst))))

(defun bad-sum (lst)
  (if (null lst)
      nil
      (+ (first lst) (bad-sum (rest lst)))))

(defun bad-countdown (n)
  (if (zerop n)
      nil
      (cons n (bad-countdown (1+ n)))))
```

Careful with the third one: fix it, but don't call the broken version with a large number. It only stops when the control stack runs out.

8.9 Write `count-conses`, which returns how many cons cells a tree is made of. `(count-conses '(a b))` gives `2`, and an atom gives `0`.

8.10 project Write `item-report`, which takes the item alist and returns a list of strings, one per item, saying where it is. Items the player carries should read differently from items lying in a room:

```
("coin is carried"
 "lamp is in the cellar"
 "key is in the kitchen"
 "letter is in the bedroom")
```

8.11 project Rewrite `item-names` so that two items read `"key and lamp"` and three read `"key, lamp and coin"`.

```
(item-names nil)              ; => ""
(item-names '(key))           ; => "key"
(item-names '(key lamp))      ; => "key and lamp"
(item-names '(key lamp coin)) ; => "key, lamp and coin"
```

8.12 project Write `loot`, which picks up everything lying in the current room. It returns `"There is nothing here to take."` when the room is empty, and a message naming what was taken otherwise. Write a recursive helper that calls `take` on each item in turn.

CHAPTER 9.

Functions as Values

Until now, a function was something you called. In Lisp a function is also a value, like a number or a list. You can store it in a variable, put it in a list, pass it to another function, and return it from a function. This chapter shows how, and why it's one of the most useful ideas in the language.

9.1. Getting hold of a function: `#'`

```
(defun twice (n)
  "Return N multiplied by two."
  (* n 2))

#'twice
```

The result looks like this:

```
#<FUNCTION TWICE>
```

`#'twice` means "the function named `twice`", the function itself, not a call to it. The `#<...>` notation is how Lisp prints objects that can't be typed back in as text.

`#'` is short for the special operator `function`, the same way `'` is short for `quote`:

```
(function twice)          ; => #<FUNCTION TWICE>
(functionp #'twice)       ; => T
(functionp 'twice)        ; => NIL
```

The last line matters. `'twice` is a symbol, the *name* of the function. `#'twice` is the function. Two different things, which the end of this chapter puts to use.

9.2. Two kinds of names

Why not just write `twice`, the way Python does? Because in Common Lisp, a symbol can name a function and a variable *at the same time*, and the two never collide:

```
(defparameter twice 100)

(list twice (twice 5))  ; => (100 10)
```

The first `twice` is in a normal argument position, so Lisp looks up the *variable*. The second is first in a list, so Lisp looks up the *function*. Each name has two separate slots: one for its value as a variable, one for its function.

That's why you need `#'`: it says "the function slot, please" in a place where Lisp would otherwise read the variable slot.

The same rule lets you use `list` as a variable name without losing the `list` function:

```
(let ((list '(1 2 3)))
  (list list (length list)))   ; => ((1 2 3) 3)
```

You'll see parameters called `list` in other people's code. It works, but it's confusing to read, so this guide uses `lst`.

Clean up the variable before going on, so it doesn't surprise you later. `makunbound` removes a variable's value:

```
(makunbound 'twice)
```

9.3. Calling a function you're given: `funcall`

A function stored in a variable can't be called by putting the variable first in a list. That would look in the function slot, which is empty. `funcall` calls whatever function you hand it:

```
(defparameter *operation* #'twice)

(funcall *operation* 21)    ; => 42
(funcall #'+ 1 2 3)         ; => 6
```

`funcall` takes the function first, then the arguments, exactly as you would write them in a normal call.

`funcall` also accepts a symbol. It looks up the function the symbol names at the moment of the call:

```
(funcall 'twice 4)          ; => 8
```

9.4. Arguments from a list: `apply`

Sometimes the arguments are already in a list. `apply` spreads the list out into separate arguments:

```
(apply #'+ '(1 2 3 4))      ; => 10
(apply #'max '(3 9 2))      ; => 9
```

`(apply #'+ '(1 2 3 4))` is the same as `(+ 1 2 3 4)`.

You can give some arguments directly before the list. The last argument must always be a list:

```
(apply #'+ 1 2 '(3 4))      ; => 10
(apply #'list 'a '(b c))    ; => (A B C)
```

How to choose:

Use	When
A normal call	You know the function when you write the code
`funcall`	The function is in a variable, the arguments are separate
`apply`	The function is in a variable, the arguments are in a list

9.5. Writing functions that take functions

Here is a function whose first parameter is itself a function:

```
(defun apply-twice (f x)
  "Call F on X, then call F on the result."
  (funcall f (funcall f x)))
```

```
(apply-twice #'twice 5)    ; => 20
(apply-twice #'reverse '(1 2 3))   ; => (1 2 3)
(apply-twice #'rest '(a b c d))    ; => (C D)
```

Inside `apply-twice`, `f` is an ordinary variable holding a function, so the body calls it with `funcall`. The caller decides *what* happens. `apply-twice` decides *how often*.

A function that takes or returns functions is called a *higher-order function*. Java has these too, through interfaces like `Function` and method references like `Math::abs`. In Lisp they need no special machinery.

9.6. `mapcar`: do it to every element

The most common higher-order function walks a list, calls a function on each element, and collects the results:

```
(mapcar #'twice '(1 2 3))         ; => (2 4 6)
(mapcar #'string-upcase '("a" "b")) ; => ("A" "B")
(mapcar #'first '((a 1) (b 2) (c 3))) ; => (A B C)
```

Compare the first line with `double-all` from Chapter 8. That recursive function, with its base case and its `cons`, did exactly what `mapcar` does with `#'twice`. `mapcar` is that pattern, written once, for any function.

With several lists, `mapcar` takes one element from each and passes them together:

```
(mapcar #'+ '(1 2 3) '(10 20 30))  ; => (11 22 33)
(mapcar #'list '(a b) '(1 2))      ; => ((A 1) (B 2))
```

The function must accept as many arguments as there are lists. If the lists have different lengths, `mapcar` stops at the end of the shortest:

```
(mapcar #'+ '(1 2 3) '(10 20))     ; => (11 22)
```

Chapter 10 covers the rest of this family: functions that keep or remove elements, find one, or combine them all into one value.

9.7. `lambda`: functions without names

Often the function you want to pass is tiny and used only once. Giving it a name with `defun` would be clutter. `lambda` makes a function on the spot:

```
(mapcar (lambda (n) (* n n)) '(1 2 3 4))   ; => (1 4 9 16)
```

`(lambda (n) (* n n))` is a function with one parameter `n` that returns `n` squared. It has the same shape as a `defun`, without the name:

```
(defun square (n) (* n n))   ← a named function
(lambda (n) (* n n))         ← the same function, unnamed
```

A `lambda` evaluates to a function object, so you don't need `#'` in front of it. You'll see `#'(lambda ...)` in older code. It means the same thing.

```
(funcall (lambda (a b) (+ a b)) 3 4)       ; => 7
((lambda (a b) (+ a b)) 3 4)               ; => 7
```

The second line puts the `lambda` itself first in the list. That's allowed, and it's the one case where something other than a name can come first. You'll rarely write it, but you'll recognise it.

Run a `lambda` on its own and you see something like:

```
#<FUNCTION (LAMBDA (A B)) {5383DB5B}>
```

The number in braces is a memory address. Yours will differ.

Everything from Chapter 5 works in a `lambda`: several body expressions, `&optional`, `&key`, `&rest`.

```
(mapcar (lambda (name &optional (greeting "Hi"))
          (concatenate 'string greeting ", " name))
        '("Ada" "Alan"))
;; => ("Hi, Ada" "Hi, Alan")
```

9.8. A `lambda` can use the variables around it

A `lambda` written inside a `let` or a function body can use the variables visible there:

```
(defun add-to-all (amount numbers)
  "Return NUMBERS with AMOUNT added to each one."
  (mapcar (lambda (n) (+ n amount)) numbers))

(add-to-all 10 '(1 2 3))    ; => (11 12 13)
```

The `lambda` has one parameter, `n`. It also uses `amount`, which isn't its own parameter. It belongs to `add-to-all`. The `lambda` can see it because it was written inside `add-to-all`, where `amount` is in scope.

A function that remembers variables from the place where it was written is called a *closure*. In Lisp every `lambda` is one, so you never have to ask for it.

9.9. Functions that return functions

A function can build a `lambda` and hand it back:

```
(defun make-adder (amount)
  "Return a function that adds AMOUNT to its argument."
  (lambda (n) (+ n amount)))
```

```
(defparameter *add-five* (make-adder 5))

(funcall *add-five* 10)             ; => 15
(mapcar (make-adder 100) '(1 2 3))  ; => (101 102 103)
```

`make-adder` has already returned when `*add-five*` is called, yet the function still knows that `amount` is `5`. The closure keeps the variable alive for as long as the function exists. Each call to `make-adder` creates a fresh `amount`, so `(make-adder 5)` and `(make-adder 100)` don't interfere.

9.10. A closure can change its variables

The variable a closure remembers is a real variable, not a frozen copy of its value. The closure can change it with `setf` or `incf`:

```
(defun make-counter ()
  "Return a function that counts up by one each time it is called."
  (let ((count 0))
    (lambda () (incf count))))

(defparameter *counter-a* (make-counter))
(defparameter *counter-b* (make-counter))

(list (funcall *counter-a*)
      (funcall *counter-a*)
      (funcall *counter-a*)
      (funcall *counter-b*))
;; => (1 2 3 1)
```

Each counter has its own `count`, created by its own call to `make-counter`. Nothing outside the closure can read or change that variable. It is private state, the job a private field does in a Java object, with no class around it.

9.11. Symbol or function?

Near the start of this chapter, `'twice` and `#'twice` turned out to be different things. Both work with `funcall`, but they behave differently when you redefine the function.

```
(defparameter *saved* #'twice)   ; the function object itself

(defun twice (n)                 ; a new, wrong definition
  (* n 3))

(list (funcall *saved* 5)         ; the old function, kept in the variable
      (funcall 'twice 5))         ; whatever TWICE means right now
;; => (10 15)
```

SBCL may print a warning that `twice` is being redefined. That's expected.

`#'twice` grabs the function as it is at that moment. Redefining `twice` later creates a new function, and `*saved*` still holds the old one. `'twice` is only a name, so `funcall` looks it up at each call and always finds the current definition.

Put the right definition back:

```
(defun twice (n)
  "Return N multiplied by two."
  (* n 2))
```

In everyday code, write `#'`. It's clearer and slightly faster. When you store functions in a table that should follow your edits while you work at the REPL, store their names. The project step below does exactly that.

9.12. `mapc`: when you only want the effect

`mapcar` collects results into a new list. If you call a function only for what it does, such as printing, the collected list is waste. `mapc` calls the function on each element the same way, and returns the original list instead of building a new one:

```
(mapc #'print '(1 2 3))
```

```
1
2
3
```

9.13. Common mistakes

9.13.1. Forgetting `#'`

```
(mapcar twice '(1 2 3))
;; error: The variable TWICE is unbound.
```

Without `#'`, `twice` in an argument position is read as a variable. Write `#'twice`.

9.13.2. Calling a parameter directly

```
(defun call-it (f)
  (f 10))
;; warning: undefined function: F
```

`(f 10)` looks for a *function* named `f`, and `f` is a variable. A function held in a variable is called with `(funcall f 10)`.

9.13.3. Quoting a `lambda`

```
(mapcar '(lambda (x) (* x x)) '(1 2))
;; error: The value (LAMBDA (X) (* X X)) is not of type (OR FUNCTION SYMBOL)
```

With a quote, the `lambda` is just a list of three elements, not a function. Write the `lambda` without a quote, or with `#'`.

9.13.4. `apply` without a list at the end

```
(apply #'+ 1 2)
;; error: The value 2 is not of type LIST
```

The last argument of `apply` must be a list. Use `funcall` when the arguments are separate.

9.13.5. Expecting `mapcar` to change the list

`mapcar` returns a new list and leaves the original alone. If you want a variable to hold the result, store it: `(setf *numbers* (mapcar #'twice *numbers*))`.

9.13.6. The wrong number of arguments

```
(mapcar #'cons '(a b c))
;; error: invalid number of arguments: 1
```

`mapcar` with one list calls the function with one argument. `cons` needs two, so it needs two lists.

9.14. Project step: a command table

So far you play the game by calling functions: `(take 'key)`, `(inventory)`. A real game reads commands such as `(take key)` and decides which function to run. Functions as values make that a short piece of code.

Start with a table, an alist that maps each command word to the name of the function that handles it:

```
(defparameter *commands*
  '((look . look)
    (inventory . inventory)
    (take . take)
    (go . go-to)
    (loot . loot))
  "Map each command word to the function that handles it.")
```

If you wrote `drop` in Exercise 7.8, add `(drop . drop)` to the list.

The values are symbols, not `#'look` and so on. `run-command` looks up the current definition every time, so when you fix a bug in `take` (as Chapter 8 did), the table uses the fixed version without being rebuilt.

```
(defun run-command (command)
  "Run COMMAND, a list such as (TAKE KEY), and return the reply."
  (let ((entry (assoc (first command) *commands*)))
    (if (null entry)
        "I don't know how to do that."
        (apply (cdr entry) (rest command)))))
```

`(first command)` is the verb, `(rest command)` is the list of its arguments, and `apply` spreads that list into the call. `(take key)` becomes `(take 'key)`, and `(look)` becomes `(look)` with no arguments. Like `funcall`, `apply` accepts a symbol and calls the function it names.

The words in the command don't need quotes inside, because the whole command is quoted once:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
```

```
(setf *current-room* (find-room 'kitchen))

(list (run-command '(take key))
      (run-command '(inventory))
      (run-command '(dance)))
;; => ("You take the key." "You are carrying: key." "I don't know how to do that.")
```

A whole session is a list of commands, and `mapcar` runs each one:

```
(defun play (commands)
  "Run each command in COMMANDS and return the list of replies."
  (mapcar #'run-command commands))
```

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))

(play '((take key) (take key) (inventory)))
;; => ("You take the key." "You already have that." "You are carrying: key.")
```

`mapcar` calls `run-command` on the commands in order, from first to last, so the second `take` sees the effect of the first.

A command with the wrong number of words, such as `(look kitchen)`, calls `look` with an argument it doesn't accept, and you land in the debugger. Press `q` to leave it (Appendix C). Later chapters show how to catch that kind of error.

Adding a command now means writing a function and adding one pair to `*commands*`. `run-command` never changes.

9.15. Exercises

9.1 Predict the result of each line, then check:

```
(funcall #'list 1 2)
(apply #'* 2 '(3 4))
(mapcar #'length '((a) (b c) ()))
(mapcar (lambda (x) (cons x x)) '(a b))
(apply-twice (lambda (s) (concatenate 'string s "!")) "hi")
```

9.2 Write `my-mapcar`, which takes a function and one list and does what `mapcar` does, using recursion and `funcall`.

9.3 Use `mapcar` and a `lambda` to turn `'("ada" "alan" "grace")` into `'("Hello, Ada" "Hello, Alan" "Hello, Grace")`.

9.4 Write `make-multiplier`, which takes a number and returns a function that multiplies its argument by that number.

9.5 Write `compose`, which takes two functions `f` and `g` and returns a function of one argument that calls `g` first, then `f` on the result. `(funcall (compose #'twice #'1+) 5)` should give `12`.

9.6 Each line below is wrong. Say what's wrong, then fix it.

```
(mapcar twice '(1 2))

(defun call-with-ten (f)
  (f 10))

(funcall '(lambda (x) (+ x 1)) 5)

(apply #'max 3 7)
```

9.7 Write `make-stepper`, which takes a start and a step and returns a function. Its first call returns the start, and each later call returns the previous value plus the step:

```
(let ((s (make-stepper 10 5)))
  (list (funcall s) (funcall s) (funcall s)))
;; => (10 15 20)
```

9.8 Use `mapcar` to build the alist `((A . 1) (B . 2) (C . 3))` from the lists `(a b c)` and `(1 2 3)`, in one short expression.

9.9 `*table-a*` and `*table-b*` below both point at `shout`. Predict what each call returns after `shout` is redefined, then check:

```
(defun shout (s) (string-upcase s))

(defparameter *table-a* (list (cons 'shout #'shout)))
(defparameter *table-b* (list (cons 'shout 'shout)))

(defun shout (s) (concatenate 'string (string-upcase s) "!"))

(list (funcall (cdr (assoc 'shout *table-a*)) "hey")
      (funcall (cdr (assoc 'shout *table-b*)) "hey"))
```

9.10 project Add short forms to the command table: `l` for `look` and `i` for `inventory`. Don't write any new functions.

9.11 project Write a `help` command that lists every command word in the table, and add it to `*commands*`. Reuse `item-names` from Exercise 8.11, which already turns a list of symbols into a readable string. The reply should look like:

```
"Commands: help, look, inventory, take, go and loot."
```

CHAPTER 10.

Working with Sequences

Chapter 8 wrote every list walk by hand. Chapter 9 showed that `mapcar` is one of those walks, written once for any function. Common Lisp has a whole family of these: functions that keep elements, remove them, find one, count them, combine them or sort them. Once you know the family, most of the recursive functions from Chapter 8 shrink to one line.

These functions work on *sequences*. A sequence is an ordered collection that you can walk from start to end. Lists are sequences, and so are strings, which are sequences of characters. The same function works on both:

```
(length '(a b c))      ; => 3
(length "abc")         ; => 3
(reverse "abc")        ; => "cba"
```

`mapcar` is the exception: it works on lists only. This chapter shows `map`, its sequence version.

10.1. Keeping and removing: `remove-if` and `remove-if-not`

`remove-if` takes a predicate and a sequence, and returns a new sequence without the elements the predicate says yes to:

```
(remove-if #'evenp '(1 2 3 4 5))                ; => (1 3 5)
(remove-if (lambda (n) (> n 10)) '(5 12 8 20))  ; => (5 8)
```

`remove-if-not` does the opposite: it keeps only the elements the predicate says yes to.

```
(remove-if-not #'evenp '(1 2 3 4 5))            ; => (2 4)
```

The name says what gets removed: `remove-if` removes the matches, `remove-if-not` removes the rest. Other languages call the second one “filter”.

On a string, the result is a string. `digit-char-p` returns a true value for a digit character such as `#\7`, and `nil` otherwise:

```
(remove-if #'digit-char-p "a1b2c3")             ; => "abc"
```

When you only want to remove one particular value, `remove` is shorter:

```
(remove 'b '(a b c b))      ; => (A C)
(remove #\space "a b c")    ; => "abc"
```

None of these change the sequence you give them. They return a new one:

```
(defparameter *numbers* (list 1 2 3 4))
(list (remove-if #'evenp *numbers*) *numbers*)
;; => ((1 3) (1 2 3 4))
```

10.1.1. `complement`

The Common Lisp standard marks the `-if-not` functions as deprecated, meaning "may be removed one day". Every implementation still has them and you'll see them everywhere, so use them freely. The alternative is `complement`, which takes a predicate and returns a new predicate that answers the opposite:

```
(funcall (complement #'evenp) 3)              ; => T
(remove-if (complement #'evenp) '(1 2 3 4))   ; => (2 4)
```

`complement` is a function that returns a function, like `make-adder` in Chapter 9.

10.2. Finding: `find`, `find-if`, `position`, `member-if`

`find` returns the first element equal to the one you ask for, and `find-if` returns the first one the predicate says yes to. Both return `nil` when nothing matches:

```
(find 3 '(1 2 3 4))              ; => 3
(find-if #'evenp '(1 3 4 5 6))   ; => 4
(find-if #'evenp '(1 3))         ; => NIL
```

`position` and `position-if` return *where* the match is, counting from zero like `nth`:

```
(position 'c '(a b c))             ; => 2
(position-if #'digit-char-p "abc7") ; => 3
(position 'z '(a b c))             ; => NIL
```

`find` has one blind spot. If the element you're looking for is `nil`, you can't tell "found `nil`" from "found nothing":

```
(find nil '(a nil b))     ; => NIL, but it was there
(position nil '(a nil b)) ; => 1, no doubt
```

`member` from Chapter 6 has an `-if` version too. It returns the rest of the list from the first match:

```
(member-if #'evenp '(1 3 4 5))   ; => (4 5)
```

10.3. Counting: `count` and `count-if`

```
(count 'a '(a b a c))            ; => 2
(count-if #'evenp '(1 2 3 4))    ; => 2
(count #\s "mississippi")        ; => 4
```

10.4. Asking about every element: `every` and `some`

`every` is true when the predicate says yes to every element. `some` is true when it says yes to at least one. `plusp` is true for numbers above zero:

```
(every #'plusp '(1 2 3))     ; => T
(every #'plusp '(1 -2 3))    ; => NIL
```

```
(some #'evenp '(1 3 4))      ; => T
(some #'evenp '(1 3 5))      ; => NIL
```

Compare the first line with `all-positive-p` from Chapter 8. That whole function is `(every #'plusp lst)`.

`every` on an empty list is `T`: no element says no. `some` on an empty list is `NIL`: no element says yes.

`some` returns the first true value the predicate gave, the same way `or` returns the first true value (Chapter 6). `digit-char-p` returns the digit's value, so:

```
(some #'digit-char-p "ab7c")   ; => 7
```

Like `mapcar`, both accept several sequences and pass one element from each. They stop at the end of the shortest:

```
(every #'< '(1 2 3) '(2 3 4))  ; => T, because 1<2, 2<3 and 3<4
```

`notany` and `notevery` are the negated versions: `(notany #'evenp lst)` means "no element is even".

10.5. The keyword arguments they share

Most of these functions accept the same keyword arguments (`&key`, Chapter 5). Learn them once and they work everywhere.

10.5.1. `:key`: look at part of each element

`:key` takes a function. Each element goes through that function before it is compared or tested:

```
(find 'b '((a 1) (b 2) (c 3)) :key #'first)      ; => (B 2)
(remove-if #'evenp '((a 1) (b 2)) :key #'second)  ; => ((A 1))
(count 'kitchen '((key . kitchen) (lamp . cellar) (box . kitchen))
       :key #'cdr)                                 ; => 2
```

The result is still the whole element, `(B 2)`, not just the part that was compared. `(find 'b alist :key #'car)` does the same job as `(assoc 'b alist)`.

10.5.2. `:test`: which equality to use

By default these functions compare with `eql`, which (like `eq`) says two strings are different unless they are the very same object. That's the string-key problem from Chapter 7 again:

```
(find "b" '("a" "b"))                  ; => NIL
(find "b" '("a" "b") :test #'string=)  ; => "b"
```

`:test` takes the function that decides whether two things count as equal. The next section lists the choices.

10.5.3. `:start`, `:end`, `:from-end` and `:count`

`:start` and `:end` limit the work to part of the sequence. `:from-end t` searches from the end. `:count` limits how many elements are removed:

```
(position #\a "banana" :start 2)      ; => 3
(position #\a "banana" :from-end t)   ; => 5
(remove 'a '(a b a c a) :count 2)     ; => (B C A)
```

Positions are always counted from the start of the whole sequence, even with `:start` or `:from-end`.

10.6. Four kinds of equality

You've used `eq` for symbols and `string=` for strings. Common Lisp has four general equality functions. Each one is looser than the one above it:

Function	True when
`eq`	Both are the very same object. Use it for symbols.
`eql`	Like `eq`, and also equal numbers of the same type, and equal characters. The default for `find`, `member`, `assoc` and the rest.
`equal`	Like `eql`, and also lists and strings with the same contents. Case matters.
`equalp`	Like `equal`, ignoring case in strings, and comparing numbers by value.

```
(list (eql #\a #\a)               ; T
      (eql 1 1.0)                 ; NIL, an integer and a float
      (equalp 1 1.0)              ; T
      (eql (list 1 2) (list 1 2)) ; NIL, two separate lists
      (equal (list 1 2) (list 1 2)) ; T, same contents
      (equal "abc" "ABC")         ; NIL
      (equalp "abc" "ABC"))       ; T
```

Don't use `eq` on numbers or characters. Whether two equal numbers are the same object is up to the implementation, so `eq` may give either answer. `eql` is always safe for them.

For lists of lists or strings, `:test #'equal` is usually what you want:

```
(member '(b 2) '((a 1) (b 2)) :test #'equal)   ; => ((B 2))
```

10.7. `map`: `mapcar` for any sequence

`map` works like `mapcar` on strings as well as lists. Its first argument says what kind of sequence to build, the same way `concatenate` does (Chapter 3). `char-upcase` turns one character into upper case, and `char-code` gives a character's number:

```
(map 'string #'char-upcase "abc")   ; => "ABC"
(map 'list #'char-code "AB")        ; => (65 66)
(map 'list #'+ '(1 2) '(10 20))     ; => (11 22)
```

10.8. `reduce`: combine everything into one value

`reduce` takes a function of two arguments and uses it to combine the elements, from left to right:

```
(reduce #'+ '(1 2 3 4))     ; => 10
(reduce #'max '(3 9 2))     ; => 9
```

The steps for the first line:

```
(+ 1 2)   => 3
(+ 3 3)   => 6
(+ 6 4)   => 10
```

The first argument of each call is the result so far, the second is the next element. That's the accumulator from Chapter 8, with `reduce` doing the walk.

10.8.1. Starting value: `:initial-value`

`:initial-value` gives `reduce` a starting point. The first call combines it with the first element:

```
(reduce #'+ '(1 2 3) :initial-value 100)   ; => 106
```

It matters most for short lists. With one element, `reduce` returns that element without calling the function. With no elements, it calls the function with *no arguments*:

```
(reduce #'+ '(7))     ; => 7
(reduce #'+ '())      ; => 0, because (+) is 0
(reduce #'max '())    ; error: invalid number of arguments: 0
```

`(+)` with no arguments is `0` and `(*)` is `1`, so those two are safe. `max` needs at least one number. When the list may be empty, give a starting value, and `reduce` returns it unchanged:

```
(reduce #'max '() :initial-value 0)        ; => 0
```

With a starting value of `nil` and a `lambda`, `reduce` can build a list. This is `my-reverse` from Exercise 8.4:

```
(reduce (lambda (acc x) (cons x acc))
        '(1 2 3)
        :initial-value nil)
;; => (3 2 1)
```

10.8.2. Other keywords: `:from-end` and `:key`

`:from-end t` combines from right to left. With `#'list` as the function you can see the grouping:

```
(reduce #'list '(1 2 3))              ; => ((1 2) 3)
(reduce #'list '(1 2 3) :from-end t)  ; => (1 (2 3))
```

With `:from-end t`, the element comes first and the result so far second, the reverse of the normal order.

`:key` works as everywhere else:

```
(reduce #'+ '((a . 1) (b . 2) (c . 3)) :key #'cdr)   ; => 6
```

10.9. `sort`

`sort` takes a sequence and a predicate of two arguments. The predicate answers "should the first one come before the second?":

```
(sort (list 3 1 2) #'<)                ; => (1 2 3)
(sort (list 3 1 2) #'>)                ; => (3 2 1)
(sort (list "pear" "apple") #'string<)  ; => ("apple" "pear")
```

`:key` sorts by part of each element:

```
(sort (list '(b . 2) '(a . 1) '(c . 3)) #'< :key #'cdr)
;; => ((A . 1) (B . 2) (C . 3))
```

Strings are sequences too. `char<` compares two characters:

```
(sort (copy-seq "hello") #'char<)       ; => "ehllo"
```

10.9.1. `sort` destroys its argument

Every function so far in this chapter left its input alone. `sort` is different: it reuses the conses of the list you give it and rearranges them. Afterwards the original variable points at one cons somewhere in the middle of the result:

```
(defparameter *nums* (list 3 1 2))
(list (sort *nums* #'<) *nums*)
;; => ((1 2 3) (3)) in SBCL
```

`*nums*` still points at the cons holding `3`, which is now the last cons of the sorted list. Other implementations may leave something else behind. The rules that follow from this:

- **Always use the value `sort` returns.** To sort a variable, write `(setf *nums* (sort *nums* #'<))`.
- **To keep the original, sort a copy.** `copy-list` makes a new list with the same elements: `(sort (copy-list *nums*) #'<)`. `copy-seq` does the same for any sequence, strings included.
- **Never sort a quoted list.** `'(3 1 2)` written in your code is a constant, and changing a constant has undefined results. That's why the examples above use `(list 3 1 2)`.

A list fresh from `mapcar`, `remove-if` or `list` belongs to nobody else, so sorting it directly is safe.

10.9.2. `stable-sort`

When two elements are equal for the predicate, `sort` may put them in either order. `stable-sort` keeps their original order:

```
(stable-sort (list '(ada . 2) '(bob . 1) '(cy . 2)) #'< :key #'cdr)
;; => ((BOB . 1) (ADA . 2) (CY . 2))
```

`ada` stays ahead of `cy` because it was ahead before. `stable-sort` destroys its argument just like `sort`.

10.10. `remove-duplicates` and `subseq`

`remove-duplicates` returns a sequence in which each element appears once. By default it keeps the *last* copy of each; `:from-end t` keeps the first:

```
(remove-duplicates '(a b a c))              ; => (B A C)
(remove-duplicates '(a b a c) :from-end t)  ; => (A B C)
(remove-duplicates "mississippi")           ; => "msip"
```

It accepts `:test` and `:key` too. `subseq`, from Chapter 3, works on lists as well as strings:

```
(subseq '(a b c d) 1 3)    ; => (B C)
```

Most of these functions have a destructive twin, such as `delete` for `remove`. They save memory and come with the same rules as `sort`. This guide uses the safe versions.

10.11. Choosing the right function

You want	Use
A result for every element	`mapcar`, `map`
Only some elements	`remove-if-not`, `remove-if`
The first matching element	`find`, `find-if`
Where it is	`position`, `position-if`
How many match	`count`, `count-if`
Yes or no for all or any	`every`, `some`
One value from all elements	`reduce`
A new order	`sort`, `stable-sort`
Each element once	`remove-duplicates`

Write a recursive function when none of these fits, for example when you walk a tree as in `count-atoms`. These functions walk only the top level of a sequence.

10.12. Common mistakes

10.12.1. Ignoring what `sort` returns

```
(sort *nums* #'<)
*nums*    ; => (3), not the sorted list
```

Store the result: `(setf *nums* (sort *nums* #'<))`.

10.12.2. Sorting a quoted list

```
(sort '(3 1 2) #'<)
```

It may seem to work, but it modifies a constant in your code. Write `(sort (list 3 1 2) #'<)`, or sort a `copy-list`.

10.12.3. Mixing up `remove-if` and `remove-if-not`

`remove-if` throws away the matches. If you get exactly the elements you wanted to keep, missing, swap to `remove-if-not`.

10.12.4. Finding strings without `:test`

```
(find "key" '("lamp" "key"))   ; => NIL
```

The default test is `eql`. Add `:test #'string=` or `:test #'equal`.

10.12.5. Confusing `:key` and `:test`

```
(find "key" '("lamp" "key") :key #'string=)
;; error: invalid number of arguments: 1
```

`:key` takes a function of one argument that picks a part of each element. `:test` takes a function of two arguments that compares. `string=` needs two, so it belongs in `:test`.

10.12.6. `reduce` on an empty list

`(reduce #'max '())` calls `(max)` and fails. Give an `:initial-value` when the list can be empty.

10.12.7. Expecting `remove` to change the variable

Like `mapcar`, `remove` and friends return a new sequence. Store it with `setf` when you need to.

10.13. Project step: shorter item functions

Three functions from Chapter 8 walk the item alist by hand. Each one fits one of this chapter's patterns.

`items-in` keeps the pairs whose place matches, then takes the item from each pair. That's `remove-if-not` followed by `mapcar`:

```
(defun items-in (pairs place)
  "Return the items in the alist PAIRS whose location is PLACE."
  (mapcar #'car
          (remove-if-not (lambda (where) (eq where place))
                         pairs
                         :key #'cdr)))
```

```
(items-in '((lamp . cellar) (key . kitchen) (box . kitchen)) 'kitchen)
;; => (KEY BOX)
```

`set-location` copies every pair, replacing one. That's `mapcar` with a `lambda` that decides, pair by pair:

```
(defun set-location (pairs item place)
  "Return a copy of the alist PAIRS with ITEM moved to PLACE."
  (mapcar (lambda (pair)
            (if (eq (car pair) item)
                (cons item place)
                pair))
          pairs))
```

```
(set-location '((lamp . cellar) (key . kitchen)) 'key 'player)
;; => ((LAMP . CELLAR) (KEY . PLAYER))
```

`items-here`, `take`, `drop`, `loot` and `inventory` call these two by name, so they use the new versions without any change.

One improvement while you're here: list the inventory in alphabetical order. `items-here` returns a fresh list built by `mapcar`, so sorting it directly is safe:

```
(defun inventory ()
  "Describe what the player is carrying, in alphabetical order."
  (let ((carried (items-here 'player)))
    (if (null carried)
        "You are carrying nothing."
        (concatenate 'string
                     "You are carrying: "
                     (item-names (sort carried #'string<
                                       :key #'symbol-name))
                     "."))))
```

Check it. The test moves two items straight into the player's hands with `set-location`, so you don't have to walk around:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *item-locations* (set-location *item-locations* 'lamp 'player))
(setf *item-locations* (set-location *item-locations* 'key 'player))
(inventory)
;; => "You are carrying: key and lamp."
```

Without the `sort`, the reply would be `"key and lamp"` the other way round, following the order of the alist. The `defparameter` alist is quoted, but nothing here sorts or changes it: `set-location` builds a new list each time.

10.14. Exercises

10.1 Predict the result of each line, then check:

```
(remove-if-not #'evenp '(5 6 7 8))
(find-if #'numberp '(a b 3 c 4))
(count-if #'symbolp '(a 1 b 2 c))
(position #\n "banana")
(reduce #'* '(1 2 3 4))
(every #'symbolp '())
(remove-duplicates '(1 2 1 3 2))
```

10.2 Rewrite each of these Chapter 8 functions with a one-line body, using this chapter's functions: `my-sum` (8.2), `my-find` (8.5), `all-positive-p` and `my-reverse` (8.4). Name them `sum2`, `find2`, `all-positive2-p` and `reverse2` so they don't replace the originals.

10.3 Use `remove-if` to drop every string shorter than four characters from `'("a" "lisp" "is" "great")`.

10.4 Given:

```
(defparameter *scores* '((ada . 90) (alan . 72) (grace . 85)))
```

Write one expression for each:

1. The alist sorted from the highest score to the lowest, leaving `*scores*` untouched.
2. The first pair with a score below 80.
3. The total of all scores.

10.5 Write `my-remove-if` recursively, using `funcall`. It takes a predicate and a list.

10.6 Write `count-vowels`, which counts the vowels a, e, i, o and u in a string, upper or lower case. `(count-vowels "Common Lisp")` gives `3`. Hint: `find` can look for a character in the string `"aeiou"`.

10.7 Predict `T` or `NIL` for each, then check:

```
(eql 2 2)
(eql 2 2.0)
(equal "a" "a")
(equal '(1 (2)) '(1 (2)))
(equalp "Lisp" "LISP")
(eql (list 1) (list 1))
```

10.8 Each line below is wrong. Say what's wrong, then fix it.

```
(sort '(3 1 2) #'<)

(reduce #'max '())

(find "b" '("a" "b"))

(remove-if #'evenp *numbers*)   ; meant to remove the even numbers
                                ; from *numbers* for good
```

10.9 Use `reduce` to find the longest string in `'("hi" "hello" "hey")`.

10.10 Write `my-every`, which takes a predicate and a list and works like `every`, using only `some`, `complement` and `not`.

10.11 project Write `empty-rooms`, which returns the names of the rooms in `*rooms*` that have no items lying in them. Use `mapcar` and `remove-if`.

10.12 project Change `help` from Exercise 9.11 so that it lists the commands in alphabetical order, each word once, even if a pair was pushed twice.

Part II review

You can now read any list-shaped data at sight. The load-bearing ideas: everything is an object built from cons cells (Ch 7); recursion follows the shape of the data (Ch 8); functions are values you can pass (Ch 9); sequences are lists with a standard vocabulary (Ch 10); and `loop` is the do-what-I-mean iteration (Ch 11). The project's alists — `*rooms*`, `*item-locations*` — were this part's running example.

If any of these feel shaky, the fix is REPL time, not re-reading: take one chapter's exercises and redo two of them from scratch.

PART III

Core Tools

> Part II taught you to think in lists: recursion, functions as values, and the sequence functions. Part III gives you the everyday tools that sit on top of those ideas: loops, formatted output, the data shapes beyond lists, macros, the condition system, and the packages-and-files layer that turns source into a program.
>
> The chapters in this part don't depend on each other as tightly as Part II's did, and you can skip around within it once you've read Chapter 11. But the project steps build on the game as you left it at the end of Part II, so the ordering is the order the game grows in.

CHAPTER 11.

Loops

Part II taught you to think in lists: recursion, functions as values, and the sequence functions. This chapter opens the everyday tools with the other way to repeat work: loops.

Recursion and `mapcar` cover a lot, but some jobs read more naturally as "do this for each element" or "count from 1 to 10". Common Lisp has small loops for the simple cases (`dolist`, `dotimes`) and one large, flexible one (`loop`).

11.1. `dolist`: once per element

```
(dolist (x '(a b c))
  (print x))
```

```
A
B
C
```

`(dolist (x list) body...)` binds `x` to each element in turn and runs the body. `x` is a local variable, like one made by `let`.

`dolist` returns `nil`. It's for effects, such as printing or changing a variable, not for building a result. To return something, add a third element, the *result form*, which is evaluated after the last pass:

```
(let ((evens nil))
  (dolist (n '(1 2 3 4 5 6) (reverse evens))
    (when (evenp n)
      (push n evens))))
;; => (2 4 6)
```

`push` puts each number at the front, so the result form reverses the list at the end. That's the accumulator pattern from Chapter 8, with `dolist` doing the walk.

11.2. `dotimes`: a number of times

```
(dotimes (i 3)
  (print i))
```

```
0
1
2
```

`(dotimes (i n) body...)` runs the body `n` times, with `i` going from `0` up to `n` minus one. It also accepts a result form:

```
(let ((total 0))
  (dotimes (i 5 total)
```

```
    (incf total i)))
;; => 10, because 0 + 1 + 2 + 3 + 4
```

11.3. Leaving early: `return` and `return-from`

`(return value)` leaves a `dolist` or `dotimes` at once, and the loop returns `value`:

```
(dolist (x '(1 3 4 5 6))
  (when (evenp x)
    (return x)))
;; => 4
```

The loop stops at `4` and never looks at `5` or `6`.

`return` leaves only the innermost loop. To leave a whole function from inside nested loops, use `(return-from name value)`, where `name` is the function's name:

```
(defun find-pair-sum (target lst)
  "Return the first two elements of LST that add up to TARGET."
  (dolist (a lst)
    (dolist (b lst)
      (when (= (+ a b) target)
        (return-from find-pair-sum (list a b))))))
```

```
(list (find-pair-sum 10 '(1 4 6))
      (find-pair-sum 100 '(1 4 6)))
;; => ((4 6) NIL)
```

A plain `return` here would leave only the inner `dolist`, and the outer one would carry on with the next `a`. When nothing matches, both loops finish and return `nil`, so the function returns `nil`.

11.4. `do`: the old general loop

You'll meet `do` in older code. It sets up variables, steps them, and stops when a test is true:

```
(print
 (do ((i 0 (1+ i)))     ; variable, start value, step
     ((= i 3) 'done)    ; stop when true, then return DONE
   (print i)))          ; the body
```

```
0
1
2
DONE
```

Each variable is written `(name start step)`. Before each pass, `do` checks the test. When it is true, `do` returns the value after it. Most programmers reach for `loop` instead, so you only need to be able to read `do`.

11.5. `loop`: one macro, many clauses

`loop` reads almost like English. It is built from *clauses*: words such as `for`, `collect` and `when`, each followed by its arguments.

```
(loop for x in '(1 2 3)
      collect (* x 10))
;; => (10 20 30)
```

"For each `x` in the list, collect `x` times ten." `collect` builds the list in order, so no `reverse` is needed. The sections below go through the clauses one group at a time.

11.5.1. Counting with `for`

```
(loop for i from 1 to 5 collect i)        ; => (1 2 3 4 5)
(loop for i from 0 below 5 collect i)     ; => (0 1 2 3 4)
(loop for i below 3 collect i)            ; => (0 1 2)
(loop for i from 10 downto 7 collect i)   ; => (10 9 8 7)
(loop for i from 0 to 10 by 5 collect i)  ; => (0 5 10)
```

Words	Meaning
`from n`	Start at `n` (`0` if left out)
`to n`	Stop after `n`: `n` is included
`below n`	Stop before `n`: `n` is left out
`downto n`	Count down, `n` included
`by n`	Step by `n` instead of `1`

11.5.2. Walking lists and strings

```
(loop for x in '(a b c) collect (list x))     ; => ((A) (B) (C))
(loop for tail on '(a b c) collect tail)      ; => ((A B C) (B C) (C))
(loop for c across "abc" collect c)           ; => (#\a #\b #\c)
```

`in` gives each element. `on` gives each tail of the list: the whole list, then its `rest`, then the `rest` of that. `across` walks a string, one character at a time.

`by` works on lists too. It takes the function used to move forward, `cdr` by default:

```
(loop for x in '(a 1 b 2 c 3) by #'cddr collect x)   ; => (A B C)
```

11.5.3. Taking elements apart

The variable after `for` can be a pattern. Each element is split up to fit it:

```
(loop for (name . score) in '((ada . 90) (alan . 72))
      collect name)
;; => (ADA ALAN)

(loop for (a b) in '((1 2) (3 4))
      collect (+ a b))
;; => (3 7)
```

The first pattern matches dotted pairs, the shape of an alist. The second matches two-element lists. This is the cleanest way to walk an alist.

11.5.4. Several `for` clauses

With two `for` clauses, both step together, and the loop stops when either one runs out:

```
(loop for x in '(a b c)
      for i from 1
      collect (cons i x))
;; => ((1 . A) (2 . B) (3 . C))
```

`for i from 1` has no end, so the list decides when to stop.

11.5.5. `for` with `=`: computed values

`for var = start then next` starts with one value and computes each later value from the previous one:

```
(loop for n = 1 then (* n 2)
      repeat 5
      collect n)
;; => (1 2 4 8 16)
```

`repeat 5` stops the loop after five passes. Without `then`, the expression after `=` is evaluated again on every pass:

```
(loop for x in '(1 2 3)
      for square = (* x x)
      collect square)
;; => (1 4 9)
```

11.5.6. Collecting results

Clause	Result
`collect x`	A list of every `x`, in order
`append x`	The lists `x` joined into one list
`sum x`	The total
`count x`	How many times `x` was true
`maximize x`	The largest `x`
`minimize x`	The smallest `x`

```
(loop for x in '(1 2 3) sum x)                ; => 6
(loop for x in '(1 2 3 4) count (evenp x))    ; => 2
(loop for x in '(3 9 2) maximize x)           ; => 9
(loop for x in '((a b) (c)) append x)         ; => (A B C)
```

`maximize` and `minimize` on an empty list give a result the standard leaves open, so check for an empty list first.

11.5.7. Choosing elements: `when`, `unless`, `if`

```
(loop for x in '(1 2 3 4 5)
      when (evenp x) collect x)
;; => (2 4)
```

A `when` clause controls only the clause right after it. To control two, join them with `and`:

```
(print
 (loop for x in '(1 2 3 4)
       when (evenp x) collect x and do (print x)))
```

```
2
4
(2 4)
```

`if` has an `else`. With `into`, each `collect` fills its own named list:

```
(loop for x in '(1 2 3 4)
      if (evenp x) collect x into evens
      else collect x into odds
      finally (return (list evens odds)))
;; => ((2 4) (1 3))
```

With `into`, `loop` no longer returns the list by itself. The `finally` clause runs once after the last pass, and `(return ...)` inside it sets the value of the whole `loop`.

11.5.8. Running code: `do`

`do` inside a `loop` runs ordinary Lisp forms on every pass. It may be followed by several forms:

```
(loop for x in '(1 2)
      do (princ "item: ")
         (princ x)
         (print 'done))
```

```
item: 1
DONE item: 2
DONE
```

`print` starts with a newline and ends with a space, which explains the odd layout. A `loop` with only `do` returns `nil`.

11.5.9. Local variables: `with`

```
(loop with total = 0
      for x in '(1 2 3)
      do (incf total x)
      finally (return total))
;; => 6
```

`with` makes a variable once, before the first pass. Here `sum` would be shorter, but `with` helps when the work doesn't fit one of the collecting clauses.

11.5.10. Stopping: `while`, `until`, `return`

```
(loop for x in '(1 2 3 -1 4)
      while (plusp x)
      collect x)
;; => (1 2 3)

(loop for x in '(1 3 4 5)
      when (evenp x) return x)
;; => 4
```

`while` stops as soon as its test is false, `until` as soon as its test is true. `return` as a clause leaves the loop with a value. You can also write the Lisp form `(return x)` inside a `do`.

11.5.11. Yes-or-no questions: `always`, `never`, `thereis`

```
(loop for x in '(1 2 3) always (plusp x))              ; => T
(loop for x in '(1 2 3) never (> x 5))                 ; => T
(loop for x in '(a 7 b) thereis (and (numberp x) x))   ; => 7
```

These are `every`, `notany` and `some` from Chapter 10, as clauses. `thereis` returns the first true value, like `some`.

11.5.12. Clause order

Put `with` and `for` clauses first, then everything else: `repeat`, `while`, `when`, `collect`, `do` and the rest. The Common Lisp standard requires this order, and some implementations complain if you break it.

11.6. Which one to use

Job	Use
A result for every element	`mapcar`
Keep, find, count with one predicate	Chapter 10's functions
Effects for each element, such as printing	`dolist`
Something `n` times	`dotimes`
Counting ranges, several lists at once, filtering and collecting together, early exit with a result	`loop`
Trees, nested structure	Recursion (Chapter 8)

These overlap. `(mapcar #'twice lst)` and `(loop for x in lst collect (twice x))` do the same job. Pick the one that reads most clearly.

11.7. Common mistakes

11.7.1. Expecting `dolist` to return results

```
(dolist (x '(1 2 3)) (* x 2))   ; => NIL
```

The body's values are thrown away. Use `mapcar`, or `(loop for x in lst collect (* x 2))`.

11.7.2. `to` versus `below`

`(loop for i from 0 to 3 collect i)` gives four numbers, `(0 1 2 3)`. To match `nth` positions in a three-element list, use `below 3`.

11.7.3. `in` on a string

```
(loop for c in "abc" collect c)
;; error: The value "abc" is not of type LIST
```

`in` walks lists. Strings need `across`.

11.7.4. Forgetting `do`

```
(loop for x in '(1 2 3) (print x))
;; error: (PRINT X) found where a LOOP keyword was expected
```

Plain Lisp forms inside a `loop` go after `do`.

11.7.5. `finally` without `return`

```
(loop with total = 0
      for x in '(1 2 3)
      do (incf total x)
      finally total)    ; => NIL
```

`finally` runs its forms, but their value is thrown away. Write `finally (return total)`.

11.7.6. `return` inside nested loops

`return` leaves only the innermost loop. To leave the function, use `return-from` with the function's name.

11.7.7. `when` controlling one clause only

In `when (evenp x) collect x do (print x)`, the `print` runs for every `x`, even or odd. Join the clauses with `and` when both belong to the `when`.

11.8. Project step: a transcript and a shorter report

`play` from Chapter 9 returns a list of replies. A real game prints each command and its reply as it goes. That's a job for `dolist`, because the work is printing, not building a list.

`terpri` prints a newline. Its name is an old abbreviation of "terminate print line".

```
(defun transcript (commands)
  "Run each command in COMMANDS, printing it and its reply."
  (dolist (command commands)
    (princ "> ")
    (princ command)
    (terpri)
    (princ (run-command command))
    (terpri)))
```

```
(defparameter *item-locations*
  '((lamp . cellar)
```

```
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))

(transcript '((take key) (take key) (inventory)))
```

```
> (TAKE KEY)
You take the key.
> (TAKE KEY)
You already have that.
> (INVENTORY)
You are carrying: key.
```

`princ` prints the command list in upper case, because that's how Lisp stores the symbols (Chapter 3). The reply strings print as written.

Next, `item-report` from Exercise 8.10 walked the alist by hand and used `caar` and `cdar` to reach inside each pair. A destructuring `loop` names the two halves directly:

```
(defun item-report (pairs)
  "Return one string per item in PAIRS, saying where it is."
  (loop for (item . place) in pairs
        collect (if (eq place 'player)
                    (concatenate 'string (item-name item) " is carried")
                    (concatenate 'string
                                 (item-name item)
                                 " is in the "
                                 (string-downcase (symbol-name place))))))
```

If you skipped Exercise 8.10, this definition is complete on its own.

```
(defparameter *item-locations*
  '((coin . player)
    (lamp . cellar)
    (key . kitchen)))
(item-report *item-locations*)
;; => ("coin is carried" "lamp is in the cellar" "key is in the kitchen")
```

11.9. Exercises

11.1 Predict the result of each line, then check:

```
(loop for i from 1 to 4 collect (* i i))
(loop for i below 4 sum i)
(loop for x in '(a b c) for i from 0 collect (list i x))
(loop for tail on '(1 2 3) collect (length tail))
(dolist (x '(1 2 3)) x)
(loop for x in '(5 8 2 9) count (> x 4))
(loop for c across "hello" count (char= c #\l))
```

11.2 Write `sum-to`, which adds the numbers from 1 to `n` using `dotimes`. `(sum-to 10)` gives `55` and `(sum-to 0)` gives `0`.

11.3 Rewrite `count-vowels` from Exercise 10.6 with `loop` and `across`.

11.4 Write `range`, which returns the integers from `start` up to, but not including, `end`. `(range 2 5)` gives `(2 3 4)`.

11.5 Write `first-repeat`, which returns the first element of a list that appears again later in the list, or `nil` if there is none. `(first-repeat '(a b c b a))` gives `A`. Hint: `for tail on`.

11.6 Write `split`, which returns a list of two lists: the numbers in a list, then everything else. `(split '(a 1 b 2 3))` gives `((1 2 3) (A B))`.

11.7 Each expression below is wrong. Say what's wrong, then fix it.

```
(loop for i from 1 to 3 (print i))

(dolist (x '(1 2 3)) (* x 2))   ; meant to return (2 4 6)

(loop for c in "abc" collect (char-upcase c))

(loop with total = 0
      for x in '(1 2 3)
      do (incf total x)
      finally total)
```

11.8 Write `take-while`, which returns the elements at the front of a list that a predicate says yes to, and stops at the first one it says no to. `(take-while #'plusp '(1 3 -1 2))` gives `(1 3)`. Use `loop` with `while`.

11.9 Use `loop` with a destructuring pattern to build the alist `((ADA . 90) (ALAN . 72))` from the two lists `'(ada alan)` and `'(90 72)`, in one expression.

11.10 Predict `T` or `NIL`, or the value, for each, then check:

```
(loop for x in '(2 4 6) always (evenp x))
(loop for x in '(1 2 3) never (minusp x))
(loop for x in '(a b 3 c) thereis (numberp x))
(loop for x in '(2 4) always (evenp x) finally (return 'checked))
```

Careful with the last one: does `finally` run after `always` stops the loop early?

11.11 project Write `describe-all`, which prints one line per room in `*rooms*`, saying which items lie there, using `dolist` and `items-here`. Lines for empty rooms should say so:

```
BEDROOM: letter
KITCHEN: key
HALLWAY: nothing
GARDEN: nothing
```

11.12 project Change `transcript` so that it prints a header line `"You are in the kitchen."` before the commands, and a footer line `"Game over."` after them, using a `let` around the `dolist`. Then write `count-unknown`, which takes a list of commands and returns how

many of them `run-command` doesn't recognise. `run-command` returns a string for every command, so compare each reply with the unknown message using `string`.

CHAPTER 12.

format

Every chapter so far has built strings with `concatenate` and `string-downcase`. That works, but the game's messages are turning into nested calls. `format` is Lisp's answer: one function that takes a *control string* with placeholders and produces text. Other languages copied it: Python's `%` formatting and its newer format mini-language both trace back to `format`.

This chapter is a reference as much as a lesson. Learn the first third; the rest you can look up as you need it.

12.1. The shape of `format`

```
(format t "Hello, world.~%")
```

```
Hello, world.
```

`format` takes a *destination*, a control string, and any number of arguments. The destination decides where the text goes:

Destination	Meaning
`t`	Print to standard output, return `nil`
`nil`	Build a string and return it
A stream	Write to that stream (Chapter 16)

So `t` is for printing and `nil` is for building. The game's message functions build strings, so they use `nil`:

```
(format nil "You take the ~A." 'key)
;; => "You take the key."
```

12.2. The tilde: directives

Everything in a control string is copied out word for word, except what starts with a tilde. A tilde begins a *directive*, a placeholder or an instruction:

```
(format nil "You see ~A and ~A." 'lamp 'key)
;; => "You see LAMP and KEY."
```

`~A` means "put the next argument here, printed for people to read". Arguments are used in order, one per placeholder.

The directive letter is case-insensitive: `~a` and `~A` are the same. Capital letters in the directives that print words choose the case of the output, as you'll see.

12.3. The essential directives

12.3.1. `~A` and `~S`

`print` and `princ` from Chapter 1 are two ways to show a value: `princ` for people, `print` for Lisp. `~A` and `~S` are `format`'s versions of the same distinction:

```
(format nil "~A" "hello")   ; => "hello"
(format nil "~S" "hello")   ; => "hello", with the quotes
(format nil "~A" 'key)      ; => KEY
(format nil "~S" 'key)      ; => KEY
```

For strings the difference matters, for symbols it doesn't. Use `~A` for messages people read, `~S` when the text must be read back by Lisp, for example in a saved file.

12.3.2. `~D`: decimal integers

```
(format nil "You have ~D coins." 42)          ; => "You have 42 coins."
(format nil "~D" -7)                          ; => -7
```

It fails on anything but an integer, on purpose: `format` doesn't guess.

12.3.3. `~F`: floats

`~-F` says how many digits to show after the decimal point:

```
(format nil "Distance: ~,2F km" 123.4567)   ; => "Distance: 123.46 km"
(format nil "~F" 0.5)                       ; => "0.5"
```

12.3.4. `~C`: characters

```
(format nil "Press ~C to quit." #\q)   ; => "Press q to quit."
```

12.3.5. `~%`: newline, `~&`: fresh line, `~~`: tilde

```
(format t "Line one.~%Line two.~%")
```

```
Line one.
Line two.
```

`~%` prints exactly `n` newlines, where `n` is a number between the tilde and the letter. `~&` is more forgiving: it prints a newline only if the output isn't already at the start of one. Ending messages with `~&` instead of `~%` is a good habit, because two messages in a row then never leave a blank line between them.

```
(format t "A.~%")
(format t "~3%")
(format t "B.~&")
(format t "C.~&")
```

```
A.

```

```
B.
C.
```

~~ prints one tilde, since a bare ~ would start a directive.

12.4. Arguments used in other ways

12.4.1. ~{}: the next argument is a list

Some directives don't print an argument themselves, but change how the next ones are treated. `~{` says "the next argument is a list, and its elements are the arguments for what follows":

```
(format nil "~{~A~}" '(a b c))       ; => "ABC"
(format nil "~{~A, ~}" '(a b c))     ; => "A, B, C, "
```

`~{~}` walks the list, running the directives between `~{` and `~}` once per element. That's `mapcar` inside a control string.

Because `~A` consumes from the list, you can interleave text and values:

```
(format nil "~{~A: ~A~%~}" '("name" "Ada" "age" "36"))
;; => "name: Ada
;;     age: 36
;;     "
```

12.4.2. ~@{}: the arguments themselves are the list

`~@{` is the same walk, but over `format`'s own arguments instead of a list:

```
(format nil "~{~A or ~}" 'x 'y)      ; wrong, no list argument
```

```
(format nil "~@{~A or ~}" 'x 'y)     ; => "X or Y or "
```

*12.4.3. Trailing separators: ~^*

Both walk forms have a companion directive that changes the last pass. `~^` means "stop here if the list is exhausted":

```
(format nil "~{~A~^, ~}" '(a b c))       ; => "A, B, C"
(format nil "~@{~A~^, ~}" 'x 'y 'z)      ; => "X, Y, Z"
```

`~^` is how you join a list with commas and no dangling separator — the exact job `item-names` did by hand in Exercise 8.11.

12.4.4. ~T: columns

`~T` moves to a column, like a tab:

```
(format t "~A~10T~A~%" 'room 'items)
(format t "~A~10T~A~%" 'kitchen 'key)
```

```
ROOM      ITEMS
KITCHEN   KEY
```

`~10T` means "move to column 10". `~@T` means "move right by that many spaces".

12.5. Width, padding and case

A number between `~` and the letter sets the *minimum width* the value is padded with spaces on the left:

```
(format nil "[~5D]" 42)    ; => "[   42]"
(format nil "[~5A]" "hi")  ; => "[   hi]"
```

The `@` modifier flips the padding to the right:

```
(format nil "[~5@A]" "hi")  ; => "[hi   ]"
```

The `:` modifier on `~D` adds commas:

```
(format nil "~:D" 1000000)   ; => "1,000,000"
```

On `~A` and `~S`, the `:` modifier prints `nil` as `()` instead of `NIL`:

```
(format nil "~A" nil)    ; => "NIL"
(format nil "~:A" nil)   ; => "()"
```

Modifiers combine: `~:@D` signs the number with `+`:

```
(format nil "~:@D" 42)   ; => "+42"
```

12.6. Conditionals inside format

`~[~]` picks one of several texts by the value of an argument. The argument is an integer, counting from zero:

```
(format nil "You have ~[no coins~;one coin~:;~D coins~]." 3 3)
;; => "You have 3 coins."
```

The `:;` clause is the default, used when the number is out of range. This one expression handles zero, one and many — the plural problem, solved in the control string. Pass `0` or `1` to see the other clauses.

`~:[~]` takes a true-or-false argument instead: two clauses, `nil` gets the first, anything else the second:

```
(format nil "The door is ~:[closed~;open~]." t)   ; => "The door is open."
```

`~:[` can also compare against nothing at all: `~:[~A~;nothing~]` takes no integer. When the argument is `nil`, the second clause prints:

```
(format nil "You see ~:[something~;nothing~]." nil)
;; => "You see nothing."
```

12.7. Recursive format: / and named functions

There's a directive that spells out numbers in words:

```
(format nil "~R" 42)        ; => "forty-two"
(format nil "~:R" 3)        ; => "third"
```

`~R`'s related trick is more important: you can write your own directive as a Lisp function and call it with `~/name`:

```
(defun format-room (stream argument colonp atp)
  "Print ARGUMENT, a room entry, as a short line."
  (declare (ignore colonp atp))
  (format stream "~A: ~A" (first argument) (second argument)))

(format nil "~/format-room/" '(kitchen "the key is here"))
;; => "KITCHEN: the key is here"
```

A `~/` directive calls a function of four arguments: the stream, the value, and two flags that say whether you wrote `:` and `@`. `declare (ignore ...)` tells the compiler the unused parameters are on purpose, which keeps it quiet. This is rare in day-to-day code, but it explains the shape: the first argument of any `format` helper function is the stream, and that's why `format` can take `nil` or `t` there.

12.8. Building messages for the game

The game's string building so far:

```
(concatenate 'string
             (item-name item)
             " is in the "
             (string-downcase (symbol-name place)))
```

The same message with `format`:

```
(format nil "~A is in the ~(~A~)." 'key 'kitchen)
;; => "key is in the kitchen"
```

`~(~A~)` lower-cases everything it wraps. With `:` it capitalises the first word instead:

```
(format nil "~:(~A~) is in the ~(~A~)." 'key 'kitchen)
;; => "Key is in the kitchen"
```

The examples above are fragments; the full message is in the project step. The important part is the pair of directive modifiers:

Directive	Effect
`~A`	As is
`~(~A~)`	Lower-case
`~:(~A~)`	First letter of each word upper-case
`~@(~A~)`	First letter upper-case, rest lower

Directive	Effect
`~:@(~A~)`	All upper-case

`~@(~A~)` is the one for starting a sentence: it gives `"Key"` from `KEY`, with any capitals inside tamed to lower case.

12.9. Common mistakes

12.9.1. Extra or missing arguments

```
(format nil "~A and ~A" 'x)
;; error: too few arguments
```

Count the placeholders and the arguments. One `~A` per argument.

12.9.2. A stray tilde

```
(format nil "100% sure")
;; error: unknown directive %
```

`%` after a space is a directive too, roughly `~%`. Write `~~` for a literal tilde, and be careful with `%` in text meant for people.

12.9.3. `format t` inside a builder

A function whose job is to *return* a string must use `format nil`. Mixing them up either prints nothing or returns `nil`:

```
(defun describe-room (room)
  (format t "You are in the ~A." room))
;; returns NIL, prints as a side effect
```

12.9.4. `~D` on the wrong type

`~D` takes an integer. `(format nil "~D" 'key)` is an error, and `(format nil "~D" 3.5)` is too. Use `~A` when the type varies.

12.9.5. Case directives without the closing)

`~(= must be closed with =~)`, just as `~{` must be closed with `~}`. An unclosed case directive reads the rest of the string as text to change.

12.10. Project step: messages in one place

The game's message functions each build strings by hand. This step rewrites them around `format`, and gathers the messages where you can see and change them.

First the helper that names a list of items with commas, no "and":

```
(defun item-list-string (items)
  "Return a readable list of the names of ITEMS, comma-separated."
  (format nil "~{~(~A~)~^, ~}" (mapcar #'symbol-name items)))
```

```
(list (item-list-string nil)
      (item-list-string '(key))
      (item-list-string '(key lamp))
      (item-list-string '(key lamp coin)))
```

Expected:

```
("" "key" "key, lamp" "key, lamp, coin")
```

The empty list gives an empty string, which the callers guard against. The `"key and lamp"` form needs two passes, which `~{~}` alone can't do, so keep the `item-names` from Exercise 8.11 for the game's player messages, and use `item-list-string` where plain commas are right: the `help` command and `describe-all`.

Now the messages, rebuilt around `format nil`:

```
(defun inventory ()
  "Describe what the player is carrying, in alphabetical order."
  (let ((carried (sort (items-here 'player)
                       #'string< :key #'symbol-name)))
    (if (null carried)
        "You are carrying nothing."
        (format nil "You are carrying: ~A." (item-names carried)))))

(defun take (item)
  "Pick up ITEM if it is in the current room."
  (cond ((eq (item-location item) 'player)
         (format nil "You already have the ~(~A~)." item))
        ((eq (item-location item) (room-name *current-room*))
         (setf *item-locations*
               (set-location *item-locations* item 'player))
         (format nil "You take the ~(~A~)." item))
        (t
         "You don't see that here.")))

(defun drop (item)
  "Put down ITEM in the current room, if the player carries it."
  (if (eq (item-location item) 'player)
      (progn
        (setf *item-locations*
              (set-location *item-locations* item
                            (room-name *current-room*)))
        (format nil "You drop the ~(~A~)." item))
      "You don't have that."))

(defun loot ()
  "Take everything lying in the current room."
  (let ((found (items-here (room-name *current-room*))))
    (if (null found)
        "There is nothing here to take."
        (progn
          (take-all found)
```

```
          (format nil "You take ~A." (item-names found))))))

(defun item-report (pairs)
  "Return one string per item in PAIRS, saying where it is."
  (loop for (item . place) in pairs
        collect (if (eq place 'player)
                    (format nil "~@(~A~) is carried" item)
                    (format nil "~@(~A~) is in the ~(~A~)"
                            item place))))
```

`take`, `drop` and `item-report` now pass symbols straight to `format`, and the case directives do the lowering. Compare `item-report` with the version from Chapter 11: no `symbol-name`, no `string-downcase`, no `concatenate`. The message reads as a sentence in the code.

Run the whole thing. This resets the items and the current room, so run it as one block:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))

(list (run-command '(take key))
      (run-command '(take key))
      (run-command '(inventory))
      (run-command '(take lamp))
      (run-command '(drop key)))
```

Expected:

```
("You take the key."
 "You already have the key."
 "You are carrying: key."
 "You don't see that here."
 "You drop the key.")
```

One more: a single function that describes a room the way a player sees it, the way `look` does but with the items in the sentence. The conditional directives handle the empty room and the plurals:

```
(defun describe-room-text (room)
  "Return a sentence describing ROOM and what lies there."
  (let ((here (items-here room)))
    (format nil "You are in the ~(~A~). ~:[There is nothing here.~;You see ~{~:(~A~)~^, ~}~]~&"
            room
            here
            (mapcar #'symbol-name here))))
```

Read the control string from the start: the room's name, a period, then `~:[` with two clauses — `nil` gets "There is nothing here.", anything else gets "You see " followed by the item walk with commas. `~&` ends the string so two descriptions never run together.

```
(defparameter *item-locations*
  '((key . kitchen)
    (letter . bedroom)))
(list (describe-room-text 'kitchen)
      (describe-room-text 'garden))
```

Expected:

```
("You are in the kitchen. You see Key."
 "You are in the garden. There is nothing here.")
```

The capital on `"Key"` comes from `~:(~A~)`. With two items, the walk would print `"Key, Letter"` — right for a list, and the exercises refine the "and".

12.11. Exercises

12.1 Predict the result of each line, then check:

```
(format nil "~A!" 'done)
(format nil "~S" "hi")
(format nil "~3D" 7)
(format nil "~,1F" 2.25)
(format nil "~A~A" 'a 'b)
(format nil "a~2%b")
```

12.2 Write `comma-list`, which takes a list of symbols and returns them as a lower-case string separated by `", "`. `(comma-list '(A B C))` gives `"a, b, c"`. One `format` expression.

12.3 Use one `format` call to produce the string `"1: one, 2: two, 3: three"` from the list `'((1 "one") (2 "two") (3 "three"))`, with no `loop` or `mapcar`.

12.4 Write `price-line`, which takes a name and a price and returns a padded line, name left-aligned in 12 characters, price right-aligned in 8 with two decimals: `(price-line 'coffee 3.5)` gives `"coffee        3.50"`.

12.5 Predict the result, then check:

```
(format nil "You have ~[no keys~;one key~:;~D keys~]." 0)
(format nil "You have ~[no keys~;one key~:;~D keys~]." 1)
(format nil "You have ~[no keys~;one key~:;~D keys~]." 5)
```

12.6 Write `pluralise`, which takes a number, a noun as a symbol and returns `"0 coins"`, `"1 coin"` or `"7 coins"`, lower-cased, in one `format` call using `~[`.

12.7 Each line below is wrong. Say what's wrong, then fix it.

```
(format nil "~A ~A" 'x)

(format nil "50% off")

(format nil "You have ~D apples." "three")

(format nil "~(HELLO~" 'world)
```

12.8 Write `bar`, which takes a number from 0 to 10 and returns a health bar: that many `"#"` characters, then the rest as `"-"`, in brackets. `(bar 7)` gives `"[#######---]"`. One `format` call with `~10@A` on a prepared string of `#` characters, plus a second prepared string of `-` characters — two pieces, one `format`.

12.9 project Rewrite `help` from Exercise 10.12 to use `item-list-string`, keeping the alphabetical order. The reply should end with a period.

12.10 project Improve `describe-room-text` so two items read `"You see Key and Letter"` and three read `"You see Key, Letter and Coin"`. You may write a helper function; the `format` call stays the same.

CHAPTER 13.

Structures, Hash Tables and Arrays

Lists are flexible, but they have costs. Finding an item in an alist takes a walk through the whole list. Reaching the third field of a room means `third` or `caddr`, and reading `(caddr room)` later tells you nothing about what that slot means. This chapter adds three new data shapes that fix those problems:

- **Structures**: named records with named fields, for data with a fixed shape, such as a room.
- **Hash tables**: look up by key in one step, however many entries.
- **Arrays**: fixed-length indexed collections, faster than lists.

13.1. Structures: `defstruct`

`defstruct` defines a new kind of record:

```
(shadow "ROOM")   ; ROOM is also a CL function name; take our own symbol
(defstruct room
  name
  description
  exits)
```

That one form does a lot:

- It defines a new type, `room`.
- It defines a constructor, `make-room`.
- It defines an accessor for each field: `room-name`, `room-description`, `room-exits`.
- It defines a predicate, `room-p`.

Each of these exists before you write a line for them. The names are derived from the structure name: `make-room`, `room-name`, `room-p`.

Creating a record uses keyword arguments, one per field:

```
(defparameter *bedroom*
  (make-room :name 'bedroom
             :description "A quiet room with a bed."
             :exits '(south)))
```

Fields you leave out get `nil`.

Reading a field calls the accessor:

```
(room-name *bedroom*)          ; => BEDROOM
(room-description *bedroom*)   ; => "A quiet room with a bed."
```

Changing a field uses `setf` on the accessor:

```
(setf (room-exits *bedroom*) '(north south))
(room-exits *bedroom*)         ; => (NORTH SOUTH)
```

This is the same `setf` expansion as `(setf (car ...))` from Chapter 7: `setf` knows how to assign to any accessor.

`room-p` tells you whether a value is a room:

```
(list (room-p *bedroom*) (room-p 'bedroom))   ; => (T NIL)
```

13.1.1. Printing and comparing

`defstruct` has options. The two most useful go right after the name:

```
(defstruct (room (:print-function print-room))
  name
  description
  exits)
```

By default a structure prints unreadably, like `#S(ROOM :NAME BEDROOM :DESCRIPTION ...)`, which is fine. A `:print-function` makes it print your way:

```
(defun print-room (room stream depth)
  (declare (ignore depth))
  (format stream "<room ~(~A~)>" (room-name room)))

(print *bedroom*)   ; => <room bedroom>
```

The print function receives the value, the stream to print on, and the current print depth, which you can usually ignore — hence the `declare` line, which tells the compiler the third parameter is deliberately unused. The body calls `format` with a stream as destination — the third destination from Chapter 12, put to work.

Because `print-room` mentions `room-name`, and `room-name` is only created by the `defstruct`, SBCL warns "undefined function ROOM-NAME" when it compiles `print-room` before the structure exists. The warning is harmless: `print-room` is only **called** when a room is printed, which happens after `defstruct` has run. If you find the warning noisy, put the `defstruct` first and the `print-room` second — then re-run the `defstruct` with `:print-function` to install the printer.

The `:type` option is a bigger change; skip it. But `:named` and the comparison options matter less than a simpler rule: to compare two structures field by field, use `equalp`, which (Chapter 10) compares structures by their contents:

```
(equalp (make-room :name 'a) (make-room :name 'a))   ; => T
(eq (make-room :name 'a) (make-room :name 'a))       ; => NIL
```

`eq` asks whether they are the same object, and each `make-room` call builds a new one.

13.1.2. Default values

A field can have a default, given as a slot option:

```
(defstruct (item (:print-function print-item))
  (name 'thing)
  (weight 1))
```

```
(item-name (make-item))   ; => THING
```

13.1.3. Why structures instead of lists

The rooms of the game are alists of fields, or lists reached with `third` and `caddr`. Structures replace both with names:

List version	Structure version
`(third room)`	`(room-description room)`
Comments to remember the order	The field names say it
One wrong `cadr` and data is misread	The accessor checks the type

When data has a fixed shape that you'll never change at runtime, a structure is the natural home for it.

13.2. Hash tables

A hash table stores key-value pairs and finds any pair in roughly one step, by computing where the key lives. The alist walk of `assoc` is linear: twice the pairs, twice the work. A hash table isn't.

```
(defparameter *caps* (make-hash-table))

(setf (gethash 'one *caps*) 1)
(setf (gethash 'two *caps*) 2)

(gethash 'two *caps*)        ; => 2, and T
```

`make-hash-table` makes one, `gethash` reads and (with `setf`) writes. Reading a missing key returns `nil`, but the REPL shows two values:

```
NIL, T      ← the value, and "was it found?"
NIL, NIL
```

`gethash` returns *two* values: the value, and a flag saying whether the key was there. That flag matters when `nil` is a legitimate stored value:

```
(setf (gethash 'nothing *caps*) nil)

(multiple-value-list (gethash 'nothing *caps*))  ; => (NIL T)
(multiple-value-list (gethash 'absent *caps*))   ; => (NIL NIL)
```

`multiple-value-list` catches all the values a form returns, as a list. Multiple values are the subject of the next section.

`gethash` also takes a default, returned when the key is absent:

```
(gethash 'absent *caps* 0)   ; => 0
```

13.2.1. Iterating: `maphash` and `loop`

`maphash` calls a function of two arguments, key and value, on every entry. The order is unspecified:

```
(maphash (lambda (k v) (format t "~A -> ~D~%" k v)) *caps*)
```

`loop` can walk a hash table too:

```
(loop for k being the hash-keys of *caps* collect k)
(loop for v being the hash-values of *caps* sum v)
```

13.2.2. Removing: `remhash` and `clrhash`

```
(remhash 'one *caps*)   ; => T, it was there
(clrhash *caps*)        ; empties the whole table
```

13.2.3. Keys and tests

`make-hash-table` takes a `:test` argument. The default is `eq`, which suits symbol keys — the game's room and item names. For string keys you must ask for `equal`, or the same string spelled twice counts as two different keys:

```
(defparameter *ages* (make-hash-table :test #'equal))

(setf (gethash "ada" *ages*) 36)

(gethash "ada" *ages*)   ; => 36, T
```

Only `eq`, `eql`, `equal` and `equalp` are valid tests. Numbers as keys need `(make-hash-table :test #'eql)`; `eq` on numbers is unsafe (Chapter 10).

13.2.4. Hash table vs alist

	Alist	Hash table
Lookup	Walk, slow on big lists	One step
Insert at the front	`push`, fast	`setf gethash`, fast
Keep an order	Yes	No, order is unspecified
Readable in your source	Yes, quoted	No, built at runtime
A key with two values (history)	Yes	No, one value per key

Use an alist for small tables you want to read in your code, a hash table for lookups that grow, and switch when an alist starts to feel slow. The game's tables stay alists: they are small, and being visible in the source has been worth more than speed.

13.3. Arrays

An array stores elements by index. `make-array` makes one:

```
(defparameter *a* (make-array 3 :initial-element 0))
```

Read and write with `aref`, whose indexes count from zero like `nth`'s:

```
(aref *a* 0)          ; => 0
(setf (aref *a* 1) 42)
*a*                   ; => #(0 42 0)
```

`#(...)` is how an array prints. Like `#'` and `#\`, the `#` signals "not an ordinary list".

`length` works on arrays too. `svref` is a faster `aref` for simple arrays such as this one, and `vector` builds one from elements:

```
(vector 'a 'b 'c)     ; => #(A B C)
```

A one-dimensional array is called a *vector*. Strings are vectors of characters — which is why `(length "abc")` and `(aref "abc" 1)` work:

```
(aref "abc" 1)        ; => #\b
```

Arrays have a fixed size. An array of 3 stays 3; to grow, make a bigger one and copy, or use `adjustable-array-p` territory, which this guide leaves for later reading. Lists and vectors cover different needs:

	List	Vector
Grow at the front	`push`, fast	No
Reach element `n`	Walk, `nth`	One step, `aref`
Fixed or growing	Grows	Fixed
Printed as	`(...)`	`#(...)`

Reach `push` for collections that grow; `aref` for collections you index by number.

13.4. Multiple values

`gethash` returned two values. Any function can: `values` returns several:

```
(defun divide (a b)
  "Return the quotient and remainder of A divided by B."
  (values (floor a b) (mod a b)))

(divide 17 5)         ; REPL shows: 3, 2
```

In ordinary code, a form with multiple values behaves as its first value. `(list (divide 17 5))` gives `(3)`. To catch the rest, `multiple-value-bind` binds them to names:

```
(multiple-value-bind (q r) (divide 17 5)
  (list q r))          ; => (3 2)
```

`floor` itself returns two values — the quotient and the remainder — which is why `divide` above works without naming `mod` twice. Other functions that return extra values: `gethash`, `floor`, `truncate` and `round`.

Common Lisp's `values` is a quiet feature: you can ignore it everywhere and catch it when you need it. Java needed a whole class or an out-parameter to return two things; here it's a `values` form and one binding form.

13.5. Common mistakes

13.5.1. Forgetting the `make-` name

`defstruct room` creates `make-room`, not `make-<room>` or `new-room`. The constructor is always `make-` plus the structure name.

13.5.2. `:test` on the wrong hash table

```
(defparameter *t* (make-hash-table))
(setf (gethash "a" *t*) 1)
(gethash "a" *t*)     ; => NIL, NIL
```

The default test is `eq`, and two `"a"` strings are different objects. Declare `:test #'equal` when the keys are strings.

13.5.3. Treating a hash table like an alist

`assoc` doesn't work on hash tables: `(assoc 'one *caps*)` fails, because a hash table isn't a list. Its walk is `maphash` or the `loop for ... being` form.

13.5.4. `aref` on a list, `nth` on an array

```
(aref '(a b c) 1)   ; error: The value (A B C) is not of type VECTOR
(nth 1 (vector 'a 'b 'c))   ; error, a vector is not a list
```

Each shape has its own accessor.

13.5.5. Expecting hash-table order

`maphash` and the `loop` walk may present entries in any order. If you need an order, collect the keys and sort them.

13.5.6. Ignoring the second value of `gethash`

`(gethash key table)` returning `nil` means either "the value is `nil`" or "the key is absent". When `nil` can be stored, bind both values with `multiple-value-bind`.

13.6. Project step: rooms as structures

The rooms of the game become structures. This is the biggest single change to the project so far, and it pays off immediately: every function that reached into a room with `second` or `third` now reads by name.

First the structure, with a print function so rooms read well:

```
(defun print-room (room stream depth)
  (declare (ignore depth))
  (format stream "<room ~(~A~)>" (room-name room)))

(defstruct (room (:print-function print-room))
  name
  description
  exits)
```

If you defined `room` in an earlier experiment, redefine it now with this block, so the accessors match. SBCL warns "undefined function ROOM-NAME" while compiling `print-room`, because the

accessor doesn't exist yet. The warning is harmless: `print-room` is only **called** when a room is printed, which is after `defstruct` has run.

Now the five rooms, with the same topology as before, built with `make-room`:

```
(defparameter *rooms*
  (list
   (make-room :name 'bedroom
              :description "A small bedroom with an unmade bed."
              :exits '(hallway))
   (make-room :name 'hallway
              :description "A narrow hallway with creaky floorboards."
              :exits '(bedroom kitchen))
   (make-room :name 'kitchen
              :description "A warm kitchen that smells of bread."
              :exits '(hallway garden cellar))
   (make-room :name 'garden
              :description "An overgrown garden."
              :exits '(kitchen))
   (make-room :name 'cellar
              :description "A damp cellar. Something drips in the dark."
              :exits '(kitchen))))
```

The old `*rooms*` held lists; this holds structures. Every function that reads a room keeps working, because they all go through `room-name`, `room-description` and `room-exits` — and those are now the structure's accessors, generated by `defstruct`, replacing the Chapter 5 functions of the same names.

`find-room` needs one change: `assoc` walked an alist, and `*rooms*` is now a plain list of structures. Chapter 10's `find` with `:key` fits exactly:

```
(defun find-room (name)
  "Return the room whose name is NAME, or NIL if there is none."
  (find name *rooms* :key #'room-name))
```

`go-to`, `look` and the rest call `find-room`, so they need nothing. Check the whole chain:

```
(list (find-room 'kitchen)
      (room-exits (find-room 'kitchen))
      (setf *current-room* (find-room 'hallway))
      (go-to 'kitchen)
      (look))
```

Expected: the kitchen structure (printed as `<room kitchen>`), the list of exits, then the go-to message and look description for the kitchen.

One new ability that structures make natural: a room can hold more than the old shape allowed. Redefine the structure with a `locked` field defaulting to `nil`, then set it on the garden:

```
(defstruct (room (:print-function print-room))
  name
  description
  exits
```

```
  (locked nil))

(setf (room-locked (find-room 'garden)) t)

(room-locked (find-room 'garden))   ; => T
```

SBCL warns about redefining the structure; that's expected. Every existing room still works — the new field defaults to `nil` for the four unlocked rooms, and `setf` on the accessor sets it for the garden. Chapter 6's hard-coded locked-garden logic can now ask the room itself; the exercises do that.

13.7. Exercises

13.1 Define `defstruct book` with fields `title`, `author` and `(year nil)`. Create one with `make-book`, read each field, set `year` with `setf`, and show that `book-p` accepts your book and rejects a symbol.

13.2 Predict the result of each line, then check:

```
(aref (vector 10 20 30) 2)
(length (vector 'a 'b))
(aref "lisp" 0)
(multiple-value-list (floor 17 5))
(multiple-value-list (gethash 'x (make-hash-table)))
```

13.3 Build a hash table `*prices*` with `:test #'equal`, keyed by strings, holding three items and prices. Then write `total`, which takes a list of item names and adds up their prices with `maphash` — or, more simply, with `reduce` and `gethash`.

13.4 Write `hash-keys`, which returns the keys of a hash table as a list, using `loop`. Then sort them, since hash-table order is unspecified.

13.5 Each line below is wrong. Say what's wrong, then fix it.

```
(assoc 'one *caps*)

(aref '(a b c) 1)

(defparameter *t* (make-hash-table))
(setf (gethash "a" *t*) 1)
(gethash "a" *t*)

(make-room 'kitchen "desc" '(west))   ; wrong call style
```

13.6 Write `oldest`, which takes a hash table mapping name symbols to ages and returns the name of the oldest person, using `loop for key being the hash-keys`, with `maximize` on the values — or a `let` that keeps the best so far.

13.7 Predict `T` or `NIL`, then check:

```
(equalp (make-room :name 'a :exits '(n)) (make-room :name 'a :exits '(n)))
(eq (make-room :name 'a) (make-room :name 'a))
```

```
(room-p (make-room))
(eql 1.0 1)
```

13.8 Write `invert`, which takes a hash table and returns a new one with keys and values swapped. Assume values are unique and symbols.

13.9 project Change `go-to` so that it checks `room-locked`: when the destination has `room-locked` true, reply `"The garden is locked."` and stay where you are. The hard-coded `eq` test on the garden's name goes away.

13.10 project Write `locked-rooms`, which returns the names of all locked rooms, using `remove-if-not` over `*rooms*`.

13.11 project Give rooms an `items` field, defaulting to `nil`, and write `take-struct` and `drop-struct`, which move item symbols between the current room's `items` field and a list `*carrying*`. Compare the feel of this with the alist version: which functions became simpler, and which became longer? Answer in one paragraph of prose.

CHAPTER 14.

Macros

You've used macros all along without the name: `when`, `unless`, `cond`, `and`, `or`, `setf`, `incf`, `push`, `defun`, `defparameter`, `loop`, `dotimes` are all macros. Now you learn what they are and how to write them. This is Lisp's most famous feature: the code you write is data, and programs can write programs.

14.1. Functions cannot do everything

The `if` from Chapter 6 evaluates only one of its two branches:

```
(if t (print 'yes) (print 'no))
```

Only `YES` prints. Now try to write `my-if` as a function:

```
(defun my-if (condition then-branch else-branch)
  (cond (condition then-branch)
    (t else-branch)))
```

Call it:

```
(my-if t (print 'yes) (print 'no))
```

```
YES
NO
YES
```

Both branches print. Lisp evaluated all three arguments *before* `my-if` ran, because that's how functions work: arguments are evaluated first. No function can receive the two branches unrun.

`if` and `when` get around this because they are *macros*: they receive their arguments as raw code and decide themselves which parts to evaluate. That's the whole idea. A macro is a function from code to code, run at compile time, whose result is the code that actually executes.

14.2. The macro's point of view

A macro call hands the macro *forms*, not values. Write a macro with `defmacro`, whose shape matches `defun`, and you can see the raw forms by having the macro return them as data:

```
(defmacro last-form (form)
  form)

(macroexpand-1 '(last-form (+ 1 2)))
```

`macroexpand-1` asks a macro what it would produce, without running the result:

```
(+ 1 2)
```

A macro that *does* something to its forms, building a new form:

```
(defmacro twice (form)
  (list 'progn form form))

(macroexpand-1 '(twice (princ "hi")))
;; => (PROGN (PRINC "hi") (PRINC "hi"))
```

`twice` receives `(princ "hi")`, builds `(progn (princ "hi") (princ "hi"))`, and that's the code that runs. The body used ordinary list functions — `list`, quoting — because the arguments are lists, the data from Part II.

```
(twice (princ "hi"))
```

```
hihi
```

14.3. Backquote: templates for code

Building lists by hand gets clumsy fast. The *backquote* character quotes a form the way `'` does, but inside it, a comma `,` unquotes — inserts the value of an expression:

```
(defparameter name 'ada)

`(hello ,name)
;; => (HELLO ADA)
```

Everything stays as written, except `,name`, which is replaced by its value. With a plain quote there are no holes at all — the whole form is data, so `name` stays the symbol:

```
'(hello name) ; => (HELLO NAME)
```

A comma outside a backquote is not syntax at all: the reader stops with "Comma not inside a backquote". That error is worth recognising, because it usually means a missing backquote rather than a stray comma.

Inside a macro, backquote is how you write the expansion as a template, with commas marking the holes:

```
(defmacro twice (form)
  `(progn ,form ,form))

(macroexpand-1 '(twice (princ "hi")))
;; => (PROGN (PRINC "hi") (PRINC "hi"))
```

Same expansion, but the code reads as the code it produces.

`,@` (comma-at) splices: it inserts a *list's elements* rather than the list itself. It's `append` where `,` is `cons`:

```
(defparameter extra '(c d))
```

```
`(a ,extra e)   ; => (A (C D) E)
`(a ,@extra e)  ; => (A C D E)
```

Now macros that earn their keep. `when` is close to:

```
(defmacro my-when (test &body body)
`(if ,test (progn ,@body) nil))
```

`&body` is `&rest` for macro bodies: same behaviour, a signal to readers and to editors that the forms are a body of code. Check it:

```
(macroexpand-1 '(my-when (plusp 5) (princ "big") (princ "!")))
;; => (IF (PLUSP 5) (PROGN (PRINC "big") (PRINC "!")) NIL)
```

And `my-unless`, the same shape with the branches swapped:

```
(defmacro my-unless (test &body body)
`(if ,test nil (progn ,@body)))
```

14.4. A macro the functions couldn't do: `while`

Here is a control structure Common Lisp doesn't have built in — a `while` loop that repeats until its test goes false. No function could write it, because the body must run many times, and a function receives its body once:

```
(defmacro while (test &body body)
  `(loop while ,test
    do ,@body))
```

```
(defparameter count 0)
(while (< count 3)
(print count)
(incf count))
```

```
0
1
2
```

The expansion is ordinary `loop` code:

```
(macroexpand-1 '(while (< n 10) (princ n)))
;; => (LOOP WHILE (< N 10) DO (PRINC N))
```

You defined syntax. Users of `while` can't tell it from something built into the language. Java gained lambdas and streams over the years, but no Java program can add new syntax to Java; the language grows only when its committee grows it. In Lisp, `defmacro` is part of the language for every programmer, and much of the standard library is itself macros written this way.

14.5. Expansion, not evaluation

`macroexpand-1` shows one step. `macroexpand` expands repeatedly, until the result is no longer a macro call — useful because your macros often expand into other macros:

```
(macroexpand '(my-when t (my-unless nil (princ "ok"))))
```

```
(IF T (PROGN (IF NIL NIL (PROGN (PRINC "ok")))) NIL)
```

`when` and `unless` expanded into `if` and `progn` here, because `my-when` and `my-unless` are macros too.

Two facts to keep straight:

The macro's parameters are *forms*, so inside the macro you call list functions (Chapter 7) and, in the template, `,` and `,@`.

Watch the difference in this macro, which counts how many forms are in its body:

```
(defmacro body-size (&body body)
  `(quote ,body))

(body-size (princ 1) (princ 2) (princ 3))
;; => ((PRINC 1) (PRINC 2) (PRINC 3))
```

The three `princ` calls never ran. The macro saw them as data and returned them as data.

14.6. Variables in macros: the hygiene trap

Macros that introduce variable names can collide with the names the caller uses. Here is a broken macro:

```
(defmacro swap-bad (a b)
  `(let ((temp ,a))
    (setf ,a ,b)
    (setf ,b temp)))
```

It looks right:

```
(defparameter *x* 1)
(defparameter *y* 2)
(swap-bad *x* *y*)
(list *x* *y*) ; => (2 1)
```

But pass a variable literally named `temp`:

```
(defparameter temp 'kept)
(swap-bad temp *x*)
temp ; => 2, not KEPT
```

The expansion was `(let ((temp temp)) ...)`, and the macro's `temp` shadowed the caller's. The caller's variable was clobbered — a bug that only shows up for certain callers, the worst kind.

The fix: use `gensym`, which generates a symbol guaranteed never to have been used before:

```
(gensym) ; => #:G1234
(gensym) ; => #:G1235
```

The `#:` shows an uninterned symbol: a symbol no package owns (packages are Chapter 16). Build the expansion so the temporary name comes from `gensym` at expansion time:

```
(defmacro swap (a b)
  (let ((temp (gensym)))
    `(let ((,temp ,a))
      (setf ,a ,b)
      (setf ,b ,temp))))
```

Notice the two levels: `let` (plain) runs at expansion time and makes `temp` name the fresh symbol; backquote with `,temp` inserts that symbol into the template.

```
(defparameter temp 'kept)
(defparameter *x* 1)
(swap temp *x*)
(list temp *x*) ; => (1 KEPT)
```

Now no caller can break it. The rule: *any variable a macro introduces in code that runs more than once should be a gensym*. Code that runs once, such as a `let` binding evaluated a single time in the expansion, is safe with an ordinary name.

The same trap exists for arguments evaluated repeatedly:

```
(defmacro square-bad (n)
  `(* ,n ,n))

(square-bad (progn (princ "evaluated ") 5))
;; prints "evaluated " twice, then 25
```

`,n` appears twice, so the form is evaluated twice. The fix binds the form once:

```
(defmacro square (n)
  (let ((v (gensym)))
    `(let ((,v ,n))
      (* ,v ,v))))
```

14.7. Destructuring in defmacro

A macro's parameter list can take lists apart, like `loop`'s patterns in Chapter 11:

```
(defmacro do-pair ((a b) list &body body)
  `(dolist (pair ,list)
    (let ((,a (first pair))
          (,b (second pair)))
      ,@body)))
```

```
(do-pair (name price) '((key 5) (lamp 10))
(format t "~A costs ~A~%" name price))
```

```
KEY costs 5
LAMP costs 10
```

The parameter `(a b)` destructures the first argument of every call, and `,@body` splices the caller's body into the `let`.

14.8. When to write a macro

Macros are powerful, and power wants restraint. The test: *could a function do it?* If yes, write a function — functions compose better, can be passed with `#'`, and debug more simply. Write a macro when you need one of these:

Control flow: evaluate some arguments and not others (`when`, `and`), or evaluate them more than once (`while`).

New binding forms: introduce variables the body can use (`do-pair`'s `a` and `b`, `with-*` style macros).

Access to forms: you need the code itself, as `setf` needs the place and `trace` needs the function name.

Everything else is a function, and Chapter 9's higher-order functions cover much of what another language would reach for macros to do.

14.9. Common mistakes

14.9.1. Calling the macro where you wanted the expansion

`macroexpand` takes a *form*, quoted:

```
(macroexpand (my-when t 1))
;; expands the literal call, then macroexpand receives 1
```

Write `(macroexpand '(my-when t 1))`, with a quote.

14.9.2. Forgetting the comma

```
(defmacro double-expr (x)
`(+ x x)) ; wrong: uses the symbol X

(macroexpand-1 '(double-expr (+ 2 3)))
;; => (+ X X) — the argument vanished
```

The comma in `(+ ,x ,x)` is what inserts the caller's form.

14.9.3. The reverse: comma outside backquote

```
(defmacro bad (x)
(+ ,x 1))
;; reader error: comma is not inside a backquote
```

A comma only means "insert" inside a backquoted form.

14.9.4. Splicing a non-list

```
(defmacro wrap (form)
`(progn ,@form))

(wrap (princ "hi"))
;; works: (princ "hi") is a form

(wrap 5)
;; error: 5 is not a list, and ,@ needs one
```

`,@` requires a list. When the argument may be one form or many, decide in the macro body.

14.9.5. A gensym that evaluates at the wrong time

```
(defmacro swap-broken (a b)
`(let ((,(gensym) ,a))
(setf ,a ,b)
(setf ,b ,(gensym))))
```

Two different `gensym` calls run at *two different moments*, and the second `gensym` here runs when the expanded code runs, not when the macro expands. Compute the fresh symbol once, in a plain `let` around the backquote, and bind it in the expansion.

14.9.6. Writing a macro where a function fits

If your macro just computes a value from its arguments' values, delete it and write a function. `defmacro` is for syntax, and the project step shows the difference on the game.

14.10. Project step: an undo macro and a command with syntax

The game gets two things from this chapter: an *undo* command, and a macro that cleans up a repeated pattern.

Undo needs a history of item alists. Each move pushes a copy of `item-locations` onto a stack; undo pops and restores:

```
(defparameter history nil)

(defun snapshot ()
  "Record the current item locations for undo."
  (push (copy-alist item-locations) history))

(defun undo ()
  "Restore the item locations from before the last move."
  (if (null history)
    "Nothing to undo."
    (progn
      (setf item-locations (pop history))
      "Undone.")))
```

`copy-alist` copies both levels of the alist, so the snapshot survives later changes to the live one (Chapter 7's sharing structure). Then every mutating command saves state first. The pattern "snapshot, then act" repeats in `take`, `drop` and `go-to` — a macro's job:

```
(defmacro with-undo (&body body)
  `(progn
    (snapshot)
    ,@body))
```

Redefine `take` to use it:

```
(defun take (item)
  "Pick up ITEM if it is in the current room."
  (cond ((eq (item-location item) 'player)
         (format nil "You already have the ~(~A~)." item))
    ((eq (item-location item) (room-name current-room))
      (with-undo
        (setf item-locations
          (set-location item-locations item 'player)))
      (format nil "You take the ~(~A~)." item))
    (t
      "You don't see that here.")))
```

Here `with-undo` is a macro only because of where the `snapshot` sits: it must run *before* the caller's forms, interleaved with them, and the caller's forms must splice into the same `progn`. A function `(with-undo (setf ...))` couldn't do that — it would receive the `setf` already evaluated.

Now the command table. Add undo to it (if Exercise 14.7 has you add `drop` too, do that after):

```
(push '(undo . undo) commands)
```

Check the undo. This block resets everything, so run it whole:

```
(defparameter item-locations
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf history nil)
(setf current-room (find-room 'kitchen))

(list (run-command '(take key))
  (run-command '(undo))
  (items-in item-locations 'kitchen)
  (run-command '(take key)))
```

Expected: the take message, `"Undone."`, `(KEY)` back in the kitchen alist, and the take message again. The undo restored not just the items but the *possibility*: the second `take` succeeds because the first one's effect was erased.

Second, a macro that reads better at the call site. Several game replies depend on the player carrying something:

```
(defun open-chest ()
  "Open the chest if the player carries the key."
  (if (member 'key (items-here 'player))
```

```
    "The chest opens."
    "The chest is locked."))
```

One of these is fine. When a dozen checks pile up, a macro names the concept:

```
(defmacro if-carrying (item then &body else)
  `(if (member ,item (items-here 'player))
    ,then
    (progn ,@else)))

(defun open-chest ()
  "Open the chest if the player carries the key."
  (if-carrying 'key
    "The chest opens."
    "The chest is locked."))
```

Note `then` is a single form (no `&body`), and `else` is a body. The indentation convention — the lone then on its own line, the else body indented under it — is what makes the call read like `if`.

Both macros expand into code you could have written by hand; check with `macroexpand` to see it. That's the healthy shape of most macros in real programs: small, readable expansions of repetitive patterns, not clever machinery.

14.11. Exercises

14.1 Predict each expansion, then check with `macroexpand-1`:

```
(twice (princ 'x))
(my-when (> 2 1) (princ 'a))
(swap p q)
(square (+ 1 2))
```

Then run each form on suitable variables and say what happens and why.

14.2 Write `my-or2`, a two-argument `or`: evaluate the first; if it is true, return it without evaluating the second; otherwise evaluate and return the second. No function can do this — say why in one comment line inside your test block.

14.3 Write `repeat`, a macro `(repeat n &body body)` that runs the body `n` times, where `n` is a form evaluated once. `(repeat (+ 1 2) (princ 'x))` prints `XXX` — and prints `"evaluated"` only once if `n` is `(progn (princ "evaluated ") 3)`.

14.4 Each macro below is broken. Say what breaks, for which caller, then fix it with `gensym` or a different shape.

```
(defmacro first-of (lst)
`(let ((head ,lst))
(first head)))

(defmacro inc-twice (x)
`(incf ,x ,x))
```

```
(defmacro unless-bad (test &body body)
`(if ,test nil ,body))
```

14.5 Write `benchmark`, a macro `(benchmark form)` that returns the result of `form`, but prints a line before and after, so the expansion is `(progn (princ "start") <form-run> (princ "end") ...)` with the result threaded through. Then explain in one comment why `benchmark` must be a macro and not a function. (Real timing needs `get-internal-run-time`, which is beyond this chapter.)

14.6 Write `for-each`, a macro `(for-each (x lst) &body body)` that works like `dolist`. Check its expansion with `macroexpand-1`, then use it to print the rooms' names.

14.7 project Add `(drop . drop)` and `(undo . undo)` to `commands`, and write a `rewind` command that undoes repeatedly while `history` is non-empty, returning how many steps it rolled back. Use `loop with`.

14.8 project Write `if-room`, a macro `(if-room name then &body else)` that evaluates `then` with the variable `it` bound to the room found by `find-room`, or the `else` body when no room has that name:

```
(if-room kitchen
(format nil "~A is found" (room-name it))
"No such room.")
```

This is Lisp's `if-let` pattern, and it shows a macro introducing a binding — impossible for a function.

CHAPTER 15.

Conditions and the Debugger

You've landed in the debugger often enough by now: a wrong argument, a missing function, `sort` on a quoted list. In most languages an error means the program dies and you read a stack trace. Common Lisp does something different: an error is not the end of the program but an event other code can respond to, and the debugger is the place where *you* are the handler. This chapter covers both sides.

15.1. Signalling: `error`, `warn`, `signal`

`error` announces that something has gone wrong and unwinds the stack looking for a handler:

```
(error "That room doesn't exist.")
;; Enters the debugger
```

The string is a *format control* — `error` calls `format` on it (Chapter 12), so directives work, and following arguments feed them:

```
(error "Can't take ~A: not here." 'key)
```

Never build the message with `concatenate` before calling `error`. Pass the parts and let `format` do it. That keeps the message machine-readable as well as human-readable, which handlers below depend on.

`warn` is `error`'s softer cousin. It prints a warning and continues; the stack is not unwound and the program carries on:

```
(warn "Low on fuel: ~D litres left." 2)
(princ 'still-running)
```

```
WARNING: Low on fuel: 2 litres left.
STILL-RUNNING
```

`signal` is the quietest: it announces an event and, if nobody handles it, returns `nil` instead of entering the debugger. It is for events that may or may not be anyone's business. Most code needs only `error` and `warn`.

15.2. Conditions are objects

A *condition* is the object that represents the problem. Every error signal creates one. The one you've met most often is `simple-error`, and its relatives cover the common cases: `type-error` when a value has the wrong type, `unbound-variable` when a name has no value, `undefined-function` when a function isn't defined, `division-by-zero`.

You can see one directly with `make-condition`, and read its fields without ever signalling it:

```
(defparameter *c* (make-condition 'type-error
                                  :datum "abc"
```

```
                                  :expected-type 'integer))

(type-error-datum *c*)            ; => "abc"
(type-error-expected-type *c*)    ; => INTEGER

(princ *c*)    ; prints something like:
               ; The value "abc" is not of type INTEGER.
```

Each condition type has accessors for its facts: `type-error` knows the bad value and the type wanted. Handlers read these instead of parsing text — the reason `error` takes a format control rather than a finished string.

New condition types are defined with `define-condition`, which reads like `defstruct` with a report instead of a print function:

```
(define-condition room-not-found (error)
  ((name :initarg :name :reader room-not-found-name))
  (:report (lambda (condition stream)
             (format stream "No room called ~(~A~)."
                     (room-not-found-name condition)))))

(make-condition 'room-not-found :name 'attic)
```

The parent class, `(error)`, decides how the condition is treated: inheriting from `error` makes it an error, inheriting from `warning` makes it a warning, inheriting from `condition` alone makes it a plain event. `:report` builds the message the debugger shows. `:initarg` and `:reader` are Chapter 13's slots again — conditions are CLOS objects under the hood, and Chapter 17 makes that explicit.

Signalling your own type:

```
(error 'room-not-found :name 'attic)
```

15.3. Handling: `handler-case`

`handler-case` runs a body with a rescue plan for each condition type. Its shape:

```
(handler-case
    body-form
  (condition-type (c) recovery-form...))
```

If `body-form` runs cleanly, its value is the result. If it signals a matching condition, the stack unwinds to this point and the recovery forms run instead:

```
(defun parse-count (text)
  "Return the integer in TEXT, or NIL."
  (handler-case (parse-integer text)
    (error (c) (declare (ignore c)) nil)))

(list (parse-count "42")
      (parse-count "  7 ")
```

```
      (parse-count "nope"))
;; => (42 7 NIL)
```

`parse-integer` is new here. It converts the digits in a string to a number, and *signals an error* when there are none — it refuses to guess. It also returns two values: the number and where it stopped reading (Chapter 13's multiple values).

Compare this with other languages' try/catch. The differences that matter:

- The *condition type* decides which clause runs, and the clause can be as specific as your own condition type. Catching `error` catches everything, which is rarely what you want — write specific clauses.
- The stack has already unwound when the handler runs: the recovery code runs where `handler-case` sits, not where the error happened. That limits what a handler can do — it can substitute a value, but it cannot "fix things and continue from the middle". The next section is the feature that fills that gap.
- The condition object `c` is bound for the recovery forms, so the handler can ask questions: `(type-error-datum c)` and so on.

Several clauses, most specific first:

```
(defun try-integer-divide (a b)
  "Return A divided by B, or a message when that can't work."
  (handler-case (/ a b)
    (division-by-zero () "Can't divide by zero.")
    (type-error (c) (format nil "Bad input: ~A." (type-error-datum c)))))

(list (try-integer-divide 10 2)
      (try-integer-divide 10 0)
      (try-integer-divide 10 'x))
```

Expected: `(5 "Can't divide by zero." "Bad input: X.")`

The clause with no parameter, `(division-by-zero () ...)`, is normal: the condition object is there if you want it, not if you don't. The clause form `(:no-error ...)` runs when the body succeeded, which is handy for functions that report either a result or a failure:

```
(defun safe-parse-or-log (text)
  (handler-case (parse-integer text)
    (error (c) (format t "~&parse failed: ~A~%" c) nil)
    (:no-error (n) (format t "~&parsed ~D~%" n) n)))
```

`:no-error`'s variable list gets the body's *values* — a multiple-value safe exit.

15.4. Restarts: repair instead of rescue

`handler-case` substitutes an answer. A *restart* offers the caller a *choice of ways to continue*, and this is what makes Lisp's system different. When the debugger opens, what you've been seeing is a menu of restarts — `ABORT`, and usually others such as `USE-VALUE` or `STORE-VALUE`.

You can define your own with `restart-case`:

```
(defun read-temperature (text)
  "Return the temperature in TEXT, or 20 when unreadable."
  (restart-case (parse-integer text)
    (use-default ()
      :report "Use the default temperature of 20."
      20)))
```

If `parse-integer` signals, no handler catches it here — but the `restart-case` has registered a way out. In the debugger, the `USE-DEFAULT` restart appears in the menu, and choosing it returns `20` — execution resumes as `if` that value had been produced.

Invoking restarts programmatically is `invoke-restart`, but the everyday tool is `find-restart` with `invoke-restart` inside a handler, or better: associating them. The full pattern — a low-level function that signals, a middle layer that offers restarts, a high-level function that chooses — is the *condition system idiom*:

```
(defun move-towards (name)
  "Change room, signalling ROOM-NOT-FOUND on a bad name."
  (let ((room (find-room name)))
    (when (null room)
      (error 'room-not-found :name name))
    (setf *current-room* room)
    (look)))

(defun move-safely (name)
  "Like MOVE-TOWARDS, but offers recovery choices."
  (restart-case (move-towards name)
    (substitute-room (other)
      :report (lambda (stream)
                (format stream "Go to ~(~A~) instead." other))
      (move-towards other))
    (stay-put ()
      :report "Stay where you are."
      "You stay put.")))
```

That `:report` lambda is overcomplicated, and shows why restarts belong with simpler arguments: the report format string is the main readable part, and the lambda hides it. A cleaner version:

```
(defun move-safely (name)
  "Like MOVE-TOWARDS, but offers recovery choices."
  (restart-case (move-towards name)
    (stay-put ()
      :report "Stay where you are."
      "You stay put.")))

(defun move-or-stay (name)
  "Move to NAME, or stay when no such room exists."
  (handler-case (move-towards name)
    (room-not-found ()
      (invoke-restart 'stay-put))))
```

Read the layers: `move-towards` knows nothing about recovery; it only announces facts. `move-safely` offers choices. `move-or-stay` decides. Any layer can be swapped without touching the others — the same separation Java's exception hierarchy aims at, but with the choice of *continuing* built in, not just unwinding.

`(invoke-restart 'stay-put)` only works when a restart of that name is in scope — inside the `restart-case`. From outside, the debugger menu offers the restarts interactively; that's the same mechanism with you at the controls.

15.5. Interactive restarts: the debugger as REPL

When an unhandled error opens the debugger in SLIME or SLY (Appendix C), you see:

- The condition and its message.
- The menu of restarts, numbered.
- The stack of calls, each inspectable.

The restarts you'll actually use:

Restart	Meaning
`ABORT`	Give up this attempt, back to the REPL (`q`)
`USE-VALUE v`	Pretend the failing expression produced `v`
`STORE-VALUE v`	Like `USE-VALUE`, for places rather than values
`RETRY`	Run the failing form again

`USE-VALUE` is the one worth knowing by name. When a lookup fails deep in a long computation, you don't restart the computation: you supply the missing value and continue. In your own code, you can offer exactly that restart with `restart-case` and `:report "Use a replacement value."`, and the debugger menu grows a choice tailored to the problem.

15.6. `ignore-errors` and `unwind-protect`

Two small forms cover the everyday cases.

`ignore-errors` returns `nil` when its body signals any error, and the body's value otherwise. It's `handler-case` with one catch-all clause and no access to the condition:

```
(list (ignore-errors (/ 1 0))
      (ignore-errors (/ 10 2)))
;; => (NIL 5)
```

It's quick and blunt. For anything where you'd like to know *what* went wrong, use `handler-case`.

`unwind-protect` isn't about errors at all, but about cleanup: its second form runs no matter how the first form exits — normally, by error, by a jump:

```
(defparameter *log* nil)

(defun careful-change (f)
  (unwind-protect
      (progn (push 'start *log*)
             (funcall f)
```

```
            (push 'end *log*))
    (push 'cleanup *log*)))

(careful-change (lambda () (error "boom")))
(reverse *log*)   ; => (START CLEANUP)
```

The `end` never ran, because `error` unwound the stack, but `cleanup` did. Any resource you open — a file (Chapter 16), a socket, a temporary state — should be closed in an `unwind-protect`. The `with-open-file` macro you'll meet next chapter has exactly that inside it.

15.7. Common mistakes

15.7.1. Catching `error` when you mean something specific

`(handler-case body (error (c) ...))` recovers from *everything*, including the typos and bugs you wanted to see. Catch the narrowest condition type that covers the case.

15.7.2. Building the message before signalling

```
(error (concatenate 'string "Can't take " (symbol-name item) "."))
```

Pass a format control: `(error "Can't take ~A." item)`. The condition keeps the pieces, and handlers can read them.

15.7.3. Invoking a restart that isn't in scope

`(invoke-restart 'stay-put)` outside the `restart-case` that defined it gives "The restart STAY-PUT is not active". The restart lives while the `restart-case` form runs; the handler must sit inside that dynamic extent.

15.7.4. Confusing `warn` with `error`

`warn` continues; `error` unwinds. If your program keeps running after something you thought was fatal, you signalled a warning.

15.7.5. Forgetting that the stack has unwound

A `handler-case` handler cannot "go back" into the failed code. It chooses a value from where it stands. Continuation-from-the-middle is the restarts' job, offered at the failing site.

15.8. Project step: failsafe commands

The command runner from Chapter 9 crashes the session on a malformed command: `(look kitchen)` calls `look` with an argument, `(take)` calls `take` with none, `(go)` likewise. This step makes `run-command` answer politely instead, and adds a `count-letters` command built on the condition system.

First, a safe runner. The current `run-command` (Chapter 9) looks up the command and applies it; the failure happens inside `apply`. Separate that into two pieces so the error-catching layer has a smaller target:

```
(defun call-command (command)
  "Call the function that handles COMMAND."
  (let ((entry (assoc (first command) *commands*)))
```

```
    (if (null entry)
        "I don't know how to do that."
        (apply (cdr entry) (rest command)))))

(defun run-command (command)
  "Run COMMAND, a list such as (TAKE KEY), and return the reply."
  (restart-case (handler-case (call-command command)
                  (program-error ()
                    "Wrong number of arguments for that command."))
    (retry-with (fixed)
      :report "Run a corrected command instead."
      (run-command fixed))))
```

`program-error` covers wrong argument counts. The `retry-with` restart isn't used by the handler — it's there for you: when this fires in the debugger, you can supply a corrected command and the session continues. That's a debugging affordance, built in four lines.

Check the graceful replies:

```
(list (run-command '(look kitchen))
      (run-command '(take))
      (run-command '(dance)))
```

Expected: the wrong-arguments message twice, and the unknown-command message. No debugger.

Second, a command that counts the letters in an item's name and demonstrates `restart-case` with a real recovery:

```
(defun letter-count (item)
  "Return the number of letters in ITEM's name, or offer zero."
  (restart-case
      (let ((entry (assoc item *item-locations*)))
        (when (null entry)
          (error 'room-not-found :name item))
        (format nil "~(~A~) has ~D letters."
                item (length (symbol-name (first entry)))))
    (guess-zero ()
      :report "Treat it as zero letters."
      (format nil "~(~A~) has 0 letters." item))))

(defun count-letters (command)
  "Handle (COUNT-LETTERS ITEM)."
  (handler-case (letter-count (second command))
    (room-not-found ()
      (invoke-restart 'guess-zero))))
```

Read the flow carefully, because this is the condition system's round trip: `letter-count` runs inside `count-letters`'s `handler-case` body. When the item isn't in the alist, `letter-count` signals `room-not-found` from inside its own `restart-case`. The handler in `count-letters` runs — *and it sits inside* the `restart-case`'s dynamic extent, because `letter-count` is the function that established the `restart-case`. So the handler can invoke the `guess-zero` restart, which resumes `letter-count` from the `restart-case` — not from `count-letters`'s point, but from `letter-count`'s. That's the

whole condition system in one example: the signalling function offers choices, the caller decides, and neither needs to know the other's code.

Add the command:

```
(push '(count-letters . count-letters) *commands*)

(defparameter *item-locations*
  '((key . kitchen) (letter . bedroom)))
(list (run-command '(count-letters key))
      (run-command '(count-letters sword)))
```

Expected: `"key has 3 letters."` and `"sword has 0 letters."` — the second through the restart path.

15.9. Exercises

15.1 Predict what each block returns or prints, then check:

```
(ignore-errors (+ 1 2))
(ignore-errors (nth 5 '(a b)))
(handler-case (/ 4 0) (division-by-zero () 'nope))
(handler-case (car 'sym) (type-error (c) (type-error-datum c)))
(warn "test") ; what does this return?
```

15.2 Write `safe-nth`, which takes an index and a list and returns the element, or `nil` when the index is out of range — using `handler-case` and `nth`'s error, not `length`.

15.3 Define a condition `invalid-move` with a `:reason` slot and a `:report`, then write `check-move`, which signals it for a direction the current room doesn't have, and `try-move`, which catches it and returns the reason.

15.4 Each snippet below is wrong. Say what's wrong, then fix it.

```
(error (format nil "Bad value: ~A" v))

(handler-case (parse-integer s)
  (error () "bad")
  (parse-error () "unparseable"))

(handler-case body
  (error (c) (invoke-restart 'my-restart)))
```

15.5 Write `to-integer`, which returns the integer in a string, or `0` with a warning when the string holds no integer. Use `parse-integer`, `handler-case` and `warn`, and make sure the warning text includes the bad string via a format directive.

15.6 Write `collect-results`, which takes a list of strings and returns a list of the `parse-integer` results of the parseable ones, skipping the rest. Use `loop`, `handler-case` and `push`, then `nreverse` it at the end — or explain in a comment why `append` in a `collect` position would also work.

15.7 project Write `run-session`, which takes a list of commands and runs them all with `run-command`, collecting replies, so that one malformed command cannot stop the session. Compare its `handler-case` with `run-command`'s: why do both need one?

15.8 project Add a `repair` restart to `call-command` for unknown commands, whose `:report` explains it substitutes `(look)`, and make `run-command` use it via `handler-case` and `invoke-restart` so that `(run-command '(dance))` runs `look` and returns its text.

CHAPTER 16.

Packages, Files and ASDF

Everything so far lived in one flat world: every symbol you made sat in the same bag, and the game's functions called each other by bare name. Small programs can live like that. This chapter adds the three tools Lisp uses to organise bigger ones: *packages*, which divide symbols into named groups; *files and streams*, which let a program read and write text on disk; and *ASDF*, which tells Lisp how to load a program spread over several files.

16.1. Packages: bags of symbols

A package is a table of symbols. Every symbol belongs to exactly one package, and every read of Lisp code happens *in* some package, which decides where each bare name goes. When you've been typing at the REPL, the prompt has been telling you this:

```
CL-USER>
```

`CL-USER` is the default package, the playground. `*package*` holds the current one:

```
*package*              ; => #<PACKAGE COMMON-LISP-USER>
(package-name *package*) ; => "COMMON-LISP-USER"
```

The symbols you've used come from two main packages: `CL`, the Common Lisp language itself (`car`, `defun`, `format`, several hundred of them), and `KEYWORD`, home of `:name` and friends. Symbol names are only strings, and two packages may both contain a symbol named `ROOM` without any confusion — the full name of a symbol is written `package:symbol`:

```
(symbol-package 'car)          ; => #<PACKAGE COMMON-LISP>
(symbol-package :key)          ; => #<PACKAGE KEYWORD>
(symbol-package 'room-name)    ; => #<PACKAGE COMMON-LISP-USER>
```

16.1.1. `defpackage` and `in-package`

A new package is defined with `defpackage`, which is itself a macro from `CL`. A package for the game's engine:

```
(defpackage :game
  (:use :cl)
  (:export :look
           :take
           :drop
           :inventory))

(in-package :game)
```

Read the parts:

- `(:use :cl)` says the `CL` package's exported symbols are visible here — otherwise you'd have to write `(cl:defun ...)` everywhere.

- `(:export ...)` lists the symbols other packages may use: the engine's *public interface*. Everything else stays private.
- `(in-package :game)` makes `game` the current package. From here on, bare names resolve into `game`.

After `in-package`, defining things is ordinary:

```
(defun internal-helper ()
  "Private to the GAME package."
  'shh)

(defun take (item)
  "Public: take ITEM."
  (format nil "taking ~(~A~) with ~A" item (internal-helper)))
```

Switch back and reach in through the interface:

```
(in-package :cl-user)

(print (game:take 'key))
```

`game:take` works because `take` was exported. `game:internal-helper` (single colon) is a read error — the symbol isn't exported, and the reader refuses to resolve it. A double colon, `game::internal-helper`, does reach an unexported symbol, and that asymmetry is deliberate: double colon says "I know this is private and I want it anyway", which is useful when debugging your own code and rude between packages you don't own.

Symbols are shared, not copied: `game:take` *is* the same symbol the `game` package calls `take`. Exporting is just permission to refer to it.

16.1.2. Name conflicts

Two packages may both export a symbol named `look`. A package that uses both must resolve the collision; if you `:use` a package whose export collides with `cl`'s, the reader complains as soon as a conflicting name is read. The everyday rules that keep this from happening:

- Use `:use :cl`, and for other packages prefer explicit listing: `(:use :cl :game.utils)` on packages whose exports you control.
- When a conflict appears, the honest fix is renaming one of the symbols, not twisting imports.

Two more bits of syntax you'll meet. `#:foo` is a symbol belonging to no package at all, made by `gensym` in Chapter 14 and used there for exactly this reason: it cannot collide with anything. And the keywords `:name` and `:key` belong to `keyword`, evaluate to themselves, and are the standard choice for names that are only names.

16.1.3. Packages and Org Babel

One practical note. Your Org Babel blocks share one Lisp session, so an `in-package` in one block affects all the others. When a chapter experiment switches packages, switch back with `(in-package :cl-user)` in the same block, or your next game function will be defined in the wrong place. The blocks below each switch back at the end.

16.2. Files and streams

A *stream* is a source or sink of characters (or bytes). You've used streams all along: `format t` writes to the standard output stream, the debugger reads from the input one. Files are streams attached to paths on disk.

16.2.1. Reading a file: `with-open-file`

```
(with-open-file (in "/tmp/notes.txt")
  (loop for line = (read-line in nil)
        while line
        collect line))
```

`with-open-file` opens the file, binds `in` to the stream, runs the body, and closes the file however the body exits — it has an `unwind-protect` from Chapter 15 inside it. Never call `open` without a closing plan; `with-open-file` is the form to reach for.

`read-line` reads one line and returns it without the newline. Its second argument `nil` means "don't signal at end of file"; the *second* value it returns then says whether the end was reached, but the `while line` test above handles the common case. For a file you didn't write, the robust walk uses the second value:

```
(with-open-file (in "/tmp/notes.txt")
  (loop for (line found) = (multiple-value-list (read-line in nil))
        while found
        collect line))
```

`read-char` reads one character, `read` reads one Lisp *object* — more on that below. All three accept the same `nil` second argument and return `nil` at the end.

16.2.2. Writing a file

`with-open-file` takes `:direction` and, for writing, whether to replace or append:

```
(with-open-file (out "/tmp/notes.txt"
                     :direction :output
                     :if-exists :supersede)
  (format out "Line one.~%")
  (write-line "Line two." out))
```

Argument	Meaning
`:direction :output`	Writing instead of the default `:input`
`:if-exists :supersede`	Replace an existing file
`:if-exists :append`	Add to the end of an existing file
`:if-exists :error`	The default: signal an error rather than clobber

Without `:if-exists`, writing over an existing file is an error — Lisp refuses to destroy your data by accident.

`write-line` writes a string and a newline; `princ` and `print` work on streams too, taking the stream as their optional argument. `format`'s third destination kind from Chapter 12 is exactly this stream.

16.2.3. Saving Lisp data: `print` and `read`

Text meant for people and text meant for Lisp are different, and Chapter 12's `~A` versus `~S` distinction now gets its reason. To *save* data, write it with `print`, which produces text Lisp can read back:

```
(defparameter *save-state*
  '((key . kitchen) (letter . bedroom)))

(with-open-file (out "/tmp/save.lisp" :direction :output
                     :if-exists :supersede)
  (print *save-state* out)
  (print 'kitchen out))

(with-open-file (in "/tmp/save.lisp")
  (list (read in) (read in)))
;; => ((KEY . KITCHEN) (LETTER . BEDROOM)) and KITCHEN
```

Each `read` takes the next object from the stream: a list, a symbol, a number, a string — anything the reader can parse. The round trip through `print` and `read` is exact, which is why `print` writes the quotes on strings and the case of symbols. This *readable data* style is one of Lisp's quiet strengths: configuration files, saved games and caches can be Lisp data, read back with no parser of your own.

`print` puts a newline before and a space after its output, so repeated `print` calls produce one object per line. `prin1` is `print` without the newlines, for packing several objects on one line.

16.3. ASDF: another system definition facility

A *system* is a program's files and how they fit together. ASDF is the standard tool that knows how to load one: which files, in what order, and what else must be loaded first.

A system is described by a `.asd` file. Suppose the game moves into a directory `~/src/game/` with two files, `packages.lisp` and `game.lisp`. The file `game.asd` sits alongside them:

```
~/src/game/
├── game.asd
├── packages.lisp
└── game.lisp
```

```
(asdf:defsystem "game"
  :description "A small adventure game"
  :serial t
  :components ((:file "packages")
               (:file "game")))
```

The parts:

- `:serial t` loads the components in the listed order, each one only after the previous — what you want nearly always at the start.
- `:components` lists the files, names without the `.lisp` ending.

- `packages.lisp` holds only the `defpackage` form, `game.lisp` everything else. Splitting the package definition into its own file is the convention, because other files' `in-package` lines need it loaded first.

To load it, ASDF must find the `.asd` file. With Quicklisp (Appendix A), the easy home is its local-projects folder; otherwise push the directory onto ASDF's registry:

```
(push "/home/you/src/game/" asdf:*central-registry*)
(asdf:load-system "game")
```

`load-system` compiles and loads each file in order, and on the next call it reloads only what changed. From then on, other code reaches the game through its package:

```
(in-package :cl-user)
(game:look)
```

ASDF scales up from here — dependencies between systems, tests, versioned releases — but the shape above is the shape everything else hangs off: one `.asd` file, files listed in order, one package per concern. Quicklisp and ASDF divide the work: Quicklisp *finds and fetches* systems from the internet, ASDF *builds* the ones you have.

16.4. Common mistakes

16.4.1. Forgetting `(:use :cl)`

```
(defpackage :game)
(in-package :game)
(defun take (x) x)

CL-USER> (game:take 1)
;; works, but in :game, even CAR is invisible:
;; (car '(1 2)) gives "The function CAR is undefined"
```

Without `:use :cl`, bare names resolve only inside the package, and the whole language vanishes. Always `(:use :cl)` unless you know why not.

16.4.2. Switching packages and forgetting to switch back

An `in-package` at the REPL is sticky. When `(in-package :game)` is run and the next block defines `take`, the game's `take` is silently replaced. The rule above: switch back inside the same block.

16.4.3. The wrong path separators

Windows paths work in Lisp with forward slashes: `"C:/Users/you/save.lisp"`. Backslashes need doubling, since `\` is an escape character in strings (Chapter 3). Forward slashes work everywhere and are the habit to keep.

16.4.4. Reading with `read` what wasn't written with `print`

`read` parses Lisp syntax, so feeding it prose gives reader errors about stray parentheses and unknown symbols. Human-readable files want `read-line`; Lisp-readable files want `print` and `read`.

16.4.5. Opening for output without :if-exists

```
(with-open-file (out "/tmp/notes.txt" :direction :output) ...)
;; the second run fails: "File exists"
```

Decide deliberately: `:supersede` to replace, `:append` to add, `:error` to refuse.

16.4.6. Streams used after the file is closed

The stream `in` exists only inside the `with-open-file` body. Saving it in a variable and reading from it later gives "stream is closed". Do all the reading in the body, or keep the *data* rather than the stream.

16.5. Project step: save and load the game

The game gains two commands, `save-game` and `load-game`, that write and restore the whole state: the item alist and the current room's name.

The state to save, gathered into one list:

```
(defun game-state ()
  "Return a single list describing the whole game state."
  (list *item-locations*
        (room-name *current-room*)))

(defun restore-state (state)
  "Set the game state from the list STATE."
  (setf *item-locations* (first state))
  (setf *current-room* (find-room (second state))))

(defparameter *save-file* "/tmp/game-save.lisp")
```

The two commands:

```
(defun save-game ()
  "Write the game state to the save file."
  (with-open-file (out *save-file*
                       :direction :output
                       :if-exists :supersede)
    (print (game-state) out))
  (format nil "Saved to ~A." *save-file*))

(defun load-game ()
  "Restore the game state from the save file."
  (handler-case
      (with-open-file (in *save-file*)
        (restore-state (read in))
        (look))
    (file-error ()
      "No saved game found.")))
```

`save-game` writes one object — the state list — and `load-game` reads one object back. The `handler-case` turns a missing file into a polite reply instead of a debugger: `file-error` is the condition type for exactly that.

Check the round trip. Walk to the kitchen, take the key, save, wreck the state, load:

```
(defparameter *item-locations*
  '((lamp . cellar) (key . kitchen) (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))

(list (run-command '(take key))
      (save-game))
```

```
;; Wreck the state, as if the session had restarted:
(setf *item-locations* nil)
(setf *current-room* (find-room 'bedroom))

(load-game)
;; => the kitchen's description

(inventory)
;; => "You are carrying: key."
```

What was destroyed is restored, because the state went to disk as readable Lisp and came back the same. Add the commands to the table:

```
(push '(save . save-game) *commands*)
(push '(load . load-game) *commands*)
```

One design point worth noticing: `game-state` and `restore-state` are the whole save format, written down as functions. When the game grows a new piece of state, those two functions change together and nothing else knows how the save file is laid out.

16.6. Exercises

16.1 Predict the result of each line, then check:

```
(symbol-package 'defun)
(symbol-package 'game:take)
(keywordp :look)
(symbol-package (first '(a b)))
```

16.2 Write a file `"/tmp/haiku.txt"` with three lines of any text using `with-open-file` and `write-line`, then read it back into a list of three strings with `read-line`.

16.3 Write `append-note`, which takes a string and adds it to the end of `"/tmp/notes.txt"`, creating the file if it doesn't exist. The `:if-does-not-exist` argument of `with-open-file` controls that; `:create` is its usual value.

16.4 Predict the result, then check:

```
(with-open-file (out "/tmp/num.lisp" :direction :output
                     :if-exists :supersede)
  (print '(1 (2 3) "four") out))
(with-open-file (in "/tmp/num.lisp")
  (read in))
```

Why does the list come back with the string's quotes intact?

16.5 Each snippet below is wrong. Say what's wrong, then fix it.

```
(with-open-file (in "/tmp/notes.txt")
  (setf *saved-lines* in))
; later:
(read-line *saved-lines*)

(with-open-file (out "/tmp/notes.txt" :direction :output)
  (write-line "overwrite?" out))
; second run

(read-line (open "/tmp/notes.txt"))
```

16.6 Write `count-lines`, which returns the number of lines in a file, using `with-open-file` and `loop`. `(count-lines "/tmp/haiku.txt")` gives `3` after Exercise 16.2.

16.7 Define a package `:shop` that uses `cl` and exports `sell` and `buy`. Inside it, define a private function `price` and the two exported functions using it. Then, from `cl-user`, show that `shop:sell` works, `shop:price` fails, and `shop::price` works.

16.8 project Extend `game-state` and `restore-state` to include `*history*`, so that undo survives a save and load. Check that after `save`, `take`, `load`, an `undo` restores the key — and say in a comment what had to change and what didn't.

16.9 project Write `save-report`, which writes the text of `describe-all` (from Exercise 11.11) to `"~/game-report.txt"`, one line per room, and returns the number of lines written. Where does `~` as a path start from, and does it work on Windows? Check with your implementation's documentation, and make the path a `defparameter` so it's easy to change.

Part III review

You can now read any list-shaped data at sight, produce text with `format`, build records and lookup tables, write macros that extend the language, handle errors with conditions and restarts, and organise code into packages, files and systems. If any of these feels shaky, the fix is REPL time, not re-reading: take one chapter's exercises and redo two of them from scratch.

The game, at the end of Part III, has rooms as structures (Chapter 13), items as an alist (Chapter 7), a command table with a failsafe runner (Chapters 9 and 15), an undo history (Chapter 14), and the ability to save and load its state (this chapter). From Part IV on, the guide adds the tools that make the game's **own** code — the helpers, not just the game — better: conditions as objects other code can program against, CLOS, packages designed around interfaces, macros that read like syntax, and the reader and printer mechanisms that let save files round-trip through `print` and `read` on data structures Lisp doesn't know about yet.

PART IV

The Lisp Way

Part I gave you the language. Part II gave you lists and recursion. Part III gave you the tools a working program uses: loops, formatted output, records and lookups, macros, conditions, packages, files.

Part IV goes *under* those tools. It explains the mechanisms whose behaviour you've been trusting:

- where names live and how binding actually works (lexical and dynamic scope, closures, special variables);
- how Lisp reads text into data, and how that single mechanism underlies the reader macros, the save-file round trip, and `format`'s printing direction;
- the condition system's other half — non-unwinding handlers, the restart toolkit, the condition hierarchy;
- what compilation and loading actually do to your code, and why `eval-when` exists;
- what types mean in a dynamically typed language, and why declarations matter anyway;
- what a symbol is as a data structure, including the plist slot you haven't touched yet;
- what multiple values are, and how they differ from lists;
- closures as objects, including the constructor idiom that makes them cheap state;
- the parts of CLOS beyond `defclass` and `defmethod`;
- packages as data structures you can query and manipulate;
- generators, built from closures and values;
- the habits of reading other people's Lisp;
- the practical interoperability layer — JSON, CSV, shelling out, C via CFFI.

None of this is new *language*. All of it is new *understanding*. At the end of Part IV, when something in Lisp surprises you, you'll have a model to reason from, not just a set of behaviours to memorise.

The project continues to grow as the chapters demand — undo histories in Chapter 17′s exercise, the game's public interface in Chapter 24′s project step, and the executable in Part VI.

CHAPTER 17.

Lexical and Dynamic Variables

Chapter 9 gave you closures: functions that remember the `let` variables around them. That memory works because of *lexical scoping*, and it is half of Lisp's binding story. The other half — *dynamic scoping*, the story behind the earmuffed names like `*print-readably*` and `*package*` — has been used quietly for chapters. This chapter makes both halves explicit and shows how they fit together.

17.1. Lexical scope: where a name lives in the text

Lexical scope is scope by *position in the source code*. When you write:

```
(defun make-adder (n)
  (lambda (x) (+ x n)))
```

the `n` inside the lambda is meaningful because the lambda is written `inside` `make-adder`'s parameter list. Where a variable can be seen is determined by where it appears in the text — by the nesting of forms — not by anything that happens at run time.

That is the only kind of scope you've met so far, and it is the only kind Java and Python have. Two consequences worth stating plainly, because Chapter 9 used them without naming them:

- *Closures close over the binding, not the value.* The lambda keeps a live reference to `n`'s binding; if the binding is later changed with `setf`, the closure sees the change (the `make-counter` example).
- *Each call makes new bindings.* Calling `make-adder` twice gives two independent `n`s, each closed over by its own lambda. This is why closures can be used as private state.

17.2. Dynamic scope: where a name lives in time

Now a second kind of binding. Every variable defined with `defvar` or `defparameter` is a *dynamic variable* (also called a *special variable* — the terms are interchangeable; `special` comes from `declare` `(special ...)`, below). You've made many of them for the game:

```
(defparameter *current-room* nil)
(defparameter *item-locations* nil)
```

A dynamic variable has a *global* value, but — and this is the part that takes getting used to — its meaning at any moment is *whoever has most recently bound it while running*. A `let` on a dynamic variable doesn't make a private variable: it temporarily *replaces* the global value for the duration of the `let` body, and restores it afterwards:

```
(defparameter *noise-level* 0)

(defun shout ()
  (format t "noise level: ~D~%" *noise-level*))

(defun loudest ()
  (let ((*noise-level* 11))
    (shout)))
```

```
(defun example ()
  (shout)          ; prints: noise level: 0
  (loudest)        ; prints: noise level: 11
  (shout))         ; prints: noise level: 0  — restored
```

The `shout` function never takes an argument and never reads a lexical `*noise-level*`; it reads *whatever binding is in force when it runs*. Inside `loudest`, the `let` has pushed a new binding onto `*noise-level*`, and everything `loudest` calls — however many functions deep — sees it. When `loudest` returns, the binding is popped and the old value is exactly as it was.

This is scope in *time* rather than scope in *text*: a dynamic variable's extent is the dynamic extent of the form that bound it. Chapter 15 said "the restart lives while the `restart-case` form runs — its dynamic extent". Same phrase, same idea; now you know it applies to variables too, and that `restart-case`'s visibility to `invoke-restart` is dynamic scoping in action.

17.3. Why earmuffs: the naming rule

The asterisks — "earmuffs" — are not decoration; they are the community's marking for "this variable is dynamic". The rule they encode:

- A dynamic variable's name *should* have earmuffs. Reading `*noise-level*`, you know its value depends on who called you.
- A lexical variable's name *should not*. Reading `noise-level`, you know it's a local or parameter, entirely under your function's control.

The convention matters because the two kinds of variable fail in different ways, and the name tells you which failure mode to expect. A lexical can be shadowed by accident (Chapter 4); a dynamic can be *rebound* by any caller up the stack. Several existing earmuffed names from the chapters — `*package*`, `*print-readably*`, `*error-output*` — are dynamic variables defined by the implementation, and their earmuffs are the same signal: "any code you call can change what this means to you, and the polite way to change it for your callees is to `let` it".

Why did Chapter 4 use `defparameter` for game state? Because the game's state *is* dynamic by nature: `look`, `take` and `inventory` all read `*current-room*` and `*item-locations*` without them being passed, and the whole program is a family of functions sharing that state. This is the honest use of dynamic variables: *global configuration and state that many functions share*, especially when you sometimes want to change it for a whole subtree of calls — which is exactly what Chapter 23's `let ((*save-file* ...))` around a test will do.

17.4. defvar vs defparameter, properly

Chapter 4 noted the mechanical difference; the real difference is about *initialisation intent*:

- `defparameter` assigns the value *every time the form runs*. Loading the file twice, or C-c C-c on the line, resets the variable. Right for values you're actively developing — the game's state, whose reset-on-reload is a feature while you iterate.
- `defvar` assigns *only if the variable is unbound*. Reloading the file leaves an existing value alone. Right for values the *user* of your code might have changed — configuration like `*save-file*` that should survive a reload, and values a library defines but expects callers to customise.

This is why libraries use `defvar` for their knobs:

```
(defvar *default-save-file* "/tmp/game-save.lisp"
  "Overridden by callers who want a different path.")

(defun save-to-default ()
  (with-open-file (out *default-save-file*
                       :direction :output
                       :if-exists :supersede)
    (print (game-state) out)))
```

A caller redefining `*default-save-file*` with `defparameter` in their own file keeps their setting across reloads of the library. With `defparameter` in the library, every reload would trample it.

One subtlety worth knowing: `defvar` also *declares the name special* — this is how a variable becomes dynamic. The declaration sticks to the name globally, for good. That leads to the trap below.

17.5. Binding vs assignment

The distinction that untangles most dynamic-variable confusion:

- *Binding* creates a new place for the name: `let`, `defparameter`, function parameters all bind.
- *Assignment* changes an existing place: `setf` assigns.

On a *lexical* variable, the difference is barely visible: `let` binds, `setf` assigns, and nothing else can see either from outside. On a *dynamic* variable, the difference is everything:

```
(defparameter *x* 1)

(defun reader ()
  *x*)

(defun assigner ()
  (setf *x* 2))

(defun binder ()
  (let ((*x* 3))
    (reader)))

(list (reader)      ; => 1
      (progn (assigner) (reader))  ; => 2  — assignment changed the global
      (binder))     ; => 3  — binding shadowed it, only during the let
```

The `binder` case is the one to internalise: `let` on a dynamic variable is not "make a local `*x*`" but "temporarily change `*x*` for everyone I call". Chapter 18's exercise used exactly this to redirect warnings: `(let ((*error-output* ...)) ...)` — a binding, not an assignment, so the rest of the program was unaffected after the `let` ended.

17.6. The dynamic variable toolkit

The remaining pieces, for completeness and for the rare times you need them:

`symbol-value` reads a dynamic variable's current value when you have the *name* as an object rather than in code:

```
(symbol-value '*x*)          ; the same as (the value of *x*)
(setf (symbol-value '*x*) 9) ; assignment by name
```

This is how code can work on variables as data — a configuration system can look up `*default-save-file*` from a string in a file. It only works for dynamic variables: lexical variables have no symbol home to look up, which is precisely what makes them unreachable from `symbol-value`.

`set` is the ancestor of `setf`: it assigns a dynamic variable by name, `(set '*x* 5)`. Old code uses it; new code uses `setf` on `symbol-value` if it means this. Know it when you read it.

`makunbound` removes a dynamic variable's global value — Chapter 9 met it as a debugging tool. Afterwards the variable is *unbound*, not `nil`: reading it signals `unbound-variable`, and `(boundp '*x*)` asks before reading.

`progv` binds dynamic variables by name at run time — the deepest end of this toolkit, and rare:

```
(progv (list '*x*) (list 42)
  *x*)    ; => 42, with *x* dynamically bound for the body
```

You'll meet it in code that interprets other Lisp code. Note what it can do that `let` cannot: the variable names don't have to be written in your source; they can be computed.

`(declare (special x))` is the manual switch that makes a name dynamic where no `defvar` has run. You should *not* need it — if you find yourself writing it, what you almost certainly want is a `defvar`. It exists for reading other people's code and for rare binding tricks; the habit to avoid is defining ordinary-looking variables with it, because the declaration leaks past lexical boundaries in confusing ways.

17.7. The standard dynamic variables you'll actually use

A short list, since you've used half of these already and knowing them as a family helps:

Variable	Meaning	Used in
`*package*`	the current package — affects how names are read	16
`*standard-output*`	where `format t`, `princ` and friends write	12
`*error-output*`	where `warn` writes	18
`*print-readably*`	print output that `read` can rebuild	18
`*print-length*`	cap on list length printed (nil = unlimited)	this ch.
`*print-level*`	cap on nesting depth printed	this ch.
`*read-base*`	what number base `read` assumes	this ch.
`*features*`	the list of implementation feature symbols	App. A
`*load-pathname*`	the file being `load`ed, if any	this ch.
`*debugger-hook*`	a function to call instead of the debugger	this ch.

Two examples of the `let` pattern in daily use. Printing large data without flooding the screen:

```
(defparameter *big-list* (loop for i below 100 collect i))
(let ((*print-length* 10)
      (*print-level* 3))
```

```
  (print *big-list*))
;; (0 1 2 3 4 5 6 7 8 9 ...) — ten elements, then ellipsis
```

And sending one function's output to a string instead of the terminal (Chapter 18's `with-output-to-string` is this pattern, prepackaged):

```
(with-output-to-string (s)
  (princ *noise-level* s))
;; => "0"
```

17.8. Lexical and dynamic together: which one wins

When a name could be both — say a `let` on a name that happens to be `defvar`'d — *dynamic wins*: the `defvar` makes every binding of that name dynamic, even `let` bindings and even function parameters. That is the trap:

```
(defvar *acc* nil)

(defun process (list)
  ;; DANGER: *acc* is dynamic, so this let is a global
  ;; temporary change, and nested/reentrant calls share it!
  (let ((*acc* nil))
    (dolist (x list)
      (push x *acc*))
    (nreverse *acc*)))
```

For this function it happens to work — but if two calls to `process` nest (because something in the `dolist` calls `process` again), they *share the same binding*, and the inner call's `pops` wreck the outer's accumulator. A lexical `let` would have given each call its own. The rule:

- Loop accumulators, temporary results, anything private to one call: *lexical* names without earmuffs.
- Shared state and configuration deliberately spanning calls: *dynamic* names with earmuffs.

The earmuff convention isn't just style; it's how you keep the two categories from colliding in the same name.

17.9. Common mistakes

17.9.1. Earmuffs without defvar/defparameter

```
(setf *result* 42)   ; *result* was never defined
```

This works — `setf` on an undefined name creates a global — but the variable is not declared special, so it is a plain global, and warnings about it appear. Worse, if some other file later runs `(defvar *result*)`, the name becomes dynamic and *every existing `let` on it changes meaning*. Always create dynamic variables with `defvar` or `defparameter`, never bare `setf`.

17.9.2. Rebinding a special by parameter

```
(defun draw-room (*current-room*) ...)
```

Since `*current-room*` is special, this parameter doesn't shadow it lexically — it dynamically *rebinds* it for the duration of the call, and everything the function calls sees the new room. That may even be what you want, but it is never what it looks like to a Java-trained eye. Keep parameters unearmuffed.

17.9.3. Assuming let restores after setf

`(let ((*x* 1)) (setf *x* 2))` leaves the *global* `*x*` as it was — the `let` bound it, `setf` assigned to the binding, and the binding dies at the `let`'s end. But `(progn (setf *x* 2) ...)` with no `let` changed the global for good. If your test suite changes a knob with `setf` and doesn't restore it, later tests run with the changed value: bind with `let` around each test instead of assigning.

17.9.4. Reading print-length as a bug

When a printed list shows `(a b c ...)`, that's `*print-length*` abbreviating, not data missing. Check whether something bound the print variables.

17.10. Project step: configurable output

The game's messages go to the terminal by `format t`. Making the game testable and scriptable means sending them *somewhere else* — a string, a file — without touching the message functions. Dynamic variables are the tool:

```
(defparameter *game-output* t
  "Where game messages go: t for the terminal, or a stream.")

(defun say (control &rest args)
  "The game's single output function."
  (apply #'format *game-output* control args))

(defun sayln (control &rest args)
  "Like SAY, with a newline."
  (apply #'format *game-output*
         (concatenate 'string control "~%") args))
```

Now every message function calls `say` instead of `format t`, and capturing a whole session of play is one `let`:

```
(defun play-to-string (commands)
  "Run COMMANDS, capture all game output in a string."
  (with-output-to-string (s)
    (let ((*game-output* s))
      (dolist (c commands)
        (sayln "~A" (run-command c))))))
```

Check:

```
(setf *current-room* (find-room 'kitchen))
(setf *item-locations* '((key . kitchen)))

(play-to-string '((look) (take key)))
```

Expected: one string holding the look text and the take text — with nothing printed to the terminal, because `*game-output*` was dynamically bound to the string stream for the duration. Note the shape: `say` and the message functions needed no changes; the *configuration* was rebound from outside. That is the dynamic variable's whole job description.

And the test-suite payoff (Chapter 23's harness gets stronger):

```
(defun look-mentions-room-p ()
  "Does the look command's output name the current room?"
  (search "kitchen"
          (string-downcase
           (with-output-to-string (s)
             (let ((*game-output* s))
               (run-command '(look)))))))
```

17.11. Exercises

17.1 Predict the three printed lines, then check:

```
(defparameter *depth* 0)

(defun show () (format t "~D " *depth*))

(defun nest (n)
  (let ((*depth* (1+ n)))
    (show)
    (when (< n 2) (nest (1+ n)))
    (show)))

(nest 0)
```

17.2 Write `with-temp-room`, a function taking a room name and a function, which runs the function with `*current-room*` dynamically bound to that room, restoring the room afterwards even if the function signals. (Two chapters' tools combine here: `let` plus `unwind-protect` — or explain why plain `let` already restores it.)

17.3 Each snippet below is wrong or misleading. Say what actually happens, then fix it.

```
(setf *counter* 0)
(defun bump () (incf *counter*))

(defun loud-say (text)
  (setf *game-output* (make-string-output-stream))
  (format *game-output* "LOUD: ~A" text))

(defvar name nil)   ; intended as a per-call local
```

17.4 Write `symbol-value` based lookup: `config-value` takes a string like `"*SAVE-FILE*"` and returns the current value of the dynamic variable it names, or `:unknown` when no such variable is bound. Use `intern`, `boundp` and `symbol-value`. What does your function return for `"CAR"`, and why?

17.5 Predict, then check:

```
(defvar *log* '())

(defun log-it (x) (push x *log*))

(defun phase-one ()
  (let ((*log* nil))
    (log-it 'a)
    (log-it 'b)))

(phase-one)
*log*
```

Where did `a` and `b` go? What name change makes the two `log-it` calls write to the global log instead?

17.6 Using `*print-length*` and `*print-level*`, write `terse`, which takes any object and returns its printed form capped at 5 elements and 2 levels of nesting. Test it on a deeply nested list.

17.7 project Add a `verbosity` knob: `defparameter *verbose* nil`, and make `take` and `drop` emit extra `say` messages only when `*verbose*` is true. Write `play-quietly`, which runs a command list with verbosity off no matter what it was outside, and `play-loudly`, which forces it on. Then explain in a comment why `say` needed to exist as a wrapper before this exercise could be done cleanly.

CHAPTER 18.

The Reader

Every block of Lisp you've run has passed through two stages on its way to becoming a running program: first the *reader* turned your typed characters into Lisp objects, then `eval` (or `compile`) made those objects do things. You've known this dimly since Chapter 2 — `quote` stops evaluation of a *list object* — but the reader itself has been invisible. This chapter makes it visible, because reader syntax is Lisp's parser, and a surprising amount of "the language" is actually just reader notation.

18.1. The reader's job

The reader takes a stream of characters and produces *data*: one Lisp object per form. That's all it does — it knows nothing about functions, variables or evaluation. When you type:

```
(+ 1 2)
```

the reader produces a three-element list containing the symbol `+`, the integer `1` and the integer `2`. Evaluation is a separate later step. This separation — data first, meaning later — is the property behind macros (Chapter 14) and behind this chapter: you can get at the data, change the reader, and add notation.

`read` is the reader's front door (met in Chapter 16 for files): it reads one object from any stream. `read-from-string` reads one object from a string:

```
(read-from-string "(a b (c 1))")
;; => (A B (C 1)), and a second value: 12, where reading stopped

(read-from-string "42")
;; => 42
```

Everything you type at the REPL, everything in your `.lisp` files, everything `read` pulls from a save file — all of it goes through the same machinery.

18.2. The standard read syntax, collected

Most of this you know; the point is to see it as one table, one system. The reader's notations:

Syntax	Produces	Met in
`123`, `-1.5`	numbers	3
`1/3`	a ratio	3
`1.0d0`	a double float	3
`"text"`	a string (\ escapes inside)	3
`#\a`, `#\Space`	a character	3
`foo`, `Foo`	a symbol (upper-cased by default)	3
`:key`	a keyword	3
`'x`	`(quote x)`	2
`(a ,b ,@c)`	quasiquote	14

Syntax	Produces	Met in
`(a . b)`	a cons cell	7
`nil`, `t`	the two constants	2
`#'fn`	`(function fn)`	9
`#:sym`	an uninterned symbol	14

The `#` (sharp-sign) dispatch family deserves its own look, because each following character is a different notation:

Syntax	Produces	Met in
`#'foo`	the function `foo`	9
`'(1 2)`	`(quote (1 2))`	2
`#x1F`, `#o17`, `#b1010`	hex / octal / binary numbers	new
`#c(1 2)`	the complex number 1+2i	new
`#p"/tmp/x"`	a pathname object	new
`#.expr`	*evaluate expr while reading*	new
`#+feature`, `#-feature`	read the next form only if a feature is (not) present	new
`#\Newline`	named characters	3
`#`	…	`#`
`#;`	comment out the next *form*	new
`#:sym`	uninterned symbol	14
`#S(room ...)`	a structure literal	new
`#(...)`	a vector literal	new
`#n`, `#n#`	label and reference (circular structure)	new

The new ones, in action:

```
(list #xff            ; => 255 — hex
      #b1010          ; => 10  — binary
      #c(3 4)         ; => #C(3 4) — complex
      #p"/tmp/")      ; => #P"/tmp/" — a pathname object
```

Pathnames are Lisp's file-name objects — Chapter 16 used strings where `(merge-pathnames #p"saves/" (user-homedir-pathname))` would be the portable spelling. Complex numbers you may never need; that the reader handles them in one notation is the illustration that *numbers are data the reader knows*.

`#.` — sharp-dot — is the startling one: the reader itself evaluates the expression, and the result becomes the object being read:

```
(list #.(+ 2 3)          ; => (5)
      #.*package*)       ; the current package object, read in as data
```

Used with restraint, it embeds computed constants in source. Used recklessly, it makes source unreadable. The respectable uses are small: `#.(get-internal-run-time)` stamped at compile time, or references to constant tables defined earlier in the same file.

`#+` and `#-` read a form only when a *feature* is present or absent. Features are symbols on the dynamic variable `*features*` (the earmuff from the variables chapter, and now you see it used):

```
#+sbcl (defparameter *implementation* :sbcl)
#+ccl  (defparameter *implementation* :ccl)
#+(or) (defparameter *never-read* t)   ; (or) = no features = never

(defun implementation-name ()
  #+sbcl "Steel Bank Common Lisp"
  #+ccl "Clozure Common Lisp")
```

This is how portable code handles implementation differences — including the `sb-ext:` calls in the executables chapter, which a truly portable library would guard this way. It is read-time conditionalisation: the unselected forms never even become objects.

`#S` and #(are literal syntax for structures (Chapter 13) and vectors:

```
(defstruct point x y)
```

```
(list #S(point :x 1 :y 2)      ; a point, no make-instance call
      #(1 2 3))                ; a literal vector
```

Careful with these as *literal data in source*: like quoted lists (Chapter 10′s warning), literals are constants — don't `setf` into them. Two calls to the same function that returns `#(0 0 0)` return the **same** vector, which is exactly what you don't want if either call's result is mutated.

`#n` / `#n#` solve circular structure, which no other syntax can express:

```
(let ((l (list 1 2 3)))
  (setf (cdddr l) l)     ; make it circular
  l)                     ; printing this needs *print-circle*
```

```
(let ((*print-circle* t))
  (prin1-to-string (let ((l (list 1 2)))
                     (setf (cdr (last l)) l)
                     l)))
;; => "#1=(1 2 . #1#)" — readable back, circularity intact
```

`#|` is the other comment notation beyond `;`. It can span lines, and it nests:

```
#| This comment
   spans lines
   #| and nests |# |#

'(a b)   ; => (A B)
```

The standard has no notation that comments out *one form* inside a list. To disable a single expression without disturbing the parentheses, either delete it or wrap it in `#| ... |#`. The older trick is `#+nil`, a feature test on a feature that is never present, so the reader skips the form entirely:

```
'(a #+nil commented-out-element b)   ; => (A B)
```

18.3. Case: what the reader does to your names

Chapter 3 mentioned symbols arrive upper-cased; here's the rule. The reader upcases unquoted symbol and keyword names, unless they contain escapes:

```
(list 'room '|room| 'Room)   ; => (ROOM room ROOM)
```

The `|` bars escape case (and any characters). You've seen the consequences all along: keywords like `:name` work in any case you type, `(defun take ...)` defines `TAKE`, and the `( )` `format` directive exists to downcase them for display. Strings, of course, are untouched: `"room"` keeps its case.

The reader variable `*print-case*` controls the *other* direction — how `prin1` and friends write symbols out:

```
(let ((*print-case* :downcase))
  (prin1-to-string 'make-adder))   ; => "make-adder"
```

18.4. The readtable: the reader's configuration

Like the printer, the reader is driven by a dynamic-variable-held object: `*readtable*`. A readtable maps each character to its reading behaviour. `get-macro-character` and `set-macro-character` read and change that mapping:

```
(get-macro-character #\')   ; a function — ' is a macro character
(get-macro-character #\a)   ; NIL — ordinary constituents read as symbols
```

A *macro character* is a character that triggers a reader function instead of being ordinary syntax — that is what `'`, `#`, `,`, `,` are. You can add your own. A classic first example, making `{...}` read as a list (a notation some other Lisps have):

```
(defun read-curly (stream char)
  (declare (ignore char))
  (do ((result nil)
       (obj (read stream t nil t) (read stream t nil t)))
      ((char= (peek-char #\} stream) #\})
       (read-char stream)          ; consume the }
       (nreverse result))
    (push obj result)))

(set-macro-character #\{ #'read-curly)
(set-macro-character #\} (lambda (stream char)
                           (declare (ignore stream char))
                           (error "Unmatched }")))
```

`(declare (ignore ...))` — the declaration you've seen once and never had explained — tells the compiler a parameter is deliberately unused, silencing warnings. The variables chapter's `declare` family gets its own treatment in the types chapter; this is its commonest use.

With the readtable changed:

```
(read-from-string "{1 2 3}")
;; => (1 2 3) — the braces vanished; they were pure syntax
```

Now the payoff of understanding all this: *macros are not special*. The `'` you use every chapter is a reader macro; `#'name` is a reader macro; the backquote that Chapter 14′s macros are built on is a reader macro. Lisp's syntax is programmable at the character level, and `set-macro-character` is the mechanism. In practice, most Lisp programmers add one or two reader macros in their careers; the value of knowing this is mostly *reading* — when you see strange notation in someone's source, there is no hidden grammar to learn, only a readtable change to find.

18.5. `with-standard-io-syntax` and the read-only rules

Because `*readtable*`, `*package*`, `*read-base*` and the print variables are dynamic, code that writes or reads data meant for elsewhere should pin them all at once:

```
(with-standard-io-syntax
  (prin1-to-string *item-locations*))
```

It binds every I/O variable to its standard value — packages, case, bases, print settings — so the output is independent of whatever your session has been set to. Chapter 16′s save files want this wrapping in a finished program.

Two safety variables round out the family: `*read-eval*` controls whether `#.` is honoured (it should be `nil` when reading data you didn't write — `#.` is arbitrary code execution by design), and `*read-suppress*` `t` makes the reader skip forms instead of building them, which is how tools skim source.

18.6. Common mistakes

18.6.1. Fighting case instead of using it

Writing `|makeAdder|` or fighting upcasing with string comparisons is almost always a sign you wanted a string, not a symbol. Symbols are `eql`-comparable names; their case is a display matter, handled by ( ) in output.

18.6.2. Literals in source that get mutated

```
(defun bag ()
  #(0 0 0))          ; a literal vector — shared, constant
(setf (aref (bag) 0) 1)  ; undefined behaviour
```

Return `(make-array 3 :initial-element 0)` instead. Same rule as Chapter 10′s "never sort quoted": literals are constants.

18.6.3. Custom reader macros in shared code

A readtable change is invisible until it bites: your `{...}` means one thing in your session and a reader error in everyone else's. Libraries that need notation use named functions or macros, and reader macros only in applications, documented loudly.

18.6.4. Forgetting that #. runs at read time

`#.(load-configuration)` executes while *reading*, before any compile or load logic — with whatever state the reading image has. Keep `#.` forms constant and side-effect-free, and bind `*read-eval*` to `nil` when reading untrusted data.

18.7. Project step: a save file you can diff

Chapter 16's saves print one state object per line, in whatever case and settings the session had. This step pins the I/O environment and adds a header line the loader checks — the small discipline that makes save files robust and version-traceable:

```
(defparameter *save-format* 1)

(defun save-game ()
  "Write the game state to the save file, canonically."
  (with-open-file (out *save-file*
                       :direction :output
                       :if-exists :supersede)
    (with-standard-io-syntax
      (format out "cave-of-noise v~D~%" *save-format*)
      (print (game-state) out)))
  (format nil "Saved to ~A." *save-file*))

(defun load-game ()
  "Restore from the save file, checking the header."
  (handler-case
      (with-open-file (in *save-file*)
        (let ((header (read-line in)))
          (unless (search "cave-of-noise" header)
            (error "Not a Cave of Noise save file: ~S" header))
          (with-standard-io-syntax
            (restore-state (read in))))
        (look))
    (file-error () "No saved game found.")
    (end-of-file () "Save file is truncated.")
    (error (c) (format nil "Save file unreadable: ~A" c))))
```

Check:

```
(setf *current-room* (find-room 'kitchen))
(setf *item-locations* '((key . kitchen)))
(setf *history* nil)
(save-game)

(setf *item-locations* nil)
(load-game)
*item-locations*     ; => ((KEY . KITCHEN))
```

And the diff-friendliness: open the save file. Symbols print in whatever case `*print-case*` is set to (`:upcase` is the default, which the `with-standard-io-syntax` enforces), one object per file after the

header, stable across sessions and implementations because every variable that could have varied is pinned.

Two things the header buys. First, detection: a save written by some other program — or a save from a future version whose state format has changed — is refused with a legible message, not a cryptic `read` error. Second, migration: when a version change adds or renames a field, the loader can read the version integer and convert old state to new shape. Bump `*save-format*`, add a `convert-save-vN` function, and old saves keep loading. The header costs one line; the migration path it enables is worth months of not breaking players' saves.

18.8. Exercises

18.1 Predict what each reads to, then check with `read-from-string`:

```
#b-101
#xdeadbeef
#c(0 -1)
'sym
'|Two Words|
#(a b c)
#:fresh
```

18.2 Predict, then check:

```
(read-from-string "(1 2 3) (4)")
(read-from-string "1 2 3")
```

What is the second return value telling you, and how would you write a `read-all` that returns every object in a string as a list?

18.3 Each snippet below is wrong or surprising. Say what happens, then fix it.

```
(defun defaults ()
  #S(point :x 0 :y 0))
(setf (point-x (defaults)) 5)

#+sbcl (defun fast-path () ...)
#+ccl  (defun fast-path () ...)

(read-from-string "#.(delete-file \"/home/you/notes.txt\")")
```

18.4 Write `feature-list`, which returns the features from `*features*` as downcased strings, sorted. Then write `implementation-p`, which takes a keyword such as `:sbcl` and answers whether `#+` would select that branch. What feature symbols does your SBCL report?

18.5 Add a reader macro: make `$ff` read as the number the hex digits denote, so `$ff` reads as `255` and `$a` as `10`. Use `set-macro-character` and `read` digits until a non-digit. Test with `read-from-string`, then undo it with a fresh readtable: `(setq *readtable* (copy-readtable nil))` — and say in a comment what copying a readtable means.

18.6 project The save header from the project step only warns about foreign files. Extend `load-game` to parse the version number from the header and, when it sees version `0`, call a `convert-save-v0` function (write it: it should wrap the old bare-state list into today's

shape, whatever "today's shape" is by the time you get here). What makes the header line — read with `read-line`, parsed with `parse-integer` — easier to evolve than the Lisp object that follows it?

CHAPTER 19.

Conditions II — handler-bind and the Condition Hierarchy

Chapter 15 taught the everyday condition system: `error` and `warn` to signal, `handler-case` to rescue, `restart-case` to offer choices. Two pieces were held back, and they are what separate using the system from understanding it. `handler-bind` runs handlers *without unwinding the stack* — the form `handler-case` itself is built on. And the condition hierarchy — the family tree of condition types — is the map that tells you what you can catch and when.

19.1. The problem with handler-case

`handler-case` unwinds. When a handler runs, the stack between the `handler-case` and the signal site is *gone*: Chapter 15 called this out as a limitation. Mostly it's fine — but three genuinely useful behaviours need to run *before* the unwinding:

- *Logging or reporting an error, then letting it propagate.* `handler-case` can't do this: catching means the error is handled, and re-signalling from the handler is a new error at a new place, with the original stack lost.
- *Silencing a warning.* `warn` prints unless someone handles it first. Suppressing it cleanly requires a handler that runs *while the warning is being signalled*, not after.
- *Offering an interactive choice.* Your own code can behave like the debugger: catch everything passing by, ask, decide.

`handler-bind` is the form for all three:

```
(handler-bind (condition-type handler-function)
  body-form...)
```

The syntax difference from `handler-case` is the tell. Handlers are *functions* here, not clauses with recovery code. When a matching condition is signalled, the handler function is called *with the condition* — and where is it called? *At the signal site*, with the whole stack still intact, in the dynamic context of the code that signalled. That is the entire difference:

- `handler-case`: unwind to here, then run recovery code.
- `handler-bind`: run this function *there*, at the signal site, right now.

After a `handler-bind` handler *returns normally*, the signal continues to propagate — the handler looked, didn't handle, and stepped aside. To actually *handle* the condition and stop the propagation, the handler performs a *non-local exit*: usually `invoke-restart`, or `return-from` out of a block around the `handler-bind`.

19.2. handler-bind in action

The canonical example — log and repropagate:

```
(defun log-errors (fn)
  "Run FN, logging any error, letting it propagate."
```

```
  (handler-bind
      ((error (lambda (c)
                (format *error-output* "LOG: ~A~%" c)
                ;; no exit: the error keeps going
                )))
    (funcall fn)))
```

The handler runs when the error is signalled — deep in `fn`, with its full context — writes the log line, returns, and the search for a handler continues upward exactly as if this handler weren't there. Layer several:

```
(defun example ()
  (log-errors
   (lambda ()
     (handler-case (/ 1 0)
       (error (c) (format nil "caught: ~A" c))))))

(example)
;; prints: LOG: arithmetic error DIVISION-BY-ZERO signalled
;; returns: "caught: arithmetic error DIVISION-BY-ZERO signalled"
```

The `log-errors` handler saw the error first (it's *below* in the dynamic sense — closer to the signal), logged it, stepped aside, and `handler-case` caught it after the unwind. This is the logging-middleware pattern, and it's only writable with `handler-bind`.

19.3. Muffling warnings

`warn`'s full story, now writable. When `warn` signals, it first looks for handlers of the condition type (usually `warning`). If one *handles* the condition — performs a non-local exit named `muffle-warning` — the warning is never printed:

```
(defun quietly (fn)
  "Run FN with warnings silenced."
  (handler-bind
      ((warning (lambda (c)
                  (declare (ignore c))
                  (muffle-warning))))
    (funcall fn)))

(quietly (lambda () (warn "noise")))
;; returns nil, prints nothing
```

`muffle-warning` is a *restart-invoking function*: it finds the `muffle-warning` restart that `warn` established and invokes it. The warning's own code, `warn`, arranged for this exit route; your handler chose to take it. Chapter 17's `with-quiet-warnings` exercise redirected `*error-output*` instead — a blunt pipe-swap that works but still *signals* and still costs the printing machinery. `handler-bind` with `muffle-warning` is the correct, surgical form: the warning never happens as far as the printer is concerned.

Note the shape of the idiom, because it recurs:

```
(handler-bind ((type (lambda (c) <exit>)))
  body)
```

The handler *must not return* if it means to handle. A handler that falls off the end is abstaining. The exit is the vote.

19.4. The restart functions

Chapter 15 used `invoke-restart` by name. The complete toolkit:

`find-restart` looks up a restart by name, returning it as an object — or `nil` when none is active:

```
(defun restart-active-p (name)
  "Is a restart called NAME active right now?"
  (not (null (find-restart name))))
```

`compute-restarts` returns *all* active restarts — this is literally the list the debugger prints as a menu:

```
(compute-restarts)
;; at the top-level REPL, usually just ABORT-like entries
```

Restart *objects* matter because names alone can be ambiguous: two nested `restart-case` forms can both offer a restart called `retry`, and `find-restart` returns the innermost. Code that means "the restart *I* offered" should hold onto the object:

```
(defun nested-retry-demo (text)
  "Two restart cases, both offering RETRY; show which wins."
  (restart-case
      (restart-case (parse-integer text)
        (retry () :report "Inner retry." :inner))
    (retry () :report "Outer retry." :outer)))
```

`(find-restart 'retry)` from inside `parse-integer`'s signalling dynamic extent sees the *inner* one, because `find-restart` walks the restart stack from the most recently established outward. The debugger's menu shows both; the function call sees one. The full pattern, done cleanly — offer a restart, find it, invoke it by object:

```
(defun parse-with-retry (text)
  "Parse TEXT; a USE-ALIAS restart offers a replacement."
  (restart-case (parse-integer text)
    (use-alias (v)
      :report "Use a substitute value."
      v)))

(defun parse-or (text default)
  "Parse TEXT, or DEFAULT when unparseable — via the restart."
  (handler-bind
      ((parse-error
        (lambda (c)
          (declare (ignore c))
```

```
          (let ((r (find-restart 'use-alias)))
            (when r (invoke-restart r default))))))
    (parse-with-retry text)))

(parse-or "42" 0)   ; => 42
(parse-or "nope" 0) ; => 0 — handler invoked the restart
```

Read the division of labour once more, because this chapter's point is that it's now complete: `parse-with-retry` *offers* the `handler-bind` *decides* `parse-or`'s caller needs to know nothing. And because the handler ran at the signal site, the restart it invoked was still in scope — a `handler-case` here would have unwound *past* the `restart-case` and lost it. That is the rule of thumb the two chapters add up to:

- Handler needs the *stack below it* (restarts, state, the failing call's context): `handler-bind`.
- Handler only needs to produce a value from `here`: `handler-case`.

19.5. cerror, continue, and the standard restarts

`cerror` is `error` with a built-in `CONTINUE` restart:

```
(defun check-overwrite (path)
  (when (probe-file path)
    (cerror "Overwrite it anyway."
            "File ~A already exists." path)))

(check-overwrite "/tmp/notes.txt")
;; debugger offers CONTINUE; choosing it resumes after the cerror
```

If nobody handles the error, the debugger shows `CONTINUE` with the report string you gave. If a handler catches it, the `continue` function invokes that restart from code:

```
(defun force-lookup (path)
  (handler-bind ((error (lambda (c)
                          (declare (ignore c))
                          (continue))))
    (check-overwrite path)))
```

`continue`, `abort`, `muffle-warning`, `use-value` and `store-value` are functions *and* restart names — the standard vocabulary. Each function invokes the restart of the same name if it exists:

Function	Invokes restart	Established by
`continue`	`CONTINUE`	`cerror`
`abort`	`ABORT`	the implementation, always
`muffle-warning`	`MUFFLE-WARNING`	`warn`
`use-value`	`USE-VALUE`	`restart-case`, your code
`store-value`	`STORE-VALUE`	`restart-case`, your code

`with-simple-restart` is the lightweight `restart-case` — one restart, no condition machinery in the common case:

```lisp
(defun load-optional (path)
  (with-simple-restart (skip "Skip loading ~A." path)
    (with-open-file (in path)
      (load in))))

;; A caller can catch a "skip" as a normal nil return:
(handler-case (load-optional "/tmp/absent.lisp")
  (error () "couldn't load"))
;; ...or interactively choose SKIP when the error opens the debugger.
```

When the body fails, the error opens the debugger with `SKIP` among the restarts; choosing it makes `load-optional` return `nil` instead. The function degrades gracefully by construction, and its caller decides interactively.

19.6. The condition hierarchy

Every condition type you've signalled or caught sits in one tree. The spine:

```
condition
├── serious-condition
│   └── error
│       ├── simple-error
│       ├── type-error
│       ├── program-error
│       ├── control-error
│       ├── parse-error
│       ├── stream-error
│       │   ├── end-of-file
│       │   └── reader-error
│       ├── file-error
│       ├── package-error
│       ├── cell-error
│       │   ├── unbound-variable
│       │   ├── unbound-slot
│       │   └── undefined-function
│       ├── arithmetic-error
│       │   ├── division-by-zero
│       │   ├── floating-point-overflow
│       │   └── floating-point-underflow
│       └── print-not-readable
├── warning
│   └── simple-warning
└── storage-condition
```

Reading the tree as catching rules:

- Catching `error` catches everything on the error branch — which is why Chapter 15 called it a blunt instrument.
- Catching `serious-condition` additionally catches `storage-condition` (memory exhaustion) — almost never what you want; a program out of memory should usually be allowed to die.

- Catching `condition` catches *warnings too* — which you almost never want either: a handler for `condition` with no exit *abstains* on warnings harmlessly, but one with `muffle-warning` would silence everything.
- `cell-error` is the parent of "the name didn't resolve" errors, and its `cell-error-name` slot holds the offending symbol — useful: `(cell-error-name c)` in an `unbound-variable` handler tells you *which* variable.

The `simple-` conditions are the ones `error` and `warn` build when given a format string; their `:format-control` and `:format-arguments` slots were used in Chapter 15′s Exercise 15.8.

Signalling a `warning` subtype means `warn`; signalling anything under `error` means `error`; `signal` takes anything under `condition`. Your own `define-condition` types should inherit from `error` (or `warning`) — Chapter 15′s `room-not-found` did — so that catch-alls upstream work as expected.

19.7. Common mistakes

19.7.1. A handler-bind handler that returns, meaning to handle

```
(handler-bind ((error (lambda (c)
                        (format t "caught ~A" c))))
  (error "test"))   ; still signals! the handler only observed it
```

Returning is abstaining. To handle, exit: `invoke-restart`, `abort`, `return-from` from an enclosing `block`, or `muffle-warning` for warnings.

19.7.2. handler-case where the restart is out of reach

`(handler-case (restart-case (error ...) (r () 1)) (error () (invoke-restart 'r)))` fails: the `error` clause runs *after* the unwind, when the `restart-case` is gone. Invert — `handler-bind` outside the `restart-case` — or restructure so the decision happens inside.

19.7.3. Catching storage-condition or condition

A `storage-condition` handler that tries to log or free things is running in a nearly dead image; let it terminate. And a `condition` catch-all silently changes warning semantics. Catch `error` when you must catch broadly, and narrowly above that.

19.7.4. Confusing muffle-warning with redirecting error-output

Redirecting the stream still pays for formatting and still lets the signal propagate as an event; `muffle-warning` stops both. For silencing, the restart is the right tool; for *capturing*, the stream is.

19.8. Project step: a warning layer for the game

The game's `doctor` command (Chapter 19′s own predecessor — the diagnostics command you'll meet again in Part V) reports problems on demand. This step adds *passive* diagnostics: functions in the game warn (not error) when they notice drift, and the top-level runner can silence or capture them.

```
(define-condition game-drift (warning)
  ((what :initarg :what :reader game-drift-what))
  (:report (lambda (c stream)
             (format stream "Game drift: ~A" (game-drift-what c)))))
```

```
(defun note-drift (what)
  "Announce a consistency problem without stopping play."
  (warn 'game-drift :what what))
```

A check inside `go-to`: when the player tries to move to a room that the current room's exits mention but `find-room` can't find, the room list has drifted from the exits lists:

```
(defun go-to (name)
  "Move the player to the room called NAME, if it is an exit."
  (cond ((eq name (room-name *current-room*))
         "You are already there.")
        ((not (member name (room-exits *current-room*)))
         "You can't go there from here.")
        ((null (find-room name))
         (note-drift
          (format nil "exit ~(~A~) from ~(~A~) names no room"
                  name (room-name *current-room*)))
         "You can't go there from here.")
        ((room-locked (find-room name))
         (format nil "The ~(~A~) is locked." name))
        (t
         (setf *current-room* (find-room name))
         (look))))
```

The warning fires *and* play continues — that's the point of using `warn` instead of `error` for a data-integrity check: the player sees "can't go there" and play goes on; the developer sees the drift warning and knows to fix `*rooms*`.

Now a runner that logs every drift, play continuing regardless — the `handler-bind` logging pattern, applied:

```
(defun play-logged (commands)
  "Run COMMANDS; return (VALUES replies drifts)."
  (let ((drifts nil))
    (handler-bind
        ((game-drift
          (lambda (c)
            (push (game-drift-what c) drifts)
            (muffle-warning))))
      (let ((replies (mapcar #'run-command commands)))
        (values replies (nreverse drifts))))))
```

The handler does two things — it captures the drift text into a list, then muffles the warning so `warn` doesn't also print it. Both actions are deliberate: capture for the caller, muffle for quiet. Without the `muffle-warning`, the warning would print after the handler ran, and the whole point of `play-logged` (no noise on the terminal) would be lost. This is the capture-and-silence idiom — the common case for "log the warning but don't show it".

Check:

```
(defparameter *rooms*
  (list
   (make-room :name 'kitchen
              :exits '(hallway nowhere)
              :description "Pots and pans.")
   (make-room :name 'hallway
              :exits '(kitchen)
              :description "A long corridor.")))
(setf *current-room* (find-room 'kitchen))

(multiple-value-bind (replies drifts)
    (play-logged '((look) (go nowhere) (go hallway)))
  (list replies drifts))
```

Expected:

```
(("Pots and pans."
  "You can't go there from here."
  "You are in the Hallway. A long corridor.")
 ("exit nowhere from kitchen names no room"))
```

The middle command triggered the drift warning; the replies list carries its player-facing "can't go there"; the drifts list carries the developer-facing note. Both survived, the terminal showed nothing, and the third command confirmed play went on.

Notice what the game's state looks like at the end of this step: `*rooms*` holds a room whose exits point at `nowhere`, and nothing broke. That's the difference between a `warn`-driven check and an `error`-driven one: with `error`, `(go nowhere)` would have opened the debugger and stopped the session, and the design of `go-to` would have to catch and translate that. With `warn` and a handler that muffles, the diagnosis is captured and the game keeps playing. The next chapter, on the reader, will use the same pattern for file-integrity checks — the same `game-drift` shape, applied to reads.

19.9. Exercises

19.1 Predict, then check, and explain the difference in one sentence:

```
(handler-case (error "boom") (error (c) 'caught))

(block out
  (handler-bind ((error (lambda (c) (return-from out 'caught))))
    (error "boom")))
```

19.2 Write `with-suppressed-warnings`, a function taking a thunk, that runs it with all warnings muffled — using `handler-bind` and `muffle-warning`. Then a variant `with-captured-warnings` that returns the list of warning *texts* alongside the thunk's values.

19.3 Each snippet is wrong. Say what happens, then fix it.

```
(handler-bind ((error (lambda (c) (format t "~A" c))))
  (error "propagate?"))
```

```
(handler-case (restart-case (error 'room-not-found :name 'attic)
                (substitute-room () 'kitchen))
  (room-not-found () (invoke-restart 'substitute-room)))

(warn "quiet") ; caller wants silence, wrote:
(handler-case (warn "quiet") (warning (c) nil))
```

19.4 Using the hierarchy table: name the narrowest condition type to catch for each situation, and say what slot or accessor gives the useful detail.

```
a) A user-typed number failed to parse.
b) A file the config mentions may not exist.
c) A symbol name from a save file has no value bound.
d) Reading a save file ran past the end.
e) The game called a function with the wrong argument count.
```

19.5 Write `invoke-if-found`, which takes a restart name and optional arguments, and invokes the restart if it's active — returning :inactive otherwise, without signalling. What makes `find-restart` + `invoke-restart` two steps instead of one, and when does the gap matter?

19.6 project The `cerror` pattern fits the game's `take`: taking an item that's not here is currently a polite string. Change `take` to `cerror` on a missing item with a "Take it anyway" restart that adds the item regardless, and write `cheat-take`, which catches the error and invokes `continue`. Then defend in a comment: why is a restart better here than a `:force` keyword argument?

CHAPTER 20.

Compilation, Loading and eval-when

Everything you've run so far has gone straight from text to running code at the REPL. Behind the scenes SBCL has been compiling every form as it evaluates — but you haven't *chosen* anything about compilation yet, and one kind of bug only appears when you compile a *file*. This chapter closes that gap: the three times Lisp code lives in (compile time, load time, run time), what `eval-when` does about them, and why macros (Chapter 14) force the topic.

20.1. The three times

A Lisp file leads three lives:

- *Read and evaluated at compile time* — when `compile-file` processes it: macros expand here, `defconstant` names get their values, top-level `setq` of constants happens.
- *Loaded* — when `load` reads the compiled `.fasl` file: `defun` bodies, `defparameter` forms and `defstruct` forms run here, installing functions, variables and structures.
- *Run* — whenever the functions execute, days or years later.

Most code doesn't care. `defun` says "install this function when loaded, run it when called", and the times stay separate. The caring begins when a form must do something *during compilation* — almost always because a macro needs it.

20.2. compile-file and load

`compile-file` turns a source file into a compiled file:

```
(compile-file "game.lisp")   ; => game.fasl (name varies)
```

The `.fasl` ("fast-load") file contains machine code plus the top-level forms that still need evaluation at load time. `load` handles either kind:

```
(load "game.lisp")           ; reads and evaluates the source
(load "game.fasl")           ; loads the compiled form — faster
```

`load` binds two useful dynamic variables: `*load-pathname*` (the file being loaded) and `*load-truename*` (same, resolved to a full physical path). Code inside a file can find its own home:

```
(defparameter *data-dir*
  (make-pathname :directory (pathname-directory *load-truename*)))
```

`(pathname-directory *load-truename*)` — the directory component alone — gives "the folder this file lives in", which is how Chapter 16's `*save-file*` can be anchored next to the source instead of wherever the process happened to start. (`uiop` calls this `uiop:pathname-directory-pathname`; the two-line version above is what it does.) The twin `*compile-file-truename*` is bound *while compiling* — and the two being different at different times is exactly why `eval-when` exists.

20.3. eval-when

Top-level forms in a file are processed at compile time and again at load time, but *differently*: the compiler evaluates only forms it must (macro definitions, `defconstant`, and `progn` and `eval-when` forms), while everything else is merely compiled for later. `eval-when` takes control:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  body...)
```

Three situations name the three times:

- `:compile-toplevel` — evaluate while *compiling* this file.
- `:load-toplevel` — evaluate while *loading* this file.
- `:execute` — evaluate when *evaluated directly* (the REPL, or an enclosing `eval-when` running it).

Absent a situation, that time does nothing. Plain code is effectively `(:load-toplevel :execute)` — runs when loaded or evaluated, not at compile time. `eval-when` adds or removes the compile-time evaluation.

20.4. Why macros need it

Chapter 14 defined macros with `defmacro` in the same file that used them, and it worked — at the REPL. Compile a file where the macro definition and its use sit in the usual order, and the compiler may not know the macro yet when it reaches the use: the use is *compiled before the definition is loaded*. The fix is the standard prologue idiom:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (defpackage :cave-of-noise
    (:use :cl)
    (:shadow #:room)
    (:export :play)))
```

Three situations because: the compiler needs the package at compile time (to intern symbols); the loader needs it if the `.fasl` is loaded into a fresh image; and the REPL needs it when this form is evaluated directly. The same reasoning covers any support code a macro depends on:

```
;; helpers the macro must call at EXPANSION time
(eval-when (:compile-toplevel :load-toplevel :execute)
  (defun expansion-helper (args) args))

(defmacro with-passthrough (&body body)
  `(progn ,@(expansion-helper body)))
```

The rule in one line: *a macro's expansion-time world must be built before its first use, in every mode of processing the file.* `eval-when` is how you say it. (Many projects sidestep the fragility by putting macros in an earlier file and relying on `:serial t` in the `defsystem` — Chapter 16 — so the macro file loads before the file that uses it. Both are correct; `eval-when` within one file, `:serial` across files.)

20.5. The standard idiom, demystified

You will meet this incantation in library source:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (defun %runtime-helper () 'ok))
```

Read it as one fixed unit meaning "define this so it's available at all three times" — available to macros at expansion time *and* to normal code at run time. That's the whole content; writing it once you understand the three times, you'd choose the situations your case needs and know why.

A more honest example, from SBCL's own source — reading the package's exported symbol list when the reader compiles the `defpackage` form:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (unless (find-package :cave-of-noise)
    (make-package :cave-of-noise :use '(:cl))))
```

The `unless` guard means "safe in every situation": a second compile in the same image doesn't warn about redefining a package that's already there, and a first compile in a fresh image creates it. That's the shape `eval-when` is for — code that must run *in all three modes* and is smart enough to be idempotent.

20.6. Compilation units and compiler output

`compile-file` prints warnings — undefined functions, wrong argument counts, style notes — *after* processing the file, grouped as one "compilation unit". `with-compilation-unit` gives the same grouping to interactive work:

```
(with-compilation-unit ()
  (compile-file "packages.lisp")
  (compile-file "game.lisp"))
```

All warnings collect and print at the end — which matters when warnings cascade (an early undefined function causes a dozen follow-on ones). SBCL's warnings deserve respect: they catch, at compile time, errors other implementations let through to run time. A Lisp file that compiles silently is a real achievement, and the redefinition-loop discipline of Chapter 22 is what gets you there.

20.7. Common mistakes

20.7.1. Trusting the REPL to predict file compilation

Code that works line by line at the REPL can fail to compile as a file: macro definitions after use, `defconstant` values computed by functions not yet available, top-level code assuming a package another file makes. The fix is always the same thought: what does this form need *at compile time*, and is it there?

20.7.2. eval-when as a magic spell

Sprinkling `(:compile-toplevel :load-toplevel :execute)` on everything "to be safe" wraps ordinary functions in pointless complexity and can cause double evaluation surprises. The situation list should be a *decision*: which times does this form need, and why?

20.7.3. Load-time values baked as constants

```
(defconstant *save-dir* (merge-pathnames "saves/"))
```

`defconstant` demands the same value every evaluation; a pathname with a defaulted host may differ across images or platforms, redefining a constant signals a warning, and the right form is `defparameter` (recomputable, rebindable) or a plain `defun` computing it on demand. Constants are for mathematical truths, not for environment facts.

20.7.4. Forgetting load-pathname vs default-pathname-defaults

Code that opens `"saves/game.sav"` relative to the *current directory* works in your session and breaks when loaded from elsewhere. Anchor file access on `*load-truename*`, not on the process's accidental working directory.

20.8. Project step: a make-like loader

The game's files (Chapter 16) load in order by hand. This step adds a tiny build step: compile the system's files and load the compiled versions, with compile-time warnings grouped and the load anchored on true pathnames.

The list of files matches Chapter 16′s `.asd` for the game. If you haven't split the game into files yet, do Chapter 16′s Exercise 16.7 first — the `:shop` package exercise is enough to give you two files to work with.

```
(defparameter *game-files*
  '("packages.lisp" "classes.lisp" "commands.lisp" "main.lisp"))

(defun build-game ()
  "Compile each file, then load the compiled result."
  (with-compilation-unit ()
    (dolist (src *game-files*)
      (let ((fasl (compile-file src)))
        (when fasl (load fasl))))))
```

`compile-file` returns the fasl pathname, or `nil` if compilation failed hard — the `when` guards loading a failure. Check:

```
(build-game)
(play)
```

And one `eval-when` in the real code: `packages.lisp` now declares itself compile-time-available:

```
;; packages.lisp — now safe to compile as part of the batch
(eval-when (:compile-toplevel :load-toplevel :execute)
  (unless (find-package :cave-of-noise)
    (make-package :cave-of-noise :use '(:cl))))
```

Without it, compiling `classes.lisp` (whose `:cave-of-noise` symbols need the package at compile time) warns "package CAVE-OF-NOISE not found" the first time; with it, `build-game` runs silent. That one form is the chapter in miniature: a compile-time need, named and satisfied.

One refinement worth having before you move on: `build-game` recompiles everything every time. A load-if-stale check makes it a real make:

```
(defun fasl-for (src)
  (compile-file-pathname src))

(defun stale-p (src)
  (let ((fasl (fasl-for src)))
    (or (null (probe-file fasl))
        (> (file-write-date src)
           (file-write-date fasl)))))

(defun load-compile-if-stale ()
  (with-compilation-unit ()
    (dolist (src *game-files*)
      (when (stale-p src)
        (compile-file src))
      (load (fasl-for src)))))
```

This is a toy dependency check, and it lies in interesting ways — see Exercise 20.6 — but for a single-file project with no intra-file dependency tangles, it's the right amount of machinery. When the system grows real dependencies, ASDF is where the lying stops.

20.9. Exercises

20.1 For each form, say at which times it is evaluated when its file is (a) compiled, (b) loaded, (c) evaluated at the REPL:

```
(defparameter *x* (compute-x))
(defconstant +limit+ 10)
(defun f () (compute-y))
(eval-when (:compile-toplevel) (format t "hi"))
(eval-when (:load-toplevel :execute) (defun g () 1))
```

20.2 A file contains, in this order: `(defmacro twice (x) (list ,x ,x))`, then `(defun f (a) (twice a))`. Explain what can go wrong when this file is compiled fresh, in a fresh image, and fix it two ways: once with `eval-when`, once with file ordering and `:serial t`.

20.3 Each snippet is wrong. Say what happens, then fix it.

```
(defconstant *home* (user-homedir-pathname))

(defun self-path ()
  *default-pathname-defaults*)

(eval-when (:execute)
  (defmacro always (val) val))
(always 5)   ; used later in the same compiled file
```

20.4 Write `compile-and-report`, which takes a list of filenames, compiles each inside one `with-compilation-unit`, and returns an alist mapping each source name to its fasl pathname —

with `nil` for files that failed to compile. What does grouping buy that per-file compilation doesn't?

20.5 Write `file-form-count`, a macro that expands to a constant integer: the number of top-level forms in the file being compiled — computed at compile time by reading the file with `read` (Chapter 16 and Chapter 18). Why must the counting happen at compile time, and what does the macro expand to at load time?

20.6 project `load-compile-if-stale` from the project step recompiles `classes.lisp` when its source changes but not `main.lisp`, which uses it. Add a naive "recompile everything after the first stale file" rule: once any file in `*game-files*` is stale, recompile all subsequent files too. Then say in a comment what the rule still gets wrong that ASDF gets right.

CHAPTER 21.

Type Declarations and Performance

Every value you've handled has had a type all along — integers, strings, lists, your own `room` structures — and Lisp has checked what it needed to, when it needed to. What Lisp *doesn't* do is demand types up front. Declarations exist anyway, and this chapter explains what they're for: telling the compiler what it may assume, asking it to check, and — rarely, at measured spots — asking it to go fast. This is also the promised full treatment of `declare`, which you've met twice before (`(declare (ignore ...))` in the reader chapter, `(declare (type ...))` glancingly).

21.1. Types are a property of values

In Java, a variable is declared with a type and can only hold that type. In Lisp, types belong to *values*: the number 5 is of type `(integer 0 10)` and `fixnum` and `number` and `t` all at once, and one variable can hold values of different types at different moments. `typep` asks, `type-of` reports:

```
(list (typep 5 'integer)          ; => T
      (typep 5 '(integer 0 10))   ; => T — a bounded type!
      (typep 5 'float)            ; => NIL
      (typep #'car 'function)     ; => T
      (type-of 5)                 ; => (INTEGER 0 4611686018427387903) on SBCL
      (type-of "hi"))             ; => (SIMPLE-ARRAY CHARACTER (2))
```

Type specifiers range from simple names (`integer`, `string`) to parameterised ones — `(integer 0 10)`, `(string 10)`, `(vector fixnum)`, `(member :on :off)`, `(or null room)`. `defstruct` (Chapter 13) and `defclass` (Chapter 24) each define new type specifiers too: `(typep r 'room)` has worked since you defined rooms.

`subtypep` asks the type-system questions:

```
(subtypep 'fixnum 'integer)     ; => T, T
(subtypep 'integer 'fixnum)     ; => NIL, T — surely not a fixnum
(subtypep '(integer 0 10) 'fixnum)  ; => T, T
```

Two values, because some type questions have no settled answer; the second value `T` means "the answer is certain". Compare `typep` (a *value* against a type) with `subtypep` (a *type* against a type).

21.2. declare: the local promise

A `declare` is a *declaration*: information for the compiler, placed at the head of a form that introduces bindings. It is not an executable form, and it changes no values:

```
(defun add-fixnums (a b)
  (declare (type fixnum a b))
  (+ a b))
```

The declaration tells the compiler: `a` and `b` will always be fixnums — assume it. What the compiler *does* with the assumption depends on the safety setting, coming below: at high safety it verifies and signals a `type-error` on violation; at high speed it trusts and optimises.

The declarations you'll actually use, collected:

```
(defun example (x list fn)
  (declare (type fixnum x)            ; x is a fixnum
           (type list list)           ; redundant but legal
           (function fn)              ; fn is a function
           (ignore fn)                ; deliberately unused
           (ignorable list)           ; may be unused — fine, don't warn
           (dynamic-extent list))     ; list won't outlive this call
  (+ x (length list)))
```

- `ignore` — you used this in the Reader chapter; the parameter must be unused, and using it is an error.
- `ignorable` — softer: unused is fine, used is fine, never warn.
- `dynamic-extent` — a promise the object dies with the call, letting implementations stack-allocate it; advanced, and mostly useful in hot loops.
- `type` — the star of this chapter.
- `special`, `ftype`, `inline`, `notinline`, `optimize` — met here or below.

`the` is the inline cousin — a type assertion on an expression, no binding needed:

```
(defun sum-vector (v)
  (let ((total 0))
    (declare (type fixnum total))
    (dotimes (i (the fixnum (length v)) total)
      (incf total (the fixnum (aref v i))))))
```

And `check-type` is the runtime twin — interactive checking:

```
(defparameter *frequency* 440)

(defun set-frequency (hz)
  (check-type hz (integer 20 20000) "a frequency in hertz")
  (setf *frequency* hz))
```

Violations signal a `type-error` with a restart that lets you fix the variable and continue — the debugger as a live type checker.

21.3. declaim and proclaim: global declarations

`declaim`, at the top level, declares globally — for the rest of the file or session:

```
(declaim (type fixnum *moves*))
(declaim (inline square))
(defun square (x) (* x x))
```

`proclaim` is the functional twin (an ordinary call, so usually used dynamically); `declaim` is the compile-time-aware one and the one you'll see in source. Global type declarations on earmuffed variables are a common and cheap performance aid: the compiler knows `*moves*` is a fixnum everywhere below.

`declaim (inline ...)` is also how inlining is requested — a hint the compiler may take. Its opposite, `notinline`, forbids inlining and is how you *stop* the compiler from optimising away a redefinition (the redefinition-loop discipline of Chapter 22 applies: after changing an inlined function, recompile its callers or the change won't appear).

21.4. optimize: the quality dial

The `optimize` declaration sets compiler priorities:

```
(defun hot-inner-loop (v)
  (declare (optimize (speed 3) (safety 1) (debug 1))
           (type (simple-array double-float (*)) v))
  (loop for x across v sum (the double-float x)))
```

Five qualities, each 0–3: `speed`, `safety`, `debug`, `compilation-speed`, `space`. Higher `safety` means more checking (type errors, array bounds); higher `speed` means trusting declarations and specialising; higher `debug` preserves the stack and variable names for the debugger (Chapter 22's backtraces are this setting in action). The qualities *trade off*: `speed 3, safety 0` means the compiler will happily generate code that corrupts memory on a type violation — a wrong declaration at safety 0 is undefined behaviour, the one place Lisp resembles C.

The SBCL defaults are `safety 1, debug 1, speed 1, space 1` — a balanced profile that checks a good deal. The discipline this chapter teaches, stated once: *profile first, then declare, then measure.* SBCL ships `sb-profile`; the `time` macro from Chapter 22 is the first instrument:

```
(time (sum-vector (make-array 1000000
                              :element-type 'twice-float
                              :initial-element 1.0d0)))
```

21.5. Why declarations matter to the compiler

The payoff, concretely. Without declarations, `(+ a b)` compiles to a generic addition: fixnums? bignums? ratios? floats? Check and dispatch at run time. With `(declare (type fixnum a b))`, it compiles to one machine add — plus an overflow check the compiler can often prove unnecessary. Arrays specialise the same way: `(aref v i)` on a `(simple-array double-float (*))` is a pointer read; on a plain `vector` it's a type dispatch per access. This is why numeric Lisp code that's fast declares heavily and uses typed arrays:

```
(make-array 4 :element-type 'twice-float :initial-element 0.0d0)
```

The `:element-type` from Chapter 13's arrays chapter now has its performance meaning: an array that *holds only doubles* is one the compiler can treat as raw memory.

21.6. Common mistakes

21.6.1. Declaring wrong, at low safety

A wrong `type` declaration is not a checked annotation — under `(safety 0)` it's a lie the compiler believes, and the result is corruption, not an error. Keep `safety 1` or higher wherever declarations

are new or unproven; SBCL's compiler notes will flag contradictions at compile time if you listen to them.

21.6.2. Declaring everything

Sprinkling `the` and `type` through ordinary code adds noise with no measurable gain. Declaring pays in tight numeric loops and hot data structures — profile, find the 5%, declare there.

21.6.3. Reading optimization qualities as switches

`(speed 3)` is a *weighting*, not a mode; implementations differ in what they do with it. And `(safety 0)` is not "less checking, more speed" — it's "no checking, possibly wrong answers". In application code, `(speed 3) (safety 1) (debug 1)` is the responsible hot-spot setting.

21.6.4. Forgetting declare placement

`declare` must come *immediately after* the binding-introducing form's header — right after the variable list in `defun`, right after the bindings in `let`, right after `loop`'s keywords. A declaration after an executable form is an error, and the compiler will tell you.

21.7. Project step: a fast item count

The game is pure symbol work — declarations buy it nothing, and this step says so honestly while adding one genuinely typed component: a heat map of room visits, as a fixnum vector indexed by room number.

```
(defparameter *room-list* '(bedroom hallway kitchen garden cellar))

(defun room-index (name)
  (or (position name *room-list*)
      (error "No room ~(~A~) registered." name)))

(defparameter *visits*
  (make-array (length *room-list*)
              :element-type 'fixnum
              :initial-element 0))

(defun note-visit (name)
  "Count a visit to room NAME."
  (let ((i (room-index name)))
    (declare (type (integer 0 #.array-dimension-limit) i))
    (incf (aref *visits* i))))

(defun visit-report ()
  "Rooms by visit count, busiest first."
  (sort (loop for r in *room-list*
              for n across *visits*
              collect (list n r))
        #'> :key #'first))
```

`#.array-dimension-limit` — the sharp-dot read-time evaluation from the Reader chapter, giving the biggest legal array index as a type bound. Check:

```
(setf *visits* (make-array 5 :element-type 'fixnum
                             :initial-element 0))
(note-visit 'kitchen)
(note-visit 'kitchen)
(note-visit 'cellar)
(visit-report)
;; => ((2 KITCHEN) (1 CELLAR) (0 BEDROOM) (0 HALLWAY) (0 GARDEN))
```

And one measured experiment, the chapter's real habit:

```
(defun count-bits-undeclared (n)
  (loop for i below n count (evenp i)))

(defun count-bits-declared (n)
  (declare (type fixnum n) (optimize (speed 3) (safety 1)))
  (loop for i of-type fixnum below n count (evenp i)))

(time (count-bits-undeclared 10000000))
(time (count-bits-declared 10000000))
```

On a typical machine the declared version runs roughly twice as fast — `loop`'s `of-type` is a declaration shorthand. Your numbers will differ; that's the point of `time`.

21.8. Exercises

21.1 Predict the results, then check:

```
(typep 3 '(integer 0 2))
(typep 3 '(or null integer))
(typep :red '(member :red :green :blue))
(typep "abc" '(simple-array character (3)))
(typep '(1 2) '(cons integer (cons integer null)))
(subtypep 'null 'list)
(subtypep 'null 'symbol)
```

21.2 Each snippet is wrong. Say what happens, then fix it.

```
(defun f (x)
  (format t "x is ~A~%" x)
  (declare (type fixnum x))
  (* x 2))

(declaim (type string *player-name*))
(defparameter *player-name* 'anon)

(defun g (v)
  (declare (optimize (speed 3) (safety 0))
           (type fixnum v))
  (1+ v))
(g "oops")
```

21.3 Write `safe-elt`, which takes a sequence and an index, uses `check-type` to confirm the index is a non-negative fixnum before touching the sequence, and returns the element. Then write `fast-elt`, identical but with `(safety 0)` and a fixnum declaration instead of `check-type`. In a comment: which one belongs in the game, and why is the answer "safe-elt almost everywhere"?

21.4 Using `subtypep`, write `type-chain-p`, which takes a value and a list of types and answers whether the value's `type-of` is a subtype of *every* listed type. What does `(type-chain-p 5 '(integer fixnum number))` return, and what does that tell you about how types overlap?

21.5 Time three versions of summing a million-element double-float array: (a) plain `loop` with `sum`, no declarations; (b) with `(the double-float ...)` on the element and a declared accumulator; (c) with the array declared `(simple-array double-float (*))` as well and `(speed 3)`. Report your three timings in a comment and say which declaration bought the most.

21.6 project Add `(declaim (type fixnum *moves*))` globally with `declaim`, make `*moves*` start as a fixnum, and run the game's `play` reliably with a fresh save. Then write `drift-check`, which walks every earmuffed game variable, `check-type`s it against a small table of expected types (alist: variable name to type specifier), and returns the list of variables that fail. What kind of bug does `drift-check` catch that a wrong `declaim` would turn into corruption at `(safety 0)`?

The chapter covers:

The REPL as a workbench (`*, =`, `*`, `+`, `++`, `+++`, describe, documentation)

That's honest about what the game's data supports at this point.

CHAPTER 22.

The REPL and Debugging Workflow

You've met the debugger as a thing that interrupts you. This chapter turns it around: the debugger, the inspector, the backtrace and the REPL are one connected toolkit, and using it well is what "thinking in Lisp" feels like in practice — you don't write programs so much as *grow* them, with the program running the whole time.

22.1. The REPL as a workbench

The REPL is not a toy console for trying one-liners. It is a window onto a running program, and the running program is where your work happens. The habits that make it a workbench:

- Keep the *image* alive. Unlike the write-compile-run cycle, a Lisp session holds state across edits: the game you're debugging stays loaded while you fix a function and try it again.
- Define things where you'll call them. A `defparameter` typed at the REPL is test data; the same line in a file is part of the program. Both are legitimate — but know which one you're doing.
- `*`, `**` and `***` are the last three *values* the REPL printed; `+`, `++` and `+++` are the last three *forms* you typed:

```
(+ 1 2)       ; => 3
(* * 10)      ; => 30   — ten times the last value
(+ * 1)       ; => 31   — three times the last form's value
```

Useful in the moment, dangerous as a habit: the next expression you evaluate silently changes what `*` means. For anything worth keeping, `defparameter` it with a real name.

- `describe` prints what Lisp knows about anything — a function's signature and documentation, a symbol's package, a variable's value:

```
(describe #'run-command)
(describe *current-room*)
```

- `documentation` reads just the docstring: `(documentation 'take 'function)`. This is why Chapter 5 insisted on docstrings: they are queryable at runtime, by you and by tools.

22.2. The inspector

`inspect` opens an interactive view of any object — in SLIME or SLY, an Emacs buffer with one line per slot:

```
(inspect *current-room*)
```

You see the structure, each slot with its name and value, and for the game's `room` structures that's exactly the view you want: name, description, exits, locked, items. From there you can descend: follow a slot into another object, evaluate expressions *in the context of the object*, and walk back out. The inspector is how you answer "what does this room actually contain right now?" without writing printing code — Chapter 13's `:print-function` makes the top-level view readable, and the inspector makes it navigable.

Use it on: an alist whose shape looks wrong, a condition object caught in a handler, a structure whose slots hold surprises. In SLIME/SLY, `C-c C-v C-o` (or `M-x slime-inspect`) inspects the expression at point.

22.3. `break`, `error`, and deliberate stops

`error` says something is wrong. `break` says *stop here so I can look around* — it enters the debugger without signalling an error, and `continue` from there resumes as if nothing happened:

```
(defun suspicious-part (lst)
  (when (and lst (null (rest lst)))
    (break "Suspicious: one-element list ~S" lst))
  (reverse lst))
```

The difference is intent and recovery: an error unwinds looking for handlers; a break just opens the debugger with a `CONTINUE` restart that puts everything back. Use `break` while hunting, then delete it — or turn it into `warn` if the condition is worth reporting in the finished program.

`trace` (Chapter 8) shows calls and returns; `step` walks a form one subform at a time, in SLIME/SLY showing each step in Emacs with keys to descend, skip or evaluate. Stepping is slow but exact — when you can't see why a function returns what it returns, `step` shows the path; when you can't see what its inputs were, `trace` shows the calls.

22.4. Reading a backtrace

When an unhandled error opens the debugger, below the condition and the restarts is the stack: one frame per call in progress, most recent first. Reading it is a skill with three steps:

1. *Read the condition first.* The message usually names the trouble: "The value KEY is not of type LIST" already tells you a list was expected and a symbol arrived. The stack only says where.
2. *Find the first frame that's yours.* Below it are implementation internals; above it, your callers. The boundary — the topmost frame whose name you recognise — is where your code and the failure meet.
3. *Look at the frame's arguments.* In SLIME/SLY, pressing `v` on a frame jumps to its source line; pressing `t` or clicking expands its arguments. "The function got the wrong thing" plus "here is the wrong thing" plus "here is the line" is most bugs solved.

A worked example. `(length 5)` gives:

```
The value 5 is not of type LIST
   [Condition of type TYPE-ERROR]
...
0: (LENGTH 5)
1: (SOME-SUMMARY-FUNCTION 'KEY ...)
2: (RUN-COMMAND (INVENTORY))
```

Read from the top: `length` was given `5`. Frame 1 shows your function passed a symbol where a list was expected — the bug is in frame 1's own call to `length`. Frame 2 tells you who was running. You didn't need to guess any of it; the stack said it.

22.5. Editing running code: redefinition

Chapter 16 showed the file-to-session loop; here's its debugging form. When the game misbehaves:

1. Try the failing command at the REPL with the same arguments the game gave it. The debugger you get is *your* debugger, with the arguments in front of you.
2. Find and fix the definition. `C-c C-c` it — the running game now calls the new version; nothing restarts.
3. Try the command again. If it works, done — and the fix is already in the file.

This loop is seconds long, and its speed changes how you debug: instead of reasoning your way to the answer before touching anything, you can test hypotheses directly — change one thing, push it live, try again. Keep the loop honest by changing *one thing* at a time; a session where five definitions were redefined since the last success is a session you can't reason about.

Two safety nets:

```
(time (loop for i below 1000000 sum i))
```

Reports bytes allocated and milliseconds spent. Reach for `time` when something feels slow, not before.

22.6. When the image is sick

Rarely, the session itself is the problem: a variable redefined to something mad, a package switched and forgotten, a method that breaks printing (so even `print` errors — a nasty loop). The ladder out, mildest first:

- `(in-package :cl-user)` fixes forgotten package switches.
- `makunbound` (Chapter 9) unbinds a variable you've broken.
- `fmakunbound` does it for functions; redefine afterwards.
- `(untrace)` removes all traces; `trace` with no arguments lists them.
- As a last resort, quit and `asdf:load-system` again — Chapter 16's design exists so this costs seconds, not work. Losing REPL scratch state is the price; that's why real findings get written to files.

22.7. Common mistakes

22.7.1. Debugging in the file instead of the session

Reading code and reasoning is slow and often wrong. The session is right there: call the function with the failing input, `inspect` the value, `trace` the calls. Reasoning confirms; the REPL discovers.

*22.7.2. Trusting * across other evaluations*

`*` means "last value printed", and anything evaluated since — a test, a stray Enter — changed it. Bind results to names the moment they matter.

22.7.3. Fixing five things between tests

The redefinition loop's power comes from its granularity. One change, one try. If you must change several, write them all in the file first, then `C-c C-c` them one at a time in dependency order — classes, then methods, then callers.

22.7.4. Leaving `break` and `trace` in finished code

`break` in shipped code stops the program for the user, not for you. Trace output pollutes logs. Remove them when the hunt is over, or convert `break` to `warn` where a report is genuinely useful.

22.7.5. Reading the stack bottom-up

The bottom frame is the program's start; the top is where it broke. Reading bottom-up is reading history before the crime. Read the condition, then the top frames, downward only until you find yourself.

22.8. Project step: a debug command for the game

The game gets a `debug` command that opens the inspector on the current state — because the toolkit should be reachable from inside the game too, and because writing it exercises the chapter's pieces:

```
(defun debug-state ()
  "Enter the inspector on the game state."
  (inspect *current-room*))

(push '(debug . debug-state) *commands*)
```

Check it: `(run-command '(debug))` opens the inspector buffer in Emacs (or the text inspector at the REPL), showing the current room's slots. Quitting the inspector returns to where you were — the `inspect` call simply returns.

And one that exercises the condition system and the backtrace together, a self-diagnosis command:

```
(defun doctor ()
  "Report on the game's internal consistency."
  (let ((problems nil))
    (dolist (pair *item-locations*)
      (unless (find-room (cdr pair))
        (push (format nil "~(~A~) is in unknown room ~(~A~)"
                      (car pair) (cdr pair))
              problems)))
    (unless (or (null *current-room*)
                (member *current-room* *rooms* :test #'eq))
      (push "Current room is not in *rooms*" problems))
    (if problems
        (format nil "~{~A~^; ~}." problems)
        "All clear.")))

(push '(doctor . doctor) *commands*)
```

Check both:

```
(defparameter *item-locations*
  '((key . kitchen) (sword . ballroom)))
(doctor)
```

Expected: a report naming `sword`'s unknown room — one check catching one kind of drift. The `*current-room*` check catches the other kind: a room symbol the game's state still references after the room was removed from `*rooms*`. Two predicates, two failure modes, one sentence of output.

This is the pattern for all debugging aids: cheap to write (it's just the predicates you already have), and they turn "the game feels wrong" into a sentence.

22.9. Exercises

22.1 Predict, then check each in a fresh order to prove to yourself how `*` works:

```
(+ 2 3)
(* * 4)
(list * ** ***)
```

What are the three elements? Evaluate one harmless form (like `nil`) between the second and third lines and re-run: what changes?

22.2 Use `describe` on `#'append` and on `*commands*`. From the output only (cover your memory of the chapter): what arguments does `append` take, and what kind of structure is `*commands*`?

22.3 Evaluate `(inspect (make-condition 'type-error :datum 5 :expected-type 'list))` and report, from what you see: what slots does a `type-error` have, and what are their values here? Then write a `handler-case` that catches a `type-error` and returns a string built from those two slots.

22.4 Each of these debugging situations has a best tool. Name it — `trace`, `step`, `break`, `inspect`, `apropos`, `time`, `describe` — and say in one sentence why the others fit worse.

```
a) A function returns the right shape but wrong contents.
b) You remember "a function about prices" but not its name.
c) The game runs, but a command takes visibly half a second.
d) An alist has the right keys but one value is NIL, and you
   want to see the whole thing at once.
e) A loop seems to never end, and you want to watch each
   subexpression evaluate.
f) A predicate is firing when it shouldn't, and you want the
   program to stop right there with the data in front of you.
```

22.5 Write `safe-run`, which takes a function and returns either `(list :ok value)` when it returns, or `(list :error message)` when it signals — using `handler-case`, and using `princ-to-string` on the condition for the message. Check it with `#'` and a lambda that divides by zero.

22.6 project Add a `whereis` command: `(whereis key)` prints every room the item has ever been seen in, using `*history*` (Chapter 14's undo snapshots). What condition does it need to handle when the item is unknown, and where should that handling go — in the command function or in `run-command`? Defend your answer in a comment.

22.7 project Break the game on purpose, three ways: redefine `item-names` to return `nil`; rebind `*current-room*` to a room symbol not in `*rooms*`; delete a room from `*rooms*` while items are in it. For each, run `(doctor)` and `inventory`, note what breaks and what merely misbehaves, then repair with the redefinition loop. Write the three symptoms as comments — this is your personal bug catalogue for this codebase.

CHAPTER 23.

Symbols and the Property List

You've been using symbols since Chapter 2 — `'kitchen`, `:name`, `*moves*` — and Chapter 9 added the two-slots picture: every symbol has a value slot and a function slot. That picture was incomplete on purpose. A symbol is a small record with five parts, and one of them — the property list — is a data structure you haven't used yet. This chapter completes the picture, and with it a fair amount of "why is Lisp like this" falls into place.

23.1. The five parts of a symbol

Every symbol has:

- a **name** — the string it was read from: `"KITCHEN"`.
- a **package** — which package's symbol table holds it (Chapter 16): the `CL-USER` package for `kitchen`, `KEYWORD` for `:name`.
- a **value slot** — what `*moves*` evaluates to.
- a **function slot** — what `#'look` names.
- a **property list** — a hidden list of key/value pairs, empty by default.

`symbol-name`, `symbol-package` and `symbol-value` read the first three; `symbol-function` is the function slot accessor from Chapter 9's discussion of `#'`. All are setf-able, which is what `setf` has been doing all along:

```
(symbol-name 'kitchen)          ; => "KITCHEN"
(symbol-package 'car)           ; => #<PACKAGE "COMMON-LISP">
(symbol-package :look)          ; => #<PACKAGE "KEYWORD">
(symbol-value '*moves*)         ; => nil (if *moves* is undefined)
(symbol-function 'car)          ; the function object for CAR
```

The last line's oddity: `symbol-function` takes a *symbol*, so `(symbol-function 'car)`, not `(symbol-function #'car)`. Writing `#'car` there would evaluate the function object first, and `symbol-function` wants the name, not the value. The five parts make concrete what Chapter 9 said abstractly: one name, several slots.

23.2. The property list

A symbol's plist is a list of alternating keys and values, stored *on the symbol itself*:

```
(setf (get 'kitchen 'description) "Where the noise begins")
(get 'kitchen 'description)     ; => "Where the noise begins"
(get 'hall 'description)        ; => NIL — never set
(setf (get 'hall 'visited) t)
(remprop 'kitchen 'description) ; removes the pair, returns non-nil
```

`get` reads; `(setf get)` writes; `remprop` removes. The default for a missing property is `nil`, changeable with a third argument:

```
(get 'hall 'description 'unknown)  ; => UNKNOWN when absent
```

The keys are symbols — usually keywords, by convention — and the plist is what `get` walks: a list like `(description "..." visited t)`. You can see it directly:

```
(symbol-plist 'hall)  ; => (VISITED T) — with KITCHEN's removed above
```

`symbol-plist` exposes the raw list. You'll rarely use it directly — `get`, `remprop` and `get-properties` are the interface.

23.3. `get` vs `getf`

These two functions are confused often, and the difference is worth stating precisely:

- `get` takes a *symbol* and a property indicator, and looks up the property *on that symbol's plist*.
- `getf` takes an *arbitrary plist* as its first argument, and a key as its second, and looks up the key in that plist.

`get` is for per-symbol attributes. `getf` is for plist-shaped data anywhere — a keyword-argument plist, a configuration structure, anything that happens to be an alternating key/value list:

```
(getf '(:name "kex" :exits nil) :name)   ; => "kex"
(getf '(:speed 3 :depth 2) :depth)       ; => 2
```

The signature tells you which you want: `(get symbol indicator)` when the symbol *is* the thing; `(getf plist key)` when the plist *is* the thing.

23.4. Plists vs alists vs hash tables

You now hold all three dictionary shapes Lisp offers natively: the alist (Chapter 7), the hash table (Chapter 13), and the plist. The differences are about *where the pairs live* and what operations are natural:

Structure	Pairs live on	Natural operations	Costs
plist	the symbols themselves	`get`, `remprop`	linear scan per lookup
alist	one list you own	`assoc`, `acons`, `push`	linear scan
hash table	a table object you own	`gethash`, `remhash`, `maphash`	O(1) average

The real distinction: a plist *smears* the association across all symbols that carry the property — `(get name 'location)` across every room symbol is the data, and no single object owns it. An alist is *one list* that can be passed, rebound, saved and rebuilt. That's why the game used alists for `*item-locations*` — state you save and restore — and why plists fit *attributes that belong to named things*:

- Symbols as *names of things* (rooms, commands, items): a plist is the thing's attributes, reachable by name alone.
- One shared table of *mappings*: alist or hash table.
- Data to *save, pass, or have several versions of*: never a plist — you can't snapshot a symbol's plist short of copying it pair by pair, and there's one plist per symbol, not one per context.

`get-properties` is the plist utility worth knowing beyond `get`:

```
(get-properties '(description "#| ... |#" visited t) '(visited description))
;; => (VISITED T) — first property present, with its value
```

The distinction from `getf`: `get-properties` reports which properties are *present*, and its first return value is the suffix of the plist starting at the first match — different from `getf`'s value-only answer. Rare in application code; worth recognising in libraries.

23.5. Plists and keyword arguments

`format` isn't the only place a plist-shaped value appears in disguise. Many libraries document "accepts a plist of options"; modern code uses real `&key` parameters (Chapter 5), but the plist style survives in older APIs:

```
(defun draw-window (title &rest options)
  (let ((width  (getf options :width 80))
        (height (getf options :height 24)))
    (list title width height)))
```

When you read `(:name "kitchen" :exits ...)` in old code, you're seeing a plist-shaped option list — a flat list of alternating keywords and values, handled with `getf`. Chapter 5's `&key` parameters are the modern spelling of the same idea; the plist is the data shape underneath.

23.6. gensym revisited

Chapter 14's gensym created "fresh symbols" — now you can say what that meant precisely. A gensym's product is a symbol with a generated name — `"G123"`, say — and no home package: `symbol-package` returns `nil`, which is the tell that it is not an interned symbol.

Its read syntax `#:g123` exists so the printer can print it back; reading `#:g123` produces a *different* uninterned symbol with the same name — which is why uninterned symbols can't round-trip (Chapter 16's print/read round trip for save files needed interned names).

```
(symbol-package (gensym)) ; => NIL
(symbol-name (gensym)) ; => "G456" — no colon, just the name
```

23.7. Common mistakes

23.7.1. Plists as primary state

Putting game state on plists and then trying to `save-game`: there's no object to hand to print — you'd have to walk every symbol and collect properties first, and restored state is spread across the package. State you save belongs in lists, structures and hash tables you own; plists for per-symbol attributes, if at all.

23.7.2. remprop on shared symbols

remprop and (setf get) mutate *the symbol*, which is shared by everyone using that name. Removing description from 'kitchen removes it for the whole image — including other code that set it. Property lists are global by nature; treat them as such.

23.7.3. get on non-symbol keys

(get 5 'x) signals an error — get requires symbol keys, because the plist lives *on symbols*. For string or number keys, use an alist with :test, or a hash table with the right test, or — nothing Lisp-standard, because plist keys are symbols by design.

23.7.4. Believing type declarations protect plists

23.8. Project step: per-command metadata

The **commands** table (Chapter 9) maps command names to functions. This step adds per-command metadata — help text and usage — where the plist genuinely fits: attributes belonging to named things, consulted by name alone.

```
(defun define-command (name fn help usage)
  "Register a command with its metadata for a help system."
  (push (cons name fn) commands)
  (setf (get name 'help) help)
  (setf (get name 'usage) usage))

(defun command-doc (name)
  "Help text and usage for NAME, or a fallback."
  (format nil "~(~A~): ~A~% usage: ~A"
    name
    (or (get name 'help) "No help available.")
    (or (get name 'usage) "unknown")))

(defun help-command ()
  "List commands with one-line help, alphabetically."
  (format t "~:{~&~(~A~)~12T~A~}~%"
    (sort (loop for pair in commands
        for name = (car pair)
        collect (list name (or (get name 'help) "")))
      #'string< :key #'first)))
```

The `:{}` directive walks a list of lists (Chapter 12); the sort with :key is Chapter 10's `sort` doing book-listing duty. Check:

```
(setf commands nil) ; start clean, for the demo
(define-command 'look #'look "Describe your surroundings." "(look)")
(define-command 'take #'take "Pick something up." "(take ITEM)")
(define-command 'go #'go-to "Move in a direction." "(go NAME)")

(help-command)
;; go Move in a direction.
;; look Describe your surroundings.
;; take Pick something up.

(command-doc 'take)
;; => "take: Pick something up.
;; usage: (take ITEM)"
```

23.9. Exercises

23.1 Predict, then check:

```
(symbol-name 'kitchen)
(symbol-package :kitchen)
(symbol-name :kitchen)
(eq 'kitchen 'KITCHEN)
(eq (intern "KITCHEN") 'kitchen)
(symbol-package (gensym))
```

23.2 Each snippet is wrong or surprising. Say what happens, then fix it.

```
(get "kitchen" 'description)

(setf (get 'kitchen 'exits) '(hall cellar))
(remprop 'kitchen 'exits)
;; ...later, elsewhere:
(get 'kitchen 'exits)

(let ((props (symbol-plist 'kitchen)))
(remprop 'kitchen 'description)
props)

(getf '(name "hall" name "cellar") :name)
```

23.3 Write `plist-p`, which answers whether a list is a valid plist (an even number of elements). Then write `plist->alist`, which converts `(a 1 b 2)` to `((a . 1) (b . 2))`, and `alist->plist` the other way. Which direction loses information, and why can that be acceptable?

23.4 Write `plist-keys`, which returns the distinct keys of a plist. Then using `get-properties`, write `plist-subset-p`, which answers whether a plist contains every key/value pair of another plist.

23.5 Write `swap`, a macro expanding to code that exchanges two variables' values, using `gensym` for the temporary. Then demonstrate — with `macroexpand-1` and a variable actually named `tmp` — why the naive version (with a plain `let` on `tmp`) is a bug.

23.6 project Add per-item flavour text as plist properties on the item symbols — `(setf (get 'key 'flavour) ...)` — and write a `describe-flavour` function that returns the flavour or a default. Then answer in a comment: why is flavour text (never saved, fixed at definition) a defensible plist use where item *locations* (Chapter 7) were not?

Flavour text is (a) an attribute of a named thing, consulted by name alone — the plist's natural shape; (b) fixed at definition time, never needs snapshotting, versioning, or save-game; (c) global by nature — the flavour of the key is the same every session, which is exactly the intent. Item **locations**, by contrast, are mutable state with a lifetime: they change during play, get saved, restored, and would be spread unownably across symbols on a plist — which is why Chapter 7 gave them an alist and Chapter 23′s common-mistakes list forbade plists for them. The dividing line, in one sentence: plists for constants about names, owned structures for state about things.

CHAPTER 24.

Multiple Values

A handful of functions you've already met have been returning *two* things, and the code has been quietly catching only one: `floor` returned a quotient and a remainder, `gethash` returned a value and a found-flag, `parse-integer` returned a number and where reading stopped, `read-from-string` returned an object and a position, `get-properties` returned a key, a value and a tail. Lisp has *multiple values* — a function result can be several objects at once — and this chapter explains the mechanism properly, because it is not what you'd guess coming from other languages, and the guessing causes real bugs.

24.1. Values are not lists

The instinct — return a list of two things — is wrong twice over. A list must be allocated, deconstructed and (for `nil` and false values) made ambiguous: a list of `(nil nil)` can't distinguish "two nils" from "a list whose car is nil" without care. Multiple values are none of that: several objects handed over in one result, with no structure built and no ambiguity. The rule that makes them work:

In any context that expects one value, extra values vanish.

```
(floor 7 2)            ; two values: 3 and 1
(+ (floor 7 2) 10)     ; => 13 — + saw only the 3; the 1 vanished
(* 2 (floor 7 2))      ; => 6 — same
(list (floor 7 2))     ; => (3) — the remainder dropped
```

This is why ordinary code can call `floor` inside arithmetic without dismantling anything: the primary value — the first one — is what every ordinary context sees. The extra values exist only for the forms that ask for them.

`values` produces multiple values:

```
(defun divide (a b)
  "Return the quotient and remainder of A divided by B."
  (values (floor a b) (mod a b)))

(divide 7 2)        ; REPL shows both: 3, 1
(+ (divide 7 2) 0)  ; => 3 — the mod vanished
```

The REPL, which wants to show you everything, prints all the values, each on its own line. Other contexts see one.

The truncation isn't a bug in your code; it's the *contract* of "function call arguments are single-valued". `(list (floor 7 2))` doesn't fail to catch `1`; it deliberately discards it, because `list`'s arguments are single-valued slots. Only the forms designed for values catch them all, and the rest of this chapter names those forms.

24.2. Receiving values

`multiple-value-bind` catches values into variables:

```
(multiple-value-bind (q r)
    (floor 7 2)
  (list q r))          ; => (3 1)
```

The syntax mirrors `let`: a list of variables, a form, a body. You don't have to name the values you don't want — bind only the first:

```
(multiple-value-bind (q)
    (floor 7 2)
  q)                   ; => 3
```

`multiple-value-list` collects *everything* into a list — the general-purpose net:

```
(multiple-value-list (floor 7 2))  ; => (3 1)
(multiple-value-list (+ 1 2))      ; => (3) — one value, one-element list
(multiple-value-list (values))     ; => NIL — zero values, empty list
(multiple-value-list (values nil)) ; => (NIL) — one nil value, one-element
```

That last pair is the distinction to keep straight. `(values)` produces *zero* values; `(values nil)` produces *one* value, which happens to be nil. Both look like `nil` in a single-value context, but `multiple-value-list` tells them apart — nil (empty) versus `(nil)` (one element). The exercise section puts this to work; for now, notice that "no value" and "the value nil" are different things, and the empty list is the signal for "no value."

Chapter 13 used `multiple-value-list` for `gethash` without explaining it; now it's explained — the found-flag was kept because the function returns two values:

```
(defparameter *visited* (make-hash-table))

(multiple-value-bind (value found-p)
    (gethash 'kitchen *visited*)
  (if found-p
      value
      "first visit"))
```

The found-flag is the single most useful extra value in Common Lisp, because it resolves the ambiguity `gethash` would otherwise have: a stored `nil` and an absent key both read as `nil` without it.

There's also `nth-value`, which extracts a specific value without binding the others:

```
(nth-value 0 (floor 7 2))   ; => 3
(nth-value 1 (floor 7 2))   ; => 1
(nth-value 2 (floor 7 2))   ; => NIL — no third value
```

It's useful when you know exactly which value you want and the others would just clutter the binding. `(nth-value 1 (gethash k table))` is "just the found-flag, please."

`values-list` is `values` turned inside out — a list in, values out:

```
(values-list '(3 1))                          ; same as (values 3 1)
(multiple-value-list (values-list '(1 2 3)))  ; => (1 2 3)
(multiple-value-list (values-list nil))       ; => NIL — zero values
```

`values` and `values-list` are duals: (values-list (multiple- value-list form)) reconstructs `form`'s values, and `(multiple-value-list (values-list list))` reconstructs `list`. They don't round-trip through any intermediate — each produces what the other consumes, in the appropriate context.

24.3. Single-value and multiple-value contexts

The vocabulary for the vanishing rule: a *single-value context* keeps the primary value; a *multiple-value context* keeps all. The full table, which is the reference to come back to:

Context	Values kept
Argument of a function call	1 (primary)
`let`/`let*` binding value form	1 (primary)
`if`/`when`/`cond` test form	1 (primary)
`progn` non-final form (value discarded)	1 (primary)
Inner `(values ...)` in `values`' args	1 (primary)
Body's last form of `defun`/`let`/`progn`	all
`multiple-value-bind`'s form	all
`multiple-value-list`'s form	all
`multiple-value-call`'s argument forms	all (each)
`multiple-value-prog1`'s first form	all
`return`/`return-from` value form	all

The table's shape is the lesson: *default behavior is truncation*. Any form that uses its argument's values all does so by explicit design, and the design is visible in the name (`multiple-value-...`) or the position (a body's last form, an exit form).

Two corners worth pointing at:

The body's last form. `(defun f () (values 1 2))` returns two values, because the function body's last form is a multiple-value context. Every function you write returns *all values of its last expression*, whether you thought about multiple values or not. That's why `(defun g () (f))` passes both values through — `(f)` is the body's last form, hence multiple-value.

`return-from`. `(return-from block (values 1 2))` exits the block with *both* values, and `(multiple-value-bind (a b) (block ...) ...)` catches them. `return` uses the same mechanism (it's `return-from nil`), and `dolist`, `dotimes` and `loop` all establish `nil` blocks, so a `return` with a values form delivers them all:

```
(multiple-value-bind (a b)
    (dolist (x '(1 2 3) (values :done 'three))
      (declare (ignore x)))
  (list a b))    ; => (:DONE THREE)
```

And with `return` mid-loop:

```
(multiple-value-bind (a b)
    (dolist (x '(1 2 3))
      (when (= x 2) (return (values :found x))))
  (list a b))    ; => (:FOUND 2)
```

The `(declare (ignore x))` in the first example is Chapter 21's declaration — the body is empty, `x` is unused, silence the warning.

24.4. `multiple-value-call` and `multiple-value-prog1`

`multiple-value-call` calls a function with *all* values of every argument form appended together:

```
(multiple-value-call #'+ (values 1 2) 3 (values 4 5))
;; => 15 — arguments were 1, 2, 3, 4, 5
```

It is the only ordinary way to *combine* values instead of truncating them, and its commonest use is passing a multiple-valued result onward untouched:

```
(multiple-value-call #'list (floor 7 2))
;; => (3 1) — same as (multiple-value-list (floor 7 2))
```

That identity is worth stating: `(multiple-value-list form)` is equivalent to `(multiple-value-call #'list form)`. Once you see it, "what does `multiple-value-call` do" stops needing a special explanation.

`multiple-value-prog1` evaluates forms and returns the *first one's* values — all of them — unlike `prog1`, which truncates:

```
(defparameter *trace* nil)

(defun floor-and-log (a b)
  "Return FLOOR's two values, logging a trace entry."
  (multiple-value-prog1
      (floor a b)                       ; all its values preserved
    (push (list 'floor a b) *trace*)))  ; side effect only

(multiple-value-list (floor-and-log 7 2))
;; => (3 1)
*trace*   ; => ((FLOOR 7 2))
```

`prog1` would have kept only `floor`'s primary value 3; the remainder would be lost, and the caller catching both values would see `nil` for the second. When the first form might return multiple values and the result matters, `multiple-value-prog1` is the correct spelling.

The pattern `multiple-value-prog1` serves: *do something before returning a multi-valued result, without touching the result.* Logging, tracing, snapshotting — the side effect runs, the values come through untouched.

24.5. Writing functions that return values

Two styles, and choosing well is most of the craft here:

Values for cheap secondary information. `floor`'s remainder, `gethash`'s flag, `read-from-string`'s position: things the caller *sometimes* wants, cheap to ignore otherwise. If the caller almost always wants it, make it a primary value or a structure; if almost never, print it or store it.

Values instead of ambiguity. The strongest use: `(gethash key table)` distinguishes "stored nil" from "absent", and your own functions can too:

```
(defun lookup-room (name)
  "Two values: the room, and whether it was found."
  (let ((r (find name *rooms* :key #'room-name)))
    (values r (not (null r)))))
```

Callers who don't care about `found-p` write `(lookup-room 'kitchen)` and use the room; careful callers bind both. The function serves both without a keyword argument.

Note what `values` *doesn't* do: it isn't a tuple type. `(values 1 2)` is not an object — there is no way to store "the two values" in a variable; you can only catch them immediately (`multiple-value-bind`) or solidify them (`multiple-value-list`). Passing values through intermediate functions needs the multiple-value forms:

```
(defun floor-with-log (a b)
  (multiple-value-prog1 (floor a b)
    (format *trace-output* "floor ~D ~D~%" a b)))

(defun twice-remainder (a b)
  (multiple-value-bind (q r)
      (floor a b)
    (declare (ignore q))
    (values q (* 2 r))))
```

The first function preserves `floor`'s two values while logging. The second catches both values and emits two of its own, the remainder doubled.

24.6. Common mistakes

24.6.1. Assuming (values a b) returns a list

```
(length (values 1 2))       ; error: 1 is not a list
(append (values 1 2) '(3))  ; error: 1 is not a list
```

Values aren't a datum; `multiple-value-list` is the bridge when you actually want the list. The error message tells you the primary value escaped (as a number) and the list operation got confused — the diagnosis, once you know values exist.

24.6.2. Losing values through innocuous forms

`(let ((q (floor 7 2))) ...)` — `q` gets 3; the remainder is gone, silently. Any single-value binding truncates. The fix is `multiple-value-bind` at the point of the call, and the discipline is knowing where values live: only in the form that produced them, until caught or dropped. Once caught into a list or bound into variables, they're ordinary data.

24.6.3. return inside dolist that you meant as a single value

```
(multiple-value-bind (a b)
    (dolist (x '(1 2))
      (return x))
  (list a b))   ; => (1 NIL)
```

The trap isn't the truncation — that's expected, `return`' delivers `x`'s one value, and `b` gets nil. The trap is believing the opposite: some readers think `return` truncates, and test `(return (values 1 2))`, expecting `(1 NIL)`, and are surprised by `(1 2)`. In fact `return` preserves multiple values (as `return-from` does). Knowing this *prevents* a class of bug where you avoid `return` because you think it truncates, when it doesn't.

The genuine trap: `let` binding value forms truncate, and they're common in the code that catches `return`:

```
(let ((pair (return-from some-block (values 1 2))))
  pair)   ; pair binds 1, not (1 2)
```

`return-from` delivers both values, but `let` keeps one. Use `multiple-value-bind` if you want both.

24.6.4. Ignoring the found-flag where it matters

`(gethash 'key table)` returning `nil` means *either* absent *or* stored-nil. Code that treats it as "absent" corrupts tables that legitimately store `nil` values. Wherever `nil` is a meaningful stored value, the two-valued call is not optional.

24.6.5. Confusing (values) with (values nil)

```
(if (multiple-value-list (values)) :present :absent)      ; => :ABSENT
(if (multiple-value-list (values nil)) :present :absent)  ; => :PRESENT
```

Both look like "nothing" in a single-value context — both are `nil` when passed to a function that expects one value. But in a multiple-value context, one returns zero values, the other returns one. The difference is only detectable by the forms designed to detect it — `multiple-value-list`, the count of a bind, and so on.

24.6.6. Expecting a toplevel to catch two values

`(defun f () (floor 7 2))`. At the REPL, `(f)` shows 3, 1. But `(mapcar #'f ...)` sees only `3` per call. `mapcar`'s function-call arguments (and results) are single-valued. If you want a mapcar-shaped operation that preserves values, you have to write it (`multiple-value-call` with careful argument plumbing) — there's no "mapcar for values" in the standard.

24.7. Project step: two-valued command dispatch

Chapter 15′s `run-command` returns one value: the reply string. When the player types a real command, the reply is "you take the key"; when they type nonsense, the reply is "I don't know how to do that". A caller can't tell those cases apart from the string alone without matching on the text. Multiple values fix this cleanly — every command returns two things: the reply, and whether the command was recognised.

First `call-command`, which knows the difference:

```
(defun call-command (command)
  "Call COMMAND's handler.  Two values: reply, known-p."
  (let ((entry (assoc (first command) *commands*)))
    (if (null entry)
        (values (format nil "No such command: ~(~A~)."
                        (first command))
                nil)
        (values (apply (cdr entry) (rest command))
                t))))
```

Then `run-command`, which wraps it with the failsafe from Chapter 15:

```
(defun run-command (command)
  "Execute COMMAND.  Two values: reply, known-p."
  (restart-case
      (handler-case (call-command command)
        (program-error ()
          (values "Wrong number of arguments for that command." t)))
    (retry-with (fixed)
      :report "Run a corrected command instead."
      (run-command fixed))))
```

The `restart-case` preserves the two values — when the body returns normally, `restart-case` returns all of its values. When `program-error` is caught, the handler returns `(values "..." t)`: the command was known, but the arguments were wrong, so it's flagged as known. That distinction is genuinely useful: "known but failed" and "unknown command" are different failures, and the caller can distinguish them by the second value.

Now `play`, which was `(mapcar #'run-command commands)` returning a list of replies. Updated for two values:

```
(defun play (commands)
  "Run COMMANDS.  Two values: replies, unknown-command list."
  (let ((replies nil)
        (unknown nil))
    (dolist (c commands (values (nreverse replies)
                                (nreverse unknown)))
      (multiple-value-bind (reply known-p)
          (run-command c)
        (push reply replies)
        (unless known-p (push (first c) unknown))))))
```

`dolist`'s result form is `(values ...)`, which returns both lists. Inside the body, `multiple-value-bind` catches each command's two values; the `unless` decides where to file the unknown ones. The `nreverse` preserves order, as usual.

Check it:

```
(setf *current-room* (find-room 'kitchen))
(setf *item-locations* '((key . kitchen)))
```

```
(multiple-value-bind (replies unknown)
    (play '((take key) (dance) (look)))
  (list replies unknown))
```

Expected:

```
(("You take the key."
  "No such command: DANCE."
  "You are in the Kitchen. A warm kitchen that smells of bread. You can see: key.")
 (DANCE))
```

The reply list carries the "No such command: DANCE." string (the player sees it); the unknown list carries `DANCE` (the developer audits it). Both come from the same command, and `play`'s single function call returns both.

Now Exercise 11.12's `count-unknown`, which compared reply strings with `string=` to detect unknowns, becomes honest:

```
(defun count-unknown (commands)
  "How many commands are unknown?"
  (multiple-value-bind (replies unknown)
      (play commands)
    (declare (ignore replies))
    (length unknown)))
```

The string-matching was always a hack — if you changed the message text, the count broke. The two-value contract is what the game meant all along: "was this command known?" is a question the dispatcher can answer, and now it does.

The `(declare (ignore replies))` is Chapter 21's declaration doing its job: the binding exists to consume a value the function doesn't need, and the declaration documents that the non-use is deliberate.

24.8. Exercises

24.1 Predict, then check:

```
(multiple-value-list (values))
(multiple-value-list (values 1))
(multiple-value-list (values 1 2 3))
(+ (values 1 2) (values 3 4))
(list (values 1 2) (values 3 4))
(values (values 1 2) (values 3 4))
```

24.2 Each snippet is wrong or surprising. Say what happens, then fix it.

```
(defun two-things () (values :a :b))
(let ((x (two-things))) x)

(multiple-value-bind (a b)
    (loop for i below 3
```

```
          finally (return (values :done i)))
  (list a b))

(let ((table (make-hash-table)))
  (setf (gethash :k table) nil)
  (gethash :k table))

(defun pass-through (form-values)
  form-values)
(multiple-value-list (pass-through (values 1 2)))
```

24.3 Write `safe-divide`, which returns two values — the quotient, and `nil` on success or `:division-by-zero` when the divisor is zero (primary value `nil` in that case) — and `safe-sum`, which sums a list of (numerator denominator) pairs using `safe-divide`, and returns `nil` and `t` as two values when any division failed. No conditions: the exercise is doing `gethash`'s flag trick by hand.

24.4 Write `compose-values`, which takes two functions `f` and `g` (`f` runs first, then `g` gets all of `f`'s values), and returns the composed function. `(funcall (compose-values #'floor #'list) 7 2)` should give `(3 1)`. Which primitive is `compose-values` just `multiple-value-call` for?

24.5 Write `first-or-default`, which takes a thunk and a default, calls the thunk, and returns its primary value or the default when it returned *zero* values. What does `(multiple-value-list (values))` return, and how does your function detect that case?

24.6 project The `save-game`/`load-game` pair (Chapters 16 and 18) currently return one value. Make both two-valued: `save-game` returns `t` and the pathname written (or `nil` and an error description on failure); `load-game` returns the state and a loaded-p flag. Then update the game's `save` and `load` commands to use the second value — no string matching on the error message.

CHAPTER 25.

Closures as Objects

Chapter 9 introduced closures with `make-adder` and `make-counter`: functions that remember the `let` variables they were born next to. That was a first pass, enough to use `lambdas` as callbacks. This chapter develops closures into what they really are — *Lisp's objects, before CLOS objects* — with three new ideas: what exactly is captured, how several closures can share one environment, and where closure state beats `defparameter` state.

25.1. What is captured, precisely

When a `lambda` references a variable from an enclosing scope, it captures the *binding* — the variable itself, not the value at creation time:

```
(defun make-counter ()
  (let ((count 0))
    (lambda ()
      (incf count))))

(defparameter *c* (make-counter))
(funcall *c*) ; => 1
(funcall *c*) ; => 2
(funcall *c*) ; => 3
```

The counter keeps counting because `count` is a live binding that outlived the `let` that created it. This is the defining property of lexical closures: the binding's extent extends as long as any closure can reach it. Contrast the value-capture instinct:

```
(defun make-counters-wrong ()
  (let ((count 0))
    (list (lambda () count)
      (lambda () (incf count)))))

(defparameter pair (make-counters-wrong))
(funcall (first pair)) ; => 1 — sees the increment...
(funcall (second pair)) ; => 2
(funcall (first pair)) ; => 2 — the SAME binding
```

Both closures share one `count`, so the "reader" sees every increment. If you wanted independent copies, each closure needs its own binding — which is the next section's subject.

25.2. Constructors: closures born in a loop

`dotimes` and `dolist` introduce fresh bindings *per iteration*, so closures created inside a loop each capture their own variable:

```
(defun make-bumpers (limit)
  "A list of closures; the i-th adds i."
  (loop for i from 1 to limit
```

```
    collect (lambda (x) (+ x i))))

(defparameter bumpers (make-bumpers 3))
(mapcar #'funcall bumpers (list 100 100 100))
;; => (101 102 103) — three separate bindings
```

No capture-in-a-loop bug in Common Lisp's case here — each iteration binds a new `i`. (In languages with per-function variable binding, this example is the classic trap; in Lisp the loop's binding is per-iteration by specification, and `loop`'s `for` clauses too.) The corresponding *real* trap in Lisp is different, and covered in Common mistakes below: capturing the *loop variable of a surrounding function* instead of the loop's own binding.

25.3. Shared environments: objects from closures

The step beyond `make-counter`: return *several* closures that share one set of bindings. That shared set is private state with no name anyone else can touch — in one word, *encapsulation*, achieved with nothing but `let` and `lambda`:

```
(defun make-account (balance)
  "A bank account as three closures over one binding."
  (values
    (lambda (amount) ; deposit
      (setf balance (+ balance amount)))
    (lambda (amount) ; withdraw
      (if (> amount balance)
        (error "Insufficient funds: ~A < ~A" amount balance)
        (setf balance (- balance amount))))
    (lambda () balance))) ; balance

(multiple-value-bind (deposit withdraw balance)
  (make-account 100)
  (funcall deposit 50)
  (funcall withdraw 30)
  (funcall balance)) ; => 120
```

Nothing outside the three closures can read or write `balance` — not by `symbol-value`, not by any accessor, because it isn't a slot anywhere; it's a variable that exists only in their shared lexical world. Compare the CLOS version you'll write in Chapter 26: a class with a `balance` slot is the same design, with the closures replaced by methods and the `let` replaced by the instance. Neither is better; the closure version has no class to define, the class version has `print-object`, inheritance and redefinition support. For small, fixed, use-and-forget state, closures are lighter.

A dispatch-table variant — one closure, behaviour chosen by message — is the style that grew into "generic functions" in other languages:

```
(defun make-registry ()
  (let ((table (make-hash-table)))
    (lambda (message &optional key value)
      (ecase message
        (:put (setf (gethash key table) value))
```

```
        (:get (gethash key table))
        (:keys (loop for k being the hash-keys of table
            collect k))))))

(defparameter reg (make-registry))
(funcall reg :put 'key "brass")
(funcall reg :get 'key) ; => "brass"
(funcall reg :keys) ; => (KEY)
```

The hash table is invisible to callers; the closure is the only door. Add a `:count` message later and no caller changes.

25.4. Closures vs defparameter state

The earmuffed globals you've used since Chapter 4 are *process-wide* state. Closure state is *instance* state: call `make-counter` twice and you have two independent counters, whereas two uses of a `defparameter` counter are one counter.

The rule of thumb:

One of something for the whole program (the game's `rooms`, `commands`): a `defparameter`. Global by intent.

A third use, small and constant in real code: closures as *parameterised behaviour*. Chapter 9's `make-adder` is the seed; `remove-if` takes a predicate, and building that predicate on the fly is the daily use:

```
(defun name-matcher (target)
  (lambda (room) (eq (room-name room) target)))

(remove-if (name-matcher 'kitchen) rooms)
```

The `lambda` captures `target`; the predicate is data and behaviour in one argument. No class, no global, no coupling.

25.5. Common mistakes

25.5.1. Capturing the wrong variable in a loop

```
(let ((fns nil))
(dolist (x '(a b c) (nreverse fns))
(push (lambda () x) fns)))

(mapcar #'funcall **) ; => (A B C) — fine, dolist binds per iteration
```

The genuine trap is when the `lambda` references an *outer* function's parameter instead of the loop's variable — then every closure sees the same (final or only) argument value. When closures over loop variables misbehave, check which binding each free variable names.

25.5.2. Closure state and save files

Closure bindings are invisible to `print`, `read` and `save-game`: there is no object to serialise. State that must survive belongs in structures, alists and hash tables reachable from named variables. Closures for *session-lifetime* state — counters, iterators, per-connection handles.

25.5.3. funcall amnesia

A closure is a function object; `(make-counter)` alone creates it and throws it away. `setf` it, pass it, store it in a table — but if nobody holds it, its bindings die with it. The `make-account` example only works because `multiple-value-bind` kept all three closures; keeping one and dropping the others orphaned their shared world.

25.5.4. Overusing closures for big state

Once a closure-based "object" grows past four or five messages, or needs `print-object`, inheritance, or redefinition during development (Chapter 22's workflow), move it to `defclass`. Closures excel small; classes scale.

25.6. Project step: a command-history object

Chapter 14's `history` is a plain list of alist snapshots, managed by the global `snapshot` and `undo` functions. This step wraps history in a closure object — private state, three operations, no global except the single handle.

```
(defun make-history ()
  "A bounded command history: :add, :recent, :clear, :undo-to."
  (let ((snapshots nil)
        (limit 20))
    (lambda (message &optional arg)
      (ecase message
        (:add
          (push arg snapshots)
          (when (> (length snapshots) limit)
            (setf snapshots (subseq snapshots 0 limit)))
          t)
        (:recent
          (subseq snapshots 0 (min (or arg limit)
              (length snapshots))))
        (:undo-to
          (when snapshots
            (let ((s (first snapshots)))
              (pop snapshots)
              s)))
        (:clear
          (setf snapshots nil)
          t)))))
```

Replace the global-history versions from Chapter 14:

```
(defparameter history (make-history))

(defun snapshot ()
  "Record the current item locations for undo."
```

```
  (funcall history :add (copy-alist item-locations)))

(defun undo ()
  "Restore the item locations from before the last move."
  (let ((s (funcall history :undo-to)))
    (if s
      (progn
        (setf item-locations s)
        "Undone.")
      "Nothing to undo.")))
```

The signatures of `snapshot` and `undo` are unchanged from Chapter 14 — callers of these functions don't need to know that the storage moved from a global list to a closure. That is the *encapsulation* the chapter promised: private state, public interface, no change to the interface.

Check:

```
(setf current-room (find-room 'kitchen))
(setf item-locations '((key . kitchen)))

(snapshot)
(setf item-locations (set-location item-locations 'key 'player))
(undo)
(item-location 'key) ; => KITCHEN
```

The `limit` trimming keeps the history bounded without any caller knowing. Note what the design avoids: `history` entries never reach `save-game` (they're closure state, by design — a fresh session starts with empty history), and no other code can reach in and mangle the list, because there is no list to reach — only the four messages. If a future chapter wants history to *survive* saves, that's the signal to convert this object to a `defclass` with a serialisable slot — and the class version's save-game support would need to expose the snapshots list, which is exactly the trade the chapter warned about.

25.7. Exercises

25.1 Predict, then check:

```
(let ((fns (loop for i from 1 to 3
collect (lambda () i))))
(mapcar #'funcall fns))

(defun make-two ()
(let ((n 0))
(values (lambda () (incf n))
(lambda () (decf n)))))
(multiple-value-bind (up down) (make-two)
(list (funcall up) (funcall up) (funcall down) (funcall up)))

(let ((x 10))
(let ((f (lambda () x)))
```

```
(setf x 99)
(funcall f)))
```

25.2 Each snippet is wrong or surprising. Say what happens, then fix it.

```
(defun make-accumulator ()
(lambda (n) (setf total (+ total n))))
(funcall (make-accumulator) 5)

(defparameter counters
(loop for name in '(a b c)
collect (list name (let ((count 0))
(lambda () (incf count))))))
(funcall (second (first counters)))

(let ((fns nil))
(dotimes (i 3)
(push (lambda () i) fns))
(mapcar #'funcall fns))
```

25.3 Write `make-average`, returning a single closure that takes a number and returns, as two values, the running average and the running count. Then extend `make-registry` (from the chapter) with a `:count` message — how many keys it holds — without changing any existing caller.

25.4 Write `make-stack`, returning four closures (`push!`, `pop!`, `empty-p`, `contents`) sharing one list binding. `pop!` should signal an error on underflow, and `contents` should return a copy of the list. What does returning a copy teach about encapsulation vs leaking?

25.5 Chapter 10′s `remove-duplicates` uses `eql` by default. Write `make-key-matcher`, which takes a `:key` function and a target value and returns a predicate suitable for `find-if`. Then use it to find a room by name without writing an inline `lambda`. Compare with the chapter's `name-matcher`: which one generalises, and why?

25.6 project Wrap the game's item-taking in a closure-based *cooldown*: a `make-cooldown` object with `:block` (records a command and answers whether the same command came within the last two calls) and `:reset`. Wire it into `run-command` so repeated identical commands get the reply "Not again." instead of executing. In a comment: is cooldown state (a) session state like history, or (b) persistent state like item locations — and what decides the answer?

CHAPTER 26.

CLOS

You've built the game with lists, alists, structures and closures. That works at small size, but it has two strains you may have felt: `room-name`, `room-exits` and friends are named by convention (Chapter 13), and every new kind of thing means more parallel functions. The Common Lisp Object System, CLOS, offers a different arrangement: *classes* give data named parts, and *generic functions* give behaviour a way to vary by the data it works on.

CLOS is not a bolted-on layer. It is part of the language standard, written in Lisp, and powerful enough that people have built other object systems on top of it. Two ideas set it apart from Java's model from the start, and the chapter keeps returning to them:

- *Methods belong to generic functions, not to classes.* A class never "owns" a method; a generic function collects methods from wherever they're defined.
- *Methods specialise on all their arguments*, not just the first. Java's `this` picks one distinguished argument; CLOS picks among all of them.

The chapter also positions CLOS against the two alternatives you already have: structures (Chapter 13) for internal records, closures (Chapter 25) for small stateful objects. CLOS is the tool for types at the *edges* of your program, where other code adds behaviour and other people extend.

26.1. Classes: `defclass`

A class is defined with `defclass`, and reads like a `defstruct` with ambitions:

```
(defclass thing ()
  ((name :initarg :name
         :reader thing-name)
   (description :initarg :description
                :reader thing-description
                :initform "an ordinary thing"))
  (:documentation "Something the player can carry or examine."))
```

The parts:

- The second slot of `defclass` — here `()` — lists *superclasses*. `thing` has none beyond the default.
- Each slot has a name and *slot options*: `:initarg` names the keyword `make-instance` accepts, `:reader` generates an accessor function, `:initform` gives a default.
- Unlike `defstruct`, nothing is exported or auto-generated beyond what you ask for. `thing-name` exists because you asked.

The class is named `thing`, not `item`, to avoid two collisions with earlier chapters: `item` is a value the game passes around (the symbol `'item` in command arguments), and `item-name` is Chapter 7's function that lowercases a symbol for messages. `thing` has neither conflict.

Instances are made with `make-instance`, and slots are read with the readers:

```
(defparameter *key-thing*
  (make-instance 'thing
                 :name 'key
```

```
                :description "a small brass key"))

(thing-name *key-thing*)          ; => KEY
(thing-description *key-thing*)   ; => "a small brass key"
(thing-description (make-instance 'thing :name 'rock))
;; => "an ordinary thing"
```

`:initform` runs each time an instance is made, so defaults like `:initform (list)` would give each instance its *own* fresh list — the problem `defstruct`'s shared defaults dodge by luck. If you want a default evaluated once for all instances, say so deliberately.

Slots are writable with `:accessor` instead of `:reader`, which generates both the reader and a `setf` place:

```
(defclass door ()
  ((locked :initarg :locked
           :accessor door-locked-p
           :initform t)))

(defparameter *cellar-door* (make-instance 'door))
(door-locked-p *cellar-door*)          ; => T
(setf (door-locked-p *cellar-door*) nil)
(door-locked-p *cellar-door*)          ; => NIL
```

`setf` works on slot accessors with no special form — another place Chapter 4's "setf knows about places" pays off.

`:reader`, `:accessor`, `:initarg` are a lot of typing compared to `defstruct`'s automatic everything. The trade is deliberate: `defstruct` is for internal, implementation-flavoured records, and `defclass` for types with an interface you'd defend. As a rule of thumb: structures for the game's scratch data, classes for the kinds of thing other code will touch.

26.2. Inheritance

The superclass list is where classes build on each other:

```
(defclass container (thing)
  ((contents :initarg :contents
             :accessor contents
             :initform nil)))

(defclass treasure (thing)
  ((value :initarg :value
          :reader value
          :initform 1)))

(defparameter *chest*
  (make-instance 'container
                 :name 'chest
                 :description "a wooden chest."
                 :contents '(gold ring)))
```

```
(thing-name *chest*)      ; => CHEST
(contents *chest*)        ; => (GOLD RING)
(typep *chest* 'thing)    ; => T
```

`container` *is a* `thing`: it has `thing`'s slots and readers, plus its own. `typep` asks the class question, and it works for any type you've met — `(typep 3 'integer)` is the same operation.

Multiple inheritance is ordinary in CLOS — a class may list several superclasses — and slot conflicts are resolved by a *class precedence list*: the class itself, then its superclasses left to right, depth-first, deduplicated. You rarely need to compute one by hand, but when two superclasses both define a slot or a method, the precedence list decides whose wins, leftmost first. `class-precedence-list`, called on an instance's class, shows it:

```
(sb-mop:class-precedence-list (class-of *chest*))
;; => (#<STANDARD-CLASS CONTAINER> #<STANDARD-CLASS THING>
;;     #<STANDARD-CLASS STANDARD-OBJECT>
;;     #<STANDARD-CLASS T>)
```

The order: `container` first (most specific), then `thing` (its superclass), then `standard-object` (the default CLOS parent), then `t` (everything). Any method selection walks this list from left to right, most specific first.

26.3. Generic functions and methods

Here is the part that's genuinely different. A *generic function* is a named thing that dispatches to methods:

```
(defgeneric describe-thing (thing)
  (:documentation "Return a sentence describing THING."))
```

`defgeneric` defines the name, the parameter list and the documentation. It has no body. Bodies come from `defmethod`:

```
(defmethod describe-thing ((t2 thing))
  (format nil "You see ~(~A~): ~A."
          (thing-name t2) (thing-description t2)))

(defmethod describe-thing ((c container))
  (format nil "You see ~(~A~): ~A It holds ~{~(~A~)~^, ~}."
          (thing-name c)
          (thing-description c)
          (contents c)))

(defmethod describe-thing ((tr treasure))
  (format nil "You see ~(~A~): ~A It's worth ~D coins."
          (thing-name tr) (thing-description tr) (value tr)))
```

Each `defmethod` adds a method to the same generic function, and the parenthesised parameter `((t2 thing))` says "this method applies when this argument is a `thing`". Calling it picks the most specific method:

```
(list (describe-thing 'table)
      (describe-thing *key-thing*)
      (describe-thing *chest*))
```

Expected: a generic error (no method for symbols — `table` is not a `thing`), the item's description, and the container's listing. Re-run with only the registered things:

```
(list (describe-thing *key-thing*)
      (describe-thing *chest*)
      (describe-thing (make-instance 'treasure
                       :name 'crown
                       :description "a tarnished crown."
                       :value 100)))
```

Expected:

```
("You see key: a small brass key."
 "You see chest: a wooden chest. It holds gold, ring."
 "You see crown: a tarnished crown. It's worth 100 coins.")
```

Each of the three kinds of thing gets its own method, and the call selects by the argument's class. `container` inherits from `thing`, so a container is also "described by thing" — but the container method is *more specific*, so it wins.

Notice what didn't happen: `thing` doesn't "have" a `describe-thing` method. The methods live on `describe-thing`, and `thing` merely appears in one's parameter specialiser. You can add a method for a class you don't own — for `number`, even — from anywhere, and you can add methods to a generic function from any file, at any time, including in the running REPL. Java's visitor pattern exists to work around this restriction; CLOS doesn't have the restriction.

Methods on several arguments — multiple dispatch — follow the same rule, with the combination of all specialisers deciding:

```
(defgeneric combine (a b))

(defmethod combine ((a number) (b number))
  (+ a b))

(defmethod combine ((a string) (b string))
  (concatenate 'string a b))

(defmethod combine ((a number) (b string))
  (format nil "~D~A" a b))

(list (combine 1 2) (combine "a" "b") (combine 3 "x"))
```

Expected: `(3 "ab" "3x")`. No `if` chains checking types; the dispatch table *is* the type logic, and each case is a method you can read and extend alone.

26.4. `call-next-method`

A method can hand work to the next one down the precedence list:

```
(defmethod describe-thing :around ((t2 thing))
  (format nil "~A (a ~(~A~))"
          (call-next-method)
          (class-name (class-of t2))))
```

Method qualifiers such as `:after`, `:before` and `:around` change when a method runs:

- `:before` methods run before the primary method, for setup.
- `:after` methods run after, and *all matching ones* run — they compose rather than choose.
- `:around` wraps everything and calls `call-next-method` to let the rest proceed.

The `:around` method above doesn't replace the primary — it runs first, calls `call-next-method` (which runs the primary chain), and appends the class name. `:before` and `:after` are the Lisp answer to what Java calls `super` calls in constructors: an `initialize-instance :after` method is the standard way to finish setting up an instance:

```
(defmethod initialize-instance :after ((c container) &key)
  (unless (listp (contents c))
    (setf (contents c) (list (contents c)))))
```

This runs whenever any `container` is made, whether or not the caller remembered a `:contents` argument — validation and repair lived in the constructor, now it lives with the class. (If you'd written `(make-instance 'container :contents '(gold))`, no change — listp is true, no wrapping. If you'd written `(make-instance 'container :contents 'gold)`, the :after fires, wraps `gold` into `(gold)`, and `(contents ...)` returns the list.)

26.5. Structures and classes: when to use which

Both give named slots. The honest comparison:

	`defstruct`	`defclass`
Accessors	automatic	only what you declare
Inheritance	single superclass	multiple, with precedence list
Methods that vary	by `typecase` or `etypecase`	by `defmethod`
Speed	usually faster, simpler	more machinery underneath
Intended for	internal records	types with a public interface

The game has used `defstruct` rooms so far, and that's fine — rooms are internal. The `thing` hierarchy above is the kind of thing that benefits from CLOS, because `describe-thing` wants to vary by type.

26.6. CLOS vs closures

Chapter 25 built account and registry objects out of closures. CLOS gives the same patterns a different shape, and choosing between them is a real design question:

	Closures	CLOS
State lives in	`let` bindings	slots
Behaviour is	one function, message-dispatched	one method per generic function

	Closures	CLOS
Adding behaviour	edit the `ecase`	define a new `defmethod`
Multiple objects	return more closures	`make-instance`, many instances
Redefinition while running	re-define the closure	re-define the class; instances auto-update
Printing	wrap the closure	`print-object`
Inheritance	compose by hand	`defclass` superclasses

For a small stateful object with three or four messages, closures are lighter — no class to define, no dispatch overhead. For a type hierarchy, multiple implementations of the same interface, or behaviour that varies by more than one argument, CLOS is the right tool.

The two also compose: a closure can be a slot value, a method can return a closure, an `initialize-instance :after` can capture a closure to use as a default method. The choice isn't exclusive; it's which shape to *lean on* for a given piece.

26.7. Common mistakes

26.7.1. Forgetting `(:use :cl)` brings CLOS in, but nothing else does

`defclass`, `defmethod`, `make-instance` are all ordinary `cl` symbols. The mistake is elsewhere: in a file with `(in-package :game)`, a method on `describe-thing` defined in `cl-user` and one defined in `game` are *two different generic functions*, because the symbols differ. When methods seem to vanish, check `(symbol-package 'describe-thing)` in both places.

26.7.2. Writing methods before the generic function

You can skip `defgeneric` — the first `defmethod` creates the generic function implicitly. But then the parameter list and documentation come from wherever the first method happened to be, and a later `defgeneric` with a different shape is an error. Define `defgeneric` first: it is the interface statement.

26.7.3. Expecting slot readers you didn't declare

`make-instance` accepted the `:initarg`, but reading the slot without a `:reader` or `:accessor` fails. CLOS generates nothing you didn't ask for.

26.7.4. Specialising on a class that isn't loaded

```
(defmethod describe-thing ((c container)) ...)
;; "The class CONTAINER is not defined"
```

The class must exist before the method that names it. Load order matters; ASDF's `:serial` from Chapter 16 exists for exactly this.

26.7.5. Confusing `typep` on classes with `eq` on symbols

`(typep *chest* 'thing)` asks about the object. `(eq *chest* 'thing)` compares the object to the symbol `thing`, which is `nil`. The quote in `typep` is part of the type notation, not a quoted object comparison.

26.7.6. Forgetting `call-next-method` in an `:around`

An `:around` method that doesn't call `call-next-method` suppresses every primary method beneath it — the description comes back as whatever the `:around` alone produced, with no item-specific information. That's occasionally intended; it's almost never what the code's author meant.

26.8. Project step: things in the game

The game's items are just symbols in `*item-locations*` — where each symbol lives. That's enough to move a key from the kitchen to the player's pocket, but not enough to *describe* the key: its weight, its flavour, whether it's fragile. The game has held that information in code, inline in `describe-room-text` and the like. This step gives it a home.

The approach: register each item symbol with a `thing` instance that carries the metadata. The two representations coexist:

- `*item-locations*` — mutable state: which room holds each item.
- `*thing-registry*` — static metadata: what each item *is*.

The link between them is the symbol (`'key`, `'chest`, etc.), which appears in both.

```
(defparameter *thing-registry*
  (make-hash-table :test #'eq)
  "Maps item symbols to their THING instances.")

(defun register-thing (sym instance)
  "Associate SYM with INSTANCE in the registry."
  (setf (gethash sym *thing-registry*) instance))

(defun lookup-thing (sym)
  "The THING instance for SYM, or NIL if unregistered."
  (gethash sym *thing-registry*))
```

Now register the game's items:

```
(register-thing 'key
  (make-instance 'thing
                 :name 'key
                 :description "a small brass key"))

(register-thing 'lamp
  (make-instance 'thing
                 :name 'lamp
                 :description "an oil lamp, dusty but intact"))

(register-thing 'chest
  (make-instance 'container
                 :name 'chest
                 :description "a wooden chest."
                 :contents '(crown gem)))

(register-thing 'crown
  (make-instance 'treasure
```

```
                :name 'crown
                :description "a tarnished crown."
                :value 100))
```

The `describe-thing` generic now has three methods, one per class, and dispatch is automatic. Add a game command that uses it:

```
(defun examine (sym)
  "Handle (EXAMINE THING): describe THING."
  (let ((instance (lookup-thing sym)))
    (if instance
        (describe-thing instance)
        (format nil "You see nothing special about the ~(~A~)."
                sym))))

(push '(examine . examine) *commands*)
```

Check:

```
(list (run-command '(examine key))
      (run-command '(examine chest))
      (run-command '(examine crown))
      (run-command '(examine sword)))
```

Expected:

```
("You see key: a small brass key."
 "You see chest: a wooden chest. It holds crown, gem."
 "You see crown: a tarnished crown. It's worth 100 coins."
 "You see nothing special about the sword.")
```

Four commands, four answers, and the middle two came from different methods of the same generic function — `container` and `treasure` dispatch to their own `describe-thing`, while `key` falls back to the `thing` method.

Now the payoff. Add a new kind of thing — a fragile thing, say — with its own method, without touching any existing code:

```
(defclass fragile (thing) ())

(defmethod describe-thing ((f fragile))
  (format nil "~A It looks like it would break if you dropped it."
          (call-next-method)))

(register-thing 'vase
  (make-instance 'fragile
                 :name 'vase
                 :description "a thin porcelain vase."))

(run-command '(examine vase))
```

Expected: “You see vase: a thin porcelain vase. It looks like it would break if you dropped it.”

The `fragile` method called `call-next-method` to get the base description and append its own sentence. No existing method was edited, no existing command was told about `fragile`, and the new type works throughout the game's interface immediately. That's what CLOS's open dispatch buys: *extension by addition*.

26.9. Exercises

26.1 Define a class `lamp` as a subclass of `thing` with a `lit` slot (default `nil`, accessor `lit-p`). Make one instance and check the accessor with `setf`.

26.2 Predict, then check:

```
(defclass a () ((x :reader x-of :initform 1)))
(defclass b (a) ((x :reader x-of :initform 2)))
(x-of (make-instance 'b))
```

Why is the answer 2? Which rule decides?

26.3 Write `defgeneric empty-thing-p` and two methods: anything returns `nil`; a `container` returns `t` when its `contents` is `nil`. Use `call-next-method` nowhere — explain in a comment why `:before` or `:after` would be wrong tools here.

26.4 Each snippet below is wrong. Say what's wrong, then fix it.

```
(defclass potion (thing)
  ((colour :initarg :colour)))

(defmethod drink ((p potion))
  (thing-colour p))

(make-instance 'potion :colour 'red)
; then:
(thing-description *potion*)

(defmethod describe-thing ((i thing)) ...)
; defined before (defclass thing ...) in the same file, loaded in order

(typep 'thing 'thing)
```

26.5 Write `defmethod combine` cases so that `(combine 'a 'b)` gives `(a b)`, `(combine "x" 5)` gives `"x5"`, and any other pair signals an error with a useful message. How many methods did you need, and what does that number tell you about multiple dispatch?

26.6 Add an `initialize-instance :after` method on `container` that sorts the contents alphabetically when they're given unsorted. Test it by making a `container` with `:contents '(zebra apple)` and reading the slot.

26.7 Define `defgeneric weight`, with `defmethod` on `thing` giving 1 and on `container` giving 1 plus the number of things it holds. Write `total-weight`, which takes a list of item symbols and sums their weights with `lookup-thing` and `reduce`. What happens with an unregistered symbol, and does that suggest where a Chapter 19 `handler-case` belongs?

26.8 project Write `list-things`, a command that lists every registered thing's name and description, sorted alphabetically. It uses `maphash` or `loop for ... being the hash-keys` to

walk the registry. In a comment: what does the class hierarchy buy over storing the description in the symbol's plist (Chapter 23)?

item-locations is an alist (Ch 7)

rooms is a list of room structures (Ch 13)

thing-registry is a hash table mapping symbols to thing instances (Ch 26)

commands is an alist (Ch 9, extended in later chapters)

history is a closure object (Ch 25)

save-file is a parameter

Chapter 26 exercises 26.1-26.8:

CHAPTER 27.

CLOS Deep Dive

Chapter 26 gave you working CLOS: `defclass`, slots with `:initarg`, `:reader`, `:accessor`, `:initform`, `defmethod`, `call-next-method`, `:before`/`:after`/`:around`, and `initialize-instance :after` as the constructor hook. This chapter goes the rest of the way — the parts of CLOS that distinguish reading other people's code from writing serious code: class precedence lists, `shared-initialize`, changing an object's class, method combination beyond `:around`, and what happens when no method matches.

27.1. Multiple inheritance and the class precedence list

A class can have several direct superclasses:

```
(defclass thing () ())
(defclass container () ())
(defclass portable (thing) ())
(defclass box (thing container) ())
```

When a method specialises on `thing` and another on `container`, and you call it on a `box` — which runs? CLOS's answer is the *class precedence list*: every class has a total order over itself and all its superclasses, with the class itself first, its direct superclasses next (left to right), then their superclasses, and so on. `class-precedence-list` shows it:

```
(sb-mop:class-precedence-list (class-of (make-instance 'box)))
;; => (#<STANDARD-CLASS BOX> #<STANDARD-CLASS THING>
;;     #<STANDARD-CLASS CONTAINER> ... STANDARD-OBJECT ... T)
```

The rule of thumb: *leftmost direct superclass wins ties*. So `(defclass box (thing container) ...)` makes `box` more `thing`-like than `container`-like: a method on `thing` beats a method on `container` for a `box` instance. Order in the superclass list is meaningful — put the "primary nature" first.

Method selection uses this list: the most specific applicable method — the one whose specialiser appears earliest in the instance's precedence list — runs first. `call-next-method` invokes the next one down. This is why Chapter 26's advice — "`:before` and `:after` for side effects, primary methods for the value" — works: the full method combination runs, in order, *all* applicable `:around`, then the most specific primary flanked by all `:before` and (in reverse) all `:after`.

27.2. shared-initialize: the real constructor chain

Chapter 26 used `initialize-instance :after` as the initialisation hook. The full story: `(make-instance 'box :x 1)` runs

```
make-instance
  → allocate-instance (fresh, empty slots)
    → initialize-instance (with the :initargs)
      → shared-initialize (fills slots from :initargs and :initforms)
        → :after methods on shared-initialize
      → :after methods on initialize-instance
```

`shared-initialize` is the generic function that actually fills slots from `:initargs` and `:initforms` — and it's called both by `initialize-instance` (fresh objects) and by `reinitialize-instance` (existing objects, re-run with new arguments):

```
(reinitialize-instance *my-lamp* :lit t)
(lit-p *my-lamp*)   ; => T — slot updated
```

The practical consequence, and the reason libraries document "`:initarg` means it's reinitialisable too": any slot you expose via `:initarg` is settable both at creation *and* at `reinitialize-instance` time, by the same keyword. If you want an option that only makes sense at birth, don't give the slot an `:initarg` — set it in `initialize-instance :after` from a different slot or argument.

The `:after` method on `initialize-instance` remains the idiomatic constructor hook (Chapter 26's derived-slot pattern); `:after` on `shared-initialize` additionally fires on reinitialisation — use it when derived slots must track re-init too.

27.3. change-class

An object's class isn't fixed for life:

```
(defclass caterpillar (thing) ())
(defclass butterfly (thing)
  ((days-flown :initform 0 :accessor days-flown)))

(defparameter *bug* (make-instance 'caterpillar
                                   :name 'bug))
(change-class *bug* 'butterfly)
(typep *bug* 'butterfly)   ; => T — same object, new class
(days-flown *bug*)         ; => 0 — new slot, :initform
```

`change-class` keeps the object's identity: every reference to it now sees a `butterfly`. Slots shared by name carry over; new slots get their `:initforms`. The `update-instance-for-different-class` generic function is the hook for migration logic (copying, transforming, defaulting).

Used sparingly — the classic fits are state machines (a connection going `connected` → `closed`) and interactive systems where an object's role genuinely changes mid-session. When in doubt, a status slot beats a class change: cheaper to print, save and query.

27.4. Method combination

Chapter 26's `:before`/`:after`/`:around` is *standard method combination*. The primary-method-plus-auxiliaries shape generalises: other combination types run *all* applicable primary methods in most-specific-first order and combine the results:

```
(defgeneric price (thing)
  (:method-combination +))

(defclass treasure-with-surcharge (treasure) ())

(defmethod price + ((t2 thing)) 1)         ; base cost
```

```
(defmethod price + ((tr treasure)) (value tr))

(defmethod price + ((ts treasure-with-surcharge)) 10)   ; extra fee

(price (make-instance 'treasure :value 100))
;; => 101 — 100 from the treasure method, 1 from thing's
(price (make-instance 'treasure-with-surcharge :value 100))
;; => 111 — both of the above, plus the surcharge
```

```
(defclass base-thing () ((base-value :initarg :base-value
                                     :initform 1
                                     :reader base-value)))

(defclass valuable (base-thing) ())

(defclass taxed (base-thing) ())

(defclass antique (valuable taxed) ())

(defgeneric price (object)
  (:method-combination +))

(defmethod price + ((o base-thing)) (base-value o))

(defmethod price + ((o valuable)) 10)
(defmethod price + ((o taxed)) 15)

(price (make-instance 'base-thing :base-value 20))
;; => 20
(price (make-instance 'valuable :base-value 20))
;; => 30
(price (make-instance 'antique :base-value 20))
;; => 45
```

That's the payoff: no `call-next-method` discipline required, and *every* applicable method contributes. The standard combinations mirror the arithmetic and list operators: `+`, `max`, `min`, `list`, `append`, `and`, `or`, `progn` (`max` is a favourite for "the best answer among all applicable methods"). `:around` methods still work with any combination, wrapping the whole combined call.

For the `antique` case, all three applicable methods run: `base-thing`'s gives 20, `valuable`'s gives 10, `taxed`'s gives 15, and `+` sums to 45. Order doesn't matter for `+` (it's commutative); order *does* matter for `list` (the list comes in most-specific-first order) and `append` (same).

Custom combinations via `define-method-combination` exist and are genuinely rare — know they're there; reach for `:around` plus a helper function first.

27.5. no-applicable-method and friends

What happens when a generic function is called with arguments no method matches? A `no-applicable-method` generic function is invoked — which by default signals an error naming the generic function and the arguments. You can define a method on it:

```
(defmethod no-applicable-method (gf &rest args)
  (format *error-output* "~A has no method for ~S~%"
          (generic-function-name gf) args)
  (invoke-restart (find-restart 'abort)))
```

— though in application code, silence is rarely the answer; the default error is usually right. The related hooks worth knowing by name only: `no-next-method` (a `call-next-method` with nowhere to go) and `compute-applicable-methods` (the dispatcher itself, specialisable for exotic effects you'll read before you write).

Also worth naming: `(defmethod foo :before ...)` on a generic function that doesn't exist yet *defines* it implicitly — CLOS creates the generic function. Chapter 26's `defgeneric` explicit form is preferred in source, as documentation and as the home of `:documentation` and options.

27.6. Common mistakes

27.6.1. Superclass order as decoration

`(defclass box (container thing) ...)` vs `(thing container)` differ in behaviour — precedence, method selection. If you don't care, you're likely to care once two `:before` methods both apply and run in the "wrong" order. Order deliberately; comment the choice when it matters.

27.6.2. Expecting :before methods to contribute values

`:before` and `:after` results are discarded. A `:before` method that computes something must store it (a slot, a special variable) for the primary to use — or be a primary method with `call-next-method` instead.

27.6.3. change-class as an update mechanism

Using `change-class` to "refresh" objects mutates identity semantics readers rely on; it also interacts badly with `save-game`'s `print`/`read` round trip (a changed class must exist at load time, in the expected shape). Prefer `change-class` for genuine role transitions, slots for status.

27.6.4. Method combination confusion: why won't my :after add to the result?

With `+` combination, `:after` methods still contribute nothing; the combination applies to *primary* methods only. The fix is usually: make the secondary behaviour a primary method (possibly on a more general class specialiser).

27.6.5. Two primary methods on the same class specialiser

You can't:

```
(defmethod price + ((tr treasure)) 1)
(defmethod price + ((tr treasure)) 2)   ; redefinition, not addition
```

The second *replaces* the first — same class, same combination type, one method per generic-function-per-class. If you want two contributions, you need two different classes (as in Chapter 27's `valuable` and `taxed` above), or one method that returns the sum of both internally.

27.7. Project step: pricing the inventory

Give the game a scoring economy, via `+` method combination — the natural fit, since an item's worth is the *sum* of contributions from every class it belongs to.

The game's `thing`, `container` and `treasure` hierarchy from Chapter 26 is the base; this chapter adds a scoring layer on top. The classes exist; now give them a price:

```
(defgeneric price (thing)
  (:method-combination +)
  (:documentation "How valuable is THING, summing contributions?"))

(defmethod price + ((t2 thing))
  1)   ; every thing is worth at least 1

(defmethod price + ((c container))
  2)   ; containers are worth a bit more

(defmethod price + ((tr treasure))
  (value tr))   ; treasures are worth their slot
```

Now the game's inventory total uses them:

```
(defun inventory-score ()
  "Total value of everything the player carries."
  (reduce #'+ (mapcar #'price (items-here 'player))))

;; where items-here returns symbols, and we look up each thing:
(defun inventory-score ()
  (reduce #'+ (mapcar (lambda (sym)
                        (price (lookup-thing sym)))
                      (items-here 'player))))
```

The first version is wrong — `items-here` returns symbols, not `thing` instances, so `(price 'key)` would try to dispatch on a symbol and fail (no method for `symbol`). The second version does the lookup.

Check:

```
(setf *item-locations* '((key . player) (crown . player)
                         (lamp . kitchen)))
(inventory-score)
```

Expected: `+ 1 100 = 101` — the key is a plain `thing` (worth 1), the crown is a `treasure` (worth 100), the lamp isn't carried.

Now the payoff: add an `antique` class combining several contributions, without touching any existing method:

```
(defclass antique (treasure) ())

(defmethod price + ((a antique))
  50)   ; antiques command a premium

(register-thing 'mirror
  (make-instance 'antique
                 :name 'mirror
```

```
                :description "a tarnished silver mirror."
                :value 200))

(setf *item-locations* (acons 'mirror 'player *item-locations*))
(inventory-score)   ; => 101 + 250 = 351
```

The `+` combination earns its keep the moment a new class joins: `defmethod price + ((a antique)) 50` adds its contribution to every antique without touching any other method — the open/closed behaviour Chapter 26 promised, finally in its purest form. The antique's price is 200 (from treasure) + 50 (from antique) + 1 (from thing) = `251`, not `200 + 50 = 250` — the `thing` method contributes too, because every antique is a thing.

That's the difference between `+` combination and inheritance-of-behaviour: inheritance gives the most `specific` method (200 for the crown's treasure method, say); combination gives the sum of all `applicable`. The two styles are both right; which one depends on whether the contributions are alternatives (`treasure` vs `container` are alternative kinds) or parts (base cost + premium + fee).

27.8. Exercises

27.1 Predict the class precedence list of `(defclass d (b c) ...)` given `(defclass a () ...)`, `(defclass b (a) ...)`, `(defclass c (a) ...)`. Check with `class-precedence-list`. Which class's methods win for a `d` instance, `b`'s or `c`'s?

27.2 Each snippet is wrong or surprising. Say what happens, then fix it.

```
(defclass widget () ((x :initarg :x :reader widget-x)))
(defmethod describe-widget :after ((w widget))
  (format nil "x is ~A" (widget-x w)))

(defclass base () ((n :initform 1 :accessor n)))
(defclass sub (base) ())
(defmethod n :after ((s sub))
  (incf (slot-value s 'n) 10))

(defgeneric area (shape))
(defmethod area ((s shape)) 0)
```

27.3 Write `describe-chain`, which takes an instance and prints each class in its precedence list with any `:before`, `:after` and primary `price` methods specialising on it. Use `apropos` or `find-method` as needed — or, simpler, keep a manual registry for this exercise. What does the output teach about why method order surprises happen?

27.4 Define a `list` method combination analogue: a `participants` generic function whose primary methods each return one participant name, combined into a list. Then add an `:around` method that records how long the full combination took (Chapter 22's `get-internal-run-time`). Where must the `:around` method specialise to wrap *every* call?

27.5 Write `safe-change-class`, which takes an instance and a new class name, verifies with `class-precedence-list` that no slots would be dropped (slots on the old class absent from the new), and refuses with a condition listing the dropped slot names. What does the refusal check teach about `change-class`'s slot policy?

27.6 project Give the game's rooms a `+` combination `weight` generic function — `room` contributes its exit count, `dark-room` adds a penalty, `visited-room` (a mixin) adds a discount — and a `safest-exit` helper that picks the exit whose destination has the lowest `weight`. In a comment: what makes `+` the right combination here and `max` the wrong one?

CHAPTER 28.

Packages and Symbols Internals

Chapter 16 taught the working day: `defpackage`, `in-package`, `export`, and why the game's symbols live behind `:cave-of-noise:` prefixes. This chapter goes under the hood — the package system as data structures you can query and manipulate. You need this not to write exotic code but to read it: every `shadow`, `unintern` and double-colon in someone's source is a sentence about these structures, and the name-conflict errors that interrupt your loading are questions these tools answer.

28.1. The symbol table, by hand

A package is, at bottom, a table from strings to symbols, with a list of exported names and a list of use-lists. Three functions are the whole API surface for one symbol:

- `find-symbol` — look up a string; two values: the symbol and its status (`:internal`, `:external`, `:inherited`, or `nil`).
- `intern` — look up or create; the reader's own operation.
- `unintern` — remove a symbol from the package (the symbol object itself survives wherever it's referenced; it's just no longer *findable by name* `here`).

```
(find-symbol "CAR" :cl)           ; => CAR, :EXTERNAL
(find-symbol "car" :cl)           ; => NIL, NIL — table is case-sensitive
(intern "THING")                  ; creates in *package* if absent
(multiple-value-bind (sym status)
    (find-symbol "LOOK" :cave-of-noise)
  (list sym status))              ; => (LOOK :EXTERNAL) if exported
```

`do-symbols`, `do-external-symbols` and `do-all-symbols` walk the tables:

```
(defun external-names (package)
  "Sorted list of a package's external symbol names."
  (let ((names nil))
    (do-external-symbols (s package)
      (push (symbol-name s) names))
    (sort names #'string<)))
```

`do-external-symbols` walks just the exports; `do-symbols` walks everything interned (with each symbol visited *twice* if the package both inherits and imports it — an implementation-dependent corner); `do-all-symbols` walks every package. The fourth macro, `with-package-iterator`, is the programmatic one you'll meet in library code that wants to produce a lazy stream of symbols rather than iterate.

28.2. export, import, and the two-way door

`export` puts a symbol's name in the package's external list. What's less obvious: `export` on a symbol the package doesn't home first imports it — the machinery is about tables, not about "publishing". The round trip:

```
(defpackage :demo (:use :cl))
(in-package :demo)

(intern "TOOL")            ; home, internal
(export 'tool)             ; now external
(unintern 'tool)           ; gone from the table entirely
(find-symbol "TOOL" :demo) ; => NIL, NIL
```

`import` puts an existing symbol `object` into another package's table — after which `(eq 'cl-user::tool 'demo::tool)` is true: one symbol, two addresses. This differs from `shadow`, which creates a *new* symbol in the package, blank, replacing the inherited name. The distinction drives every name-conflict resolution you'll ever click through in the debugger.

28.3. Name conflicts, resolved by hand

A conflict happens when a name becomes findable two ways in one package: `use-package` over a package that exports `foo` while you've also interned your own `foo`; or `import` of a symbol whose name is taken. The system signals a correctable error whose restarts offer the resolution — Chapter 19′s restarts, working for you at load time. By hand, there are three `defpackage` options:

```
(defpackage :mine (:use :cl) (:shadow :list))
;; :LIST now names MY package's fresh symbol, not CL:LIST

(defpackage :mine2 (:use :cl)
  (:shadowing-import-from :mylib :merge))
;; :MERGE resolves in favour of MYLIB's symbol for this name

(defpackage :mine3 (:use :cl)
  (:import-from :mylib :helper))
;; bring MYLIB:HELPER in; error if the name is taken
```

Option	Effect on the conflicting name
`:shadow`	my package gets its own fresh symbol
`:shadowing-import-from`	the named package's symbol wins, and is *shadowing* — later `use-package` conflicts with it resolve silently
`:import-from`	bring the named package's symbol in (error if the table already has the name)

The shadowing concept: a package can hold a symbol that shadows an inherited one — `find-symbol` returns it, and future `use-package`'s that would conflict with it are legal, the shadowing symbol winning. That's what makes `:shadowing-import-from` the tool for deliberately overriding part of a used package.

The four `defpackage` options for bringing names in, summarised:

- `(:use pkg)` — import all of `pkg`'s exports, so they can be found by name, and any name conflict is an error at `use-package` time.
- `(:import-from pkg name...)` — import the specific `symbols name...`; error if the name is taken.
- `(:shadowing-import-from pkg name...)` — same, but the imported symbols override the current table's names; no error if the name is taken.

- `(:shadow name...)` — the package gets its own symbol for each name, deliberately unrelated to whatever that name meant elsewhere.

28.4. Nicknames, renaming, deleting

`:nicknames` in `defpackage` gives short handles — `(defpackage :cave-of-noise (:nicknames #:con) ...)` makes `con:look` legal. `rename-package` renames or re-nicknames in place (data and compiled code referencing the old name keep working — they hold the package `object`, not the string). `delete-package` removes one that's empty (no other package uses it); symbols home in it become garbage unless referenced elsewhere.

Nicknames are for typing comfort at the REPL and in qualified names — code inside the package never uses them. Renaming is for a library that changed its name and wants to keep the old one working.

28.5. Reading foreign symbols, and keyword reality

The remaining oddity is the bare `:sym` — that's not package syntax at all. `:sym` reads as the symbol named `SYM` *in the KEYWORD package* — keywords are simply the symbols home in `:keyword`, all externally exported by definition, self-evaluating because their value slot holds themselves. The whole story:

```
(eq :look (intern "LOOK" :keyword))   ; => T
(symbol-package :look)                ; => #<PACKAGE "KEYWORD">
:look                                 ; self-evaluating
```

And the inverse notations, rarely needed but met in macros and printouts: `#:sym` (uninterned, Chapter 23) and `cl-user::foo` with the `::` reaching internals.

28.6. Designing a package layout for a system

Chapter 16 split the game into files; the deeper design question is *packages per file vs one package*. The practical wisdom:

- One package per interface, not per file. `packages.lisp` defining one `:cave-of-noise` package is right for an application; a library exposing `:mylib` and `:mylib.driver` splits when the parts serve different readers.
- `(:use :cl)` always; `use` of your own other packages only where the dependency is real — every used package's exports become part of your package's unqualified namespace, which is coupling.
- Export deliberately: the export list is your API contract, and `find-symbol` can test it. A symbol exported by accident is a commitment to support it.

ASDF's `:package-inferred-system` (Chapter 20) takes the per-file-package style to its conclusion: one package per file, named after the file, used by explicit `:use` or `:import-from` lines at the top. You'll meet it reading modern libraries; the skill of *reading* `:import-from` lists as a dependency declaration is what this chapter buys.

28.7. Common mistakes

28.7.1. Fighting case in package names

`(find-symbol "car" :cl)` fails — tables hold upcased names. `(find-symbol (string-upcase "car") :cl)` is the working form; the same for `intern` and every package function taking a string designator. The reader upcases names; the table stores them upcased; string comparisons must too.

28.7.2. :use lists as decoration

Long `:use` lists invite conflicts and hide provenance: when you read `merge` in the code, which package's `merge` is it? The modern default is `(:use :cl)` plus explicit `:import-from` for the handful of foreign names you want unqualified — provenance readable at the top of the file.

28.7.3. unintern in live code

`unintern` of a symbol that functions still reference leaves those references holding a symbol findable nowhere by name — redefinitions then create a different symbol with the same name, and the old code silently diverges from the new. Unintern only in package-fiddling sessions, never in program logic.

28.7.4. Treating keywords as "just strings"

Keywords are symbols: they intern (a table grows with every `new` keyword), they never garbage-collect in practice, and building keyword names from user input and interning them leaks and invites conflicts. Fixed vocabularies only: keywords for options, not for data.

28.7.5. Confusing single vs double colon

`:pkg:sym` means "external symbol `sym` in `pkg`" — and signals a reader error if `sym` is internal (not exported). `:pkg::sym` means "any symbol `sym` in `pkg`, exported or not". The single colon is the strict form; the double colon is the escape hatch. Writing `:pkg::look` when `look` is exported works, but it's a lie: the reader can't tell you cared, and readers of the code can't tell the symbol's status without checking. `:pkg:look` documents "this is API"; `:pkg::look` documents "I'm reaching into internals".

28.8. Project step: the game's API surface

The game exports whatever Chapter 16 felt like exporting. This step makes the export list deliberate and testable — an API you can verify.

The package definition, refined:

```
(defpackage :cave-of-noise
  (:use :cl)
  (:shadow #:room)
  (:nicknames #:con)
  (:export :play
           :look :take :drop :inventory
           :save-game :load-game
           :run-command))
```

Nine exports, all functions a caller would reasonably want. Deliberately not exported: `*rooms*`, `*item-locations*`, `*commands*`, `find-room`, `call-command`, `describe-thing`, `*thing-registry*` — the machinery the exports rest on. Callers use the API; internals stay internal.

The query that turns "what does my package expose?" from memory into a REPL operation:

```
(defun unexported-api ()
  "Symbols the package defines but does not export."
  (let ((internal nil))
    (do-symbols (s :cave-of-noise)
      (multiple-value-bind (found status)
          (find-symbol (symbol-name s) :cave-of-noise)
        (when (and found (eq status :internal))
          (pushnew s internal))))
    (sort internal #'string< :key #'symbol-name)))
```

Check:

```
(in-package :cl-user)
(con:look)
;; => the current room's description — the nickname works

(length (unexported-api))   ; => a number, probably 30-something
(member 'run-command (unexported-api))   ; => NIL — it's exported
(member '*rooms* (unexported-api))       ; => (*ROOMS*) — internal
```

Running `unexported-api` after a long development session routinely surfaces three or four symbols you meant to keep private but accidentally exported, or symbols you've been referring to by `::` that should be part of the API. That's the audit: names that should be public are in `:export`; names that shouldn't be are internal and reachable only by `::`.

Two more tests worth adding to the audit habit:

```
(defun export-documented-p ()
  "Exported symbols lacking documentation — an audit subset."
  (let ((undocumented nil))
    (do-external-symbols (s :cave-of-noise)
      (when (and (fboundp s)
                 (null (documentation s 'function)))
        (push s undocumented)))
    (nreverse undocumented)))
```

Every exported function should have a docstring — the export list is the promise, and the docstring is the promise's terms.

And one deliberate import, the shape modern packages prefer:

```
;; hypothetical: a future cave-of-noise that uses cl-ppcre
;; for one helper
(defpackage :cave-of-noise
  (:use :cl)
  (:shadow #:room)
  (:import-from :cl-ppcre :scan-to-strings)
  (:nicknames #:con)
  (:export :play
           :look :take :drop :inventory
```

```
           :save-game :load-game
           :run-command))
```

One name imported, by name, at the top — readable as a dependency, not as a `:use` that would pull the whole of `cl-ppcre` into the namespace.

28.9. Exercises

28.1 Predict, then check:

```
(find-symbol "LIST" :cl)
(find-symbol "list" :cl)
(eq 'cl:car (intern "CAR" :cl))
(symbol-package 'cl:car)
(symbol-package 'car)              ; evaluated in CL-USER
(multiple-value-list (find-symbol "RUN-COMMAND" :con))
```

28.2 Each snippet is wrong or surprising. Say what happens, then fix it.

```
(defpackage :p1 (:use :cl) (:export :run))
(in-package :p1)
(defun run () :p1s)
(in-package :cl-user)
(use-package :p1)
run

(defpackage :p2 (:use :cl :p1))
(in-package :p2)
(defun run () :p2s)

(intern "TEMP" :p1)
(unintern 'temp :p1)
p1::temp
```

28.3 Write `where-does-name-live`, which takes a string and a package and returns, as a keyword or `nil`: `:external`, `:internal`, `:inherited` or `:absent`. Then write `find-anywhere`, which searches a package *and* its use-lists in precedence order and returns the first package in which the name is findable. What does the order tell you about how the reader resolves unqualified names?

28.4 Write `package-diff`, which takes two packages and returns three values: names external to both, names external to only the first, names external to only the second. Use it on `:cl` and `:sb-ext` (or another implementation package you have) and report the counts in a comment.

28.5 Write `with-temp-package`, a macro that creates a fresh package with a `gensym` name, evaluates a body with `*package*` bound to it, and deletes the package after — body forms intern there. Demonstrate with a body that defines a function and calls it. What does this exercise assume about `delete-package`'s requirements, and how does your macro satisfy them?

28.6 project Split the game's single package into two: `:cave-of-noise.data` (for `*rooms*`, `*item-locations*`, `save-game`/`load-game`) and `:cave-of-noise` (commands, `play`), with the

main package `:use`-ing `:cl` and `:import-from`-ing only the data names it needs. Which names did the main package actually need, and what does the final `:import-from` list tell you about where the real coupling between the halves lies?

Part IV review

Part IV was the part that looked inside. Every chapter took something you had been using on trust and showed you the mechanism underneath:

- Chapter 17 split **where a name lives in the text** from **when it lives at all**: lexical scope, dynamic binding, and the earmuffs that announce which one you are looking at.
- Chapter 18 turned the reader from a black box into a configurable stage — read syntax, case conversion, readtables, and `with-standard-io-syntax` as the rule that makes a save file mean the same thing tomorrow.
- Chapter 19 took the one-shot deal of `handler-case` apart and rebuilt error handling as `handler-bind`, the condition hierarchy, and restarts you choose from rather than catch.
- Chapter 20 named the three times — read, compiled, loaded — and made `eval-when` the ordinary consequence it is.
- Chapter 21 turned declarations into promises: a local `declare`, a global `declaim`, `(optimize ...)` as a dial, and the measurements that show what they buy.
- Chapter 22 made the REPL a workbench: the inspector, deliberate `break`s, backtraces read from the bottom, redefinition of running code, and what to do when the image itself is sick.
- Chapter 23 took a symbol apart into its five parts and drew the line that matters: plists for constants about names, owned structures for state about things.
- Chapter 24 showed that `values` are not lists — `multiple-value-bind` and `multiple-value-call`, the single-value contexts, and the `return` trap that silently eats the second value.
- Chapter 25 gave closures a job: constructors that run once, shared environments as objects, and the habit of asking whether state belongs in a closure or in a `defparameter`.
- Chapters 26 and 27 did CLOS twice — the model, then the corners: `call-next-method`, the class precedence list, `shared-initialize`, `change-class`, and method combination as the alternative to inheriting behaviour.
- Chapter 28 opened the symbol table by hand, so that a package became an interface you design rather than a `defpackage` you copy.

The game, at the end of Part IV, is no longer a handful of rooms and a list of symbols. Items are `thing` instances registered under their symbols in `*thing-registry*`, while `*item-locations*` still says where each one is; `price` is a generic function whose `+` combination adds base value, kind and premium without touching the methods already written; `run-command` returns two values; the command history is a closure; and the package exports an eight-name API — `play`, the five commands, `save-game`, `load-game`, `run-command` — that the rest of the world is allowed to touch. Everything else is private on purpose.

If any of this feels shaky, the fix is REPL time, not re-reading: take one chapter's exercises and redo two of them from scratch. Part V takes this program out of the image — talking to other systems, generating data instead of collecting it, reading code you did not write, an executable you can hand to someone, and the tests that keep it honest.

PART V

Building Programs

Part IV gave you the mechanisms. When something in Lisp surprises you now, you have a model to reason from.

Part V puts the pieces together into a real program. It is shorter than Part IV because the concepts are already in place; what's new is the practice of *shipping*: talking to the outside world, processing data streams lazily, reading code you didn't write, wrapping the game into an executable, testing it, and knowing where the culture goes next.

The chapters:

- **Chapter 29: Interoperability** — JSON, CSV, shell commands, C via CFFI. How the game exchanges data with the world outside the image, and why the seam is thin on purpose.
- **Chapter 30: Iterators and Generators** — the closure pattern from Chapter 25 applied to streams, with the design trade-offs made explicit.
- **Chapter 31: Reading Other People's Lisp** — the audit habits, applied first to your own game, then to any Quicklisp library.
- **Chapter 32: Building Executables** — SBCL's `save-lisp-and-die`, the build script, the cross-platform story.
- **Chapter 33: Testing** — what to test, how to wire it into the build, and why the flag-vs-condition rule from Chapters 24 and 19 shapes what's worth testing.
- **Chapter 34: The Bigger Picture** — libraries, community, where to go after this book.

Every chapter uses the game as its running example. By the end, the game is a binary that runs on a machine that has no Lisp installed, exports its state to JSON and CSV, and has a test suite that refuses to let a broken build ship.

CHAPTER 29.

Interoperability

Lisp has run your whole game so far without leaving the image — but real programs talk to the world: config files in JSON, data in CSV, command-line tools, C libraries. This chapter is the passport control: shelling out, reading and writing the two data formats you'll meet most, and calling C — with an honest account of where the walls are.

The chapter's guiding principle: *the seam is thin on purpose*. Your Lisp code stays Lisp; the boundary converts between the outside world's shapes and yours in as few places as possible. Every interop layer, if it's large, is a sign the boundary wasn't thought through. Done right, the interop is one file with a handful of functions; done wrong, it's sprinkled through the codebase.

29.1. Shelling out: uiop:run-program

`uiop` (bundled with ASDF, loaded everywhere) is the portable shell:

```
(uiop:run-program '("ls" "-l") :output :string)
;; => the ls output, as one string

(uiop:run-program '("date") :output :string)
```

The argument list is *not* passed through a shell — no quoting games, no injection: each string is one argument, word for word. That's safer than the shell and it's the first thing to know. Options that matter:

```
(uiop:run-program '("grep" "noise" "log.txt")
                  :output :string
                  :error-output :string
                  :ignore-error-status t)

(multiple-value-bind (out err code)
    (uiop:run-program '("false") :output :string
                                 :ignore-error-status t)
  (list out err code))   ; exit status code is the third value
```

Without :ignore-error-status `t`, a non-zero exit signals an error — Chapter 15's `subprocess-error`, catchable like any other. For streaming output line by line instead of collecting it all, the :output `:string` version collects; for large output, stream to a file or use the callback form:

```
(uiop:run-program '("tail" "-n" "100" "cave.log")
                  :output :string)
;; just the last 100 lines, collected
```

The exact keyword for line-by-line processing has drifted across `uiop` versions — `M-.` on `uiop:run-program` and read its docstring; this is the reading skill of Chapter 31 applied within the hour. For input, :input "string" pipes it to the process's stdin.

29.2. Reading and writing CSV

CSV looks trivial and has exactly one trap: quoted fields containing commas. A quick-and-dirty split on commas breaks on real data; the library exists for the general case. For *well-behaved* CSV you produce yourself, Chapter 27′s tools suffice:

```
(defun write-csv (rows path)
  "Write ROWS (lists of strings) as naive CSV."
  (with-open-file (out path :direction :output
                            :if-exists :supersede)
    (dolist (row rows)
      (format out "~{~A~^,~}~%" row))))

(write-csv '(("room" "visits")
             ("kitchen" "3")
             ("cellar" "1"))
           "/tmp/visits.csv")
```

`~{~A~^,~}` is Chapter 12′s iteration with `~^` cutting the final comma. Reading well-behaved CSV back:

```
(defun read-csv (path)
  "Read naive CSV — no quoted commas — as a list of lists."
  (with-open-file (in path)
    (loop for line = (read-line in nil nil)
          while line
          collect (uiop:split-string line :separator ","))))
```

For CSV from the outside world, use the library: `(ql:quickload :cl-csv)`, whose `(cl-csv:read-csv file)` handles quoting, embedded newlines, and the rest. The lesson generalises: *parse formats you produce; library-parse formats you consume.*

The one shared concern worth noting: both naive CSV and cl-csv write strings, not Lisp objects. If a field can be a number or a symbol or a list, the writer must decide how to render it (`princ-to-string`? `write-to-string`? something schema- specific?) and the reader must decide how to parse it back. That's the schema problem, and CSV doesn't solve it for you.

29.3. Reading and writing JSON

JSON's shapes map onto Lisp's — objects to alists (or plists, or hash tables, library-configurable), arrays to lists, strings/numbers/booleans to their Lisp twins. The main library is `jonathan` or `cl-json`; both degrade JSON's `true`/`false` into `t`/`nil` or keywords, and `null` into `nil` — the ambiguity you should expect at the edges:

```
(ql:quickload :cl-json)

(cl-json:decode-json-from-string
 "{\"room\": \"kitchen\", \"visits\": 3, \"items\": [\"key\"]}")
;; => ((:ROOM . "kitchen") (:VISITS . 3) (:ITEMS "key"))
```

Note the shape: an *alist with keyword keys* — Chapter 7′s `assoc` with Chapter 23′s keyword conventions, ready-made:

```
(assoc :visits **)   ; => (:VISITS . 3) — cdr gives 3
```

Encoding:

```
(cl-json:encode-json-to-string
 '((:room . "hall") (:lit . t)))
;; => {"room":"hall","lit":true}
```

Watch two walls. First, `null` vs absence: JSON distinguishes `"k": null` from missing `k`; alist decode makes both `nil` — if your consumer cares, the library has options, or you handle the edge yourself. Second, keyword interning: JSON keys become keywords, so keys from *user data* intern symbols (Chapter 23′s keyword warning — fixed vocabularies only). For a save file talking to a web client, define the schema first, decode against it.

`jonathan` is the modern alternative and worth knowing:

```
(ql:quickload :jonathan)
(jonathan:parse "{\"a\": 1, \"b\": [true, null]}")
;; => #<HASH-TABLE :TEST EQUAL ...>
```

Jonathan decodes objects to hash tables by default (which preserves `null` distinctly as `+null+` — a sentinel — rather than collapsing it to `nil`), and has options for alist or plist output. The choice between the two libraries is habit and use-site; both are good.

29.4. Calling C: CFFI

The C foreign-function interface is the one Quicklisp dependency every implementation supports:

```
(ql:quickload :cffi)
```

The pattern, complete for a real (POSIX) function:

```
(cffi:define-foreign-library libc
  (:unix (:or "libc.so.6" "libc.so"))
  (t (:default "libc")))

(cffi:use-foreign-library libc)

(cffi:defcfun ("getpid" c-getpid) :int)

(c-getpid)   ; => the process id, as a Lisp integer
```

Three forms, one each for: where the library `is` (`define-foreign-library`, with per-platform alternatives — Chapter 18′s `#+` done by the library), *load it* (`use-foreign-library`), and one C function's `signature` (`defcfun`: C name, Lisp name, return type). Types map the languages: `:int`, `:double`, `:string` (converted automatically, copy in each direction), `:pointer` for the rest.

Structs and pointers, one step deeper:

```
(cffi:defcstruct timeval
  (tv-sec :long)
  (tv-usec :long))

(cffi:with-foreign-object (tv '(:struct timeval))
  (setf (cffi:foreign-slot-value tv '(:struct timeval) 'tv-sec) 0)
  (cffi:foreign-slot-value tv '(:struct timeval) 'tv-sec))
;; => 0
```

`with-foreign-object` is `with-open-file`'s shape for raw memory: allocate on the stack for the body's duration, free automatically. Memory you allocate yourself with `cffi:foreign-alloc` you must `foreign-free` — the resource discipline of every `with-` form, minus the automation.

An honest boundary note: CFFI is readable after this chapter but serious C interop (callbacks, memory management across the boundary, threads) is its own book chapter in other books. The realistic division of labour: call a C library for something Lisp lacks (a hardware binding, a fast codec), keep your logic in Lisp, and treat the FFI layer as a thin, tested seam — a dozen `defcfun` lines in one `ffi.lisp` file, never scattered.

29.5. When to leave the image, and when not to

The game's needs so far have been satisfied in-image, and most programs' are. The decision guide:

- *Text and data formats* — Lisp-native for producing; libraries for consuming (CSV, JSON above).
- *Other programs' functionality* — `uiop:run-program` for batch tools, but remember every shell-out is a fork and a parse boundary: fine for `gzip`, wrong for a per-item loop.
- *C libraries* — only for capability Lisp lacks, and via Quicklisp-wrapped bindings first: check Quicklisp for `cl-sqlite`, `cl-postgres`, `cffi` bindings before writing your own.
- *Performance* — Chapter 21 first; the FFI is not the escape hatch beginners hope: crossing the boundary costs conversion, and SBCL compiles Lisp to machine code already.

29.6. The three failure modes of an interop layer

If you take one mental model from this chapter, take this one: interop fails in three distinct ways, and each wants a different defensive shape.

Format mismatch. The outside world produces data in a shape your Lisp didn't expect — a CSV row with the wrong number of fields, a JSON object missing a required key, a shell command returning empty stdout. These are expected failures in the sense of Chapter 24: the caller usually knows what to do (fill defaults, skip the record, try the next file). Defend them with validate-then-use — the shape `game-state-json` uses in the project step below.

Environment failure. The file doesn't exist, the command isn't installed, the library isn't on the PATH. These are Chapter 15 conditions: catchable, and the right answer is almost always to surface them with a specific condition type so the caller can distinguish "missing save file" from "corrupt save file".

World inconsistency. The bytes on disk or on the wire aren't what the format says they should be — a truncated JSON document, a CSV with an unterminated quote, a response that times out mid-stream. These are the hardest, because the format's own parser usually signals a very generic error that doesn't say *where* it gave up. The reader's `reader-error` is the canonical example.

Defensive interop catches these at the seam with a translated, informative condition — never lets them escape as a bare `end-of-file` in the middle of some business logic.

The project step's `import-state-json` (Exercise 29.6) shows all three: it checks field presence (format mismatch), catches file absence (environment failure), and if the JSON is corrupt, the `cl-json` parser signals a jonathan.error:- `invalid-json` or cl-json-equivalent condition (world inconsistency) that the caller translates.

29.7. Common mistakes

29.7.1. Building shell command strings

```
(uiop:run-program (format nil "grep ~A log.txt" term)
                  :output :string)     ; WRONG: through the shell,
                                       ; injection, quoting bugs
```

The list form — `'("grep" term "log.txt")` with the term as one `element` — is the safe call: no shell, no re-parsing. If your term contains spaces or special characters, the list form does the right thing; the string form does whatever the shell's parsing decides.

29.7.2. Assuming JSON round-trips

Decode then encode does not always return the original: `null` collapses, key order changes, `1.0` may become `1`, whitespace and Unicode escapes may differ. Treat JSON as an interface format with a schema, not as a serialization of Lisp data — Chapter 16′s `print`/`read` round trip is the Lisp-native serialization, and it stays inside the image for a reason.

29.7.3. Forgetting exit codes

`run-program` signals on non-zero exit by default, but a tool that prints an error and exits `zero` will pass silently. Check the output, not just the absence of an error, when the tool's failure mode is quiet.

29.7.4. CFFI string conversions in loops

Each `:string` conversion copies. Passing a Lisp string into C per iteration of a million-item loop is a million copies — restructure to pass buffers, or do the loop in Lisp.

29.7.5. Trusting input from outside the image

Every byte read from `read`, `parse-integer`, or a JSON parser is untrusted until you've validated it. The reader is particularly dangerous — Chapter 18′s `#.` executes arbitrary Lisp at read time. Bind **read-eval** to `nil` before reading anything you didn't write yourself.

29.8. Project step: the game exports its data

A real interop seam, small and useful: the visit report from Chapter 21 leaves the image as CSV, and the game's current state exchanges with a hypothetical web front end as JSON.

The CSV export: `visit-report` returns a list of (count room-symbol) lists. That's the data to write.

```
(defun export-visits-csv (path)
  "The visit report as CSV."
  (write-csv
   (cons '("room" "visits")
```

```
        (mapcar (lambda (entry)
                  (list (string-downcase
                          (symbol-name (second entry)))
                        (write-to-string (first entry))))
                (visit-report)))
  path))
```

Note the conversions: the room's symbol is downcased to a string for readability; the count is `write-to-string`'d so the CSV writer receives strings as it expects. CSV is a string-in, string-out format; the conversion lives here, at the seam.

The JSON export — and its first draft that the chapter warns against:

```
;; Wrong: tries to serialise the whole state, including
;; non-JSON-able parts.
(defun game-state-json-WRONG ()
  (cl-json:encode-json-to-string
   (game-state)))    ; game-state includes *history*, which
                     ; is a closure — JSON encoding fails
```

The problem: `game-state` (Chapters 16 and 24) returns a list containing **item-locations**, **current-room**'s name, and `*history*` — a *closure object*. JSON can't encode closures, symbols, or hash tables directly. Even before it fails, the shape of `game-state` is Lisp's serialisation format (Chapter 18′s `print`/`read`), not JSON's.

The correct shape — define a schema at the boundary:

```
(defun game-state-json ()
  "The exchangeable slice of state, as a JSON string."
  (cl-json:encode-json-to-string
   `((:room . ,(symbol-name (room-name *current-room*)))
     (:items . ,(mapcar #'symbol-name
                        (items-in *item-locations*
                                  (room-name *current-room*))))
     (:carrying . ,(mapcar #'symbol-name
                           (items-here 'player))))))
```

Check:

```
(setf *current-room* (find-room 'kitchen))
(setf *item-locations* '((lamp . cellar)
                         (key . kitchen)
                         (letter . bedroom)))

(game-state-json)
;; => "{\"room\":\"kitchen\",\"items\":[\"key\"],\"carrying\":[]}"
```

Notice what the schema *includes*: the room's name as a string, the items in that room as strings, the player's carried items as strings. Notice what it *omits*: `*history*` (closure state, not meaningfully exchangeable), the `*thing-registry*` (structured metadata, not player state), `*visits*` (session-

local heat map, not relevant to a fresh consumer), and the *internal representation* of anything (symbols become strings; the room's structure is flattened to its name).

The schema is the seam's contract. Every field in it is a promise: "this is what the game exports". Every field not in it is a deliberate omission, not an oversight. When a web front end arrives, it consumes exactly these three fields; when a second version of the game arrives, the schema is what stays compatible.

And the import seam, which the exercises build out:

```
(defun import-state-json (json-string)
  "Restore state from a JSON exchange.  Signals on bad input."
  (let* ((alist (cl-json:decode-json-from-string json-string))
         (room-name (cdr (assoc :room alist)))
         (item-names (cdr (assoc :items alist)))
         (carrying (cdr (assoc :carrying alist))))
    ;; validate before touching state:
    (unless (and room-name (stringp room-name))
      (error "Bad import: :room missing or not a string"))
    (unless (listp item-names)
      (error "Bad import: :items missing or not a list"))
    (unless (listp carrying)
      (error "Bad import: :carrying missing or not a list"))
    (let ((room (find-room (intern (string-upcase room-name)))))
      (unless room
        (error "Bad import: room ~S does not exist." room-name))
      ;; validation complete — only now touch state:
      (setf *current-room* room)
      (setf *item-locations*
            (mapcar (lambda (pair)
                      (cons (intern (string-upcase (first pair)))
                            (intern (string-upcase (second pair)))))
                    *item-locations*))
      (dolist (name item-names)
        (setf *item-locations*
              (set-location *item-locations*
                            (intern (string-upcase name))
                            room-name-sym-of room)))
      (t t))))
```

That draft got tangled in the item-location setting. The honest version separates the schema fields from the storage:

```
(defun import-state-json (json-string)
  "Restore state from a JSON exchange.  Signals on bad input."
  (let* ((alist (cl-json:decode-json-from-string json-string))
         (room-name (cdr (assoc :room alist)))
         (item-names (cdr (assoc :items alist)))
         (carrying (cdr (assoc :carrying alist))))
    ;; validation first — see Exercise 29.6
    (unless (and room-name (stringp room-name))
      (error "Bad import: :room missing or not a string"))
```

```
    (unless (and (listp item-names)
                 (every #'stringp item-names))
      (error "Bad import: :items missing or malformed"))
    (unless (and (listp carrying)
                 (every #'stringp carrying))
      (error "Bad import: :carrying missing or malformed"))
    (let ((room (find-room
                 (intern (string-upcase room-name)))))
      (unless room
        (error "Bad import: room ~S does not exist." room-name))
      ;; mutation phase:
      (setf *current-room* room)
      (dolist (name item-names)
        (setf *item-locations*
              (set-location *item-locations*
                            (intern (string-upcase name))
                            (room-name room))))
      (dolist (name carrying)
        (setf *item-locations*
              (set-location *item-locations*
                            (intern (string-upcase name))
                            'player)))
      t)))
```

The shape is what matters: *validate every field before mutating any state*. A partially-applied import (state half-changed, then a validation error) leaves the game in an inconsistent state that no `handler-case` can recover from cleanly. Validation-then-mutation makes the mutation phase atomic in practice: every `error` fires before the first `setf`.

The intern `(string-upcase ...)` pairs are the boundary's case-and-package conversion, stated plainly where they belong: in the seam, once. If a future version of the game changes the symbol names (a rename, a pluralization), this is the one place to update.

Round-trip check:

```
(defun json-roundtrip-test ()
  (let ((json (game-state-json)))
    (setf *current-room* (find-room 'bedroom))
    (setf *item-locations* '((letter . bedroom)))
    (import-state-json json)
    (list (room-name *current-room*)
          (items-in *item-locations* 'kitchen))))
```

Expected: `(KITCHEN (KEY))` — the room and the item back where the JSON said they were.

29.9. Exercises

29.1 Predict, then check:

```
(uiop:run-program '("echo" "hello world") :output :string)
(uiop:run-program '("echo" "a;b|c") :output :string)
```

```
(cl-json:decode-json-from-string "[1, 2, [3]]")
(cl-json:decode-json-from-string "{\"a\": null, \"b\": 0}")
```

29.2 Each snippet is wrong or surprising. Say what happens, then fix it.

```
(uiop:run-program (format nil "ls ~A" dir) :output :string)

(cl-json:decode-json-from-string
 "{\"key\": \"x\"}")
(assoc "key" *)

(uiop:run-program '("gzip" "missing.txt") :output :string)

(defun json->state (json)
  (setf *current-room*
        (find-room (cdr (assoc :room json)))))
```

29.3 Write `csv->json`, which reads a CSV file (first row headers) and returns its JSON string form: each row an object keyed by the headers, numeric-looking fields as numbers. Say in a comment what your naive number test misses, and when `cl-csv`'s parser becomes necessary.

29.4 Using `cffi:defcfun` against libc: bind `abs` (one `:int`, returns `:int`) and `getenv` (one `:string`, returns `:string`), call both, and explain what `getenv` returning a `:null-pointer` means and how `cffi:null-pointer-p` guards it. Why does `abs` need no memory management while a hypothetical `malloc` binding would?

29.5 Write `run-and-parse`, which shells out to a command of your choice that emits one JSON object per line (or simulate one with a helper script), decodes each line, and returns a list of the decoded alists. What does the line-at-a-time design buy over decoding one giant array — and when does it lie?

29.6 project Add `import-state-json`, the inverse seam to `game-state-json`: decode, validate each field (room exists in `*rooms*`, items exist in the registry), and apply it to the live state — signalling a `bad-import` condition (Chapter 19) naming the first invalid field when validation fails, *before* any state is touched. In a comment: why is validate-then-apply the only safe order here, and which Chapter 24 pattern does the "first invalid field" return value use?

CHAPTER 30.

Iterators and Generators

Chapter 25 built closures with private state. Chapter 24 made functions return several values. Put the two together and you get an idea Lisp handles without any library: a *generator* — a function that produces a stream of values one at a time, remembering where it left off. This chapter builds them by hand, uses them for lazy processing, and looks at the one widespread library approach (`series`) — with an honest word about when ordinary eager lists remain the right answer.

30.1. Why generators exist

Chapter 10′s sequence functions all build complete results. `mapcar` over a million-element list allocates a million-element list before returning. `remove-if` walks the whole input before returning. `reduce` is better — it collapses to one value — but it still needs the input to exist.

For many problems that's fine: the data is already in memory, the collection fits, and eager processing reads clearly. But for three categories of problem, eager processing either doesn't fit or wastes work:

The generator pattern is the closure-based answer. It isn't the only answer (SBCL has full call/cc via `cl-cont`, and the `series` library compiles generator chains to loops), but it's the one you can build with Chapters 24 and 25 and the one every Lisp programmer should be able to write from memory.

30.2. A generator from closures and values

The simplest generator counts:

```
(defun make-counter-gen (start)
  "A generator of successive integers, starting at START."
  (let ((n start))
    (lambda ()
      (values (incf n) t))))
```

```
(defparameter next (make-counter-gen 0))
(multiple-value-list (funcall next)) ; => (1 T)
(multiple-value-list (funcall next)) ; => (2 T)
(multiple-value-list (funcall next)) ; => (3 T)
```

Two values, following Chapter 24′s flag pattern: the item, and "there is one". The flag is always `t` here — this generator is infinite — but the protocol requires it, because generators can end, and the caller must be able to tell.

The protocol, stated precisely:

Calling it returns *two values*: the next item, and a flag.

Flag `t` means "an item was produced".

Flag `nil` means "the generator is exhausted" — the item value is undefined (usually `nil`, but callers shouldn't read it).

Once exhausted, a generator stays exhausted — calling it again returns `(nil nil)`.

A generator over a list:

```
(defun make-list-gen (list)
  (lambda ()
    (if list
      (values (pop list) t)
      (values nil nil))))
```

```
(defparameter *g* (make-list-gen '(a b c)))
(multiple-value-list (funcall *g*)) ; => (A T)
(multiple-value-list (funcall *g*)) ; => (B T)
(multiple-value-list (funcall *g*)) ; => (C T)
(multiple-value-list (funcall *g*)) ; => (NIL NIL) — exhausted
(multiple-value-list (funcall *g*)) ; => (NIL NIL) — stays exhausted
```

`pop` on the captured binding — the generator *consumes* the list it holds, one `funcall` per step, state held nowhere but in the closure (Chapter 25′s encapsulation, again). The fifth call is the protocol's stickiness: exhausted generators don't come back to life.

30.3. map-gen: processing lazily

Chapter 10′s `mapcar` builds the whole result list before you look at any of it. A generator version transforms one step at a time:

```
(defun map-gen (fn gen)
  "A generator of (FUNCALL FN item) for each item of GEN."
  (lambda ()
    (multiple-value-bind (item ok)
      (funcall gen)
      (if ok
        (values (funcall fn item) t)
        (values nil nil)))))
```

```
(defparameter *g*
  (map-gen #'1+ (make-list-gen '(1 2 3))))
(multiple-value-list (funcall *g*)) ; => (2 T)
(multiple-value-list (funcall *g*)) ; => (3 T)
(multiple-value-list (funcall *g*)) ; => (4 T)
(multiple-value-list (funcall *g*)) ; => (NIL NIL) — exhausted
```

Composition works because generators are functions returning functions — the closure-composition habit of Chapter 25, now with a stream flowing through. Filter too:

```
(defun filter-gen (pred gen)
  "A generator of the items of GEN satisfying PRED."
  (lambda ()
    (loop
      (multiple-value-bind (item ok)
        (funcall gen)
```

```
        (cond ((not ok) (return (values nil nil)))
          ((funcall pred item) (return (values item t))))))))
```

The `loop` inside `filter-gen` keeps pulling until it finds a match or the source runs dry — laziness needs this "pull until satisfied" shape, and it's the first generator that consumes *variable* amounts of its source per step. A source of 1000 odd numbers produces zero output from this filter, and one call to the filter is 1000 calls to the source.

```
(defparameter evens
  (filter-gen #'evenp (make-counter-gen 0)))
(multiple-value-list (funcall evens)) ; => (2 T)
(multiple-value-list (funcall evens)) ; => (4 T)
```

Note the counter starts at 0, `incf` makes the first value 1, 1 is odd (filtered), 2 is even (kept) — so the first `funcall` on `evens` walks the counter through 1 and 2 before returning. If the counter were a million-element loop, that first `funcall` would do two steps. It's doing exactly the work asked.

30.4. The consumer side: for-each and collect

A generator is useless without consumers. Three, and that's the complete vocabulary:

```
(defun for-each (fn gen)
  "Call FN on every remaining item of GEN; return the count."
  (let ((n 0))
    (loop
      (multiple-value-bind (item ok)
        (funcall gen)
        (if (not ok)
          (return n)
          (progn (funcall fn item) (incf n)))))))

(defun collect-gen (gen)
  "Every remaining item of GEN, as a fresh list."
  (let ((items nil))
    (for-each (lambda (item) (push item items)) gen)
    (nreverse items)))

(defun take-gen (n gen)
  "A generator of at most N items of GEN, then exhausted."
  (lambda ()
    (if (<= n 0)
      (values nil nil)
      (multiple-value-bind (item ok)
        (funcall gen)
        (if ok
          (progn (decf n) (values item t))
          (values nil nil))))))
```

`for-each` consumes the whole generator, calling `fn` on each item. `collect-gen` is the bridge back to eager lists — process lazily through a pipeline of `map-gen`/`filter-gen`, then `collect-gen` at the

end when you genuinely want everything. `take-gen` is the opposite of `filter-gen`'s "consume until satisfied" — it stops asking after `n` items, closing the pipeline from the consumer side.

The pipeline:

```
(collect-gen
  (filter-gen (lambda (c) (member c '(#\a #\e #\i #\o #\u)))
    (map-gen #'identity
      (make-list-gen
        (coerce "caveofnoise" 'list)))))
;; => (#\a #\e #\o #\o #\i #\e)
```

Nothing but one element ever exists at a time between the stages — for a 12-character string that's a nicety; for a stream of a million lines it's the whole design.

30.5. Generators from thin air: infinite sources

`make-counter-gen` is already infinite. The point of lazy processing is that infinite sources are usable when consumers can stop early:

```
(defun first-few (gen n)
  (collect-gen (take-gen n gen)))

(first-few (filter-gen #'evenp (make-counter-gen 0)) 5)
;; => (2 4 6 8 10) — from an infinite stream, finite work
```

The `take-gen` closure shares `n` with itself only — a private countdown. Combined, these small functions give you *streams* in the full sense: sources (from lists, counters, or eventually I/O), transformers, takers, collectors.

The test of understanding: trace `(first-few ... 5)`. `take-gen` produces up to 5. Each `funcall` on `take-gen` `funcall`'s `filter-gen`, which walks `make-counter-gen` until it finds an even number. The counter goes 1 (skip), 2 (keep), 3 (skip), 4 (keep), …, through 10. Ten `funcall`'s on the counter, five on the take, one `collect-gen` at the end. No more work is done than the answer requires.

30.6. Generators over the game: a file source

The natural game source is the transcript — Chapter 11′s command log — processed as a stream rather than held as a list. And the natural extension is a generator over a *file*, streaming lines without holding the file in memory:

```
(defun make-line-gen (path)
  "A generator of lines from the file at PATH, with :close."
  (let ((in (open path :if-does-not-exist nil)))
    (lambda (message)
      (ecase message
        (:next
          (if (null in)
            (values nil nil)
            (multiple-value-bind (line eof-p)
              (read-line in nil nil)
              (if eof-p
```

```
                (progn (close in) (setf in nil)
                  (values nil nil))
                (values line t)))))
        (:close
          (when in (close in) (setf in nil)))))))
```

The generator takes a message — `:next` or `:close` — instead of zero arguments. That's a protocol extension: most generators are zero-arg, but resource-owning generators need an explicit cleanup path. The `:next` message returns the protocol's two values; `:close` returns nothing useful, but it frees the resource.

The weakness of the zero-arg version is now visible: a generator that opened a file and was never driven to exhaustion would leak its stream until the image died. The `:close` message is the fix — callers who abandon a generator should `:close` it. The discipline is the same as `with-open-file`'s (`unwind-protect` guarantees), stated in protocol form instead of macro form.

The consumer that uses it:

```
(defun replay-streamed (path)
  "Replay a transcript, stopping at the first unknown command."
  (let ((gen (make-line-gen path)))
    (unwind-protect
      (loop
        (multiple-value-bind (line ok)
          (funcall gen :next)
          (if (not ok)
            (return t)
            (let ((cmd (read-from-string line)))
              (multiple-value-bind (reply known-p)
                (run-command cmd)
                (unless known-p
                  (error "Unknown command: ~A" cmd))
                (format t "~&~A" reply))))))
      (funcall gen :close))))
```

`unwind-protect` (Chapter 15) guarantees the `:close` on any exit, error included. The consumer stops early on unknown commands, and the `:close` still runs. That's the whole resource discipline in one function.

30.7. series, briefly

The `series` library (installable via Quicklisp: `(ql:quickload "series")`) is the compiled-to-eager alternative: you write *series expressions* that look like lazy pipelines, and a code-writing macro phase fuses them into a single loop with no intermediate structures at all:

```
(ql:quickload "series")
(use-package :series)

(collect
  (choose-if #'evenp (scan '(1 2 3 4 5 6))))
;; => (2 4 6) — compiled into one loop, zero intermediate lists
```

`scan` turns a sequence into a series, `choose-if` is `filter`, `collect` is the eager sink; the remarkable part is that `(collect (choose-if ... (scan ...)))` is a single loop after macroexpansion — the compiler never sees an intermediate allocation. Series is elegant, standards-adjacent (an appendix to the standard's process, never ratified), and — honestly — not where a newcomer should start.

The hand-built generators above teach the model; series optimises it; ordinary `loop` with `for ... being the elements of` covers most eager needs. Reach for series when profiling shows intermediate-list allocation mattering, or when the pipeline is complex enough that the fused-loop performance matters. Reach for hand-built generators when the source is genuinely streaming (files, sockets, incremental computation) and the pipeline is small enough to reason about directly.

30.8. Common mistakes

30.8.1. funcalling a generator expecting one value

```
(mapcar #'funcall (list g1 g2)) ; keeps primaries only
;; any exhaustion flags are dropped — and worse, one funcall
;; per generator advances it once, hidden inside mapcar
```

Generators have *effect*: every call consumes. Call them in contexts where the two values are caught, and never "innocently" (for instance inside `mapcar` over a list of them).

30.8.2. Abandoned generators holding resources

The `make-line-gen` warning restated: a generator that opened a file and is never driven to exhaustion never closes it. Either design exhaustion-guaranteed use, add an explicit `:close` message, or keep the `with-open-file` outside and generate from an in-memory chunk.

30.8.3. Laziness as decoration

Wrapping a 12-element list in `make-list-gen` then `collect-gen` adds machinery for zero benefit. Lazy processing pays when: the source is unbounded or expensive, the consumer may stop early, or intermediate allocation is measured to matter. Otherwise eager lists are the simpler, correct tool — and Chapter 10′s sequence functions remain the default vocabulary.

30.8.4. Shared generators

One generator handed to two consumers `takes turns` — each `funcall` advances the single cursor. Two independent views need two generators (re-call the constructor), or an explicit `tee` that buffers — which is a real design task, not a freebie.

30.8.5. Mixing the flag into the value

A consumer that ignores the flag — `(let ((item (funcall gen))) ...)` — sees `nil` when the generator is exhausted, and treats that as "an item whose value is nil". The bug shows up as `nil` leaking into data that should have ended the stream. *Every* `funcall` on a generator should be inside a `multiple-value-bind` with the flag checked, or inside one of the standard consumers (`for-each`, `collect-gen`, `take-gen`).

30.9. Project step: a streaming transcript replay

Chapter 11′s `transcript` ran a fixed command list eagerly. This step streams the log file: read the saved transcript (one command per line), execute lazily, stop at the first unknown command — so a long replay can be aborted without running everything.

First write a transcript file to replay:

```
(defun write-transcript (path commands)
  "Write COMMANDS to PATH as a replayable transcript."
  (with-open-file (out path :direction :output
      :if-exists :supersede)
    (dolist (cmd commands)
      (with-standard-io-syntax
        (print cmd out)))))
```

Then the source:

```
(defun make-transcript-gen (path)
  "A generator of commands from the transcript at PATH."
  (let ((in (open path :if-does-not-exist nil)))
    (lambda (message)
      (ecase message
        (:next
          (if (null in)
            (values nil nil)
            (multiple-value-bind (line eof-p)
              (read-line in nil nil)
              (if (or eof-p (zerop (length line)))
                (progn (close in) (setf in nil)
                  (values nil nil))
                (values (read-from-string line) t)))))
        (:close
          (when in (close in) (setf in nil)))))))
```

And the consumer:

```
(defun replay-streamed (path)
  "Replay a transcript, stopping at the first unknown command."
  (let ((gen (make-transcript-gen path)))
    (unwind-protect
      (loop
        (multiple-value-bind (cmd ok)
          (funcall gen :next)
          (if (not ok)
            (return t)
            (multiple-value-bind (reply known-p)
              (run-command cmd)
              (unless known-p
                (error "Unknown command in transcript: ~A"
                  cmd))
              (format t "~&~A" reply)))))
      (funcall gen :close))))
```

Check:

```
(write-transcript "/tmp/cave-replay.lisp"
  '((look) (take key) (inventory)))
(replay-streamed "/tmp/cave-replay.lisp")
```

Expected: three replies printed, and the `:close` runs at the end (whether via the loop's natural exit, an error, or an interrupt).

The design point the step teaches: once a source is a generator, *stopping early is free* — a loop that returns before exhaustion costs nothing for the unread remainder, and adding a "replay only the first N" option is a `take-gen` away. Chapter 11's eager `transcript` had to run every command because it took a list; the streaming version has no such obligation.

30.10. Exercises

30.1 Predict, then check:

```
(defparameter *g* (make-list-gen '(a b)))
(multiple-value-list (funcall *g*))
(multiple-value-list (funcall *g*))
(multiple-value-list (funcall *g*))
(multiple-value-list (funcall *g*))

(collect-gen (take-gen 3 (map-gen #'* (make-counter-gen 1))))

(collect-gen (filter-gen (lambda (x) (> x 3))
(take-gen 2 (make-counter-gen 0))))
```

30.2 Each snippet is wrong or surprising. Say what happens, then fix it.

```
(defparameter src (make-list-gen '(1 2 3 4)))
(list (collect-gen src) (collect-gen src))

(defun lazy-all-p (pred gen)
(loop
(multiple-value-bind (item ok) (funcall gen)
(if (not ok) (return t)
(unless (funcall pred item) (return nil))))))
;; caller wants to keep checking gen afterwards

(let ((g (make-line-gen "/tmp/big.txt")))
(funcall g)) ; one line read, then g abandoned
```

30.3 Write `gen-take-while`, which yields items while `pred` holds and stops (exhausted) at the first failure — the lazy cousin of Chapter 10's `remove-if-not` truncation. Then write `gen-append`, which yields all of one generator, then all of another. What does `gen-append` of two infinite generators produce, and what does that tell you about where laziness ends?

30.4 Write `zip-gens`, which takes two generators and yields, as three values per call, their next items together with the ok flag — the protocol stretch for two-item zips. Then use it with `zip-gens` over two counters to generate the squares 1, 4, 9, 16, … *without* computing each square directly. (That's the diagonal of a multiplication table.)

30.5 Write `make-fib-gen`, an infinite generator of Fibonacci numbers, using two closure bindings and no recursion. Then write `fib-sum-below`, which sums Fibonacci numbers below a limit *without collecting any list* — the sum accumulated inside a loop that stops early. Which side of the protocol does the stopping belong to, and why?

30.6 project Write `make-room-viz-gen`: a generator that yields each room reachable from the current one, breadth-first, without repeats, using the closure-owned visited list. Walk it with `for-each` to print a map — stopping after five rooms with `take-gen`. In a comment: what does the generator version buy over Chapter 8′s recursive traversal, and what does the visited state being closure-private make *hard* (a later caller wanting to know how far the walk went)?

CHAPTER 31.

Reading Other People's Lisp

You can now write everything this guide has covered — but a great deal of your Lisp life will be *reading*: Quicklisp libraries, other people's systems, your own code from eight months ago. Library source has conventions this guide hasn't named, and a shape that's invisible until you've seen it once. This chapter is the guided tour: how real systems are laid out, what the comment conventions mean, what that `declaim` prologue is doing, and how to evaluate an unfamiliar library before trusting it.

31.1. Get a real library and open it

The tour needs a specimen. Pick a small, well-regarded one:

```
(ql:quickload :split-sequence)
(asdf:system-source-directory :split-sequence)
```

That last call returns the directory where Quicklisp unpacked the library. Open it in Emacs (`C-x C-f` then the path, or `C-x d` for dired — Appendix C covers both). Reading source in Emacs, `M-.` on any symbol jumps to its definition (with `slime` or `sly` running, and it works across the whole image — library code included); `M-,` jumps back. This one key is most of "reading Lisp in Emacs".

The reason this works: the Lisp image holds the *definitions* of every function, not just the text. `M-.` asks the running Lisp where `split-sequence` was defined, and the answer points at the file and line. That's a completely different lookup from Emacs's tag-based systems in other languages — the image is the authority.

31.2. The shape of a system

Most Quicklisp libraries look like this on disk:

```
split-sequence/
├── README.md
├── LICENSE
├── split-sequence.asd
├── package.lisp
├── split-sequence.lisp
└── tests/
└── split-sequence-tests.lisp
```

The conventions, top to bottom:

README — one screen: what it does, a usage example. If the README has no example, the tests are the example.

.asd — the `defsystem` from Chapter 16. Read it first: the `:components` list *is* the table of contents, and `:depends-on` *is* what you're pulling in.

package.lisp — the API contract: one `defpackage`, and the `(:export ...)` list is literally every name the library promises you.

one to three source files — most libraries are small. split-sequence is one file of 150 lines. cl-ppcre is large; most are not.

The proportion to internalise: *a Common Lisp library is often smaller than its documentation.* A feature you'd schedule a sprint for in Java is frequently 200 readable lines here.

31.3. Comment conventions: the four (and five) semicolons

The comment heading conventions, standardised by tradition and used across Quicklisp:

```
;;;; split-sequence.lisp
;;;
;;; Chapter-level banner: file title and purpose. Four
;;; semicolons, alone at the top (and section separations).

;;; Section heading — three semicolons, for a group of
;;; definitions that belong together.

(defun split-sequence (delimiter seq &key #| ... |#)
  ;; Two semicolons: comment on the code at the same
  ;; indentation as the form it describes.
  (let ((start 0))
    ;; ... sometimes a paragraph of rationale here
    (do ((i 0 (1+ i)))
      (#| ... |#)
      ;; One semicolon: a note on one line, running text,
      ;; often at the end of the line it explains. ; like this
      #| ... |#)))
```

So: `;;;;` file banners, `;;;` section headings, `;;` block comments beside code, `;` end-of-line asides. Emacs's auto-fill indents them by count; that's why the counts matter — `M-;` inserts the right one at point. Docstrings (Chapter 5) carry the *interface* documentation; comments carry the *why*, and the four-level convention tells you how deep in the structure you are.

31.4. The typical file prologue

Open a random library file and the first twenty lines are usually a prologue you can now read completely:

```
;;;; package.lisp
(defpackage :split-sequence
  (:use :cl)
  (:export :split-sequence :split-sequence-if
    :split-sequence-if-not))

;;;; split-sequence.lisp
(in-package :split-sequence)

(declaim (inline split-sequence-from-end))
(defun split-sequence-from-end (#| ... |#)
  #| ... |#)
```

```
(declaim (ftype (function (t t &key (:start index)
                           (:test function))
                          list)
                split-sequence))
```

Reading it with this guide's chapters: the `defpackage` contract (Ch 28), `in-package` (Ch 16), `declaim (inline)` (Ch 21), `declaim (ftype ...)` declaring a function's *type* — the `ftype` declaration you saw listed in Chapter 21 and have now met in the wild: it tells the compiler and the reader the function's signature; the compiler checks calls below against it. None of it is exotic anymore — that is the point of the tour.

Also common in prologues, now readable:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  #| ... |#) ; Ch 20

(setf (documentation 'split-sequence 'function)
  "The real docstring, set programmatically.") ; Ch 22
```

31.5. Docstring and naming idioms

Library code follows habits you can adopt:

Exported symbols all have docstrings, written as complete sentences: `"Return a list of subsequences..."`. `(documentation 'split-sequence 'function)` reads them.

Internal names carry a `marker`: `%compute-table`, `%parse`, or `nread-` — the percent prefix is the community's spelling of "private, don't call". You'll also see `package-name-` prefixes on internals from older code.

Predicates end in `-p` or `-p` of one `word` (`emptyp`, `boundary-p`); destructive functions are `n`-prefixed or warned in the docstring ("may destroy LIST").

Keywords for options, `&key` for everything tweakable, defaults in the signature — the interface style this guide's `&key` chapter became.

31.6. Reading macros you'll meet

Chapter 14 covered `defmacro`, but libraries use a small set of established macro names. Recognising them on sight is the reading skill:

`with-...` — a dynamic-scope resource or context (Ch 15′s `unwind-protect` pattern). `with-open-file` is the model; `with-standard-io-syntax` (Ch 18), and every library's `with-connection`, `with-database` — same shape.

`do-...` — an iteration protocol walking something (`do-symbols` Ch 28, `do-hash-table` in libraries).

`define-...` — adds definitions to the system (`define-condition`, and libraries' `define-command`, `define-hook`) — usually a macro writing `defun` or `defmethod` calls for you.

`assert` family and `check-type` (Ch 21) inline.

`loop` keywords you haven't used: `minimize`, `maximize`, `sum`, `count`, `thereis`, `never`, `always` — Chapter 11′s `loop` clause set, extended; all one vocabulary.

When a library macro is unfamiliar, the reflex is now mechanical: `macroexpand-1` it at the REPL with sample arguments (Ch 14) — the expansion is usually plain code you can read straight.

31.7. Evaluating a library before trusting it

A checklist, in order of cheapness:

License — usable in your context?

Dependencies — `:depends-on` in the `.asd`. Each is something you also trust and ship. (uiop and alexandria are nearly universal and effectively standard.)

Tests — does `tests/` exist? Can you run them? `(asdf:test-system :split-sequence)` works for systems that define an `:test-operation`.

Maintenance — the dists directory's dates, or the repo: stale is not dead in Lisp (stable standard, stable code), but a decade of silent issues is a signal.

Read the source — the final and most Lisp-native step. 150 lines, the whole story, and `M-.` makes it walkable.

The cultural note to carry: in Lisp, *the source is the documentation of last resort, and reading it is normal*. Hyperspec lookup (Appendix B) for the standard, `M-.` for the library — the reading loop is the same skill at both layers.

31.8. Common mistakes

31.8.1. Calling internal symbols

Reaching `(library::%parse ...)` because it does what you need couples you to an implementation detail the author may rearrange. If the function is genuinely useful, it's a missing export — fork, patch and ask upstream, or wrap defensively, but don't `::` in committed code.

31.8.2. Copying library code as if it were yours

The LICENSE file governs; MIT/BSD need the copyright notice preserved in copies. Beyond law: a small copied function becomes *your* maintenance burden; a dependency stays the upstream's.

31.8.3. Assuming stasis means abandonment

A library unchanged for years on a stable standard is often *done*, not dead. The signal to fear isn't silence — it's a tracker full of ignored runtime bugs.

31.9. Project step: reading the game as a stranger

The discipline of this chapter, applied to your own system: open `cave-of-noise` as if you'd never seen it, and audit it against the tour's checklist:

Does the `.asd` read as a table of contents?

Does `packages.lisp` hold the full API contract, and does every exported symbol have a docstring?

Are the comment conventions used consistently — `;;;;` file banners present?

Are internals marked (`%` prefix or naming convention) and reachable only via `::`?

Does `(asdf:test-system :cave-of-noise)` exist? If the guide's project has no tests file yet, write the first one:

```
;;;; tests.lisp
(defpackage :cave-of-noise/tests
  (:use :cl :cave-of-noise)
  (:import-from :cave-of-noise
                ;; state the tests read and rebind
                #:*item-locations* #:*current-room* #:*history*
                #:*save-file* #:*game-output*
                ;; functions and classes they call
                #:find-room #:item-location #:register-thing
                #:describe-thing #:make-history #:save-game
                #:load-game #:restore-state #:price #:treasure #:thing
                ;; command and data symbols: identity matters
                #:undo #:key #:kitchen #:player)
  (:export :run-tests))

(in-package :cave-of-noise/tests)

(defun run-tests ()
  "A handful of invariants, reported as a list of (name . ok)."
  (let ((results nil))
    (flet ((check (name got want)
             (push (cons name (equal got want)) results)))
      (check "item-location-roundtrip"
        (progn
          (setf item-locations '((key . kitchen)))
          (set-location item-locations 'key 'hallway)
          (item-location 'key))
        'hallway)
      (check "empty-history-safe"
        (progn (funcall history :clear)
          (funcall history :recent))
        nil))
    (nreverse results)))
```

And wire it into the `.asd`:

```
(defsystem :cave-of-noise
  #| ... |#)

(defsystem :cave-of-noise/tests
  :depends-on (:cave-of-noise)
  :components ((:file "tests")))
```

Check:

```
(load (merge-pathnames "tests.lisp"
    (asdf:system-source-directory :cave-of-noise)))
(cave-of-noise/tests::run-tests)
;; => (("item-location-roundtrip" . T)
;; ("empty-history-safe" . T))
```

The audit's real deliverable is the habit: after this chapter, you can't unsee a missing file banner or an undocumented export — in anyone's code, including the person who wrote it eight months ago.

31.10. Exercises

31.1 Load a small library from Quicklisp and, from the source directory alone, answer: how many source files, what does the `:components` list order, and what does the export list promise? Write three sentences — the library's contract — without reading any implementation file.

31.2 Each of these prologue lines is met in the wild. Say what each does and which chapter's machinery it is:

```
(declaim (ftype (function (string) list) parse-header))
(eval-when (:compile-toplevel) (pushnew :mylib features))
(declaim (declaration my-pragma))
(setf (documentation 'run 'function) "Run it.")
(in-package :cl-user)
```

31.3 Write `system-report`, which takes an ASDF system name and returns an alist: `:directory`, `:files` (source component names in order), `:depends-on`, `:license` if the system object carries it. Which of these does `asdf:system-source-directory` give you directly, and which does `asdf:system-registered-p` vs `asdf:find-system` distinguish?

31.4 Pick three exported functions from the library of Exercise 31.1 and read their docstrings with `documentation`. Rate each: example present? argument roles named? failure mode stated? Then write the docstring your `run-command` *should* have had, and compare with what it has.

31.5 Find one macro in the library you don't know. `macroexpand-1` it with plausible arguments, and read the expansion. Write three sentences on what it does, and say whether the expansion used machinery you already know (with-style cleanup, gensym, ecase dispatch) — and which.

31.6 project Finish the audit: list every symbol the game defines with neither a docstring nor an internal-name marker. For each, either write the docstring or rename to the `%` convention. Then write `undocumented-exports`, which walks the game's package and returns the exported symbols lacking function or variable documentation — the audit as a reusable query, in the spirit of Chapter 28′s `unexported-api`.

CHAPTER 32.

Building Executables

Everything so far lived inside a Lisp session. For sharing — or just for the satisfaction of double-clicking — the game can become a standalone program: a file that runs with no REPL, no Emacs, no Lisp installed on the other machine (well, almost — read on).

32.1. Images: what a Lisp session actually is

A running SBCL session is called an *image*: the whole heap of definitions, packages, data and state, alive in memory. Everything the game has been since Chapter 1 is in that image: the functions, the structures, the closure-captured state, the package layout, the loaded libraries, the current value of every earmuffed variable. That's what makes Lisp development feel like editing a living thing — you change a function and the running program uses the new version — and it's what makes deployment an interesting question. What do you *ship*?

Two answers, both from the same function: `save-lisp-and-die`.

32.2. Saving an image to disk

`sb-ext:save-lisp-and-die` writes the image to disk and exits the current process. Two forms of the result:

```
(sb-ext:save-lisp-and-die "/tmp/game.core"
:toplevel #'main)
```

This writes a *core file* — a raw dump of the image, to be loaded by an SBCL that's already installed:

```
sbcl --core /tmp/game.core
```

The core file is small-ish (a few megabytes for a stripped image, tens for one with all Quicklisp loaded) but useless without `SBCL`. Whoever runs it needs to have SBCL installed.

The other form:

```
(sb-ext:save-lisp-and-die "game"
:executable t
:toplevel #'main)
```

This writes an executable — an image with SBCL's runtime bundled in, self-contained:

```
./game # on Linux
game.exe # on Windows — SBCL writes the right kind
```

The executable is large — typically 40 to 80 megabytes — because the runtime is included. But it runs on any machine of the same architecture with no Lisp installed. That's the deployment story: *a Lisp program deployed to a user is an executable, and the executable is the whole language runtime plus your code.*

Two more `save-lisp-and-die` options worth knowing:

:compression `t` — compresses the image with zstd. The executable halves in size, at the cost of slightly slower startup. On SBCL builds with `sb-core-compression` (most modern ones), this is worth it.

:save-runtime-options `t` — remembers the runtime options SBCL was started with. Useful for tuning the executable's runtime (stack size, GC behaviour). Skip it unless you know you need it.

32.3. The entry point: `main`

The `:toplevel` function is the executable's first mover. Its job, in order:

Handle `--help`, `--version`, and any other early exits.

For the game, that's:

```
(defun main ()
  "Entry point for the deployed executable."
  (handler-case
    (progn
      (multiple-value-bind (state loaded-p)
        (load-game)
        (if loaded-p
          (restore-state state)
          (progn
            ;; no save yet — start fresh
            (setf current-room (find-room 'kitchen))
            (setf item-locations nil)
            (setf history (make-history)))))
      (play)
      (save-game) ; best-effort save on clean exit
      (sb-ext:exit :code 0))
    (error (c)
      (format error-output "cave: ~A~%" c)
      (sb-ext:exit :code 1))))
```

Read it carefully; each piece is doing something.

The `multiple-value-bind` on `load-game` is Chapter 24′s two-value contract, put to work: `loaded-p` tells us whether there was a save to adopt. When there isn't (first run), we set up fresh state.

The sb-ext:exit :code `0` at the end is essential. Without it, SBCL would fall through into its default REPL after `main` returns, and the executable would hang waiting for input. `sb-ext:exit` closes the process immediately.

The two exit codes matter for scripting and CI: 0 means "ran to completion", non-zero means "something went wrong". A shell script that launches `./game` can test `$?` and know whether the run succeeded.

32.4. Building from a script

`save-lisp-and-die` is a single Lisp call, but it shouldn't be typed at the REPL each release. It lives in a build script that the shell runs. A file `build.sh` next to the `.asd`:

```
#!/bin/sh
sbcl --noinform --non-interactive \
  --eval '(asdf:load-system "cave-of-noise")' \
  --eval '(sb-ext:save-lisp-and-die "game"
           :executable t :compression t
           :toplevel (function cave-of-noise:main))'
```

Reading the flags:

`--noinform` — skip SBCL's startup banner.

`--non-interactive` — exit after the `--eval` forms rather than dropping into a REPL.

The script's structure is a template: load the system, then save. Every release of every SBCL program looks like this in its build script. The variations are in what the `load` does (test suite before build? version bump? cleanup?) and in what options the `save` takes (`:compression`, `:toplevel`, `:executable`).

A minimal Makefile wrapper, for projects that prefer it:

```
game: build.sh
sh build.sh

clean:
rm -f game
```

That's the whole release process: `make` builds, make `clean` wipes.

One build-script discipline worth stating: build from a fresh `image`, not from the session you've been developing in. The build.sh above starts a new SBCL, which has none of your REPL experiments, redefinitions, or in-progress debugging state. That's exactly what you want — the executable should reflect the files, not the session.

32.5. Command-line arguments

A real program takes arguments: a save file path, a `--version` flag, a command to run without starting the game loop. SBCL exposes the process's arguments in `sb-ext:*posix-argv*`, a list of strings:

```
sb-ext:*posix-argv*
;; => ("/home/you/src/game/game" "--version")
```

The first element is the program's name (as invoked); the rest are the arguments, in order. A typical `main` inspection:

```
(defun main ()
  "Entry point, with argument handling."
  (let ((args (rest sb-ext:*posix-argv*)))
    (cond
      ((member "--help" args :test #'string=)
        (format t "Usage: game [--help] [--version] [--save-file PATH]~%")
        (sb-ext:exit :code 0))
      ((member "--version" args :test #'string=)
```

```
        (format t "The Cave of Noise, version 1.0~%")
        (sb-ext:exit :code 0))
      ((member "--save-file" args :test #'string=)
        (let ((path (second (member "--save-file" args
                  :test #'string=))))
          (when path
            (setf save-file path))))
      ;; fall through to game
      (t nil))
    (run-game))) ; the play/save/exit wrapper from before
```

Two things:

:test `#'string` on the `member` calls — arguments are strings, `member` defaults to `eql`, and "help" is not `eql` to "–help".

The dispatch shape (search for the flag, act, exit or fall through) is standard; it doesn't need a library. If the argument set gets complex — dozens of options, subcommands, required/optional distinctions — reach for `unix-opts` or `clingon` from Quicklisp. For a game with a save-file option and a version flag, the `cond` above is the whole argument parser.

32.6. Cross-platform notes

`sb-ext:save-lisp-and-die` :executable `t` produces a native executable for the platform it runs on. Build on Linux, get a Linux binary; build on Windows, get a Windows executable; to support both, build on both. The build is not cross-platform; the source is.

The Windows-specific concerns:

The generated `.exe` opens a console window by default. That's what you want for a game that reads typed commands; for a GUI application, `sb-ext:*save-lisp-and-die*` has a `:windows-subsystem` option (or you use a different launcher).

Absolute paths in the code should use forward slashes ("C:/Users/you/save.lisp"), not backslashes, because backslash is the escape character in Lisp strings. The filesystem accepts both.

For implementations other than SBCL, `sb-ext:save-lisp-and-die` doesn't exist. CCL has `ccl:save-application` (similar shape), ECL uses `ecl:build-program`, CLISP has `ext:saveinitmem`. If you want portability, isolate the call behind a thin function (Chapter 31′s seam discipline):

```
(defun save-executable (path toplevel)
  "Save the current image as an executable at PATH."
  #+sbcl (sb-ext:save-lisp-and-die path
    :executable t
    :compression t
    :toplevel toplevel)
  #+ccl (ccl:save-application path :toplevel-function toplevel)
  ;; etc. — one branch per implementation
)
```

The `#+` conditionals (Chapter 18) select at read time, so only the branch matching your implementation is even a program in the executable. The other branches never exist as code.

32.7. Common mistakes

32.7.1. Saving from the wrong image

`save-lisp-and-die` dumps *the current process*. If you call it from your development session — the one you've been debugging in, with experimental functions, half-loaded libraries, and a mutated game state — the executable contains all of that. The build script's fresh SBCL is the antidote: it starts clean, loads the files, saves. The executable reflects the source, not your session.

32.7.2. Forgetting `sb-ext:exit`

An executable whose `main` returns falls through to SBCL's default REPL. The program "hangs" (from the user's perspective) because it's waiting for input from a REPL they don't know exists. Every exit path in `main` needs `sb-ext:exit`, and every handler needs it too:

```
(defun main ()
(play)) ; WRONG — play returns, then REPL
```

versus:

```
(defun main ()
  (play)
  (sb-ext:exit :code 0)) ; right — play returns, program exits
```

32.7.3. Exit code 0 for failure

An executable that exits 0 after a crash is a script's worst nightmare: the shell thinks everything worked. The `handler-case` in `main` exits 1 on error — that's the distinction scripts rely on. Return 0 for "did what it should", non-zero for "didn't", and let the caller decide what to do.

32.7.4. Losing the debugger

The `handler-case` in `main` catches all errors and translates them to stderr + exit 1. That's right for a deployed executable — the user isn't a Lisp programmer, and a full debugger would be noise. But during development, you want the debugger. Two ways to have both:

Build two executables, one with `handler-case`, one without, from two `toplevel` functions. `main-production` wraps, `main-development` doesn't.

Gate on an environment variable or a command-line flag (`--debug`). `main` checks it and chooses.

For the game, `main-production` is what ships; during development, the REPL is more useful than the executable.

32.7.5. Forgetting :compression

An uncompressed executable is 60–100 MB; compressed is 30–50. The startup cost of decompression is a fraction of a second. For distribution, compress; the only reason not to is if startup latency really matters (it doesn't, for a game).

32.8. Project step: release the game

The release, end to end. The build script:

```
#!/bin/sh
build.sh — the whole build
```

```
set -e # stop on first error
sbcl --noinform --non-interactive
--eval "(asdf:load-system "cave-of-noise")"
--eval "(sb-ext:save-lisp-and-die "game"
:executable t
:compression t
:toplevel #'cave-of-noise:main)"
```

The `main` from earlier — with the argument handling, the load-game check, the play-save-exit sequence — is what `:toplevel` points at.

The release steps, in order:

sh `build.sh` builds `./game`.

That's a minimal release test: the executable exists, plays, saves, reloads, handles flags. The full test suite (Chapter 33) checks more, but this sequence catches the common deployment failures — wrong toplevel, missing save directory, unhandled error — before the executable ships.

The final polish — a version string and a welcome message:

```
(defparameter version "1.0")

(defun welcome ()
  "Print the game's opening banner."
  (format t "The Cave of Noise, version ~A.~%" version)
  (format t "Type (look) to begin, (help) for commands,~%")
  (format t "(quit) to exit.~%"))
```

And `play` calls `welcome` at its start. Version in one place, bumped per release, visible to players: the small discipline that separates programs from projects. Chapter 33's tests will read `version` too, so the version the game reports and the version the tests exercise are the same string.

32.9. Exercises

32.1 Build the smallest possible executable: a toplevel that prints "hello from a saved image" and exits 0. Build it with a script (or the two forms from a running SBCL), run it from a shell, and record its file size in a comment. Compare with `(defparameter #strong[payload] (make-array 1000000 :initial-element 0))` defined before saving: how much did the payload change the size, and why so little?

32.2 Write a `main` that takes `--help`, `--version` and –room `NAME` arguments: the last sets the starting room instead of the kitchen. Test all three from a shell. What does –room `attic` do when `attic` doesn't exist, and which chapter's tool handles it?

32.3 Each snippet is wrong. Say what happens, then fix it.

```
(sb-ext:save-lisp-and-die "game" :toplevel #'play)

(defun main ()
(format t "usage: ...")
;; and nothing else, invoked with --help
```

```
)

(sb-ext:save-lisp-and-die "x" :executable t :toplevel #'main)
(quit)
```

32.4 Write a `build.sh` that also runs the test suite before building. Add a flag to skip tests for a quick build (`--skip-tests` argument). Write the shell script and the `main` changes needed to test it.

32.5 Add a `--replay FILE` flag to `main` that reads a transcript from FILE and replays it through `replay-streamed` (Chapter

instead of starting the interactive loop. Exit 0 on success, 1 on the first unknown command. Why does this flag belong in `main` and not in `play`?

32.6 project Wrap `save-lisp-and-die` in a function `build-executable` that takes the output path, the toplevel function, and a keyword `:compress` (default `t`). Show how the function can be called from a fresh SBCL (`sbcl --eval '(build-executable "game" '"'cave-of-noise:main)`) so `build.sh` can be reduced to one `--eval`. What does this buy over the direct `save-lisp-and-die` call in the script, and what does it cost?

version is "1.0"

history is a closure (Chapter 25)

rooms is a list of `room` structures (Chapter 13)

thing-registry is a hash table (Chapter 26)

CHAPTER 33.

Testing

A program you can't test is a program you can't change. The game has grown for thirty-two chapters — features added, bugs fixed, structures converted from alists to structs to closures — and every one of those changes was an opportunity for a regression. This chapter is about catching them before your players do.

Lisp's testing story is unusually simple, and unusually native. Tests are functions; assertions are conditions; running tests is calling functions. No JUnit-style framework ceremony, no annotations, no build-tool integration needed to start. The two most-used libraries — `fiveam` and `prove` — add structure for large test suites, but the hand-rolled version from Chapter 31′s project step is already a test framework in miniature, and for the game's size it's enough to build on.

33.1. Why test at all

The question a beginner asks — "why write a test when I can just run the code?" — has a clear answer once the program grows past a page: *you can run the code once, but not a thousand times.* Tests are the answer to "does this still work?" asked without re-running everything manually.

The game has three specific reasons to test:

- *State transitions.* `take` modifies **item-locations** `undo` restores from history. Every change to either could break the other. A test that takes an item, undoes, and checks the item is back in the room is cheap and catches a whole class of regressions.
- *Format round-trips.* `save-game` and `load-game` are inverse operations; `game-state-json` and `import-state-json` are another pair. Testing that round trip in isolation catches most persistence bugs.
- *Cross-chapter invariants.* The `thing-registry` from Chapter 26 holds descriptions; the `item-locations` from Chapter 7 holds positions. A regression in one — an item registered but not tracked, or tracked but not registered — is the kind of bug that passes casual play but shows up in-game eventually. A test that lists registered things and asserts each has a location is a one-line check of the invariant.

The three reasons have a common shape: *any invariant you can state in a sentence is a test waiting to be written.*

33.2. The minimal test framework

Chapter 31′s `run-tests` is already the pattern:

```
(defun run-tests ()
  "A handful of invariants, reported as a list of (name . ok)."
  (let ((results nil))
    (flet ((check (name got want)
             (push (cons name (equal got want)) results)))
      (check "item-location-roundtrip"
             (progn
               (setf *item-locations* '((key . kitchen)))
```

```
              (set-location *item-locations* 'key 'hallway)
              (item-location 'key))
            'hallway)
      (check "empty-history-safe"
            (progn (funcall *history* :clear)
                   (funcall *history* :recent))
            nil))
    (nreverse results)))
```

Two design choices in the `check` helper are worth unpacking, because they determine what the suite can do:

Checks return `(name . ok)` pairs, not signalling `errors`. A failing check is data, not an exception. That lets `run-tests` run every check regardless of failures and report them all, instead of stopping at the first — the behaviour you want when a change broke three things at once and you want to see the whole list.

The `check` helper is a `flet` local, not a `global`. Test assertions are local to the test function; making them global means test code and library code share a namespace, and a bad test helper can shadow the very functions it's testing. `flet` keeps the scope tight.

Both choices scale: `fiveam` adds assertion types (`is-true`, `is-false`, `is-equal`, `is-error`), assertion messages, fixtures, and a runner, but the underlying model — a test is a function; a check is a comparison; results are collected — is exactly what the hand-rolled version shows.

33.3. What to test

The first discipline: *test behaviour through the public interface, not internals.* Tests that call `items-here` directly are testing the machinery; tests that call run-command '`(take key)` and check the reply are testing the behaviour players see. The two kinds of tests have different stability profiles:

- Internal tests (call `items-here`, `set-location`) break when the internals change. They're cheap to write, useful during development of one function, and should be discarded or moved into a "unit test" file when the function stabilises.
- Behaviour tests (call run-command '`(take key)`, `save-game`, `look`) are what players exercise. They break when the contract changes, which is exactly when you want to be notified.

For the game, the behaviour test set is short:

```
;; Behaviour: taking an item moves it from the room to the player.
;; Behaviour: undo restores the item to the room.
;; Behaviour: save/load round-trips the state.
;; Behaviour: unknown commands return a reply without signalling.
;; Behaviour: examine of a registered thing returns its description.
;; Behaviour: examine of an unregistered thing returns a generic line.
;; Behaviour: price of a treasure returns its value.
;; Behaviour: inventory lists carried items alphabetically.
```

Eight tests, each one a sentence. Run them after every change; if all eight pass, the game's basics are intact. That's the *regression gate* — not the full specification of the game, but the subset of it whose breakage would surprise you.

33.4. The flag-vs-condition rule applied to tests

The flag-vs-condition distinction from Chapters 24 and 19 determines which behaviours *can* be tested cleanly and which need special handling:

- *Flag-returning functions* — `run-command` (two values: reply, known-p), `save-game` (success-p, path), `load-game` (state, loaded-p) — are tested by checking both values. `(multiple-value-bind (reply known-p) (run-command '(dance)) ...)` lets the test assert "reply is a string, known-p is nil" in one go.
- *Condition-signalling functions* — `safe-divide` (if you wired the Chapter 19 version), `class-change-drops-slots` — are tested with `handler-case` inside the test:

```
(defun expect-condition (condition-type thunk)
  "Run THUNK, expecting CONDITION-TYPE.  Returns T if signalled."
  (handler-case (progn (funcall thunk) nil)
    (condition-type (c) (declare (ignore c)) t)))
```

The `(progn ... nil)` is the trick: if the thunk returns normally, the `progn` returns `nil` and the test fails because the expected condition never fired. If the thunk signals the expected type, the handler runs and returns `t`. If the thunk signals a different type, the handler doesn't match, and the condition propagates — the test fails with the wrong condition visible, not with a misleading "expected condition, got value".

- *Value-returning functions* — `item-names`, `item-phrase`, `comma-list` — are the easy case: call with an input, compare with `equal`.

The rule the three cases encode: *test the function's own contract, not what a caller does with it.* A flag-tester asserts the flag; a condition-tester asserts the condition; a value-tester asserts the value. The mismatch — testing a flag-returning function by expecting a condition — is a common beginner error, and the test's diagnostic is baffling when it fires.

33.5. Test doubles: closures over state

Some tests need to control what a function sees without changing the game's actual state. The **game-output** variable from Chapter 17 is exactly this kind of seam:

```
(defun run-command-output (command)
  "Run COMMAND with output captured; return the string."
  (with-output-to-string (s)
    (let ((*game-output* s))
      (run-command command))))
```

`run-command-output` is a test helper — not part of the game's API, just a convenience for tests that care about what was `printed` rather than what was `returned`. Testing `look` through it:

```
(check "look-mentions-room"
       (search "kitchen"
               (string-downcase
                (run-command-output '(look))))
       0)   ; SEARCH returns 0 when the substring is at the start
```

The `search` check is interesting: it returns the index of the match, so any result other than `nil` means "found". The test asserts `0` specifically because `look` should start with the room name; a `search` returning 5 would mean the name appears but not where the player expects. The assertion is stricter than "contains", which is the point — tests can encode formatting expectations that a `stringp` check can't.

The general pattern: closures (Chapter 25) make test doubles free. A function that reads from **game-output** can be tested by binding it to a string stream; a function that reads from **save-file** can be tested by binding it to `/tmp/test-save`. No mocking framework needed; the dynamic variables are the seams, and `let` is the tool.

33.6. Wiring tests into the build

Chapter 31's project step wired a small suite into the game's ASDF system. This chapter strengthens it to a real gate:

```
;; tests.asd
(defsystem "cave-of-noise/tests"
  :depends-on ("cave-of-noise")
  :components ((:file "tests"))
  :perform (asdf:test-op (op system)
             (declare (ignore op system))
             (unless (every #'cdr (cave-of-noise/tests::run-tests))
               (error "Test suite failed."))))
```

The `:perform` clause on `asdf:test-op` is what `asdf:test-system` calls. Running it:

```
(asdf:test-system "cave-of-noise/tests")
;; => either silent success, or an error naming the failed tests
```

The build script's test step becomes:

```
sbcl --noinform --non-interactive \
     --eval "(asdf:test-system \"cave-of-noise/tests\")" \
     --eval "(asdf:load-system \"cave-of-noise\")" \
     --eval "(cave-of-noise::build-executable ...)"
```

Three `--eval` forms: test, load, build. A failed test signals from `test-system`, SBCL's `--non-interactive` mode propagates the error as a non-zero exit, and `set -e` in the shell stops the script before the build runs. Three layers of gate, one intent: *no release with a red test.*

The three-layer structure is worth noting as a pattern. Each layer is independent (the shell's `set -e` doesn't know about the tests; the tests don't know about the build), and each is necessary — take any one away and the whole sequence can ship a broken binary. That's the normal shape for build automation: many small checks, each doing its part.

33.7. Growing the suite across chapters

The natural discipline is *one test per bug fixed.* When play reveals a bug — a command that returns the wrong string, a save that loses a field, an undo that leaves state half-restored — the fix is two changes:

1. Fix the code.
2. Add a test that would have caught the bug.

The second change is what makes the test suite grow into something useful. It's also the hardest habit to form, because the bug is already fixed and the test feels like afterthought. But the test's purpose isn't to prove the bug exists *now* — it's to ensure the bug *doesn't exist again*, after some future change. And every future change to that part of the code is now protected.

The game's test suite grows the same way. If Chapter 24′s two-valued `run-command` broke something that a caller relied on, the test that catches it would be added then. The suite isn't a chapter's deliverable; it's the shape the codebase takes as it grows.

33.8. When to reach for fiveam

The hand-rolled framework from Chapter 31 is enough for under about thirty tests. Beyond that, three things start to matter:

- *Better assertion messages.* `fiveam`'s `is-equal` prints a diff when it fails; the `check` helper above prints "got vs want" only if you wrote it that way. The ergonomics of failure matter when a test suite has fifty tests and you're reading the failing one.
- *Test discovery and organisation.* `fiveam` groups tests into suites, runs subsets (`run! 'my-suite`), and reports summaries. The hand-rolled version is one list of `(name . ok)` pairs; you organise manually.
- *Fixtures.* "Set up this state before every test in this suite" is a common need once tests start sharing setup code. `fiveam` has fixtures; the hand-rolled version would grow a macro for it, which is the sign you're building fiveam badly.

The transition point is when the test suite has its own structure — multiple files, shared setup, several suites. For the game at thirty-two tests, hand-rolled is fine. For the game at fifty, reach for the library.

The `run-tests` shape survives the transition: `fiveam`'s `run!` returns a similar result, and the build script's check is the same. The framework is a tool; the discipline (test the contract, one test per bug, green before shipping) is the habit, and it transfers.

33.9. Common mistakes

33.9.1. Testing internals

A test that calls `(items-here 'kitchen)` directly is testing `items-here`; a test that calls run-command '`(look)` and searches the reply is testing `look`. The second survives `items-here` being renamed, rewritten, or moved to another package. The first breaks every time. Test through the interface; save internal tests for during the development of one function, then delete them.

33.9.2. Tests that depend on order

```
(check "take-key" ...)     ; mutates *item-locations*
(check "undo-take" ...)    ; assumes the previous check ran
```

If `take-key` is skipped or `undo-take` is run alone, `undo-take` fails. The fix is to set up the state each test needs at the start of each check — a `let` or `setf` reset — so each check is independent. Tests

that depend on order are tests that pass in the suite and fail when debugged individually, which is the worst kind of debugging.

33.9.3. Silent test failures

A check that returns `(name . t)` when the value is wrong — because the comparison used `eq` where `equal` was needed, or because the "value" was a side-effect-producing form that always returns `t` — is worse than no test. The symptom: the suite says "green" and the bug ships anyway. `check`'s `(equal got want)` is doing the right comparison; using `eq` on strings or lists is the classic mistake (Chapter 10's four equalities, in a test context).

The disproof: deliberately break the code a test covers, run the test, watch it fail. If it doesn't fail, the test isn't testing what you thought. Every test suite should have at least one deliberate-break session where the developer sees the tests do their job.

33.9.4. Trusting the debugger during tests

Running a test in the REPL, watching it enter the debugger on failure, and manually inspecting state — that's normal during test development. But the suite as run by `asdf:test-system` must not enter the debugger. A test that signals an error should be caught by the framework or by an explicit `handler-case` in the test; letting the debugger open turns "test suite" into "interactive test session", and the build script hangs waiting for input.

The `check`-based framework is debugger-safe: assertions compare values and return pairs, no error signalling. Tests that use `expect-condition` are debugger-safe for the expected condition; an unexpected condition is the problem the framework must catch. Wiring `run-tests` into a `handler-case` in `test-op`:

```
(defun run-tests-safely ()
  "Run the test suite; signal on any unexpected condition."
  (handler-case (run-tests)
    (error (c)
      (list (cons "test-runner-error"
                  (format nil "Test runner crashed: ~A" c))))))
```

— so the framework catches its own failures and reports them as `(name . nil)` pairs, the same as test failures.

33.9.5. Over-testing

Tests have a cost: writing them, maintaining them, waiting for them. A test that asserts "string-upcase of "a" is "A"" is testing the language, not your code. A test that asserts `(1+ 2)` is `3` is testing SBCL. Tests are for your decisions — the invariants you chose — not for the language's. Delete any test you can't imagine ever failing.

33.9.6. Over-mocking

The game's tests use the game's actual functions with `let`-bound variables. That's the right amount of substitution: the functions are the game; the variables are the configuration. A test that replaces `run-command` with a stub to test `play` isn't testing `play` — it's testing the stub. Replace configuration (variables) when you need control; never replace the code under test.

33.10. Project step: the game's test suite

Write the eight behaviour tests, wire them into the ASDF system, and make the build script refuse to ship without them. The whole file:

```
;;;; tests.lisp
;;;;
;;;; The game's behaviour test suite.  Eight tests, each
;;;; asserting one sentence of the game's contract.

(defpackage :cave-of-noise/tests
  (:use :cl :cave-of-noise)
  (:import-from :cave-of-noise
                ;; state the tests read and rebind
                #:*item-locations* #:*current-room* #:*history*
                #:*save-file* #:*game-output*
                ;; functions and classes they call
                #:find-room #:item-location #:register-thing
                #:describe-thing #:make-history #:save-game
                #:load-game #:restore-state #:price #:treasure #:thing
                ;; command and data symbols: identity matters
                #:undo #:key #:kitchen #:player)
  (:export :run-tests))

(in-package :cave-of-noise/tests)

(defun run-tests ()
  "Run every check.  Returns (name . ok) pairs."
  (let ((results nil))
    (flet ((check (name got want)
             (push (cons name (equal got want)) results)))

      (check "take-moves-item"
             (progn
               (setf *item-locations* '((key . kitchen)))
               (setf *current-room*
                     (find-room 'kitchen))
               (let ((*game-output* (make-string-output-stream)))
                 (run-command '(take key))
                 (item-location 'key)))
             'player)

      (check "undo-restores-item"
             (progn
               (setf *item-locations* '((key . kitchen)))
               (setf *current-room*
                     (find-room 'kitchen))
               (let ((*history* (make-history)))
                 (run-command '(take key))
                 (run-command '(undo))
                 (item-location 'key)))
             'kitchen)
```

```
      (check "save-load-roundtrip"
             (let ((*save-file* "/tmp/test-roundtrip.lisp")
                   (*item-locations*
                    '((key . kitchen))))
               (setf *current-room*
                     (find-room 'kitchen))
               (save-game)
               (setf *item-locations* nil)
               (multiple-value-bind (state loaded-p)
                   (load-game)
                 (and loaded-p
                      (progn
                        (restore-state state)
                        (equal *item-locations*
                               '((key . kitchen)))))))
             t)

      (check "unknown-command-returns"
             (multiple-value-bind (reply known-p)
                 (run-command '(xyzzy))
               (declare (ignore reply))
               known-p)
             nil)

      (check "examine-registered-thing"
             (progn
               (register-thing
                'probe (make-instance 'thing
                                      :name 'probe
                                      :description "a test probe"))
               (search "test probe"
                       (multiple-value-bind (reply known-p)
                           (run-command '(examine probe))
                         (declare (ignore known-p))
                         reply)))
             (search "test probe"
                     (describe-thing
                      (make-instance 'thing
                                     :name 'probe
                                     :description "a test probe"))))

      (check "examine-unregistered-thing"
             (multiple-value-bind (reply known-p)
                 (run-command '(examine nonexistent))
               (declare (ignore known-p))
               (not (null (search "nothing special"
                                  (string-downcase reply)))))
             t)

      (check "price-of-treasure"
             ;; 42 from the treasure method, plus 1 from thing's
             (price (make-instance 'treasure
```

```
                                  :name 'test
                                  :value 42))
             43)

      (check "inventory-alphabetical"
             (progn
               (setf *item-locations*
                     '((zebra . player)
                       (apple . player)
                       (mango . player)))
               (multiple-value-bind (reply known-p)
                   (run-command '(inventory))
                 (declare (ignore known-p))
                 (not (null (search "apple, mango and zebra"
                                    (string-downcase reply))))))
             t))

    (nreverse results)))
```

Check:

```
(cave-of-noise/tests::run-tests)
;; => (("take-moves-item" . T)
;;     ("undo-restores-item" . T)
;;     ("save-load-roundtrip" . T)
;;     ("unknown-command-returns" . T)
;;     ("examine-registered-thing" . T)
;;     ("examine-unregistered-thing" . T)
;;     ("price-of-treasure" . T)
;;     ("inventory-alphabetical" . T))
```

The eight checks, and what each is doing:

- `take-moves-item` — Chapter 7′s invariant, tested through Chapter 9′s command dispatch.
- `undo-restores-item` — Chapter 14′s invariant, using the Chapter 25 closure-based history.
- `save-load-roundtrip` — Chapters 16/18/24′s round trip, tested with the Chapter 24 two-valued `load-game`.
- `unknown-command-returns` — Chapter 24′s contract: the second value is `nil` for unknown commands.
- `examine-registered-thing` — Chapter 26′s CLOS dispatch, tested through Chapter 9′s command interface.
- `examine-unregistered-thing` — the fallback path in `examine`.
- `price-of-treasure` — Chapter 27′s method combination, tested directly at the generic function.
- `inventory-alphabetical` — Chapter 10′s sort, tested at the player-visible output.

Each `progn` inside the check sets `up` the state the check needs, so no check depends on another. That's the "independent tests" discipline from the common mistakes list, applied.

The ASDF system:

```
(defsystem "cave-of-noise/tests"
  :depends-on ("cave-of-noise")
  :components ((:file "tests"))
  :perform (asdf:test-op (op system)
             (declare (ignore op system))
             (let ((results
                     (cave-of-noise/tests::run-tests)))
               (unless (every #'cdr results)
                 (error "Test suite failed: ~{~A~^, ~}"
                        (remove-if #'cdr results))))))
```

The `:perform` clause on `test-op` is what `asdf:test-system` invokes. The check: if any `cdr` is nil, the failure message lists which tests failed — same shape as `check`'s `(name . ok)` pairs, one line of code, and the caller sees the failure names, not just "tests failed".

And the build script (already shown in Chapter 32's Exercise 32.4, but for completeness):

```
#!/bin/sh
set -e
sbcl --noinform --non-interactive \
     --eval "(asdf:test-system \"cave-of-noise/tests\")" \
     --eval "(asdf:load-system \"cave-of-noise\")" \
     --eval "(cave-of-noise::build-executable \
              \"game\" #'cave-of-noise:main)"
```

Test, load, build. Every step is a gate; the script stops on the first failure. That's the release discipline: a build that can't fail the test suite isn't a build you can trust.

33.11. Exercises

33.1 Add three more checks to `run-tests`, one per category from the chapter: a state transition (`drop`), a format round-trip (`game-state-json` / `import-state-json`, if you did Chapter 29's project step), and a cross-chapter invariant (every registered thing is also in **item-locations** somewhere, OR carried OR both, OR is a 'defunct' thing kept for later — pick the invariant that holds for your game, and defend the choice in a comment).

33.2 Each of these tests is broken. Say what's wrong, then fix it.

```
(defun check (name got want)
  (push (cons name (eq got want)) results))

(defun run-tests ()
  (let ((results nil))
    (flet ((check (name got want)
             (push (cons name (equal got want)) results)))
      (check "string-upcase-of-a"
             (string-upcase "a") "A")
      (check "plus-is-associative"
             (+ 1 (+ 2 3)) (+ (+ 1 2) 3))
      (check "test-failure-detected"
             (handler-case (error "should fail")
               (error () :caught))
```

```
              :caught))))

(check "test-failure-detected" (error "should fail") nil)
```

33.3 Write `fiveam`-style `is-equal`, `is-true`, `is-error`, and `is-values` macros (or functions, for the first two) that a `fiveam`-style test can call. The macro versions add assertion messages; the function versions just return (name . ok) pairs. What does the macro version buy, and what does it cost?

33.4 Write `run-tests-with-filter`, which takes a list of test names and runs only those. What does this let you do that the full suite doesn't, and when does the answer change (as the suite grows, as the code matures)?

33.5 Take one of the game's bugs — either a real one you've found, or the deliberate `item-names` bug from Chapter 22′s Exercise 22.7 — and add a test that catches it. Deliberately break the code, run the test, confirm it fails. Then fix the code, run again, confirm it passes. That's the discipline the chapter describes: *the test proves itself by failing.*

33.6 project Convert the game's test suite to `fiveam` (or `rove`, or `prove` — pick one). The conversion should be mechanical: each `check` becomes an `is`/`is-equal` form, each test becomes a `(test NAME "docstring" ...)` form, and the `test-op` calls `(fiveam:run! 'cave-of-noise/tests)` instead of `run-tests`. What did the framework buy you — be honest: if the answer is "nothing" for a suite this small, that's a useful finding too.

CHAPTER 34.

The Bigger Picture

The game is built, tested, and shippable. This chapter is the map of what comes next — the ecosystem you can draw from, the references you'll consult, the community you'll join, and the final honest assessment of what "thinking in Lisp" actually means.

34.1. Quicklisp, properly

You've loaded libraries throughout — `fiveam`, `cl-json`, `cl-csv`, `cffi`. Quicklisp's job is larger than "download libraries": it's the curated distribution, updated monthly, with a stability guarantee that makes the ecosystem work. The commands:

```
(ql:system-apropos "json")   ; find systems by substring
(ql:quickload :cl-json)      ; load (and download) one
(ql:who-depends-on :cl-json) ; who uses it?
(ql:update-dist "quicklisp")  ; update the distro index
(ql:update-all-dists)        ; update installed libraries
```

Two habits worth adopting early:

- *Pin your dependencies.* `quickload` fetches the current distro; months later, the same call fetches newer versions. When a project matters, record which distro version it used — Quicklisp's `ql-dist` can lock to a specific distro, and for shared work the heavy-duty tool is `Qlot`, which pins per-project versions in a file. Start by simply noting the distro version in your project's README.
- *Prefer many small libraries over one framework.* Lisp culture composes focused libraries rather than adopting large all-embracing ones — the same instinct as the Unix tools, and the reason Quicklisp has a dozen JSON libraries instead of one.

34.2. The ecosystem, by category

A short field guide, by task. All load with `ql:quickload`. The list isn't exhaustive — it's the "first choice" by category, the libraries you'd find in most modern Lisp projects.

Task	Library	Note
Files and paths	uiop	ships with ASDF; portable path & OS ops
Command-line args	unix-opts, clingon	typed options, subcommands, help
JSON	jonathan, cl-json	read and write JSON
CSV	cl-csv	quoted fields, embedded newlines
Regular expressions	cl-ppcre	compiled to Lisp code; fast
Web servers	hunchentoot, clack	the veteran / the server abstraction
Web framework	caveman2	routing, templates, sessions on clack
HTTP client	dexador, drakma	requests with headers, forms, SSL
HTML templating	djula, cl-who	Django-style / s-expressions
Databases (Postgres)	postmodern, mito	native protocol / ORM

Task	Library	Note
Databases (SQLite)	cl-sqlite	embedded, file-based
Threads	bordeaux-threads, lparallel	thin portable / parallel map-car
Dates and times	local-time	timestamps, formatting, arithmetic
Iteration	iterate, series	loop alternative / fused pipelines
Pattern matching	trivia, optima	destructuring `match`
Common data structures	fset, cl-containers	immutable sets / heaps, queues
Utility belt	alexandria	small correct helpers, everywhere

The pattern across all categories: small, composable, standard-loving libraries rather than frameworks that own your program. You assemble; nothing assembles you.

Two of these deserve a longer look, because they show the culture's shape.

`alexandria` is the foundation of the modern Lisp ecosystem — not because it's powerful, but because it's the opposite. A couple of hundred small functions, each correcting some tiny corner of the standard that the standard doesn't reach. `read-file-into-string`, `ensure-list`, `plist-alist`, `hash-table-keys` — none of these is a big idea; each is a small, correct tool that saves you a line. Every project starts with `(ql:quickload :alexandria)` because the cost of depending on it is zero and the savings are steady.

`cl-ppcre` is the opposite: a substantial, ambitious library that compiles regular expressions to Lisp code. You'll meet it in any Lisp program that touches text, and its API shows what the "compile to Lisp" idiom looks like in practice:

```
(ql:quickload :cl-ppcre)

(cl-ppcre:scan-to-strings "\\d+" "abc123def")
;; => "123", #("123")

(cl-ppcre:split "\\s+" "hello world of lisp")
;; => ("hello" "world" "of" "lisp")

(cl-ppcre:regex-replace "noise" "the cave of noise" "silence")
;; => "the cave of silence"
```

The `scan-to-strings` returns the matched substring and a vector of capture groups; `split` takes a regex and a string; `regex-replace` returns a new string with the matched portion replaced. Three functions that cover most text-processing needs, with no framework to learn.

34.3. The Hyperspec habit

The Common Lisp `HyperSpec` is the language standard, hypertext-browsed: every operator's full specification, every detail you've seen hedged as "implementation-defined" or "unspecified" settled in print. The habit to build: when any question survives the REPL's answers, read the Hyperspec entry.

Appendix B installed the lookup: `C-c C-d h` on any symbol opens the right page. Use it for three kinds of question:

1. /"What does this operator actually do in edge case X?" — the Hyperspec's Description and Examples sections. What `(sort nil #'<)` does, what (parse-integer "12 ") does with trailing whitespace, what `(values)` returns as distinct from `(values nil)` — all specified, all discoverable.
2. /"Is this standard, or implementation-specific?" — the Definition section says. If you're using something whose spec page you can't find, it isn't standard; you're relying on a specific implementation's extension. Chapter 18′s `#+sbcl` conditionals exist for this reason.
3. /"What conditions can this signal?" — the Exceptional Situations section names them, which connects every Hyperspec entry back to Chapter 19′s condition system. Knowing that `parse-integer` signals `parse-error` and not `end-of-file` is what makes a `handler-case` correct.

Reading the Hyperspec is a skill. The entries have a fixed shape — Syntax, Arguments and Values, Description, Examples, Exceptional Situations, Notes — and the "Side Effects" section, when present, is the one that surprises people. "Sort modifies its argument in place" is in the Side Effects of the `sort` page, which is why Chapter 10′s rule "always use the return value" exists.

The Glossary section is the standard's own index of ideas: *proper name*, *fresh*, *literal*, *stable*, *home package*. Reading those definitions once, at the point when the game has given you the context to understand them, is worth an afternoon. They are the standard's own answer to "what does it mean that Lisp works this way".

34.4. Where the reading goes

Books, in the order this guide leaves you ready for them:

- *Practical Common Lisp* (Seibel) — the classic second book, free online. Real systems (a spam filter, an HTML generator, a streaming MP3 server) built chapter by chapter. Half of it will feel like reviews of what you've just done; the other half will be new. Its introduction to macros and CLOS is gentler than this guide's; its projects are more substantial.
- *Paradigms of Artificial Intelligence Programming* (Norvig) — the classic demonstration of Lisp's style. Norvig writes Lisp the way good Lisp reads: dense, clear, idiomatic. Read the first half (through the pattern matcher), skip the AI chapters if you're not interested, come back to them if you are. The exercises are exceptional.
- *Let Over Lambda* (Hoyte) — macros at full depth. This is a hard book, and deliberately so: it treats macro writing as a discipline with its own idioms and its own pitfalls. Worth reading after you've written a dozen macros of your own and have felt where they hurt.
- *Object-Oriented Programming in Common Lisp* (Keene) — CLOS as a design language. After Chapter 26′s and Chapter 27′s working knowledge, this is the book that turns CLOS into a design tool rather than a feature.
- *The Art of the Metaobject Protocol* (Kiczales et al.) — how CLOS is built, and how to build like it. The book that changed how a generation of programmers thought about object systems. Readable when CLOS feels like home and you want to know what's underneath.

Online references worth bookmarking:

- The Common Lisp `Cookbook` — task recipes that stay current with the libraries above. When you know what you want to do and not what to call, this is the fastest lookup.
- The `CLiki` — the community wiki, with library listings and how-tos. Older and less reliable than the Cookbook, but sometimes the only place a specific library's quirks are documented.

- The Common Lisp `HyperSpec` — already installed; the habit is worth building before any book, because it's the reference you'll use for years.

34.5. The community

Lisp's community is small and old, in the way a language that's been stable for forty years is. That has consequences: documentation is patchier than Python's or Java's (because the language doesn't change, no one's racing to write the new-version tutorial), community discussions move slower (because there's less to discuss), and library maintenance can be idiosyncratic (because the libraries work, and their authors have moved on to other projects).

The consequences the other way are just as real: the libraries are stable. The language is stable. Solutions published in 1995 still work in 2025. A library you adopt today won't be abandoned by a maintainer who's "moved on to Rust" — it'll be here, tested against every Lisp implementation, in another decade.

Where to find the community:

- The `#lisp` and `#commonlisp` channels on Libera.Chat (IRC, still, after all these years — the Lisp community never left).
- `r/lisp` and `r/Common_Lisp` on Reddit, which have different cultures worth visiting both of.
- The `#lisp` Discord server (more active with newcomers).
- Various Planet Lisp-style blog aggregators, still active.
- The monthly Quicklisp release announcement (subscribe to see what's new).

The culture rewards people who read the standard, write docstrings, publish small systems, and don't make noise. You are now equipped to do all three. The first is what this chapter's Hyperspec section is for; the second Chapter 22′s rules; the third is a `ql:quickload` and a README away.

34.6. What "thinking in Lisp" means, honestly

The book's title promised that you'd learn to think in Common `Lisp`. That's a phrase that deserves unpacking, because it's easy to teach syntax and hard to teach the thing under the syntax. Here's what this guide means by it.

Thinking in Lisp isn't fluency in writing `Lisp.` A programmer fluent in Java, given a specification and a week, writes Java. A programmer fluent in Lisp, given the same, writes Lisp the way Java is written, with `loop` where Java has for-loops and `defclass` where Java has class declarations. The code compiles; the tests pass; the programmer is fluent. And the code isn't Lisp.

Thinking in Lisp is fluency in the shapes Lisp thinks `in.` When you're stuck on a problem, what you reach for — the first idea that comes to mind about how to structure the solution — that's the test. For a Java programmer it's "what class do I write?". For a Lisp programmer it's one of these:

- "What data structure represents this?" — the alist, the plist, the hash table, the structure, the class, the closure — the choice determines the function that walks it, and Lisp makes all six available.
- "What's the function that produces this from the data?" — Lisp's functional core means most problems are "transform this sequence into that one", and the sequence functions from Chapter 10 handle most cases without a loop.

- "What could go wrong, and what should the caller be able to do about it?" — the condition system's question, and its answer shapes the function's signature, its signalling, and its restarts.
- "What's the macro that would make this read better?" — the notation question, which only Lisp even asks, because only Lisp lets you answer it.
- "What's the REPL experiment that tells me what's actually happening?" — the discovery question, which replaces "reasoning about the code" with "asking the running program".

The chapters in this book taught these questions by demonstrating them: when Chapter 7 chose an alist for `*rooms*`, when Chapter 8′s `set-location` copy-and- rebuild beat `push`'s aliasing, when Chapter 17′s `*game-output*` dynamic variable replaced an argument-threading problem, when Chapter 26′s CLOS hierarchy arrived at the shape where `describe-thing` varied by class without an `if`. Each of those is a choice from the shape-vocabulary, made because the shape was the right one for the problem. That's the thinking the book was after.

You won't know whether it "took" immediately. The test comes in the next program you start, or the next non-Lisp program you read — do you reach for a class when a closure would do? Do you write a for-loop when a `mapcar` reads more clearly? Do you let a bug hide behind a comment when a condition would surface it? If yes, the shapes are becoming yours. If not, the shapes are still Java-shaped, and the language is a dialect of Java that happens to look different.

The book's whole structure was in service of this: Part I gave you the atoms (numbers, symbols, lists); Part II gave you the shapes those atoms build (conses, recursion, functions-as-values); Part III gave you the tools (loops, `format`, structures, macros, conditions, packages); Part IV gave you the why of those tools (binding, reading, compiling, types, symbols-as-data, closures, classes, packages-as-data); Part V gave you the discipline of shipping (interoperability, streaming, reading strangers' code, executables, tests).

You can now read, understand, write, and extend Common Lisp. The "grok" the guide promised in its introduction is the shapes part — the vocabulary that comes to mind when a new problem arrives. That comes with practice, not with the book.

The best thing the book can do now is suggest what to practice on.

34.7. Three suggestions, in order

1. **Extend the game past the book.** The game you've built is small — five rooms, four item types, one player. It's also *yours* in a way a book's exercise never is: you chose the room descriptions, you chose which items go where, you chose how `describe-thing` reads. Add a shop where buying and selling happen; add a monster that moves between rooms; add a save format that survives a schema change; add a room that only opens when the player carries the key. Every addition is a chapter's tools re-applied. And every one teaches something the chapters couldn't, because they had to be ordered.
2. **Write something small and new.** A backup tool, a log analyser, a text adventure generator, a budget calculator. Start with `ql:quickproject` (which scaffolds an `.asd`, a `package.lisp`, a `tests/` directory, and a README), one package, one `.asd`, one file — the Chapter 28 shape, fresh. Do the whole discipline: docstrings, tests, a build script, a README. The game taught you how; a new project teaches you why — because the shapes feel natural this time, and the code reads like Lisp instead of like Lisp-looking Java.

3. **Read one good codebase.** Pick a small Quicklisp library you use — `split-sequence`, `iterate`, `cl-ppcre` if you have patience — and read it end to end. Keep the Hyperspec open beside it. Notice how the author solves the problems you've solved, and where the solutions differ. Notice the docstrings, the comment conventions, the test suite. Reading with the standard at hand is how the remaining rough edges — yours and the language's — get polished off. Chapter 31 was a tour; this is the practice.

The three suggestions have a shape: *more game*, *new game*, *someone else's game*. That's deliberate. The first teaches extension, the second teaches inception, the third teaches continuation. Together they're the full life of a programmer in any language, and they're what the book's whole structure was pointing at.

The running image, the file, and the REPL: they were the whole method. Every chapter ended in one of them — a `C-c C-c` into a live game, a `with-open-file` to a disk, a REPL experiment that answered a question no amount of reading could. That method is Lisp's method, and it is what the book was teaching all along. Enjoy the language — it rewards exactly the curiosity that got you this far.

34.8. Exercises

34.1 Explore the distro: run `(ql:system-apropos "test")` and pick one system you've never heard of. Quickload it and skim its README (find it with `(ql:where-is-system "name")`, then look in that directory). Write one sentence on what it does and whether you'd reach for it.

34.2 Read the Hyperspec entry for a function you use but don't fully understand — `loop` or `format` or `decode-json` if you like adventure. Write down one sentence you didn't know before the reading, and say which chapter of this guide it corrects or sharpens.

34.3 Pick a library from the ecosystem table by category, load it with `ql:quickload`, and write three sentences on what it does and whether you'd reach for it in a real project. Then answer: what does the library's README lack that a project's README should have? (The answer is usually the same thing Chapter 31′s audit checklist calls out.)

34.4 Convert `item-names` from Chapter 8 to a generator that yields one name at a time. Compare with the eager version: when would you prefer each? Then convert the generator back to eager processing — what did the conversion cost in code complexity, and what did it buy?

34.5 Add a `--version` flag to the game's `main` (Chapter 32) that reports both the game's version and the Quicklisp distro version, so `./game --version` tells a user both. The distro version comes from `ql-dist:`; `M-.` on `(asdf:system-version= (asdf:find-system :quicklisp))` — find the right accessor and use it.

34.6 project Write the game's `README.md`. A one-page file that a stranger can open and run the game from. Use Chapter 31′s audit checklist as the requirement list: what the game does, a usage example, how to build it, how to run the tests. Then answer — in a comment in the README — which parts you had to re-read a chapter to write. Those are the parts of the game you don't yet understand, and they're your own "next thing to learn" list.

Part V review

Part V put the game into the world. The chapters:

- Chapter 29 built the interop seam: JSON and CSV for exchanging data, `uiop:run-program` for shelling out, CFFI for calling C — and the discipline of keeping the seam thin on `purpose`, with three failure modes (format mismatch, environment failure, world inconsistency) each defended differently.
- Chapter 30 built generators from closures and multiple values: the two-value protocol, the source/transform/consume vocabulary, the resource discipline of `:close`, and the honest boundary where laziness ends (append of two infinite sources, the value of eager lists).
- Chapter 31 taught reading as a skill: the shape of a system, the comment conventions, the prologue idioms, and the audit checklist for evaluating unfamiliar code.
- Chapter 32 shipped the executable: `save-lisp-and-die`, the build script, the entry point, the argument handling, the release test.
- Chapter 33 made the discipline concrete: test the behaviour not the internals, one test per bug, green tests gate the build.
- Chapter 34 (this chapter) is the map of what comes next: the ecosystem, the references, the community, and the closing statement about what "thinking in Lisp" means.

The game, at the end of Part V, is a standalone binary that runs on a machine with no Lisp installed, exports its state to JSON and CSV, has a test suite that refuses to ship a broken build, and is built by a script that tests before it builds. That's a program, not a collection of functions.

The appendices that follow are the reference apparatus: the installation and Emacs setup guides, the answers to the exercises, a consolidated table of the project's redefinitions across chapters, the SBCL error messages you will meet, the full source of the finished game, a glossary of the standard's own terms, and a concept index in chapter order.

APPENDIX A.

Installing SBCL

SBCL (Steel Bank Common Lisp) is the program that runs Lisp code. You type Lisp, and SBCL runs it.

A.A. Linux

Use your package manager:

```
# Debian / Ubuntu / Mint
sudo apt install sbcl

# Fedora
sudo dnf install sbcl

# Arch
sudo pacman -S sbcl
```

Check it worked:

```
sbcl --version
```

You should see something like `SBCL 2.x.x`. Any recent version is fine for this guide.

A.B. macOS

macOS needs Homebrew for the compiler, then is identical to Linux:

```
brew install sbcl
curl -O https://beta.quicklisp.org/quicklisp.lisp
sbcl --load quicklisp.lisp
```

At the `(quicklisp-quickstart:install)` prompt, answer as the Linux section says; the `ql:add-to-init-file` step writes to `~/.sbclrc` exactly as on Linux. Two macOS notes: the Homebrew SBCL is current — skip any web page telling you to build from source — and Gatekeeper may warn the **first** time you run a `sb-ext:save-lisp-and-die` executable; right-click → Open once clears it permanently.

A.C. Windows

1. Go to https://www.sbcl.org/platform-table.html and download the Windows installer (`.msi`) for x86-64.
2. Run it and accept the defaults.
3. Open a **new** Command Prompt (Win key, type `cmd`, Enter) and run:

```
sbcl --version
```

If you get "'sbcl' is not recognized", SBCL isn't on your PATH. You can still use it from Emacs by giving the full path (see Appendix B). Its location is usually something like `C:\Program Files\Steel Bank Common Lisp\sbcl.exe`. Check that folder to confirm.

A.D. Trying SBCL on its own

Type `sbcl` in a terminal. You get a prompt `*`. Type:

```
* (+ 1 2)
3
* (exit)
```

That's a REPL (Read, Eval, Print, Loop). Like Python's `>>>` prompt, but you'll mostly use it from inside Emacs.

A.E. Quicklisp (the library installer)

You won't need libraries until Part V. Installing now saves you a detour later.

1. Download https://beta.quicklisp.org/quicklisp.lisp into your home folder.
   - Linux: `curl -O https://beta.quicklisp.org/quicklisp.lisp`
   - Windows: save it from the browser into `C:\Users\YourName\`
2. In a terminal, from that folder, run `sbcl --load quicklisp.lisp`
3. At the `*` prompt, type these one at a time:

```
* (quicklisp-quickstart:install)
* (ql:add-to-init-file)
```

When it asks, press Enter. Then type `(exit)`.

From now on, SBCL loads Quicklisp automatically every time it starts.

A.F. Common mistakes

- **"sbcl: command not found" right after installing on Windows:** close and reopen the Command Prompt. Old windows don't see the new PATH.
- **Typing `exit` without parentheses:** SBCL thinks you mean a variable called `exit` and opens the debugger. Type `0` or `(abort)` and press Enter to get out, then type `(exit)`.
- **Running the Quicklisp install twice:** it complains that Quicklisp is already installed. That's harmless. It means the first run worked.

APPENDIX B.

Setting Up Emacs for Common Lisp

Your Emacs config was written for you, so step 1 finds out what you have. Then you add a small block of settings.

B.A. Step 1: Find your config file

In Emacs, press `C-h v`, type `user-init-file` and press Enter.

A help window shows a file path. Note it down:

- Path contains `doom` → you use **Doom Emacs**. See "If you use Doom" below.
- Path contains `spacemacs` or the file is `.spacemacs` → see "If you use Spacemacs" below.
- Anything else (often `~/.emacs.d/init.el`, `~/.config/emacs/init.el` or `~/.emacs`) → plain Emacs. Follow step 2.

Close the help window with `q`.

B.B. Step 2: Check whether a Lisp mode is already installed

Your config might already include SLIME or SLY. These are the two packages that connect Emacs to Lisp. Only one should be active.

Press `M-x` and type `sly`. If a command called exactly `sly` shows up, SLY is installed. Press `C-g` to cancel.

- **SLY is there:** use the SLY version in step 3.
- **SLY isn't there:** use the SLIME version (the guide's default).

B.C. Step 3: Add the settings

Open your config file (`C-x C-f`, then type the path from step 1). Go to the very end (`M->`) and paste **one** of these blocks.

B.C.A. SLIME version (default)

```
;;; ---- Common Lisp setup ----
(require 'package)
(add-to-list 'package-archives
             '("melpa" . "https://melpa.org/packages/") t)
(package-initialize)
(unless package-archive-contents
  (package-refresh-contents))
(unless (package-installed-p 'use-package)
  (package-install 'use-package))
(require 'use-package)

(use-package slime
  :ensure t
  :init
  (setq inferior-lisp-program "sbcl")
  :config
```

```
  (slime-setup '(slime-fancy)))

(with-eval-after-load 'org
  (org-babel-do-load-languages
   'org-babel-load-languages
   '((emacs-lisp . t)
     (lisp . t)))
  (setq org-babel-lisp-eval-fn #'slime-eval)
  ;; Don't ask "Evaluate this block?" for Lisp blocks
  (setq org-confirm-babel-evaluate
        (lambda (lang _body) (not (string= lang "lisp")))))
```

B.C.B. SLY version (only if step 2 found SLY)

Same as above, but replace the `use-package slime` part with this, and use `#'sly-eval` instead of `#'slime-eval`:

```
(use-package sly
  :ensure t
  :init
  (setq inferior-lisp-program "sbcl"))
```

With SLY, wherever this guide says `M-x slime`, type `M-x sly` instead.

B.C.C. Windows: if `sbcl` isn't on your PATH

Replace `"sbcl"` with the full path, using forward slashes:

```
(setq inferior-lisp-program
      "C:/Program Files/Steel Bank Common Lisp/sbcl.exe")
```

B.D. Step 4: Restart and test

1. Save the config file (`C-x C-s`) and quit Emacs (`C-x C-c`).
2. Start Emacs again. The first start after this change downloads SLIME (or SLY), so it may take up to a minute. Wait until it finishes.
3. Type `M-x slime` (or `M-x sly`) and press Enter. A window titled `*slime-repl sbcl*` (or `*sly-mrepl*`) opens with a prompt like `CL-USER>`. Lisp is now running.
4. Open this file (`C-x C-f` then `thinking-in-lisp.org`).
5. Put the cursor in the block below and press `C-c C-c`.

```
(+ 1 2)
```

You should see:

```
#+RESULTS:
: 3
```

If you see `3`, you're set. Go back to "How to Use This Guide" and start Part I. If not, read "Common mistakes" at the end of this appendix.

B.E. Hyperspec lookup

The Common Lisp HyperSpec is the standard, browsed like a manual. Install it so `C-c C-d h` works from any symbol:

1. Download the HyperSpec (the Common Lisp HyperSpec mirror of your choice — the landing page offers a tarball) and unpack it somewhere permanent, e.g. `~/docs/HyperSpec/`.
2. Tell Slime where it lives, in your init file:

```
(setq common-lisp-hyperspec-root
      "file:///home/you/docs/HyperSpec/"
      common-lisp-hyperspec-symbol-table
      "/home/you/docs/HyperSpec/Data/Symbol-Table.text")
```

(Windows users: forward slashes work in these paths even on Windows — `C:/docs/HyperSpec/`.)

Now, with Slime connected: point at any symbol, `C-c C-d h` opens the Hyperspec page; `C-c C-d d` does the same for spec dict entries. A later chapter builds the **habit** this is the **keypress**.

B.F. If you use Doom Emacs

Doom keeps your settings in `~/.config/doom/` (or `~/.doom.d/`). If either folder exists, you use Doom, even if step 1 showed a different path. Don't add the block from step 3. Do this instead:

1. Open `~/.config/doom/init.el`. Find the `:lang` section and remove the `;;` in front of `common-lisp`, so the line reads:

```
       common-lisp       ; if you've seen one lisp, you've seen them all
```

Make sure `org` in the same section has no `;;` in front of it. 2. Open `~/.config/doom/config.el` and add at the end:

```
(after! org
  (org-babel-do-load-languages
   'org-babel-load-languages
   '((emacs-lisp . t)
     (lisp . t)))
  (setq org-babel-lisp-eval-fn #'sly-eval)
  (setq org-confirm-babel-evaluate
        (lambda (lang _body) (not (string= lang "lisp")))))
```

1. In a terminal, run `doom sync` (on Windows it may be `~/.config/emacs/bin/doom sync`), then restart Emacs.

Doom uses **SLY**, so use `M-x sly` wherever this guide says `M-x slime`. Doom also uses Vim-style keys by default. Read "If your keys behave like Vim" in Appendix C.

B.G. If you use Spacemacs

1. Open `~/.spacemacs` (press `SPC f e d` as a shortcut).
2. Find the list called `dotspacemacs-configuration-layers` and add `common-lisp` and `org` to it if they're missing:

```
 dotspacemacs-configuration-layers
 '(
   common-lisp
   org
   ;; ... whatever is already here ...
   )
```

1. In the same file, find the function `dotspacemacs/user-config` and add this inside it, before its final closing parenthesis:

```
(with-eval-after-load 'org
  (org-babel-do-load-languages
   'org-babel-load-languages
   '((emacs-lisp . t)
     (lisp . t)))
  (setq org-babel-lisp-eval-fn #'slime-eval)
  (setq org-confirm-babel-evaluate
        (lambda (lang _body) (not (string= lang "lisp")))))
```

1. Restart Emacs (`SPC q R`). Spacemacs uses SLIME, so `M-x slime` works as written.

B.H. Common mistakes

- **"No SLIME connection" or "Not connected" when pressing `C-c C-c`:** Lisp isn't running yet. Type `M-x slime` (or `M-x sly`) and try again. You need this once per Emacs session.
- **"No org-babel-execute function for lisp":** Org doesn't know about Lisp blocks yet. The settings didn't load. Check that you saved the config file, then restart Emacs.
- **"Searching for program: No such file or directory, sbcl":** Emacs can't find SBCL. On Windows, use the full-path setting from step 3. On Linux, check that `sbcl --version` works in a terminal.
- **The block says `common-lisp` or `cl` instead of `lisp`:** Org only recognises `lisp`. Write `#+begin_src lisp`.
- **Emacs asks "Evaluate this lisp code block?" every time:** the `org-confirm-babel-evaluate` line didn't load. Type `yes` for now and check the config.
- **SLY says SLIME is also loaded and asks to disable it:** your config has both. Answer yes, and keep only one of them in the config.
- **The result is `nil` but you expected printed text:** add `:results output` to the block's first line (see "How to Use This Guide").

APPENDIX C.

Emacs Survival Guide

This is the smallest set of keys that lets you work comfortably. Learn the first section today. The rest will come with use.

C.A. Reading key names

- `C-x` means: hold **Control**, press `x`.
- `M-x` means: hold **Alt**, press `x`. (M stands for "Meta".) If Alt doesn't work, press and release **Esc**, then press `x`.
- `C-x C-f` means: Control-x, then Control-f. You can keep Control held down for both.
- `C-x b` means: Control-x, release, then plain `b`.
- `RET` is Enter, `SPC` is the space bar, `TAB` is Tab.

C.B. The keys you need today

Key	What it does
`C-g`	Cancel. Use it whenever you're confused or stuck
`C-x C-f`	Open a file (or create a new one)
`C-x C-s`	Save
`C-/`	Undo
`C-x C-c`	Quit Emacs
`M-x`	Run any command by name

`C-g` is the most important key in Emacs. If a prompt appears at the bottom of the screen and you don't know what it wants, press `C-g` until it goes away.

C.C. Buffers and windows

A **buffer** is anything Emacs has open: a file, the Lisp REPL, a help page. A **window** is a section of the screen that shows a buffer.

Key	What it does
`C-x b`	Switch to another buffer (type part of its name)
`C-x o`	Move the cursor to the other window
`C-x 1`	Make the current window fill the screen
`C-x 0`	Close the current window (the buffer stays open)
`C-x 2`	Split the screen top and bottom
`C-x 3`	Split the screen left and right
`C-x k`	Close a buffer

A good layout for this guide: `C-x 3`, the guide on the left, the REPL on the right.

C.D. Moving around

Arrow keys, Page Up/Down, Home and End all work. These are faster once you know them:

Key	What it does
`C-a`	Start of line
`C-e`	End of line
`M-<`	Start of file
`M->`	End of file
`C-s`	Search forward (repeat `C-s` for the next match, `RET` to stop)
`C-r`	Search backward

C.E. Copy, cut and paste

Emacs uses its own keys for these:

Key	What it does
`C-SPC`	Start selecting (then move the cursor)
`M-w`	Copy the selection
`C-w`	Cut the selection
`C-y`	Paste

Selecting with the mouse also works, followed by `M-w` or `C-w`.

If you'd rather use `C-c`, `C-x` and `C-v` like other programs, type `M-x cua-mode`. It stays on until you restart Emacs.

C.F. Org mode keys

Key	What it does
`TAB`	On a heading: fold or unfold it
`S-TAB`	Fold or unfold the whole file (Shift-Tab)
`C-c C-n`	Jump to the next heading
`C-c C-p`	Jump to the previous heading
`C-c C-c`	Inside a code block: run it
`C-c '`	Inside a code block: edit it in its own window
`C-c C-,`	Insert a new block (press `s` for a source block)
`C-c C-v k`	Delete a block's result

To make a new Lisp block for an exercise answer: press `C-c C-,`, then `s`, then type `lisp` after `#+begin_src`.

C.G. Lisp keys

These work in the REPL, in `.lisp` files, and in the window that `C-c '` opens for a code block.

Key	What it does
`RET`	In the REPL: run what you typed
`M-p`	In the REPL: bring back the previous input
`M-n`	In the REPL: the next input
`C-x C-e`	Run the expression just before the cursor

Key	What it does
`C-c C-c`	In a Lisp buffer: compile the definition under the cursor
`C-c C-k`	Compile and load the whole `.lisp` file
`C-M-f`	Jump forward over one whole expression
`C-M-b`	Jump backward over one whole expression
`C-M-k`	Delete the next whole expression

When the cursor is on a parenthesis, Emacs highlights its partner. Watch for this. It's how you check that your parentheses match.

C.H. The debugger window

When your code has an error, a window called `*sldb ...*` (SLIME) or `*sly-db ...*` (SLY) opens. It shows the error message and a list of numbered choices called *restarts*.

For now:

- Read the first line. It tells you what went wrong.
- Press `q` to close the window and go back to work.

A later chapter teaches you what the other choices do. They're one of the best features of Lisp.

C.I. Debugger keys

When an error opens the SLDB buffer (the debugger of a later chapter), the keys that matter:

Key	Action
`v`	view the current frame's source
`n` / `p`	next / previous frame
`t`	toggle details of the current frame
`1`–`9`	invoke the numbered restart from the menu
`a`	invoke ABORT (the standard escape restart)
`q`	quit — abort out of the debugger
`i`	inspect the current frame's condition

The Inspector buffer: `TAB` moves between parts, `RET` descends into one, `l` goes back up. And the trace buffer from `trace`: `g` refreshes. Two habits cover 80% of use: `n` down the frames reading the backtrace, `1` on the restart you decided on — everything else is exploration.

C.J. If your keys behave like Vim

Doom, Spacemacs and many AI-written configs turn on **Evil**, which makes Emacs work like the Vim editor. Signs: typing letters doesn't insert text, or the cursor is a solid block and the bottom bar shows `<N>` or `NORMAL`.

The basics:

- Press `i` to start typing text (insert mode).
- Press `Esc` to stop typing and go back to moving around (normal mode).
- Press `:w` then `RET` to save.
- The `C-` and `M-` keys in this guide mostly still work, including `C-c C-c` and `C-c '`.

- Doom and Spacemacs also use `SPC` in normal mode as a menu key. Press it and wait: a menu of choices appears.

If you'd rather not use Vim keys, type `M-x evil-mode` to turn them off until the next restart.

C.K. If Lisp gets stuck

If running a block never finishes (for example, a loop that never ends):

1. Switch to the REPL buffer (`C-x b`, then type `repl`).
2. Press `C-c C-c` there to interrupt Lisp. The debugger opens.
3. Press `q`.

If that doesn't help, run `M-x slime-restart-inferior-lisp` (SLY: `M-x sly-restart-inferior-lisp`). This starts a fresh Lisp. Everything you defined is gone, so re-run the earlier blocks.

C.L. Common mistakes

- **Pressing `C-z` by habit (undo in other programs):** it minimises Emacs, or with Evil switches key styles. Undo is `C-/`. If the keys suddenly feel different, press `C-z` again.
- **Pressing `C-x` and then waiting:** Emacs is waiting for the second key. Finish the combination or press `C-g`.
- **Typing into the `#+RESULTS:` area by accident:** don't bother fixing it. Run the block again and the result is rewritten.
- **Closing the REPL buffer with `C-x k`:** Lisp may still be running in the background. Run `M-x slime` again; if it asks whether to create an additional connection, answer no.
- **Losing your place in a large file:** press `S-TAB` until you see only the top-level headings, move to the one you want, then press `TAB`.

APPENDIX D.

Answers to Exercises

Each answer has a solution, test lines to paste under your own answer, and the result you should see.

D.A. How to use these answers

Each answer gives a solution, a **check** to run, and the **expected** result. To check your own answer:

1. Run your answer block.
2. Make a new block under it, paste the check lines, and run it.
3. Compare with the expected result.

For exercises where the answer is a single expression, the check is simply running your expression. Your code may look different from the solution and still be right. Only the result has to match.

D.B. Chapter 1

1.1

```
(* 7 24 60)
```

Expected: `10080`

`*` takes any number of arguments, so you don't need to nest `(* 7 (* 24 60))`. That version works too.

1.2

```
(* (- 10 4) (+ 3 2))
```

Expected: `30`

1.3

```
(- (/ 100 (* 2 5)) 3)
```

Expected: `7`

Work from the inside out: `(* 2 5)` is `10`, `(/ 100 10)` is `10`, and `(- 10 3)` is `7`.

1.4

```
(princ "Welcome, traveller.")
```

Expected: `Welcome, traveller.`

You need `:results output`. Without it, the block shows the value that `princ` returns, which is the string with quotes.

1.5

Wrong	Problem	Fixed
`(5 * 3)`	The operator must come first	`(* 5 3)`
`(+2 3)`	`+2` reads as the number 2. A space is needed	`(+ 2 3)`
`(+ 1 (* 2 3)`	Missing a closing parenthesis	`(+ 1 (* 2 3))`
`(* 2, 4)`	Commas don't separate arguments	`(* 2 4)`

D.C. Chapter 2

2.1

Expression	Result	Why
`'(+ 2 3)`	`(+ 2 3)`	Quoted, so not evaluated
`(list '+ 2 3)`	`(+ 2 3)`	`list` builds the same list from its arguments
`(+ 2 3)`	`5`	A normal call

The first two give the same list in two different ways. This is the point of the exercise: code and data look the same in Lisp.

2.2

```
(list (third '(a b c d))
      (nth 2 '(a b c d))
      (first (rest (rest '(a b c d)))))
```

Expected: `(C C C)`

Any three working ways are fine. `(second (rest '(a b c d)))` is another.

2.3

```
(first (last '(red green blue)))
```

Expected: `BLUE`

`(third '(red green blue))` also works. `(last ...)` alone gives `(BLUE)`, a list, which is the trap this exercise sets.

2.4

```
(reverse (append '(1 2 3) '(4 5 6)))
```

Expected: `(6 5 4 3 2 1)`

Another way: `(append (reverse '(4 5 6)) (reverse '(1 2 3)))`.

2.5

Expression	Result	Why
`(listp nil)`	`T`	`nil` is the empty list
`(null '(nil))`	`NIL`	A list holding one element is not empty
`(length '((a b) (c d e)))`	`2`	Two elements, each of which is a list
`(rest '(only))`	`NIL`	Nothing after the first element

The second one catches many people. `(nil)` is a list with one element, which happens to be `nil`. It isn't empty.

2.6

```
(second (second '(garden (kitchen shed))))
```

Expected: `SHED`

The inner `second` gives the exits, `(KITCHEN SHED)`. The outer one picks the second exit.

D.D. Chapter 3

3.1

```
(+ 2/3 1/4)
```

Expected: `11/12`

```
(float (+ 2/3 1/4))
```

Expected: `0.9166667`

3.2

Expression	Result	Why
`(/ 10 4)`	`5/2`	Integer division is exact and reduces 10/4 to 5/2
`(/ 10 4.0)`	`2.5`	One float makes the result a float
`(mod 10 4)`	`2`	$10 = 2 \times 4 + 2$
`(isqrt 50)`	`7`	$7 \times 7 = 49$ fits in 50. $8 \times 8 = 64$ doesn't

3.3

```
(subseq "I love Lisp" 7)
```

Expected: `"Lisp"`

Count from zero: `I` is 0, the space is 1, `love` is 2 to 5, the second space is 6, and `L` is 7. `(subseq "I love Lisp" 7 11)` also works.

3.4

```
(concatenate 'string
             (string-upcase "hello")
             ", "
             (string-upcase "world")
             "!")
```

Expected: `"HELLO, WORLD!"`

This is shorter: upper-case once, after joining.

```
(string-upcase (concatenate 'string "hello" ", " "world" "!"))
```

3.5

```
(list (digit-char-p #\7)
      (char-code (char-upcase #\b)))
```

Expected: `(7 66)`

`(char-upcase #\b)` gives `#\B`, and the code of `B` is `66`, one more than `A` (65).

3.6

Expression	Result	Why
`(eq 'Apple 'APPLE)`	`T`	Symbol names become upper case when read
`(string= "Apple" "APPLE")`	`NIL`	Strings keep their case. `string=` checks it
`(string-equal "Apple" "APPLE")`	`T`	`string-equal` ignores case
`(symbol-name :north)`	`"NORTH"`	The colon isn't part of the name

3.7

```
(concatenate 'string
             "You are in the "
             (string-capitalize
              (symbol-name
               (first '(garden "An overgrown garden." (kitchen shed)))))
```

```
            ". "
            (second '(garden "An overgrown garden." (kitchen shed))))
```

Expected: "You are in the Garden. An overgrown garden."

It's the same shape as the kitchen example in the chapter, with the garden list in place of the kitchen list. Typing the room twice is still clumsy. Chapter 4 fixes that.

D.E. Chapter 4

4.1

```
(defparameter *apples* 12)
(defparameter *pears* 8)
(+ *apples* *pears*)
```

Expected: `20`

```
(setf *pears* 20)
(+ *apples* *pears*)
```

Expected: `32`

4.2 Expected: `NORMAL`

Line	`*mode*` after it	Why
`(defvar *mode* 'easy)`	`EASY`	First definition
`(setf *mode* 'hard)`	`HARD`	`setf` changes it
`(defvar *mode* 'easy)`	`HARD`	Already defined, so `defvar` skips it
`(defparameter *mode* 'normal)`	`NORMAL`	`defparameter` always sets it
`(defvar *mode* 'easy)`	`NORMAL`	Already defined again

If you ran these in a session where `*mode*` already existed, the first line also does nothing. Restart Lisp to see the table exactly.

4.3

```
(let ((width 8)
      (height 5))
  (list (* width height)
        (* 2 (+ width height))))
```

Expected: `(40 26)`

4.4

```
(let* ((celsius 30)
       (fahrenheit (+ (* celsius 9/5) 32)))
  (list celsius fahrenheit))
```

Expected: `(30 86)`

`9/5` is a ratio, so the result stays exact: 30 × 9/5 is 54, plus 32 is 86. A plain `let` fails here, because `fahrenheit` uses `celsius`.

4.5

```
(defparameter *steps* 0)
(incf *steps*)
(incf *steps*)
(incf *steps*)
(incf *steps* 10)
```

Expected: `13`

The last `incf` returns the new value, so the block shows it. Adding `*steps*` as a final line gives the same result.

4.6

Wrong	Problem	Fixed
`(let (total 10) total)`	Each binding needs its own parentheses	`(let ((total 10)) total)`
`(let ((a 3) (b (* a a))) b)`	`b` uses `a` from the same `let`	`(let* ((a 3) (b (* a a))) b)`
`(* '*apples* 2)`	The quote gives the name, not the value	`(* *apples* 2)`

Check the fixed versions:

```
(list (let ((total 10)) total)
      (let* ((a 3) (b (* a a))) b)
      (* *apples* 2))
```

Expected: `(10 9 24)` (with `*apples*` at 12 from Exercise 4.1)

4.7 Expected: `11`

The outer `x` is 5. The inner `let` binds a new `x` to $5 \times 2 = 10$. `setf` changes the inner `x` to 11, and the body returns it.

With `(list x)` after the inner `let`:

```
(let ((x 5))
  (let ((x (* x 2)))
    (setf x (+ x 1)))
  (list x))
```

Expected: `(5)`

The `setf` changed only the inner `x`. Once the inner `let` ends, the outer `x` is visible again, and it was never touched. It is still `5`.

4.8

```
(setf *current-room* *kitchen*)

(let ((name (first *current-room*))
      (exits (third *current-room*)))
  (list name (length exits)))
```

Expected: `(KITCHEN 2)`

The exits are the third element of a room, `(HALLWAY GARDEN)`, and `length` counts them.

D.F. Chapter 5

5.1

```
(defun square (n)
  (* n n))
```

Check:

```
(list (square 4) (square -3) (square 1/2))
```

Expected: `(16 9 1/4)`

5.2

```
(defun average3 (a b c)
  "Return the average of A, B and C."
  (/ (+ a b c) 3))
```

Check:

```
(list (average3 1 2 3) (average3 1 2 4) (float (average3 1 2 4)))
```

Expected: `(2 7/3 2.3333333)`

The result is exact. 7/3 isn't an error, it's the true average.

5.3

```
(defun greet (name)
  (concatenate 'string "Hello, " (string-upcase name) "!"))
```

Check:

```
(greet "ada")
```

Expected: `"Hello, ADA!"`

5.4

```
(defun power-of (base &optional (exponent 2))
  (expt base exponent))
```

Check:

```
(list (power-of 5) (power-of 2 10) (power-of 3 0))
```

Expected: `(25 1024 1)`

5.5

```
(defun make-greeting (name &key (greeting "Hello") (punctuation "!"))
  (concatenate 'string greeting ", " name punctuation))
```

Check:

```
(list (make-greeting "Ada")
      (make-greeting "Ada" :greeting "Welcome")
      (make-greeting "Ada" :punctuation "?" :greeting "Ready"))
```

Expected: `("Hello, Ada!" "Welcome, Ada!" "Ready, Ada?")`

The keywords can come in any order. The third call gives `:punctuation` first and still works.

5.6

```
(defun count-args (&rest things)
  (length things))
```

Check:

```
(list (count-args) (count-args 'a) (count-args 1 2 3 4 5))
```

Expected: `(0 1 5)`

With no arguments, `things` is the empty list, so its length is 0.

5.7

Wrong	Problem	Fixed
`(defun add (a b) (+ a b c))`	`c` isn't a parameter. SBCL warns "undefined variable: C"	`(defun add (a b) (+ a b))`
`(defun (twice n) (* 2 n))`	The name and the parameter list are separate parts	`(defun twice (n) (* 2 n))`
`(add 1)`	`add` needs two arguments: "invalid number of arguments: 1"	`(add 1 2)`

Check the fixed versions:

```
(defun add (a b) (+ a b))
(defun twice (n) (* 2 n))
(list (add 1 2) (twice 21))
```

Expected: `(3 42)`

5.8

```
(defun exit-count (place)
  (length (room-exits place)))

(defun room-title (place)
  (string-capitalize (symbol-name (room-name place))))
```

Check (run the project blocks from Chapter 5 first):

```
(list (exit-count *hallway*)
      (exit-count *garden*)
      (room-title *kitchen*))
```

Expected: `(2 1 "Kitchen")`

You can now use `room-title` inside `describe-room`, which makes it shorter:

```
(defun describe-room (place)
  "Return the text shown when the player is in PLACE."
  (concatenate 'string
               "You are in the " (room-title place) ". "
               (room-description place)))
```

Small functions that each do one thing, built on each other, are how Lisp programs are written.

D.G. Chapter 6

6.1

```
(defun sign (n)
  (cond ((< n 0) 'negative)
        ((= n 0) 'zero)
        (t 'positive)))
```

Check:

```
(list (sign -4) (sign 0) (sign 9))
```

Expected: `(NEGATIVE ZERO POSITIVE)`

`(zerop n)` works in place of `(= n 0)`.

6.2

Expression	Result	Why
`(if nil 1 2)`	`2`	`nil` is false
`(if 0 1 2)`	`1`	`0` is not `nil`, so it's true
`(when (> 1 2) 'yes)`	`NIL`	The test fails, so the body doesn't run
`(unless (> 1 2) 'yes)`	`YES`	`unless` runs when the test fails
`(if (member 3 '(1 2 3)) 'found 'missing)`	`FOUND`	`member` returns `(3)`, which is true

6.3

```
(defun my-abs (n)
  (if (< n 0)
      (- n)
      n))
```

Check:

```
(list (my-abs -7) (my-abs 0) (my-abs 3/2))
```

Expected: `(7 0 3/2)`

`(- n)` with one argument negates, as in the Chapter 3 table.

6.4

```
(defun grade (score)
  (cond ((>= score 90) 'a)
        ((>= score 80) 'b)
        ((>= score 70) 'c)
        (t 'f)))
```

Check:

```
(list (grade 95) (grade 90) (grade 85) (grade 72) (grade 40))
```

Expected: `(A A B C F)`

The order does the work. A score of 85 fails the first test and passes the second, so you never need to write "between 80 and 89".

6.5

```
(defun day-type (day)
  (case day
    ((saturday sunday) 'weekend)
    ((monday tuesday wednesday thursday friday) 'weekday)
    (otherwise 'unknown)))
```

Check:

```
(list (day-type 'sunday) (day-type 'wednesday) (day-type 'funday))
```

Expected: `(WEEKEND WEEKDAY UNKNOWN)`

6.6 First block. `if` got four arguments after the test, so SBCL complains that there are too many. Group the two prints with `progn`:

```
(let ((x 5))
  (if (> x 0)
      (progn
        (print "positive")
        (print "big"))
      (print "negative")))
```

Expected output:

```
"positive"
"big"
```

Second block. Without the extra parentheses, `cond` takes `>` as the test and tries to look up a variable called `>`: "The variable > is unbound". Each clause needs its own pair:

```
(cond ((> 5 0) 'yes))
```

Expected: `YES`

Third block. It returns `0`, not an error. `case` can't match strings, so the `"yes"` clause never matches. Use `cond` with `string=`:

```
(let ((answer "yes"))
  (cond ((string= answer "yes") 1)
        (t 0)))
```

Expected: `1`

6.7 Check all five at once:

```
(list (or nil "" 5)
      (and 1 0 nil)
      (or (member 'x '(a b)) 'none)
      (and 'a 'b)
      (not 0))
```

Expected: `("" NIL NONE B NIL)`

Expression	Result	Why
`(or nil "" 5)`	`""`	`""` is the first true value, so `or` stops there
`(and 1 0 nil)`	`NIL`	One argument is `nil`
`(or (member 'x '(a b)) 'none)`	`NONE`	`member` gives `nil`, so `or` moves on
`(and 'a 'b)`	`B`	All true, so `and` returns the last value
`(not 0)`	`NIL`	`0` is true, so `not` gives `nil`

6.8

```
(defun fizzbuzz-word (n)
  (cond ((zerop (mod n 15)) "FizzBuzz")
        ((zerop (mod n 3)) "Fizz")
        ((zerop (mod n 5)) "Buzz")
        (t (princ-to-string n))))
```

Check:

```
(list (fizzbuzz-word 1)
      (fizzbuzz-word 3)
      (fizzbuzz-word 5)
      (fizzbuzz-word 15)
      (fizzbuzz-word 7))
```

Expected: `("1" "Fizz" "Buzz" "FizzBuzz" "7")`

The test for 15 must come first. If the test for 3 came first, 15 would give `"Fizz"`. `(and (zerop (mod n 3)) (zerop (mod n 5)))` also works as the first test.

6.9

```
(defun can-go-p (name)
  "Return T if NAME is an exit of the current room, NIL otherwise."
  (if (member name (room-exits *current-room*))
      t
      nil))
```

Check (run the Chapter 4 to 6 project blocks first):

```
(setf *current-room* *hallway*)
(list (can-go-p 'kitchen) (can-go-p 'garden))
```

Expected: `(T NIL)`

A shorter version is `(not (null (member ...)))`: `null` turns the result into `t` or `nil` the wrong way round, and `not` flips it back.

6.10

```
(defparameter *garden-unlocked* nil)

(defun go-to (name)
  "Move the player to the room called NAME, if it is an exit."
  (cond ((eq name (room-name *current-room*))
         "You are already there.")
        ((and (eq name 'garden) (not *garden-unlocked*))
```

```
          "The garden door is locked.")
         ((member name (room-exits *current-room*))
          (setf *current-room* (find-room name))
          (look))
         (t
          "You can't go there from here.")))
```

Check:

```
(setf *current-room* *kitchen*)
(setf *garden-unlocked* nil)
(list (go-to 'garden)
      (progn
        (setf *garden-unlocked* t)
        (go-to 'garden)))
```

Expected:

```
("The garden door is locked."
 "You are in the Garden. An overgrown garden.")
```

The new clause goes **before** the `member` clause. If it came after, the `member` clause would match first and the lock would never be checked.

Arguments to `list` are evaluated left to right, so the first `go-to` runs while the door is still locked, and the `progn` unlocks it before the second one.

Part II rebuilds `go-to` from scratch, so it doesn't matter whether you keep this version or the chapter's.

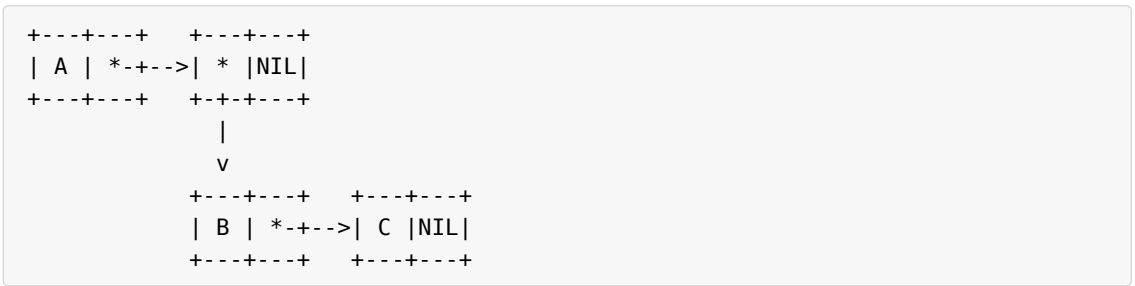
D.H. Chapter 7

7.1

```
(list (cons 1 (cons 2 (cons 3 nil)))
      (cons 'a (cons (cons 'b (cons 'c nil)) nil)))
```

Expected: `((1 2 3) (A (B C)))`

The diagram for `(A (B C))`:



The outer list has two conses, so two elements. The car of the second one points to the inner list `(B C)`.

7.2

```
(list (cons 'a 'b)
      (cons '(a) '(b))
      (cons nil nil)
      (cdr '(a))
      (car '((x y) z)))
```

Expected: `((A . B) ((A) B) (NIL) NIL (X Y))`

Expression	Result	Why
`(cons 'a 'b)`	`(A . B)`	The cdr isn't a list, so it's a dotted pair
`(cons '(a) '(b))`	`((A) B)`	The element `(A)` goes on the front of `(B)`
`(cons nil nil)`	`(NIL)`	A one-element list whose element is `nil`
`(cdr '(a))`	`NIL`	Only one cons, and its cdr is the end
`(car '((x y) z))`	`(X Y)`	The first element is itself a list

7.3

```
(list (caddr '(a b c d))
      (cddr '(a b c d))
      (cdar '((a . 1) (b . 2))))
```

Expected: `(C (C D) 1)`

Read `cdar` right to left: the car gives `(A . 1)`, and its cdr is `1`.

7.4

```
(defun swap-pair (pair)
  (cons (cdr pair) (car pair)))
```

Check:

```
(list (swap-pair '(1 . 2)) (swap-pair '(key . kitchen)))
```

Expected: `((2 . 1) (KITCHEN . KEY))`

7.5

```
(defparameter *s* '(x))
(push 'y *s*)
(push 'z *s*)
(list (pop *s*) *s*)
```

Expected: `(Z (Y X))`

After both pushes, `*s*` is `(Z Y X)`. `pop` returns `Z` and leaves `(Y X)`. The arguments of `list` run left to right, so `*s*` is read after the `pop`.

7.6

```
(defparameter *capitals*
  '((tunisia . tunis)
    (france . paris)
    (japan . tokyo)))

(defun capital-of (country)
```

```
  (or (cdr (assoc country *capitals*))
      'unknown))
```

Check:

```
(list (capital-of 'japan) (capital-of 'tunisia) (capital-of 'peru))
```

Expected: `(TOKYO TUNIS UNKNOWN)`

7.7

Wrong	Problem	Fixed
`(cons '(1 2) '(3 4))`	`cons` adds `(1 2)` as one element	`(append '(1 2) '(3 4))`
`(cdr (assoc "b" '(("a" . 1) ("b" . 2))))`	`assoc` can't compare strings	`(cdr (assoc 'b '((a . 1) (b . 2))))`
`(push 'x '(a b))`	`push` needs a variable to store the result	`(cons 'x '(a b))`

Check the fixed versions:

```
(list (append '(1 2) '(3 4))
      (cdr (assoc 'b '((a . 1) (b . 2))))
      (cons 'x '(a b)))
```

Expected: `((1 2 3 4) 2 (X A B))`

If you really want `push`, put the list in a variable first: `(let ((l '(a b))) (push 'x l))` also gives `(X A B)`.

7.8

```
(defun drop (item)
  "Put down ITEM in the current room, if the player carries it."
  (if (eq (item-location item) 'player)
      (progn
        (push (cons item (room-name *current-room*)) *item-locations*)
        (concatenate 'string "You drop the " (item-name item) "."))
      "You don't have that."))
```

Check. This resets the items, so run it as one block:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))
(list (drop 'key)
      (take 'key)
      (progn
        (go-to 'hallway)
        (drop 'key))
      (item-location 'key))
```

Expected:

```
("You don't have that." "You take the key." "You drop the key." HALLWAY)
```

The key was taken in the kitchen and dropped in the hallway. The `progn` moves the player first, then drops the key, and returns the drop message.

7.9

```
(push '(shed "A wooden shed full of old tools." (garden)) *rooms*)
(push '(garden "An overgrown garden." (kitchen shed)) *rooms*)
```

Check:

```
(setf *current-room* (find-room 'garden))
(go-to 'shed)
```

Expected: `"You are in the Shed. A wooden shed full of old tools."`

The new garden replaces the old one because `assoc` returns the *first* pair with a matching key. The new garden is at the front of `*rooms*`, so `find-room` finds it before the old one. The old garden is still in the list, hidden.

The `setf` must come *after* the pushes. If `*current-room*` was set to the garden earlier, it still holds the old garden list, whose exits don't include the shed, and `go-to` answers "You can't go there from here."

Each push adds to `*rooms*` again, so running this block twice puts two copies at the front. That's harmless, because only the first one is ever found. Re-run the `defparameter` for `*rooms*` from the project step to start clean.

D.I. Chapter 8

8.1

```
(defun my-length (lst)
  "Return the number of elements in LST."
  (if (null lst)
      0
      (+ 1 (my-length (rest lst)))))
```

Check:

```
(list (my-length nil) (my-length '(a)) (my-length '(a b c d)))
```

Expected: `(0 1 4)`

The empty list is the base case: zero elements. Otherwise, one for the first element plus the length of the rest.

8.2

```
(defun my-sum (lst)
  "Return the sum of the numbers in LST."
  (if (null lst)
      0
      (+ (first lst) (my-sum (rest lst)))))
```

Check:

```
(list (my-sum nil) (my-sum '(5)) (my-sum '(1 2 3 4)))
```

Expected: `(0 5 10)`

The base case is `0`, because adding nothing gives zero. Compare that with `my-length`: the same shape, a different "nothing" value.

8.3

```
(list (count-atoms '(a (b) c))
      (countdown 3)
      (factorial 4)
      (items-in '((lamp . cellar) (key . kitchen)) 'kitchen)
      (double-all-into '(1 2) nil))
```

Expected: `(3 (3 2 1) 24 (KEY) (4 2))`

Expression	Result	Why
`(count-atoms '(a (b) c))`	`3`	`a`, `b` and `c`: three atoms, at any depth
`(countdown 3)`	`(3 2 1)`	Counts down and stops at zero
`(factorial 4)`	`24`	$4 \times 3 \times 2 \times 1$
`(items-in ... 'kitchen)`	`(KEY)`	Only the pair whose value is `kitchen`
`(double-all-into '(1 2) nil)`	`(4 2)`	The accumulator is built backwards, so no `reverse` here

The last one is the accumulator pattern without its `reverse`. Running `double-all` instead gives `(2 4)`, because `double-all` reverses the result of the helper.

8.4

```
(defun my-reverse-into (lst result)
  "Cons each element of LST onto RESULT."
  (if (null lst)
      result
      (my-reverse-into (rest lst)
                       (cons (first lst) result))))

(defun my-reverse (lst)
  "Return LST in the opposite order."
  (my-reverse-into lst nil))
```

Check:

```
(list (my-reverse nil) (my-reverse '(1 2 3)) (my-reverse '(a)))
```

Expected: `(NIL (3 2 1) (A))`

No `reverse` is needed at the end, because walking from the front and consing onto the accumulator already reverses the list. That's the whole job here, so the helper's result **is** the answer.

8.5

```
(defun my-find (item lst)
  "Return ITEM if it is in LST, NIL otherwise."
  (cond ((null lst) nil)
        ((eq (first lst) item) item)
        (t (my-find item (rest lst)))))
```

Check:

```
(list (my-find 'b '(a b c))
      (my-find 'a '(a b c))
      (my-find 'z '(a b c))
      (my-find 'x nil))
```

Expected: `(B A NIL NIL)`

Compare this with the built-in `member` from Chapter 6: `member` returns the rest of the list from the match onward, and `my-find` returns just the item. Both are useful.

8.6

```
(defun count-symbol (sym tree)
  "Return how often SYM appears in TREE, at any depth."
  (cond ((null tree) 0)
        ((consp tree) (+ (count-symbol sym (car tree))
                         (count-symbol sym (cdr tree))))
        ((eq tree sym) 1)
        (t 0)))
```

Check:

```
(list (count-symbol 'a '(a (b a) c a))
      (count-symbol 'a '(x y))
      (count-symbol 'a 'a)
      (count-symbol 'a nil))
```

Expected: `(3 0 1 0)`

8.7

```
(defun flatten (tree)
  "Return the atoms of TREE as one flat list."
  (cond ((null tree) nil)
        ((consp tree) (append (flatten (car tree))
                              (flatten (cdr tree))))
        (t (list tree))))
```

Check:

```
(list (flatten '(a (b (c)) d))
      (flatten '())
      (flatten '(1 (2 (3 (4))))))
```

Expected: `((A B C D) NIL (1 2 3 4))`

Two recursive calls again, one for each half of the cons. The last clause wraps the atom in a list, because the answer must always be a list: `(flatten 'a)` would otherwise return the atom `A`, and `append` would refuse it.

`append` copies every cons of the lists it is given (Chapter 7), so this version is fine for the sizes in this guide, and slow on very large trees.

8.8 **Wrong:** `bad-length` throws away the recursive result, so it returns `0` for every list. The recursive call has to add one:

```
(defun fix-length (lst)
  (if (null lst)
      0
      (+ 1 (fix-length (rest lst)))))
```

Wrong: `bad-sum` has the wrong base case. Adding a number to `nil` gives "The value NIL is not a NUMBER" as soon as the recursion ends:

```
(defun fix-sum (lst)
  (if (null lst)
      0
      (+ (first lst) (fix-sum (rest lst)))))
```

Wrong: `bad-countdown` uses `(1+ n)`, so it walks away from zero and never reaches the base case. SBCL prints "Control stack exhausted". The input must get smaller:

```
(defun fix-countdown (n)
  (if (zerop n)
      nil
      (cons n (fix-countdown (1- n)))))
```

Check all three:

```
(list (fix-length '(a b c))
      (fix-sum '(1 2 3))
      (fix-countdown 3))
```

Expected: `(3 6 (3 2 1))`

With `(1- n)` in place, the third one walks down from `3` to `0`, which is four calls. The broken version needs a million calls to fill the stack, so never run it to see what happens.

8.9

```
(defun count-conses (tree)
  "Return the number of cons cells in TREE."
  (cond ((null tree) 0)
        ((consp tree) (+ 1 (count-conses (car tree))
                           (count-conses (cdr tree))))
        (t 0)))
```

Check:

```
(list (count-conses '(a b))
      (count-conses '((a) b))
      (count-conses 5)
      (count-conses nil))
```

Expected: `(2 3 0 0)`

`(a b)` is two conses, as Chapter 7's box diagram shows. `((a) b)` has one cons for the outer pair, one for the inner `(a)`, and one for the chain `(b)`: three. An atom has none, and `nil` isn't a cons either.

8.10

```
(defun item-report (pairs)
  "Return one string per item in PAIRS, saying where it is."
  (cond ((null pairs) nil)
        ((eq (cdar pairs) 'player)
         (cons (concatenate 'string
                            (item-name (caar pairs))
                            " is carried")
               (item-report (cdr pairs))))
        (t
         (cons (concatenate 'string
                            (item-name (caar pairs))
                            " is in the "
                            (string-downcase
                             (symbol-name (cdar pairs))))
               (item-report (cdr pairs))))))
```

Check. This resets the items and marks one as carried, so run it as one block:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(push '(coin . player) *item-locations*)
(item-report *item-locations*)
```

Expected:

```
("coin is carried"
 "lamp is in the cellar"
 "key is in the kitchen"
 "letter is in the bedroom")
```

Three clauses, three jobs: stop at the end, one wording for items the player carries, another for items in a room. The order of the result follows the order of the alist, because the recursion walks it from the front.

8.11

```
(defun item-names (items)
  "Return a string listing ITEMS by name."
  (cond ((null items) "")
        ((null (cdr items))
```

```
         (item-name (first items)))
        ((null (cddr items))
         (concatenate 'string
                      (item-name (first items))
                      " and "
                      (item-name (second items))))
        (t
         (concatenate 'string
                      (item-name (first items))
                      ", "
                      (item-names (rest items))))))
```

Check:

```
(list (item-names nil)
      (item-names '(key))
      (item-names '(key lamp))
      (item-names '(key lamp coin))
      (item-names '(key lamp coin box)))
```

Expected:

```
("" "key" "key and lamp" "key, lamp and coin" "key, lamp, coin and box")
```

`cddr` is "the rest of the rest", from Chapter 7. When it is `nil`, the list has exactly two elements left, and that's the moment to write "and" instead of a comma. The last two elements are always handled by that clause, whichever call reaches them.

8.12

```
(defun take-all (items)
  "Take every item in ITEMS."
  (cond ((null items) nil)
        (t (take (first items))
           (take-all (rest items)))))

(defun loot ()
  "Take everything lying in the current room."
  (let ((found (items-here (room-name *current-room*))))
    (if (null found)
        "There is nothing here to take."
        (progn
          (take-all found)
          (concatenate 'string
                       "You take "
                       (item-names found)
                       ".")))))
```

Check. Reset the items and walk to the kitchen first:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
```

```
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))
(list (loot) (loot) (inventory))
```

Expected:

```
("You take key."
 "There is nothing here to take."
 "You are carrying: key.")
```

The second `loot` finds nothing, because the first one moved the key into the player's hands.

`take-all` walks `found`, the list that `items-here` built before anything was taken. `take` replaces `*item-locations*` with a new list, but `found` is a separate list, so the walk isn't affected.

This answer needs the fixed `take` from the end of the Chapter 8 project step. With the old `take`, the second `loot` still finds the hidden `(KEY . KITCHEN)` pair and says "You take key." again.

D.J. Chapter 9

9.1

```
(list (funcall #'list 1 2)
      (apply #'* 2 '(3 4))
      (mapcar #'length '((a) (b c) ()))
      (mapcar (lambda (x) (cons x x)) '(a b))
      (apply-twice (lambda (s) (concatenate 'string s "!")) "hi"))
```

Expected: `((1 2) 24 (1 2 0) ((A . A) (B . B)) "hi!!")`

Expression	Result	Why
`(funcall #'list 1 2)`	`(1 2)`	Same as `(list 1 2)`
`(apply #'* 2 '(3 4))`	`24`	Same as `(* 2 3 4)`
`(mapcar #'length ...)`	`(1 2 0)`	One length per inner list
`(mapcar (lambda (x) ...) ...)`	`((A . A) (B . B))`	Each symbol consed onto itself
`(apply-twice ...)`	`"hi!!"`	One `"!"` per call, two calls

9.2

```
(defun my-mapcar (f lst)
  "Return a list of the results of calling F on each element of LST."
  (if (null lst)
      nil
      (cons (funcall f (first lst))
            (my-mapcar f (rest lst)))))
```

Check:

```
(list (my-mapcar #'twice '(1 2 3))
      (my-mapcar #'first '((a 1) (b 2)))
      (my-mapcar #'twice nil))
```

Expected: `((2 4 6) (A B) NIL)`

This is `double-all` from Chapter 8 with `(twice (first lst))` replaced by `(funcall f (first lst))`. Notice that the recursive call passes `f` along unchanged.

9.3

```
(mapcar (lambda (name)
          (concatenate 'string "Hello, " (string-capitalize name)))
        '("ada" "alan" "grace"))
```

Expected: `("Hello, Ada" "Hello, Alan" "Hello, Grace")`

9.4

```
(defun make-multiplier (factor)
  "Return a function that multiplies its argument by FACTOR."
  (lambda (n) (* n factor)))
```

Check:

```
(list (funcall (make-multiplier 3) 7)
      (mapcar (make-multiplier 10) '(1 2 3)))
```

Expected: `(21 (10 20 30))`

The same shape as `make-adder`: the returned `lambda` remembers `factor`.

9.5

```
(defun compose (f g)
  "Return a function that calls G on its argument, then F on the result."
  (lambda (x) (funcall f (funcall g x))))
```

Check:

```
(list (funcall (compose #'twice #'1+) 5)
      (funcall (compose #'first #'rest) '(a b c)))
```

Expected: `(12 B)`

`(1+ 5)` is `6`, doubled is `12`. The second one builds `second` out of `first` and `rest`.

9.6 **Wrong:** `twice` without `#'` is read as a variable, and there is no such variable. Write `(mapcar #'twice '(1 2))`.

Wrong: `(f 10)` looks for a function named `f`. `f` is a variable holding a function, so call it with `funcall`:

```
(defun call-with-ten (f)
  (funcall f 10))
```

Wrong: the quote turns the `lambda` into a plain list. Remove it: `(funcall (lambda (x) (+ x 1)) 5)`.

Wrong: the last argument of `apply` must be a list. Either `(apply #'max '(3 7))` or `(funcall #'max 3 7)`.

Check the fixes:

```
(list (mapcar #'twice '(1 2))
      (call-with-ten #'twice)
```

```
      (funcall (lambda (x) (+ x 1)) 5)
      (apply #'max '(3 7)))
```

Expected: `((2 4) 20 6 7)`

9.7

```
(defun make-stepper (start step)
  "Return a function that returns START, then START+STEP, and so on."
  (let ((next start))
    (lambda ()
      (let ((current next))
        (incf next step)
        current))))
```

Check:

```
(let ((s (make-stepper 10 5)))
  (list (funcall s) (funcall s) (funcall s)))
```

Expected: `(10 15 20)`

The inner `let` saves the value to return before `incf` moves `next` forward. Without it, the first call would return `15`.

9.8

```
(mapcar #'cons '(a b c) '(1 2 3))
```

Expected: `((A . 1) (B . 2) (C . 3))`

With two lists, `mapcar` passes one element from each to `cons`, which builds one dotted pair per position.

9.9 Expected: `("HEY" "HEY!")`

`*table-a*` stored the function object from the first definition, and redefining `shout` doesn't touch it. `*table-b*` stored the name, so `funcall` finds the new definition at the moment of the call. That's why the game's `*commands*` table stores names.

9.10

```
(push '(l . look) *commands*)
(push '(i . inventory) *commands*)
```

Check:

```
(equal (run-command '(i)) (run-command '(inventory)))
```

Expected: `T`

Both short forms point at existing functions, so `run-command` needs no change. Run the `push` lines once only: running them again adds duplicate pairs. They do no harm here, because `assoc` stops at the first match, but the `help` command of the next exercise would list them twice. To start clean, rerun the `defparameter` of `*commands*` from the project step.

9.11

```
(defun help ()
  "List the command words."
```

```
  (concatenate 'string
               "Commands: "
               (item-names (mapcar #'car *commands*))
               "."))

(defparameter *commands*
  '((help . help)
    (look . look)
    (inventory . inventory)
    (take . take)
    (go . go-to)
    (loot . loot))
  "Map each command word to the function that handles it.")
```

Check:

```
(run-command '(help))
```

Expected: `"Commands: help, look, inventory, take, go and loot."`

`(mapcar #'car *commands*)` gives the list of command words, and `item-names` turns any list of symbols into a string, so it works on commands as well as on items. If you added `drop` or the short forms from Exercise 9.10, they appear in the list too.

The table is rebuilt with `defparameter` here so that `help` appears first. A `push` would also work.

D.K. Chapter 10

10.1

```
(list (remove-if-not #'evenp '(5 6 7 8))
      (find-if #'numberp '(a b 3 c 4))
      (count-if #'symbolp '(a 1 b 2 c))
      (position #\n "banana")
      (reduce #'* '(1 2 3 4))
      (every #'symbolp '())
      (remove-duplicates '(1 2 1 3 2)))
```

Expected: `((6 8) 3 3 2 24 T (1 3 2))`

Expression	Result	Why
`(remove-if-not #'evenp ...)`	`(6 8)`	Keeps the even numbers
`(find-if #'numberp ...)`	`3`	The first number, not all of them
`(count-if #'symbolp ...)`	`3`	`a`, `b` and `c`
`(position #\n "banana")`	`2`	Counting from zero: b, a, n
`(reduce #'* ...)`	`24`	$1 \times 2 \times 3 \times 4$
`(every #'symbolp '())`	`T`	No element says no
`(remove-duplicates ...)`	`(1 3 2)`	Keeps the last copy of each by default

10.2

```
(defun sum2 (lst)
  (reduce #'+ lst))

(defun find2 (item lst)
  (find item lst))

(defun all-positive2-p (lst)
  (every #'plusp lst))

(defun reverse2 (lst)
  (reduce (lambda (acc x) (cons x acc)) lst :initial-value nil))
```

Check:

```
(list (sum2 '(1 2 3 4)) (sum2 nil)
      (find2 'b '(a b c))
      (all-positive2-p '(1 2 3)) (all-positive2-p '(1 -2))
      (reverse2 '(1 2 3)))
```

Expected: `(10 0 B T NIL (3 2 1))`

`sum2` handles the empty list because `(+)` is `0`. `find2` is `find` under a new name, which tells you `my-find` was a hand-written `find`.

10.3

```
(remove-if (lambda (w) (< (length w) 4))
           '("a" "lisp" "is" "great"))
```

Expected: `("lisp" "great")`

10.4

```
(defparameter *scores* '((ada . 90) (alan . 72) (grace . 85)))

(list (sort (copy-list *scores*) #'> :key #'cdr)
      (find-if (lambda (s) (< s 80)) *scores* :key #'cdr)
      (reduce #'+ *scores* :key #'cdr)
      *scores*)
```

Expected:

```
(((ADA . 90) (GRACE . 85) (ALAN . 72))
 (ALAN . 72)
 247
 ((ADA . 90) (ALAN . 72) (GRACE . 85)))
```

The last line shows `*scores*` unchanged, thanks to `copy-list`. Sorting `*scores*` directly would also modify a quoted constant.

10.5

```
(defun my-remove-if (pred lst)
  "Return LST without the elements PRED says yes to."
  (cond ((null lst) nil)
        ((funcall pred (first lst))
         (my-remove-if pred (rest lst)))
```

```
        (t
         (cons (first lst) (my-remove-if pred (rest lst))))))
```

Check:

```
(list (my-remove-if #'evenp '(1 2 3 4 5))
      (my-remove-if #'evenp nil))
```

Expected: `((1 3 5) NIL)`

Compare it with `items-in` from Chapter 8: the same three clauses, with a predicate in place of the `eq` test.

10.6

```
(defun count-vowels (s)
  "Return the number of vowels in the string S."
  (count-if (lambda (c) (find c "aeiou"))
            (string-downcase s)))
```

Check:

```
(list (count-vowels "Common Lisp")
      (count-vowels "AEIOU")
      (count-vowels "xyz"))
```

Expected: `(3 5 0)`

`string-downcase` deals with upper case once, so the list of vowels needs only lower-case letters. `find` returns the character when it is a vowel and `nil` otherwise, which is all `count-if` needs.

10.7

```
(list (eql 2 2)
      (eql 2 2.0)
      (equal "a" "a")
      (equal '(1 (2)) '(1 (2)))
      (equalp "Lisp" "LISP")
      (eql (list 1) (list 1)))
```

Expected: `(T NIL T T T NIL)`

`(eql 2 2.0)` is `NIL` because one is an integer and the other a float. The last one is `NIL` because each `list` call builds its own cons, and `eql` asks whether they are the same object.

10.8 **Wrong:** `sort` modifies the quoted constant `'(3 1 2)`. Write `(sort (list 3 1 2) #'<)`.

Wrong: `reduce` on an empty list calls `(max)`, which needs at least one argument. Give a starting value: `(reduce #'max '() :initial-value 0)`.

Wrong: `find` compares with `eql`, and two strings are different objects. Write `(find "b" '("a" "b") :test #'string=)`.

Wrong: `remove-if` returns a new list and leaves `*numbers*` alone. Store the result: `(setf *numbers* (remove-if #'evenp *numbers*))`.

Check the fixes:

```
(defparameter *numbers* (list 1 2 3 4))
(list (sort (list 3 1 2) #'<)
      (reduce #'max '() :initial-value 0)
      (find "b" '("a" "b") :test #'string=)
      (setf *numbers* (remove-if #'evenp *numbers*))
      *numbers*)
```

Expected: `((1 2 3) 0 "b" (1 3) (1 3))`

10.9

```
(reduce (lambda (a b)
          (if (> (length b) (length a)) b a))
        '("hi" "hello" "hey"))
```

Expected: `"hello"`

`a` is the longest string so far, `b` the next one. The `lambda` keeps whichever is longer. With `>`, a tie keeps the earlier string.

10.10

```
(defun my-every (pred lst)
  "Return T if PRED says yes to every element of LST."
  (not (some (complement pred) lst)))
```

Check:

```
(list (my-every #'plusp '(1 2 3))
      (my-every #'plusp '(1 -2 3))
      (my-every #'plusp '()))
```

Expected: `(T NIL T)`

"Every element says yes" means "no element says no". `(complement pred)` is the "says no" test, `some` looks for one, and `not` flips the answer. The empty list gives `T`, as `every` does.

10.11

```
(defun empty-rooms ()
  "Return the names of the rooms with no items lying in them."
  (remove-if (lambda (name) (items-here name))
             (mapcar #'car *rooms*)))
```

Check:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(empty-rooms)
```

Expected: `(HALLWAY GARDEN)`, with the rooms in the order your `*rooms*` lists them.

`(mapcar #'car *rooms*)` gives the room names, and `items-here` returns `nil` for an empty room, so `remove-if` drops every room that has something in it. The lamp is in the cellar, which isn't one of the rooms, so it doesn't count.

10.12

```
(defun help ()
  "List the command words in alphabetical order."
  (concatenate 'string
               "Commands: "
               (item-names
                (sort (remove-duplicates (mapcar #'car *commands*))
                      #'string<
                      :key #'symbol-name))
               "."))
```

Check:

```
(run-command '(help))
```

Expected, with the table from Exercise 9.11:

```
"Commands: go, help, inventory, look, loot and take."
```

If you added the short forms or `drop`, they appear in their alphabetical places. `mapcar` and `remove-duplicates` both build new lists, so `sort` can rearrange the result without touching `*commands*`.

D.L. Chapter 11

11.1

```
(list (loop for i from 1 to 4 collect (* i i))
      (loop for i below 4 sum i)
      (loop for x in '(a b c) for i from 0 collect (list i x))
      (loop for tail on '(1 2 3) collect (length tail))
      (dolist (x '(1 2 3)) x)
      (loop for x in '(5 8 2 9) count (> x 4))
      (loop for c across "hello" count (char= c #\l)))
```

Expected: `((1 4 9 16) 6 ((0 A) (1 B) (2 C)) (3 2 1) NIL 3 2)`

Expression	Result	Why
Squares, 1 to 4	`(1 4 9 16)`	`to` includes 4
`sum` below 4	`6`	$0 + 1 + 2 + 3$
Two `for` clauses	`((0 A) (1 B) (2 C))`	Both step together
`for tail on`	`(3 2 1)`	Tails of lengths 3, 2, 1
`dolist`	`NIL`	`dolist` returns `nil` without a result form
`count`	`3`	`5`, `8` and `9` are above `4`
`across` + `count`	`2`	Two `l` in “hello”

11.2

```
(defun sum-to (n)
  "Return the sum of the integers from 1 to N."
  (let ((total 0))
    (dotimes (i (1+ n) total)
      (incf total i))))
```

Check:

```
(list (sum-to 10) (sum-to 0))
```

Expected: `(55 0)`

`dotimes` counts from `0`, so the loop must run `n` plus one times to include `n` itself: `(1+ n)`. The sum starts at `0`, and the first pass adds `0`, which changes nothing. With `loop` the same job is `(loop for i from 0 to n sum i)`, or from 1, since adding 0 is idle work.

11.3

```
(defun count-vowels (s)
  "Return the number of vowels in the string S."
  (loop for c across (string-downcase s)
        count (find c "aeiou")))
```

Check:

```
(list (count-vowels "Common Lisp") (count-vowels "xyz"))
```

Expected: `(3 0)`

`across` walks the string one character at a time, and `count` adds one for every character that `find` locates in `"aeiou"`. The `loop` version is shorter than the `count-if` version and does the same work.

11.4

```
(defun range (start end)
  "Return the integers from START up to, not including, END."
  (loop for i from start below end
        collect i))
```

Check:

```
(list (range 2 5) (range 0 3) (range 5 5))
```

Expected: `((2 3 4) (0 1 2) NIL)`

`below end` is the whole job: it stops before `end`, which is what "up to, but not including" means. When `start` is not below `end`, the loop runs no passes and `collect` builds an empty list. `(range 5 5)` shows that: no error, just `nil`.

11.5

```
(defun first-repeat (lst)
  "Return the first element of LST that appears again later."
  (loop for tail on lst
        when (member (first tail) (rest tail))
          return (first tail)))
```

Check:

```
(list (first-repeat '(a b c b a))
      (first-repeat '(a b c))
      (first-repeat '(x x y)))
```

Expected: `(A NIL X)`

Walking the tails, the question for each tail is "does the rest of the list hold another copy of my first element?" `member` answers that, returning the rest of the list from the match, and `return` as a clause leaves the loop with the element itself.

The second test is worth reading closely. `(first-repeat '(a b c))` checks `a` against `(b c)`, then `b` against `(c)`, then `c` against `nil`. None match, the loop ends, and a `loop` with no `return` gives `nil`.

11.6

```
(defun split (lst)
  "Return two lists: the numbers in LST, then everything else."
  (loop for x in lst
        if (numberp x) collect x into numbers
        else collect x into others
        finally (return (list numbers others))))
```

Check:

```
(list (split '(a 1 b 2 3)) (split '(1 2)) (split nil))
```

Expected: `(((1 2 3) (A B)) ((1 2)) (NIL))`

With `into`, the two lists are named variables that `finally` hands back. Elements that fail `numberp` go to `others` through the `else`.

11.7 **Wrong:** plain forms after `for` need `do`: `(loop for i from 1 to 3 do (print i))`.

Wrong: `dolist` throws its body's values away. Either `(mapcar (lambda (x) (* x 2)) '(1 2 3))` or `(loop for x in '(1 2 3) collect (* x 2))`.

Wrong: `in` walks lists, and `"abc"` is a string. Use `across`: `(loop for c across "abc" collect (char-upcase c))`.

Wrong: `finally` without `return` gives `nil`. Write `finally (return total)`.

Check the fixes:

```
(print (loop for i from 1 to 3 do (print i)))
(print (loop for x in '(1 2 3) collect (* x 2)))
(print (loop for c across "abc" collect (char-upcase c)))
(print (loop with total = 0
             for x in '(1 2 3)
             do (incf total x)
             finally (return total)))
```

Expected output:

```
1
2
3
(1 2 3)
(A B C)
6
```

Each `print` of a result ends with a space and no newline, which is why the last three lines sit where they do.

11.8

```
(defun take-while (pred lst)
  "Return the front elements of LST that PRED says yes to."
  (loop for x in lst
        while (funcall pred x)
        collect x))
```

Check:

```
(list (take-while #'plusp '(1 3 -1 2))
      (take-while #'plusp '(-1 1))
      (take-while #'plusp '(1 2 3)))
```

Expected: `((1 3) NIL (1 2 3))`

`while` tests before each pass. When it fails at `-1`, the loop stops and never collects `2`, even though `2` is positive. That is what distinguishes `take-while` from `remove-if-not`: one keeps the front run, the other keeps everything that matches.

11.9

```
(loop for name in '(ada alan)
      for score in '(90 72)
      collect (cons name score))
```

Expected: `((ADA . 90) (ALAN . 72))`

Two `for` clauses step in step, `cons` builds each pair, and `collect` gathers them. The destructuring pattern of Exercise 11.9's question goes the other way: it takes an alist apart, this one builds one.

11.10

```
(list (loop for x in '(2 4 6) always (evenp x))
      (loop for x in '(1 2 3) never (minusp x))
      (loop for x in '(a b 3 c) thereis (numberp x))
      (loop for x in '(2 4) always (evenp x) finally (return 'checked)))
```

Expected: `(T T 3 CHECKED)`

Expression	Result	Why
`always`	`T`	Every element is even
`never`	`T`	No element is negative
`thereis`	`3`	Returns the value, not just `T`
The last one	`CHECKED`	The list ran out, so `always` never said no

The last one is the subtle case. On `'(2 4)`, `always` never fails, the list ends, and `finally` runs as usual, returning `CHECKED`. On a list containing an odd number, `always` leaves the loop early and returns `nil` without running `finally`. So `finally` after `always` only runs when the loop survived to the end.

11.11

```
(defun describe-all ()
  "Print one line per room, saying what lies in it."
```

```
  (dolist (entry *rooms*)
    (let ((name (room-name entry))
          (here (items-here (room-name entry))))
      (princ name)
      (princ ": ")
      (if (null here)
          (princ "nothing")
          (princ (item-names here)))
      (terpri))))
```

Check:

```
(defparameter *item-locations*
  '((key . kitchen)
    (letter . bedroom)))
(describe-all)
```

Expected output, in the order `*rooms*` lists them:

```
BEDROOM: letter
HALLWAY: nothing
KITCHEN: key
GARDEN: nothing
CELLAR: nothing
```

`(room-name entry)` is the function from Chapter 5, which pulls the name out of a room entry, so `dolist` walks `*rooms*` and the `let` names the two things each pass needs. `princ` prints the symbols in upper case and the strings as written, exactly as in `transcript`.

Five lines, not four: `*rooms*` as defined in Chapter 7's project step lists bedroom, hallway, kitchen, garden, and cellar, in that order. If your output has a different order or a missing line, check what your `*rooms*` holds. Exercise 7.9 pushed a shed onto it, if you did that exercise; re-run the Chapter 7 project step's `defparameter` to restore the original five-room list.

11.12

```
(defun transcript (commands)
  "Print a header, run each command in COMMANDS, then a footer."
  (princ "You are in the ")
  (princ (room-name *current-room*))
  (princ ".")
  (terpri)
  (dolist (command commands)
    (princ "> ")
    (princ command)
    (terpri)
    (princ (run-command command))
    (terpri))
  (princ "Game over.")
  (terpri))

(defun count-unknown (commands)
```

```
  "Return how many commands in COMMANDS are not recognised."
  (loop for command in commands
        count (string= (run-command command)
                       "I don't know how to do that.")))
```

Check:

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))
(transcript '((take key) (dance)))
```

Expected output:

```
You are in the kitchen.
> (TAKE KEY)
You take the key.
> (DANCE)
I don't know how to do that.
Game over.
```

```
(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(setf *current-room* (find-room 'kitchen))
(count-unknown '((dance) (take key) (fly)))
```

Expected: `2`

`count-unknown` compares each reply with the exact unknown message using `string`, which is why that message must be spelled the same way in both places. A cleaner design would return a distinctive value for unknown commands, and Chapter 13's `case` or a condition (Chapter 15) would carry that idea further.

D.M. Chapter 12

12.1

```
(list (format nil "~A!" 'done)
      (format nil "~S" "hi")
      (format nil "~3D" 7)
      (format nil "~,1F" 2.25)
      (format nil "~A~A" 'a 'b)
      (format nil "a~2%b"))
```

Expected: ("DONE!" "\"hi\"" " 7" "2.2" "AB" "a

b")

Expression	Result	Why
`~A!`	`"DONE!"`	Symbols print in upper case
`~S` on “hi”	`"\"hi\""`	The string with its quotes; the backslashes are how the quotes show inside another string
`~3D` on 7	`"  7"`	Width 3, padded on the left
`~,1F` on 2.25	`"2.2"`	One digit after the point, rounded
`~A~A`	`"AB"`	Two arguments, no space unless you type it
`a~2%b`	Two newlines between a and b	

The second result printed as `"\"hi\""` is worth a second look: the string `format` built contains a quote character, and when the REPL shows that string, it shows its own quotes and escapes.

12.2

```
(defun comma-list (symbols)
  "Return SYMBOLS as a lower-case string, separated by commas."
  (format nil "~{~(~A~)~^, ~}" (mapcar #'symbol-name symbols)))
```

Check:

```
(list (comma-list '(a b c)) (comma-list '(x)) (comma-list nil))
```

Expected: `("a, b, c" "x" "")`

`~{~}` walks the list, `~(~A~)` lower-cases each element, and `~^` puts the separator only between elements. The `mapcar` is there because `symbol-name` must run before the walk; a `:key` style shortcut doesn’t exist inside `format`.

12.3

```
(format nil "~{~D: ~A~^, ~}" '((1 "one") (2 "two") (3 "three")))
```

Expected: `"1: one, 2: two, 3: three"`

The walk gives one inner list per pass, and `~D` and `~A` take the two halves apart in order — the same trick the destructuring `loop` did in Chapter 11, expressed inside the control string. `~^` keeps the commas off the last pass.

12.4

```
(defun price-line (name price)
  "Return NAME and PRICE as padded columns."
  (format nil "~12A~8,2F" name price))
```

Check:

```
(list (price-line 'coffee 3.5) (price-line 'tea 12))
```

Expected: `("coffee 3.50" "tea 12.00")`

`~12A` pads the name on the right to width 12, and `~8,2F` shows the price with two decimals in a field of at least 8. The directive reads "width, digits F": the 2 after the comma is the number of decimals from earlier in this chapter, and the 8 before it is the width.

12.5

```
(list (format nil "You have ~[no keys~;one key~:;~D keys~]." 0)
      (format nil "You have ~[no keys~;one key~:;~D keys~]." 1)
      (format nil "You have ~[no keys~;one key~:;~D keys~]." 5))
```

Expected:

```
("You have no keys." "You have one key." "You have 5 keys.")
```

Clause 0 is `"no keys"`, clause 1 is `"one key"`, and the `:;` clause catches everything else, where `~D` prints the number.

12.6

```
(defun pluralise (n noun)
  "Return N and NOUN as a lower-case phrase with the right plural."
  (format nil "~D ~[~(~A~)~:;~(~A~)s~]" n (if (= n 1) 0 1) noun))
```

Check:

```
(list (pluralise 0 'coin) (pluralise 1 'coin) (pluralise 7 'coin))
```

Expected: `("0 coins" "1 coin" "7 coins")`

`~[` chooses by an integer, but the argument is a number of any size, not an index. The `if` turns it into an index: `0` means "singular clause", `1` means "default (plural) clause". This is the standard way to collapse any count into a yes-or-no choice for `~[`. The `~(~A~)` around the noun lower-cases it, since `COIN` would otherwise print in capitals.

12.7 **Wrong:** two placeholders, one argument. Add an argument or drop an `~A`.

Wrong: the `%` starts a directive. Write `(format nil "50~~ off")`.

Wrong: `~D` takes an integer, and `"three"` is a string. Either pass `3` or use `~A`.

Wrong: `~(= is never closed with =~)`. Also, a case directive doesn't take an argument, so `'world` would be left over. What was meant is `(format nil "~(HELLO~)")`, giving `"hello"`.

12.8

```
(defun bar (n)
  "Return a health bar of N filled segments out of 10."
  (format nil "[~A~A]"
          (make-string n :initial-element #\#)
          (make-string (- 10 n) :initial-element #\-)))
```

Check:

```
(list (bar 7) (bar 0) (bar 10))
```

Expected: `("[#######---]" "[----------]" "[##########]")`

`make-string` builds a string of `n` copies of a character. The tempting one-liner, `(format nil "[~10@A]" (make-string n :initial-element #\#))`, does *not* work: `format`'s padding is always spaces, and there is no directive to change the fill character. Padding with anything else means building the second piece yourself, which the version above does. The exercise's real lesson is that `format` is not a do-everything; when the padding character matters, you write the pieces and let `format` join them.

12.9

```
(defun help ()
  "List the command words in alphabetical order."
  (format nil "Commands: ~{~(~A~)~^, ~}."
          (sort (remove-duplicates (mapcar #'car *commands*))
                #'string< :key #'symbol-name)))
```

Check:

```
(run-command '(help))
```

Expected, with the table from Exercise 9.11:

```
"Commands: go, help, inventory, look, loot, take."
```

The period after the walk comes from the control string. This reply doesn't use "and" — one walk, one separator, as `item-list-string` explained in the project step. The `item-names` version from Exercise 10.12 remains the better reply for people; this one is the cheaper one to maintain. If you added the short forms or `drop`, they appear in their alphabetical places.

12.10

```
(defun item-phrase (items)
  "Return ITEMS as a sentence fragment: \"Key and Letter\" and so on."
  (cond ((null (cddr items))
         (format nil "~{~:(~A~)~^ and ~}" (mapcar #'symbol-name items)))
        (t
         (format nil "~{~:(~A~)~^, ~} and ~:(~A~)"
                 (mapcar #'symbol-name (butlast items))
                 (first (last items))))))

(defun describe-room-text (room)
  "Return a sentence describing ROOM and what lies there."
  (let ((here (items-here room)))
    (format nil "You are in the ~(~A~). ~:[There is nothing here.~;You see
~A.~]~&"
            room
            here
            (if here (item-phrase here) ""))))
```

Check:

```
(defparameter *item-locations*
  '((key . kitchen)
    (letter . bedroom)
    (coin . hallway)))
```

```
(push '(lamp . cellar) *item-locations*)
(list (describe-room-text 'garden)
      (describe-room-text 'kitchen)
      (describe-room-text 'cellar))
```

Expected:

```
("You are in the garden. There is nothing here."
 "You are in the kitchen. You see Key."
 "You are in the cellar. You see Lamp and Coin.")
```

Two new functions appear here, both old friends under other names: `butlast` returns the list without its last element, and `last` returns the last element as a list, which is why `(first (last ...))` takes the element itself. The two-item case has no comma, the three-item case gets `"Key, Letter and Coin"`, and the walk in `item-phrase` handles any longer list the same way.

Note the split of duties: `item-phrase` decides the wording, `describe-room-text` decides the sentence. That's the same shape as `item-names` and `inventory` from earlier chapters, and it keeps each function small enough to check on its own.

D.N. Chapter 13

13.1

```
(defstruct book
  title
  author
  (year nil))

(defparameter my-book
  (make-book :title "Paradises Lost" :author "Ursula K. Le Guin"))

(list (book-title my-book)
  (book-year my-book))
;; => ("Paradises Lost" NIL)

(setf (book-year my-book) 1974)

(list (book-year my-book)
  (book-p my-book)
  (book-p 'book))
;; => (1974 T NIL)
```

The `year` field defaults to `nil` because of the slot option, and `book-p` knows the difference between a real book and the symbol named `book`.

13.2

```
(list (aref (vector 10 20 30) 2)
  (length (vector 'a 'b))
  (aref "lisp" 0)
  (multiple-value-list (floor 17 5))
  (multiple-value-list (gethash 'x (make-hash-table))))
```

Expected: `(30 2 #\l (3 2) (NIL NIL))`

Expression	Result	Why
`aref` on 2	`30`	Indexes count from zero
`length` of vector	`2`	`length` works on any sequence
`aref` on "lisp"	`#\l`	Strings are vectors of characters
`floor`	`(3 2)`	Two values: $17 = 5 \times 3 + 2$
`gethash` on empty	`(NIL NIL)`	No value, and the key was never there

13.3

```
(defparameter prices (make-hash-table :test #'equal))

(setf (gethash "lamp" prices) 10.0)
(setf (gethash "key" prices) 2.5)
(setf (gethash "letter" prices) 0.5)

(defun total (names)
  "Return the total price of the items named in NAMES."
  (reduce (lambda (sum name)
      (+ sum (gethash name prices 0)))
    names
    :initial-value 0))
```

Check:

```
(total '("lamp" "key"))
```

Expected: `12.5`

The `:initial-value 0` makes the empty list safe, as Chapter 10 recommended. A `reduce` walks the names and asks the hash table for each price; the hash table's speed doesn't matter at three entries and matters at thirty thousand.

13.4

```
(defun hash-keys (table)
  "Return the keys of TABLE as a list."
  (loop for key being the hash-keys of table
    collect key))

(sort (hash-keys caps) #'string< :key #'symbol-name)
```

`collect` gathers whatever order `maphash`-style walking produces, and `sort` imposes one afterwards. Without the sort, two runs might print the keys differently.

13.5 Wrong: `assoc` walks lists; `caps` is a hash table. Use `(gethash 'one caps)`.

Wrong: `aref` needs a vector. `(nth 1 '(a b c))` for the list, or `(aref (vector 'a 'b 'c) 1)` for the array.

Wrong: the default test is `eq`, and two `"a"` strings are different objects. Either `(make-hash-table :test #'equal)` or use symbol keys.

Wrong: `make-room` takes keyword arguments: `(make-room :name 'kitchen :description "desc" :exits '(west))`.

Check the fixes:

```
(defparameter caps (make-hash-table))
(setf (gethash 'one caps) 1)
(defparameter ages (make-hash-table :test #'equal))
(setf (gethash "a" ages) 1)

(list (gethash 'one caps)
  (nth 1 '(a b c))
  (aref (vector 'a 'b 'c) 1)
  (gethash "a" ages)
  (room-name (make-room :name 'kitchen)))
```

Expected: `(1 B B 1 KITCHEN)`

13.6

```
(defun oldest (ages)
  "Return the name of the oldest person in the hash table AGES."
  (let ((best-name nil)
        (best-age -1))
    (maphash (lambda (name age)
        (when (> age best-age)
          (setf best-name name)
          (setf best-age age)))
      ages)
    best-name))
```

Check:

```
(defparameter ages (make-hash-table))
(setf (gethash 'ada ages) 36)
(setf (gethash 'alan ages) 41)
(setf (gethash 'grace ages) 45)
(oldest ages)
```

Expected: `GRACE`

`maphash` visits entries in unspecified order, so the answer can't depend on the walk finding the biggest first: the `let` variables keep the best seen so far, and ties go to whoever was seen first. This is the accumulator pattern on a hash table.

13.7

```
(list (equalp (make-room :name 'a :exits '(n))
    (make-room :name 'a :exits '(n)))
  (eq (make-room :name 'a) (make-room :name 'a))
  (room-p (make-room))
  (eql 1.0 1))
```

Expected: `(T NIL T NIL)`

`equalp` compares structures field by field, `eq` asks for the same object and each `make-room` builds a fresh one, `room-p` accepts any room even with every field `nil`, and `eql` still refuses to mix integers with floats.

13.8

```
(defun invert (table)
  "Return a new hash table with TABLE's keys and values swapped."
  (let ((result (make-hash-table)))
    (maphash (lambda (k v) (setf (gethash v result) k)) table)
    result))
```

Check:

```
(defparameter caps (make-hash-table))
(setf (gethash 'one caps) 1)
(setf (gethash 'two caps) 2)

(list (gethash 1 (invert caps))
  (gethash 2 (invert caps)))
```

Expected: `(ONE TWO)`

A fresh table, one `maphash` to fill it, and the assumption that no value appears twice holds. If two keys shared a value, one of them would be lost in the swap — the same warning as `remove-duplicates` keeping one copy.

13.9

```
(defun go-to (name)
  "Move the player to the room called NAME, if it is an exit."
  (cond ((eq name (room-name current-room))
         "You are already there.")
    ((not (member name (room-exits current-room)))
      "You can't go there from here.")
    ((room-locked (find-room name))
      (format nil "The ~(~A~) is locked." name))
    (t
      (setf current-room (find-room name))
      (look))))
```

Check:

```
(setf current-room (find-room 'kitchen))
(list (go-to 'garden)
  (setf (room-locked (find-room 'garden)) nil)
  (go-to 'garden)
  (go-to 'hallway))
```

Expected:

```
("The garden is locked."
NIL
"You are in the Garden. An overgrown garden."
"You can't go there from here.")
```

The third `go-to` unlocks the garden before trying again; the fourth asks to walk from the garden to the hallway, which isn't an exit. Notice the order of clauses: the "not an exit" check comes before the locked check, so unlocking a room the player can't reach (like a

hypothetical locked cellar from the bedroom) still answers "can't go there", not "locked". The chapter 6 hard-coded `eq name 'garden` test is gone; the knowledge lives in the room's slot now.

13.10

```
(defun locked-rooms ()
  "Return the names of the locked rooms."
  (mapcar #'room-name
    (remove-if-not #'room-locked rooms)))
```

Check, with the garden locked again:

```
(setf (room-locked (find-room 'garden)) t)
(locked-rooms)
```

Expected: `(GARDEN)`

`remove-if-not` keeps the rooms whose `room-locked` is true, and `mapcar` reduces them to names. `room-locked` works directly as a predicate because `nil` is false and `t` is true (Chapter 6).

13.11 The natural way to give rooms an `items` field is to add it to the existing structure, one more redefinition:

```
(defstruct (room (:print-function print-room))
  name
  description
  exits
  (locked nil)
  (items nil))
```

Now the two commands, with the game's alist-based state replaced by a plain list `carrying` and the room's own `items` field:

```
(defparameter carrying nil)

(defun take-struct (item)
  "Take ITEM from the current room's items field."
  (if (member item (room-items current-room))
    (progn
      (setf (room-items current-room)
        (remove item (room-items current-room)))
      (push item carrying)
      (format nil "You take the ~(~A~)." item))
    "You don't see that here."))

(defun drop-struct (item)
  "Put ITEM down in the current room."
  (if (member item carrying)
    (progn
      (setf carrying (remove item carrying))
      (push item (room-items current-room))
```

```
      (format nil "You drop the ~(~A~)." item))
    "You don't have that."))
```

Check:

```
(setf (room-items (find-room 'kitchen)) '(key lamp))
(setf current-room (find-room 'kitchen))

(list (take-struct 'key)
  (take-struct 'key)
  (drop-struct 'key)
  (room-items (find-room 'kitchen))
  carrying)
```

Expected: `("You take the key." "You don't see that here." "You drop the key." (KEY LAMP) NIL)`

The prose answer is yours to write, but here is what the comparison shows. `take` and `drop` became shorter: each one edits one room's field, and the whole `set-location` copying dance from Chapter 8 disappears, because a structure's field is one mutable slot instead of a shared list that must be rebuilt. What got longer is the data side: rooms must be built with `make-room`, and they can no longer be written as a quoted alist in one line. The general trade: mutable fields simplify the functions that change things and complicate the places where data is created and read back.

Write your own paragraph here if the comparison sparked other thoughts — that's the point of the exercise. The connection to Chapter 27's plist advice is worth noticing: the items live on the room because the room is a named, owned structure; putting them on the symbol `kitchen` via `setf (get 'kitchen 'items)` would be a plist mistake for the same reasons there discussed.

D.O. Chapter 14

14.1 Expansions:

```
(twice (princ 'x))
;; => (PROGN (PRINC 'X) (PRINC 'X))

(my-when (> 2 1) (princ 'a))
;; => (IF (> 2 1) (PROGN (PRINC 'A)) NIL)

(swap p q)
;; => (LET ((#:G1234 P)) (SETF P Q) (SETF Q #:G1234))

(square (+ 1 2))
;; => (LET ((#:G1235 (+ 1 2))) (* #:G1235 #:G1235))
```

The gensym numbers differ on every run; that's the point.

Running them, with `(defparameter p 1)` and `(defparameter q 2)` set first: `twice` prints `XX` and returns `X` (the value of `princ`'s last argument); `my-when` prints `A` and returns `A` — the

`if`'s then branch is the `progn`'s value, which is `princ`'s return, which is the argument; `swap` exchanges `p` and `q` to `2` and `1`; and `square` evaluates `(+ 1 2)` once and returns `9`.

```
(defparameter p 1)
(defparameter q 2)
(swap p q)
(list p q)
```

Expected: `(2 1)`

```
(twice (princ 'x))
(terpri)
(princ (my-when (> 2 1) (princ 'a)))
```

Expected output:

```
XX
A
```

The first line is the two `princ` calls from the expansion; the second is the value `my-when` returned, which the REPL did not print because `:results output` shows only what was printed. If you want the return values, drop `:results output` and wrap each form in `list`.

14.2

```
(defmacro my-or2 (a b)
  "Return A if true, otherwise B — without evaluating B unless needed."
  (let ((v (gensym)))
    `(let ((,v ,a))
       (if ,v ,v ,b))))
```

Check:

```
(list (my-or2 nil 7)
      (my-or2 3 7)
      (my-or2 nil nil))
```

Expected: `(7 3 NIL)`

The comment the exercise asks for:

No function can do this: a function receives both arguments already evaluated, so (my-or-fn (expensive-a) (expensive-b)) would run expensive-b even when expensive-a was true. It also couldn't return a's value as the answer — it can only return what its body chose to return, and by then a's form has already been run to produce the argument.

The `gensym` stops a caller variable named `v` from colliding, and also evaluates `a` only once — the two purposes, in one binding.

14.3

```
(defmacro repeat (n &body body)
  (let ((count (gensym))
        (limit (gensym)))
    `(let ((,limit ,n))
```

```
      (dotimes (,count ,limit)
        ,@body))))
```

Check:

```
(repeat (+ 1 2) (princ 'x))
(terpri)
(repeat (progn (princ "evaluated ") 3) (princ 'y))
```

Expected output:

```
XXX
evaluated YYY
```

The limit form is bound to `,limit` once, in the outer `let` — the `dotimes` then reads a variable, not the form. That's what makes the "evaluated " appear once, not three times. A `loop` version could use `below ,n` directly, which also evaluates once (a `loop` clause evaluates its bound form once, at loop setup), but the `dotimes` version with the explicit `let` makes the once-ness visible in the expansion — which is the teaching point.

14.4 **`first-of`** — the `let` binding `head` collides with a caller variable named `head`. A caller's `(first-of head)` would read its own `head` inside the macro's binding, not the outer one, and the result would be wrong in a way that depends on the caller's local names. Fix:

```
(defmacro first-of (lst)
  (let ((v (gensym)))
    `(let ((,v ,lst))
       (first ,v))))
```

`inc-twice` — `,x` appears twice, so the form is evaluated twice. Worse, the expansion's *meaning* shifts with the argument: passing a variable is fine, but passing `(incf n)` increments `n` by one and then adds the result of that to itself; passing an array reference like `(aref *a* 0)` evaluates the access twice. Fix — bind once, then incf the binding:

```
(defmacro inc-twice (x)
  (let ((v (gensym)))
    `(let ((,v ,x))
       (incf ,v ,v)
       ,v)))
```

Note `incf` on `,v` is safe because `v` is a gensym the caller can never pass in. What `inc-twice` means is "add twice the value of `x` to some variable"? No — as written it adds `x` to itself, which is only meaningful when `x` is a place (a `setf`-able expression). The fix above preserves the `place` semantics by evaluating `x` once and then setting it. If the intent was purely "double the value, no side effect", a function would be correct and simpler.

`unless-bad` — `,body` without the `@` inserts the body as *one list*, so `(if test nil ((princ 1) (princ 2)))` compiles the body as a function call whose head is `(princ 1)`. Fix:

```
(defmacro unless-fixed (test &body body)
  `(if ,test nil (progn ,@body)))
```

14.5

```
(defmacro benchmark (form)
  `(progn
     (princ "start ")
     (let ((result ,form))
       (princ "end ")
       result)))
```

Check:

```
(benchmark (+ 1 2))
```

Expected: `3`, with `"start end "` printed to the REPL window (add `:results output` and call `princ` on the result to see both together, since the block otherwise shows only the value `3`).

The comment the exercise asks for:

benchmark must be a macro because it needs to wrap the form — put code before and after it, in the same lexical scope — without evaluating the form to pass it in. A function (benchmark (heavy-computation)) would run heavy-computation before benchmark was called; its "start" line would print after the work had already been done. The macro's expansion is what runs, in the caller's environment.

14.6

```
(defmacro for-each ((x lst) &body body)
  `(dolist (,x ,lst)
     ,@body))
```

Expansion:

```
(macroexpand-1 '(for-each (r *rooms*) (princ (room-name r))))
```

Expected:

```
(DOLIST (R *ROOMS*) (PRINC (ROOM-NAME R)))
```

`R` is the pattern variable's name, inserted directly. No gensym is needed here — `dolist` binds `r` as its own local, so a caller's `r` is *shadowed* by the loop binding, not clobbered. The macro-hygiene rule from the chapter is about macros that introduce bindings around the caller's body (like `swap`); a macro that delegates to a standard looping form inherits that form's already-correct scoping.

Usage:

```
(for-each (room *rooms*)
  (princ (room-name room))
  (terpri))
```

Expected output (with the five-room topology from Chapter 13):

```
BEDROOM
HALLWAY
KITCHEN
GARDEN
CELLAR
```

14.7

```
(defun rewind ()
  "Undo until the history is empty; return a summary message."
  (loop with steps = 0
        while *history*
        do (undo)
           (incf steps)
        finally (return
                  (format nil "Rewound ~D step~:p." steps))))

(defparameter *commands*
  '((look . look)
    (inventory . inventory)
    (take . take)
    (go . go-to)
    (loot . loot)
    (drop . drop)
    (undo . undo)
    (rewind . rewind)))
```

The `:p` in `~:p` is `format`'s plural directive: it prints `"s"` for any number but `1`, and nothing for `1`. Check:

```
(defparameter *item-locations*
  '((lamp . cellar) (key . kitchen) (letter . bedroom)))
(setf *history* nil)
(setf *current-room* (find-room 'kitchen))

(list (run-command '(take key))
      (run-command '(take lamp))
      (rewind))
```

Expected: two take messages and `"Rewound 2 steps."` — the rewind rolled back both moves. Run `rewind` on an empty history and it says `"Rewound 0 steps."` — `~:p` prints no `s` there, because 0 isn't 1, but the message reads naturally either way.

Note what the `while *history*` test does: it stops when the history is empty, so `undo`'s "Nothing to undo." branch is never taken inside the loop. The count reflects moves actually rolled back.

14.8

```
(defmacro if-room (name then &body else)
  "If (find-room NAME) succeeds, evaluate THEN with IT bound to it."
  (let ((room-var (gensym)))
    `(let ((,room-var (find-room ,name)))
       (if ,room-var
           (let ((it ,room-var))
```

```
             ,then)
           (progn ,@else)))))
```

Check:

```
(list (if-room kitchen
          (format nil "~A is found" (room-name it))
        "No such room.")
      (if-room attic
          "Found"
        "No such room."))
```

Expected: `("KITCHEN is found" "No such room.")`

The `gensym` hides the room from the caller. `it` is deliberately a readable name, introduced *for* the caller in `then` — a caller with its own variable named `it` is shadowed by the macro's binding, which is the caller's own doing, since `it` is the macro's contract. This is the standard `if-let` shape, and it's exactly the kind of form — a new binding visible in one branch — that no function can create.

D.P. Chapter 15

15.1

```
(list (ignore-errors (+ 1 2))
      (ignore-errors (nth 5 '(a b)))
      (handler-case (/ 4 0) (division-by-zero () 'nope))
      (handler-case (car 'sym) (type-error (c) (type-error-datum c))))
```

Expected: `(3 NIL NOPE SYM)`

`(ignore-errors (+ 1 2))` has no error to ignore, so it returns `3`. `(ignore-errors (nth 5 '(a b)))` — `nth` tolerates an index past the end (it walks to `nil` and returns `nil`), so there's no error to ignore and the answer is just `nil`. `(car 'sym)` is a type error, and the handler pulls the datum out of the condition — `SYM`, the thing that wasn't a list.

The `warn` form: `(warn "test")` prints a warning line and returns `nil`. The return value is `nil`, not the printed text; `warn`'s job is the message, not the value.

15.2

```
(defun safe-nth (index lst)
  "Return the INDEX-th element of LST, or NIL when out of range."
  (handler-case (nth index lst)
    (type-error () nil)))
```

Check:

```
(list (safe-nth 1 '(a b c))
      (safe-nth 5 '(a b c))
      (safe-nth 0 nil)
      (safe-nth -1 '(a b c)))
```

Expected: `(B NIL NIL NIL)`

`nth` tolerates indexes past the end — it walks until the list runs out and returns `nil` — but a *negative* index is rejected with a `type-error` saying the index is not of type `(UNSIGNED-BYTE ...)`. The handler catches that and returns `nil`, which is the right answer for "I want `nil` when the index doesn't point at an element". If your implementation answers `A` for a negative index, it is not following the standard — SBCL and CCL both signal.

Which means the exercise is really testing whether you noticed that "out of range" has two senses: too big (silently `nil`) and negative (signals). `safe-nth` catches only the signal, so it answers `nil` for both.

15.3

```
(define-condition invalid-move (error)
  ((reason :initarg :reason :reader invalid-move-reason))
  (:report (lambda (condition stream)
             (format stream "You can't do that: ~A."
                     (invalid-move-reason condition)))))

(defun check-move (direction)
  "Signal INVALID-MOVE unless DIRECTION is an exit."
  (unless (member direction (room-exits *current-room*))
    (error 'invalid-move
           :reason (format nil "no exit ~(~A~)" direction)))
  direction)

(defun try-move (direction)
  "Return DIRECTION when it works, else the reason it doesn't."
  (handler-case (check-move direction)
    (invalid-move (c) (invalid-move-reason c))))
```

Check:

```
(setf *current-room* (find-room 'kitchen))
(list (try-move 'hallway) (try-move 'north))
```

Expected: `(HALLWAY "no exit north")`

The condition carries its facts in slots, and the handler reads them through the reader — never by parsing the report text, which exists only for people.

15.4 **Wrong:** `format` already ran, so `error` receives a finished string and treats it as a format control containing no directives — it works by accident, but the condition loses the pieces. Write `(error "Bad value: ~A" v)`.

Wrong: clause order. `error` catches everything, so the specific `parse-error` clause can never run. Most specific first:

```
(handler-case (parse-integer s)
  (parse-error () "unparseable")
  (error () "bad"))
```

Wrong: `invoke-restart` reaches restarts in scope *where the error was signalled*. With no `restart-case` inside `body`'s code, the restart isn't active, and the handler fails with "restart MY-RESTART is not active". Wrap the body:

```
(handler-case (restart-case body
                (my-restart () 'fell-back))
  (error (c) (declare (ignore c)) (invoke-restart 'my-restart)))
```

15.5

```
(defun to-integer (text)
  "Return the integer in TEXT, or 0 with a warning."
  (handler-case (parse-integer text)
    (parse-error ()
      (warn "No integer in ~S." text)
      0)))
```

Check:

```
(list (to-integer "42") (to-integer "nope"))
```

Expected output and value:

```
WARNING: No integer in "nope".
(42 0)
```

`~S` prints the bad string with quotes, which distinguishes `"7"` from `7` in the warning — the reason `~S` exists next to `~A`.

15.6

```
(defun collect-results (strings)
  "Return the integers parsed from the parseable strings."
  (let ((found nil))
    (loop for s in strings
          do (handler-case (push (parse-integer s) found)
               (parse-error () nil)))
    (nreverse found)))
```

Check:

```
(collect-results '("1" "no" "2" "3x" "3"))
```

Expected: `(1 2 3)`

`push` builds backwards, `nreverse` puts it straight without copying. `parse-integer` on `"3x"` stops after `3` by default and returns `3` with a second value of `1` — it doesn't signal unless *nothing* is there. The `append` alternative the exercise hints at: `(setf found (append found (list value)))` keeps order without the final reverse, at the cost of copying every time — fine here, slow on long lists.

15.7

```
(defun run-session (commands)
  "Run every command, collecting replies."
  (loop for command in commands
        collect (handler-case (run-command command)
                  (error () "That command failed."))))
```

Check:

```
(defparameter *commands*
  '((look . look) (take . take) (go . go-to)))
(setf *current-room* (find-room 'kitchen))
(defparameter *item-locations* '((key . kitchen)))

(run-session '((look) (take) (dance)))
```

Expected: the look text, "Wrong number of arguments for that command.", and "I don't know how to do that."

Why those three: `(look)` returns the description; `(take)` signals `program-error` (wrong argument count) inside `apply`, and `run-command`'s own `handler-case` catches it and returns the "wrong number of arguments" string; `(dance)` is an unknown command, so `call-command` returns "I don't know how to do that." as a plain string, no signal at all.

So in this example, `run-session`'s `handler-case` never fires. What it's for is the case `run-command`'s own handler misses — a bug in the command table itself, or a command function that signals a condition of a type `run-command`'s handler doesn't name. Both layers need a handler because they protect different things:

- `run-command`'s `handler-case` knows about command-shaped problems (wrong argument counts) and answers specifically.
- `run-session`'s is the safety net at the top: one bad command must not end the session, whatever the shape of the failure.

Catch-all at the top, narrow catches inside — the same discipline `move-or-stay` showed in the chapter.

15.8

```
(defun call-command (command)
  "Call the function that handles COMMAND, offering repair."
  (let ((entry (assoc (first command) *commands*)))
    (restart-case
        (if (null entry)
            (error 'simple-error
                   :format-control "Unknown command: ~(~A~)."
                   :format-arguments (list (first command)))
            (apply (cdr entry) (rest command)))
      (repair ()
        :report "Run LOOK instead."
        (look)))))

(defun run-command (command)
  "Run COMMAND, repairing unknown commands as LOOK."
  (handler-case (call-command command)
    (program-error ()
      "Wrong number of arguments for that command.")
    (simple-error ()
      (invoke-restart 'repair))))
```

Check:

```
(defparameter *commands*
  '((look . look) (take . take) (go . go-to)))
(setf *current-room* (find-room 'kitchen))
(run-command '(dance))
```

Expected: the kitchen's description — `look`'s reply.

`error` with a symbol names your own condition type; with a string it builds a `simple-error`, and the directives ride along as `:format-control` and `:format-arguments`. The handler catches it and invokes `repair`, whose restart body runs `look`. From the debugger menu, the same `repair` restart appears by its `:report` text — one mechanism, two ways to choose.

D.Q. Chapter 16

16.1

```
(defpackage :game
  (:use :cl)
  (:export :look :take))
(in-package :game)
(defun take (item) (format nil "taking ~A" item))
(in-package :cl-user)

(list (symbol-package 'defun)
      (symbol-package 'game:take)
      (keywordp :look)
      (symbol-package (first '(a b))))
```

Expected: the `COMMON-LISP` package, the `GAME` package, `T`, and `COMMON-LISP-USER`.

`(first '(a b))` is `a`, read in whichever package was current when you ran the block — `cl-user` here. Symbols land in the package that was current *when they were read*, which is why the same name can exist in two packages as two different symbols.

16.2

```
(with-open-file (out "/tmp/haiku.txt"
                     :direction :output
                     :if-exists :supersede)
  (write-line "An old silent pond" out)
  (write-line "a frog jumps into the pond" out)
  (write-line "splash, silence again" out))

(with-open-file (in "/tmp/haiku.txt")
  (loop for line = (read-line in nil)
        while line
        collect line))
```

Expected: `("An old silent pond" "a frog jumps into the pond" "splash, silence again")`

One stream, two trips: write with `:direction :output`, then read with the default `:input`. `read-line` strips the newlines, so the list holds the three lines as bare strings.

16.3

```
(defun append-note (text)
  "Add TEXT to the end of /tmp/notes.txt."
```

```
  (with-open-file (out "/tmp/notes.txt"
                       :direction :output
                       :if-exists :append
                       :if-does-not-exist :create)
    (write-line text out)))
```

Check:

```
(append-note "first note")
(append-note "second note")

(with-open-file (in "/tmp/notes.txt")
  (loop for line = (read-line in nil)
        while line
        collect line))
```

Expected: `("first note" "second note")` — and running the `append-note` lines again grows the file, which is what `:append` means.

16.4 The result of the `read` is `(1 (2 3) "four")`.

`print` writes Lisp-readable text: it puts the quotes on the string, because without them, `read` would see the symbols `four` and the round trip would lose the type. This is the `print` / `princ` distinction from Chapter 1 doing real work: `princ` writes for people, `print` writes for `read`.

16.5 **Wrong:** the stream is saved in `*saved-lines*`, but `with-open-file` closes it when the body ends. The later `read-line` fails with a closed-stream error. Read everything inside the body, and store the *lines*:

```
(with-open-file (in "/tmp/notes.txt")
  (setf *saved-lines*
        (loop for line = (read-line in nil)
              while line
              collect line)))
```

Wrong: no `:if-exists`. The second run signals `file-error`, because the file exists and Lisp won't clobber it without permission. Add `:if-exists :supersede` (replace) or `:append` (add).

Wrong: `open` with no one to close it. The stream leaks: on Linux it stays open until the Lisp image exits, on Windows a leaked writer can block other programs from the file. Wrap it:

```
(with-open-file (in "/tmp/notes.txt")
  (read-line in))
```

16.6

```
(defun count-lines (path)
  "Return the number of lines in the file at PATH."
  (with-open-file (in path)
    (loop for line = (read-line in nil)
          while line
          count line)))
```

Check:

```
(count-lines "/tmp/haiku.txt")
```

Expected: `3`

The `count` clause adds one for every non-`nil` line, doing the counting the `collect` version's `length` would do. One caveat: a file whose last line has no trailing newline still counts, because `read-line` returns that final text; a file ending in a newline doesn't gain a phantom empty line, since `read-line` returns `nil` after the last real one.

16.7

```
(defpackage :shop
  (:use :cl)
  (:export :sell :buy))
(in-package :shop)

(defun price (item)
  "Private: the price of ITEM."
  (if (eq item 'key) 5 10))

(defun sell (item)
  (format nil "Selling ~A for ~D coins." item (price item)))

(defun buy (item)
  (format nil "Buying ~A for ~D coins." item (price item)))

(in-package :cl-user)

(list (shop:sell 'key)
      (ignore-errors (shop:price 'key))
      (shop::price 'key))
```

Expected: `"Selling KEY for 5 coins."`, `NIL`, and `5`.

`shop:price` fails because `price` was never exported: the single colon demands a public symbol. The double colon in `shop::price` means "reach in anyway", and works — handy for your own debugging, bad manners between strangers' packages.

16.8

```
(defun game-state ()
  "Return a single list describing the whole game state."
  (list *item-locations*
        (room-name *current-room*)
        *history*))

(defun restore-state (state)
  "Set the game state from the list STATE."
  (setf *item-locations* (first state))
  (setf *current-room* (find-room (second state)))
  (setf *history* (third state)))
```

Check. Take the key, save, wreck everything, load, undo:

```
(defparameter *item-locations*
  '((lamp . cellar) (key . kitchen) (letter . bedroom)))
```

```
(setf *history* nil)
(setf *current-room* (find-room 'kitchen))
(snapshot)
(run-command '(take key))
(save-game)

(setf *item-locations* nil)
(setf *history* nil)
(setf *current-room* (find-room 'bedroom))

(load-game)
(undo)
(item-location 'key)
```

Expected: `KITCHEN` — the key is back in the kitchen, because `undo` restored the snapshot that had travelled through the save file too.

What changed: one element in `game-state`, one `setf` in `restore-state`. What didn't: `save-game`, `load-game` and every other function, because they pass the whole state around as one opaque object. That's the payoff of writing the format as functions rather than spreading `print` calls through the code.

The subtle point: `*history*` is a list of alists, and `print` writes each one as Lisp data. When the save file is read back, the history list and each of its alist snapshots come back fresh, distinct objects — `equal` to the originals but not `eq` to them. That's exactly what you want from a save file: independent copies, not shared structure that would be corrupted by later play.

16.9

```
(defparameter *report-path* "/tmp/game-report.txt")

(defun save-report ()
  "Write the room report to *REPORT-PATH*; return lines written."
  (let ((lines nil))
    (dolist (entry *rooms*)
      (let ((here (items-here (room-name entry))))
        (push (format nil "~(~A~): ~A"
                      (room-name entry)
                      (if here (item-names here) "nothing"))
              lines)))
    (setf lines (nreverse lines))
    (with-open-file (out *report-path*
                         :direction :output
                         :if-exists :supersede)
      (dolist (line lines)
        (write-line line out)))
    (length lines)))
```

Check:

```
(defparameter *item-locations*
  '((key . kitchen) (letter . bedroom)))
```

```
(save-report)

(with-open-file (in *report-path*)
  (loop for line = (read-line in nil)
        while line
        collect line))
```

Expected: `5` from `save-report`, and the five lines back from the file, in room order.

The lines are collected first and written second, so the count and the file always agree — writing and counting in one pass would need a counter threaded through the writing loop, and the split is simpler to read.

On the tilde: on Linux and macOS, many Lisps expand a leading `~/` to your home directory. On Windows, what `~` means varies between implementations, and the honest habit is the one this exercise forced: put the path in a `defparameter` at the top of your program, so the one place it lives is easy to find and change.

D.R. Chapter 17

17.1 The lines print: `1 2 3 3 2` — five numbers on one line, since each level prints on entry and on exit.

```
(defparameter *depth* 0)

(defun show () (format t "~D " *depth*))

(defun nest (n)
  (let ((*depth* (1+ n)))
    (show)
    (when (< n 2) (nest (1+ n)))
    (show)))

(nest 0)
```

Walking it: `n=0` binds `*depth*` to 1, prints `1`, calls `nest 1` which binds to 2 and prints `2`, calls `nest 2` which binds to 3 and prints `3`, doesn't recurse, prints `3` on exit; back in `nest 1`, its binding is in force again, so `2` prints; back in `nest 0`, `1`. Each `let` pushes a binding and each return pops it — the stack of bindings mirrors the call stack, which is exactly what "dynamic extent" means. Notice that `show` takes no argument and needs none: the same function prints 1, 2, 3, 3, 2 because its caller rebound the variable it reads.

17.2

```
(defun with-temp-room (room-name fn)
  "Run FN with *CURRENT-ROOM* bound to ROOM-NAME; restore after."
  (let ((*current-room* (find-room room-name)))
    (funcall fn)))
```

The comment the exercise asks for:

Plain let already restores **current-room** even if fn signals — that is what dynamic extent means. The binding is unwound by the same machinery that unwinds the stack for an error

(Chapter 15's unwind-protect semantics are built into every binding form). The temptation to add unwind-protect is a category error in the useful direction: unwind-protect is needed for actions (closing files), not for bindings, which the language restores for you.

Check:

```
(setf *current-room* (find-room 'bedroom))

(with-temp-room 'cellar
  (lambda () (princ (room-name *current-room*))))
;; prints: CELLAR

(room-name *current-room*)   ; => BEDROOM — restored
```

And with a signalling body, to confirm the restore survives it:

```
(ignore-errors
  (with-temp-room 'garden
    (lambda () (error "boom"))))
(room-name *current-room*)   ; => BEDROOM — still restored
```

That second check is the point: `ignore-errors` catches the error, the `let` has unwound, and `*current-room*` is back to what it was before the call. No `unwind-protect` needed.

17.3 **Wrong:** `*counter*` is created by a bare `setf`, so it's a plain global with a misleading earmuffed name. SBCL warns; more importantly, nothing marks the name as special, so any later `defvar` on the same name would silently change its behaviour. Fix:

```
(defparameter *counter* 0)
(defun bump () (incf *counter*))
```

Wrong: `loud-say` *assigns* `*game-output*`, so after the call every later message in the whole program goes to a stream nobody can read — the change is permanent, not scoped. And the stream is never drained. The fix is a binding, not an assignment:

```
(defun loud-say (text)
  "Return TEXT, shouted, as a string."
  (with-output-to-string (s)
    (let ((*game-output* s))
      (format *game-output* "LOUD: ~A" text))))
```

The output is the return value; `*game-output*` is untouched after the call returns.

Wrong: `name` has no earmuffs but is `defvar`'d — it *is* dynamic despite the name, and every `let` on `name` anywhere in the program is now a global temporary change. The comment "intended as a per-call local" names the confusion directly: per-call locals are lexical `let` bindings or parameters, never `defvar`. Fix: delete the `defvar` and use a parameter or plain `let`:

```
(defun name-of (room)
  (let ((name (room-name room)))
    (string-capitalize (symbol-name name))))
```

17.4

```
(defun config-value (name-string)
  "Return the value of the dynamic variable named by NAME-STRING."
  (let ((sym (intern name-string)))
    (if (boundp sym)
        (symbol-value sym)
        :unknown)))
```

Check:

```
(defparameter *save-file* "/tmp/game-save.lisp")
(list (config-value "*SAVE-FILE*")
      (config-value "*NOTHING-LIKE-THIS*")
      (config-value "CAR"))
```

Expected: the save path, `:unknown`, and `:unknown` — no error.

The `"CAR"` result is the subtle one. `car` is a function, not a variable, so `boundp` returns `nil` and the function returns `:unknown`. That's the triple from Chapter 9 made concrete: a symbol has a value slot, a function slot, and a plist, and the three are queried by different functions. `boundp` and `symbol-value` concern only the value slot, so a symbol whose function slot is occupied can still be unbound as a variable — and vice versa.

The `intern` call is the reader's own machinery, used by hand: `intern` looks the string up in a package's table, creating the symbol if absent. Passing `"*SAVE-FILE*"` with its asterisks matters — the string must be the symbol's exact (upcased) name, case and punctuation included.

17.5

```
(defvar *log* '())

(defun log-it (x) (push x *log*))

(defun phase-one ()
  (let ((*log* nil))
    (log-it 'a)
    (log-it 'b)))

(phase-one)
*log*    ; => NIL — empty!
```

`a` and `b` went into the `let`'s binding, which vanished when `phase-one` returned. The global `*log*` was never touched. That's the sandbox reading of a dynamic `let`: whatever is written inside is written into a temporary world, and the temporary world is discarded at the end.

The fix depends on intent, and there are two honest answers:

Intent 1 — entries should survive the call. Don't rebind `*log*`; write to the global directly:

```
(defun phase-one ()
  (log-it 'a)
  (log-it 'b))
```

Then `*log*` accumulates `a` and `b` across calls, in reverse order of insertion because `push` prepends.

Intent 2 — entries should be collected per phase and returned. Read `*log*` before the `let` ends:

```
(defun phase-one ()
  (let ((*log* nil))
    (log-it 'a)
    (log-it 'b)
    (reverse *log*)))    ; => (A B)
```

The sandbox was exactly right; what was missing was a way to *empty* it back out. Chapter 23′s tests will do this deliberately, binding `*save-file*` with `let` so a test can't corrupt the player's real save — the same mechanism, used for isolation this time.

The name change the question asks about: if the function's intent was "collect locally and return", the natural name for the local is a plain `log` (no earmuffs), and a lexical `let` — not the global `*log*`. Rebinding the earmuffed name **inside** the function is only right when callers of `log-it` from within `phase-one` are the point.

17.6

```
(defun terse (object)
  "The printed form of OBJECT, capped at 5 elements, 2 levels."
  (let ((*print-length* 5)
        (*print-level* 2))
    (format nil "~S" object)))
```

Check:

```
(terse '(1 2 3 4 5 6 7))
;; => "(1 2 3 4 5 ...)"

(terse '(1 (2 (3 (4))) 5))
;; => "(1 (2 #) 5)"
```

Truncated elements show as `...`, truncated levels as `#`. Using `~S` rather than `~A` keeps it *readable* — quotes on strings — which for logs is usually what you want to see. The two variables are independent dials: `*print-length*` caps width, `*print-level*` caps depth, and both default to `nil` (unlimited) — why printing everything has worked all book.

17.7

```
(defparameter *verbose* nil
  "When true, TAKE and DROP narrate their internal steps.")

(defun take (item)
  "Pick up ITEM if it is in the current room."
  (cond ((eq (item-location item) 'player)
         (format nil "You already have the ~(~A~)." item))
```

```
        ((eq (item-location item) (room-name *current-room*))
         (when *verbose*
           (sayln "~&(debug) picking up ~(~A~)" item))
         (setf *item-locations*
               (set-location *item-locations* item 'player))
         (format nil "You take the ~(~A~)." item))
        (t
         "You don't see that here.")))

(defun drop (item)
  "Put down ITEM in the current room, if the player carries it."
  (if (eq (item-location item) 'player)
      (progn
        (when *verbose*
          (sayln "~&(debug) dropping ~(~A~)" item))
        (setf *item-locations*
              (set-location *item-locations* item
                            (room-name *current-room*)))
        (format nil "You drop the ~(~A~)." item))
      "You don't have that."))

(defun play-quietly (commands)
  "Run COMMANDS with verbosity forced off."
  (let ((*verbose* nil))
    (play-to-string commands)))

(defun play-loudly (commands)
  "Run COMMANDS with verbosity forced on."
  (let ((*verbose* t))
    (play-to-string commands)))
```

The comment the exercise asks for:

play-quietly and play-loudly call run-command through code that was never told about verbosity — the flag is read deep inside take, by whichever **verbose** binding is in force at the moment. That only works because **verbose** is dynamic: the let at the top rebinds it for the whole subtree of calls. Had take taken a :verbose keyword instead, every caller between play-quietly and take — run-command, call-command, transcript, the whole chain — would have to accept :verbose and pass it along. That is the "threading the argument" problem, and dynamic variables are the answer to "a setting that distant code needs but no intermediate caller should care about" — which is precisely why the standard's **print-** variables are dynamic too.

The clean-`--say` point the comment closes on: before this chapter, the game's messages went to `format t` — there was no single seam where `*verbose*` could choose between "print" and "don't print". `say` and `sayln` created that seam. The wrapper existed before the knob needed it, and the knob is what made the wrapper worthwhile.

D.S. Chapter 18

18.1

```
(list #b-101        ; => -5
      #xdeadbeef    ; => 3735928559
      #c(0 -1)      ; => #C(0 -1) — the imaginary number -i
      ''sym         ; note two quotes: (QUOTE SYM)
      '|Two Words|  ; => |Two Words| — one symbol, case preserved
      #(a b c)      ; => #(A B C) — a vector
      '#:fresh)     ; => #:FRESH — uninterned, prints with #:
```

The `''sym` row is the sleeper: `read-from-string "''sym"` gives the two-element list `(QUOTE SYM)`, because `'` itself reads as the `quote` form. Nested twice, you get `quote` wrapped in `quote` — which evaluates to the list `(quote sym)`, not to `sym`. Most "why does my quote do nothing" bugs are one quote deep.

`#b-101` is `-(#b101)`, so `-5`. The sign goes before the digits; the reader accepts it because `read` delegates to `parse-integer` internally, and `parse-integer` allows a leading minus.

18.2 `(read-from-string "(1 2 3) (4)")` reads *one* object, `(1 2 3)`, and returns `12` as its second value — the index where the next read would continue. `(read-from-string "1 2 3")` reads `1` with second value `2`. The reader is always one-object-per- call; the second value is how you walk a string of many.

```
(defun read-all (string)
  "Every object in STRING, as a list."
  (loop with pos = 0
        for (obj next) = (multiple-value-list
                           (read-from-string string nil nil
                                             :start pos))
        while obj
        collect obj
        do (setf pos next)))

(read-all "(a) 42 \"x\"")
;; => ((A) 42 "x")
```

The `:start` argument moves the window; `nil` as the eof-value makes the walk stop on empty input. This is the skeleton of every Lisp source-processing tool — `read`, `read-from-string`, and the `:start`/`:end` keywords, repeated until the stream runs out.

Two corners worth a comment. First, `while obj` stops at the first `nil`-valued object, so `read-all "nil 1 2"` returns `nil` rather than `(nil 1 2)`. If that matters, test the `next` position against the string's length instead. Second, when the reader hits a genuine syntax error (unbalanced parenthesis, stray comma), it *signals* — a bare `read-from-string` loop will hit the debugger, not return a partial list. Production code wants a `handler-case` around each `read-from-string`, treating a reader error as "stop here" — Chapter 19's `reader-error` condition, caught and translated. The example above leaves that out because the exercise is about the position mechanism, not about error handling — Chapter 19 comes next.

18.3 **Surprising:** `#S(point ...)` in a `defun` is a *literal* — the same point object every call, and mutating it is undefined behaviour, so `(point-x (defaults))` may be `5` on the next call too. Fix: return a fresh structure:

```
(defun defaults ()
  (make-point :x 0 :y 0))
```

Surprising: the two `defun` forms are *both selected* if your implementation reports both features — `#+` is read-time conditional per form, and nothing makes the branches mutually exclusive. SBCL reports `:sbcl` and not `:ccl`, so under SBCL the second form is skipped; under an implementation reporting both, you'd get two definitions and a redefinition warning. The intent — one function with implementation-specific bodies — wants the conditional *inside* the body, one `defun` with `#+` lines:

```
(defun fast-path ()
  #+sbcl (sbcl-specific-thing)
  #+ccl  (ccl-specific-thing))
```

Surprising: the `#.` form runs `delete-file` *while reading* — the moment `read-from-string` sees it, the file is gone, before any evaluation you planned. This is why `*read-eval*` exists to be turned off:

```
(let ((*read-eval* nil))
  (read-from-string "#.(delete-file \"/home/you/notes.txt\")"))
;; => reader error: #. is not allowed in this context
```

The fix, for reading untrusted data, is a binding of `*read-eval*` to `nil` around the `read`. The lesson the exercise teaches: `#.` is code execution from text, and it fires at read time, not at eval time.

18.4

```
(defun feature-list ()
  "All feature symbols, as sorted downcased strings."
  (sort (mapcar (lambda (f) (string-downcase (symbol-name f)))
                *features*)
        #'string<))

(defun implementation-p (name)
  "Would #+NAME select code on this implementation?"
  (member name *features*))
```

Check:

```
(implementation-p :sbcl)                    ; => T on SBCL
(member "sbcl" (feature-list) :test #'string=)  ; => same answer
```

The two tests answer the same question by different doors. The second asks "is there a feature whose *name* is 'sbcl'"; the first asks "is `:sbcl` one of the symbols in the list". They agree because both are looking at the same symbol `:sbcl` — the second just converted it to a string for the comparison. `implementation-p` is the right spelling for feature-conditional *decisions* `feature-list` is the right spelling for *inspection* at the REPL.

A typical SBCL reports `:sbcl`, `:sb-core-compression` or similar, `:unix` (or `:win32`), `:x86-64` (or `:arm64`), `:64-bit`, `:little-endian`, `:ansi-cl`, `:common-lisp`, and more. The features are how one source tree covers every implementation and platform — and how Appendix A's install instructions can say "on your platform" in Lisp itself.

18.5

```
(defun read-hex (stream char)
  "Reader macro: $hex reads as the hexadecimal number."
  (declare (ignore char))
  (let ((chars nil))
    (loop for c = (peek-char nil stream nil nil)
          while (and c (digit-char-p c 16))
          do (push (read-char stream) chars))
    (parse-integer (coerce (nreverse chars) 'string)
                   :radix 16)))

(set-macro-character #\$ #'read-hex)

(read-from-string "$ff")   ; => 255
(read-from-string "$a")    ; => 10
(read-from-string "$10")   ; => 16
```

`digit-char-p` with radix `16` accepts the hex digits — the same predicate `parse-integer` uses internally, which is why the loop and the conversion agree on what a hex digit is. `(coerce ... 'string)` turns the list of characters back into a string, which is what `parse-integer` wants.

Undo, and the comment the exercise asks for:

```
(setq *readtable* (copy-readtable nil))
```

copy-readtable with argument nil returns a copy of the standard readtable — nil means "the standard one, not the current one". It is a fresh object: the session's readtable was mutated by set-macro-character, and the fresh copy is not. Assigning **readtable** to it reverts every macro character at once, back to what the standard defines. In source, you would wrap the customisation in (let ((**readtable** (copy-readtable nil))) …) so the change has dynamic extent — same mechanism as every other earmuffed knob (Chapter 17), applied to the reader instead of to output.

The subtle bit: `(copy-readtable)` with no arguments copies the *current* readtable, which is what you want for "restore after my change" when you saved the old one. `(copy-readtable nil)` copies the standard, which is what you want for "revert all the way". Exercise 18.5 uses the standard-copy form because it means "undo everything a previous exercise or session might have changed", which is the honest hygiene between REPL experiments.

18.6

```
(defun header-version (header)
  "The version integer from a save file header line, or 0."
  (let ((v-pos (search " v" header)))
    (if v-pos
        (parse-integer header :start (+ v-pos 2)
                              :junk-allowed t)
        0)))
```

```
(defun convert-save-v0 (state)
  "Old saves were (item-locations room); today's add history."
  (if (and (= 3 (length state))
           (listp (third state)))
      state                     ; already today's shape
      (list (first state)       ; item locations
            (second state)      ; room name
            nil)))              ; history didn't exist in v0

(defun load-game ()
  "Restore from the save file, converting old versions."
  (handler-case
      (with-open-file (in *save-file*)
        (let* ((header (read-line in))
               (version (header-version header)))
          (unless (search "cave-of-noise" header)
            (error "Not a Cave of Noise save file: ~S" header))
          (let ((state (with-standard-io-syntax (read in))))
            (when (= 0 version)
              (setf state (convert-save-v0 state)))
            (restore-state state)))
        (look))
    (file-error () "No saved game found.")
    (end-of-file () "Save file is truncated.")
    (error (c) (format nil "Save file unreadable: ~A" c))))
```

Check the v0 path:

```
(with-open-file (out *save-file* :direction :output
                                 :if-exists :supersede)
  (format out "cave-of-noise v0~%")
  (with-standard-io-syntax
    (print '((key . kitchen) kitchen) out)))
(load-game)
;; history becomes nil; items and room restored
(item-location 'key)   ; => KITCHEN
```

The comment the exercise asks for:

The header is a string line, read with read-line and inspected with string tools. It can carry any text, be extended with more fields, and be parsed defensively — the reader never touches it as Lisp syntax. The Lisp object after it, by contrast, is all or nothing: read either builds a valid object or signals, and there is no partial read to salvage a broken file. A string envelope around a Lisp payload is the standard shape for versioned data formats in Lisp — the same layering JSON files use, with Lisp data inside. The header's job is to say what the payload *is* so a version mismatch is a legible error rather than a confusing reader failure; the payload's job is to be data.

The `:junk-allowed t` in `header-version` is what makes it forgiving: `(parse-integer "1 extra text" :junk-allowed t)` returns `1` and ignores the rest. For a header, forgiving is right — the version is the leading digits, anything after is for humans.

D.T. Chapter 19

19.1 The `handler-case` form returns `CAUGHT`. The `handler-bind` form also returns `CAUGHT` — but by a different route:

```
(handler-case (error "boom")
  (error (c) (declare (ignore c)) 'caught))
;; => CAUGHT — unwound to the handler-case, recovery code ran

(block out
  (handler-bind ((error (lambda (c)
                          (declare (ignore c))
                          (return-from out 'caught))))
    (error "boom")))
;; => CAUGHT — the handler ran at the signal site, and
;;    return-from performed the exit
```

The sentence the exercise wants: both handle the error, but `handler-case` exits *by unwinding first*, while `handler-bind`'s handler is called *in place* and must perform its own exit — here a `return-from` through the `block` standing where the unwind would have landed. Same observable result, different mechanism; the mechanism is what matters when the handler needs the stack below it, as Exercise 19.3′s second snippet shows.

19.2

```
(defun with-suppressed-warnings (thunk)
  "Run THUNK with warnings silenced; return its values."
  (handler-bind
      ((warning (lambda (c)
                  (declare (ignore c))
                  (muffle-warning))))
    (funcall thunk)))

(defun with-captured-warnings (thunk)
  "Run THUNK; return (VALUES result warning-texts)."
  (let ((texts nil))
    (handler-bind
        ((warning
          (lambda (c)
            (push (princ-to-string c) texts)
            (muffle-warning))))
      (values (funcall thunk) (nreverse texts)))))
```

Check, using Chapter 15′s `to-integer`:

```
(with-suppressed-warnings (lambda () (to-integer "nope")))
;; => 0, nothing printed

(multiple-value-bind (v ws)
    (with-captured-warnings
      (lambda () (list (to-integer "42") (to-integer "nope"))))
  (list v ws))
```

Expected: `((42 0) ("WARNING: No integer in \"nope\"."))`

The capture must happen *inside* the handler, before `muffle-warning` — because `muffle-warning` performs a non-local exit that ends the handler immediately. After `muffle-warning`, there is no "after" in the handler to run. If you tried to capture after muffling, the capture line would never execute. The order in the snippet — push, then muffle — is the only correct one.

A second subtlety: the warnings from Chapter 15's `to-integer` are raised by `warn`, and `warn` itself establishes the `muffle-warning` restart. The handler's `muffle-warning` call finds that restart and invokes it, which is why the warning is suppressed rather than printing and then being seen.

19.3 **Wrong:** the handler prints and returns — which abstains — so the error *does* propagate, opening the debugger. If the intent was "print and stop", the handler needs an exit. Fix:

```
(block out
  (handler-bind ((error (lambda (c)
                          (format t "~A" c)
                          (return-from out nil))))
    (error "propagate?")))
;; prints the message, returns NIL, no debugger
```

Wrong: the `room-not-found` handler runs after the unwind — the `restart-case` is *inside* the failing form, so it's gone by the time the handler runs, and `invoke-restart` signals "restart SUBSTITUTE-ROOM is not active". Fix: put `handler-bind` outside the `restart-case`:

```
(handler-bind ((room-not-found
                (lambda (c)
                  (declare (ignore c))
                  (invoke-restart 'substitute-room))))
  (restart-case (error 'room-not-found :name 'attic)
    (substitute-room () 'kitchen)))
;; => KITCHEN
```

Wrong: `handler-case` on a warning catches it *after printing* — `warn` prints when no handler muffled it, then the condition propagates, and the `handler-case` returns `nil`. The warning already appeared, which is the opposite of the caller's intent. Fix: `handler-bind` with `muffle-warning`, as in Exercise 19.2:

```
(handler-bind ((warning #'muffle-warning))
  (warn "quiet"))
;; prints nothing
```

The general shape the three snippets teach: `handler-case` is for "catch and substitute" — it works only when the handler's job is to produce a value at the `handler-case`'s location, with the stack already unwound. `handler-bind` is for everything else: log, muffle, invoke a restart, or perform any action while the stack is still around. When in doubt, `handler-bind` is the tool that will do the job; `handler-case` is the one that won't.

19.4 a) `parse-error` — `parse-integer` signals it. The useful detail is the string you passed in, which you already have as the argument; `parse-error` itself has no standard slot for it. Some

implementations add one via `simple-condition`'s `:format-control` and `:format-arguments`, but portable code catches the condition and uses its own argument.

b) `file-error` — `file-error-pathname` names the file that couldn't be opened. Chapter 16's `load-game` already uses this shape; the save-file loading is the canonical example.

c) `unbound-variable` — one of the `cell-error` subtypes. `cell-error-name` gives the symbol that had no value, which is the useful detail. `cell-error` covers three siblings: the variable (`unbound-variable`), the slot (`unbound-slot`), and the function (`undefined-function`).

d) `end-of-file` — one of the `stream-error` subtypes. `stream-error-stream` names the stream. In practice this rarely needs catching: reading with `read-line`'s `nil` second argument returns `nil` at end-of-file instead of signalling, and that's the pattern Chapter 16 teaches. Catch `end-of-file` only for files where a premature end is itself a problem — a truncated save, for instance.

e) `program-error` — it has no useful standard slot; the argument-count story lives in the message. Catch it for control (the failsafe `run-command`), not for data.

The pattern across all five: `condition type` decides *whether* to catch; `condition slots` decide *what to do* about it. The narrowest type you can name is the one you should name — the broader the type, the less you can say about the condition when the handler runs.

19.5

```
(defun invoke-if-found (name &rest args)
  "Invoke the restart called NAME, or return :inactive."
  (let ((restart (find-restart name)))
    (if restart
        (apply #'invoke-restart restart args)
        :inactive)))
```

The gap matters *between* the two steps: `find-restart` can fail (name misspelled, restart out of scope) and return `nil` quietly, while `invoke-restart` on a missing name *signals an error*. Two steps let the caller decide which failure mode it wants — lookup-then-test for optional restarts, direct `invoke-restart` for restarts whose absence is a bug.

One more subtlety: between `find-restart` and `invoke-restart` the restart set could in principle change, if some code ran between the calls. In `invoke-if-found` as written, nothing does — `apply` is the very next form. So the two-step pattern is race-free in practice. In a loop that calls `invoke-if-found` repeatedly on a changing restart set, the check would fire on stale data; but restarts don't come and go that fast, and callers who need to reason about restart scope know it's a dynamic-extent question, not a mutation question.

19.6

```
(defun take (item)
  "Take ITEM; a CONTINUE restart takes it from nowhere."
  (let ((entry (assoc item *item-locations*)))
    (cond
      ((eq (cdr entry) 'player)
       (format nil "You already have the ~(~A~)." item))
      ((eq (cdr entry) (room-name *current-room*))
       (setf *item-locations*
             (set-location *item-locations* item 'player))
       (format nil "You take the ~(~A~)." item))
```

```
      (t
       (cerror "Take it anyway."
               "There is no ~(~A~) here." item)
       (setf *item-locations*
             (set-location *item-locations* item 'player))
       (format nil "You take the ~(~A~) from nowhere." item)))))

(defun cheat-take (item)
  "Take ITEM regardless of where it is."
  (handler-bind
      ((error (lambda (c)
                (declare (ignore c))
                (let ((r (find-restart 'continue)))
                  (when r (invoke-restart r))))))
    (take item)))
```

Check:

```
(defparameter *item-locations*
  '((lamp . cellar) (key . kitchen)))
(setf *current-room* (find-room 'kitchen))

(list (take 'lamp)
      (cheat-take 'lamp)
      (item-location 'lamp))
```

Expected: `("There is no lamp here." ... KITCHEN)` on a fresh save — wait, precisely: the first `take 'lamp` signals `cerror` because the lamp is in the cellar, and no handler is bound, so the debugger opens. Running the whole list in one block therefore stops at the first element. Two separate checks are the honest way:

```
(defparameter *item-locations*
  '((lamp . cellar) (key . kitchen)))
(setf *current-room* (find-room 'kitchen))

(cheat-take 'lamp)
;; => "You take the LAMP from nowhere."

(item-location 'lamp)
;; => PLAYER
```

The comment the exercise asks for:

A restart is chosen by the caller at run time — the debugger offers “Take it anyway” to a human, and cheat-take offers the same choice programmatically — with no change to take’s signature. A :force keyword would hard-code the decision into every call site that might want it, and every intermediate function (run-command, call-command, the command table’s dispatch) would have to accept :force and pass it along — the threading-arguments problem that dynamic variables (Chapter 17) and restarts both solve, in their different ways. The restart also documents itself in the debugger’s menu, where a keyword is invisible until

you read the source. When the choice belongs to the moment, not the code path, a restart is the right shape.

The `handler-bind` in `cheat-take` is deliberately broader than "catch the take error specifically". It catches any error and checks whether a `CONTINUE` restart is active. That works because `cerror` is the only place in `take` that establishes a `CONTINUE` restart — but if `take`'s implementation later grew a different `cerror` (say, prompting about an ambiguous name), the handler would continue through it too. In a real program, catch the specific condition type; here, catching `error` and gating on `find-restart` keeps the exercise's focus on the restart mechanism.

D.U. Chapter 20

20.1 a) Compiled: `defparameter` is *compiled* (its `compute-x` call is not run — the compiler only records that the value expression runs at load); `defconstant`: its value `10` is evaluated at compile time (the compiler must know constants); `defun`: compiled, body not run; the `eval-when (:compile-toplevel) format`: run *during compilation* — "hi" appears in the compiler output; the second `eval-when`: not evaluated at compile time (its situations don't include `:compile-toplevel`).

b) Loaded: all five run at load time — `defparameter` computes `compute-x`, `defconstant` re-evaluates `10`, `defun` installs `f`, the `eval-when` (compile-toplevel only) does *nothing* at load, and the second `eval-when` installs `g`.

c) REPL: everything evaluates in place except the `(:compile-toplevel)-only` form, which does nothing — `:execute` is absent, and direct evaluation is the execute situation. Direct evaluation of the `defparameter` computes `compute-x` and binds `*x*`; `defconstant` binds `+limit+`; `defun` installs `f`; the first `eval-when` is skipped; the second installs `g`.

The pattern worth carrying: the `(:compile-toplevel)-only` form is invisible outside compilation — a form that runs "when the compiler goes by", which is why it's the tool for compile-time side effects like declaring a feature, or for `(eval-when (:compile-toplevel) (declaim (special *foo*)))` to affect downstream compile-time behaviour without affecting run time.

The `defconstant` case deserves a note. At compile time the value `10` is recorded for the compiler's constant-folding machinery; at load time it's re-evaluated and re-checked against the compiled-in value. If the value *differs* between compile and load — say because `compute-x` depends on the current directory — the load signals a `defconstant` violation. That is why the earlier chapter's advice, "constants only for values that truly never change," is really about *this* failure mode.

20.2 Compiling the file, the compiler reaches `(twice a)` inside `f` *first*. Macro use means: call the macro function at compile time. But `defmacro` installs the macro function at *load* time, which hasn't happened — so the compiler either errors ("TWICE is undefined function") or — worse — treats `twice` as an undefined function call and compiles `f` wrong, reporting a warning that's easy to dismiss. The compiled `f`, once loaded, is broken or missing its expansion.

Fix one — `eval-when` makes the macro available at compile time:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (defmacro twice (x) `(list ,x ,x)))

(defun f (a) (twice a))
```

Fix two — split files, order them, and let the system load them serially:

```
;; macros.lisp
(defmacro twice (x) `(list ,x ,x))

;; game.lisp
(defun f (a) (twice a))

;; the .asd, from Chapter 16
(asdf:defsystem "game"
  :serial t
  :components ((:file "macros") (:file "game")))
```

When compiling `game.lisp`, `macros.lisp` has already been loaded (its fasl), so `twice` exists. Both fixes build the expansion-time world before the first use — the difference is whether you say it in the file (`eval-when`) or in the build order (`:serial`).

A third fix, worth knowing by name: `defsystem` with `:depends-on` on the macro's own system, if the macro lives in a separate project. The principle is the same — establish the expander before its first use — but the file (or system) that defines the macro must be a load-time (and compile-time) dependency of the file that uses it.

20.3 **Wrong:** `*home*` is not constant across images and platforms — different users, different hosts, differently defaulted pathnames. Compiling the file evaluates it once (Chapter 20's rule for constants), and loading elsewhere can signal "the value of **HOME** is not constant". Fix:

```
(defparameter *home* (user-homedir-pathname))
```

Environment facts are parameters, recomputed per image.

Wrong: `*default-pathname-defaults*` is the process's current working directory merged into pathnames — it changes as the program `chdir`s and differs per invocation. `(self-path)` answers "where am I running from", not "where was my file loaded from". Fix:

```
(defun self-path ()
  (or *load-truename* *compile-file-truename*))
```

— the file's actual home, bound only while it's being loaded or compiled, hence the `or`. At load time `*load-truename*` is bound and `*compile-file-truename*` is `nil`; while the compiler processes the file the opposite is true; after both, neither is bound and `(self-path)` returns `nil`, which is the honest answer.

Wrong: `(:execute)` only means the `defmacro` runs at the REPL. Compiling the file: the macro is *not* defined at compile time, so `always`'s later use in the same file fails to expand — same bug as Exercise 20.2. Loading the compiled file: the `defmacro` runs (installing the

macro for *future* interactive use, too late for this file), and the recorded use of `always` was compiled as a plain function call. Fix:

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (defmacro always (val) val))
```

All three cases are the same shape of mistake: a form that needs to be available at compile time was only guaranteed at execute time, or vice versa — a value meant for run time was baked in at compile time.

20.4

```
(defun compile-and-report (filenames)
  "Compile each file in one unit; return (src . fasl-or-nil) pairs."
  (with-compilation-unit ()
    (loop for src in filenames
          collect (cons src (compile-file src)))))
```

Check (with two files from Chapter 16′s Exercise 16.7 in the current directory):

```
(compile-and-report '("packages.lisp" "classes.lisp"))
;; => (("packages.lisp" . #P"packages.fasl")
;;     ("classes.lisp"  . #P"classes.fasl"))
```

Grouping collects every warning from every file and prints them once, at the end, as one unit — so a cascade of warnings caused by one early error reads as a group, with the root cause visible among its echoes instead of scattered across five separate reports. Per-file compilation buries the relationship: you see "undefined function FOO" six times and have to notice that FOO's definition was the thing in the *first* file that failed.

The limit of grouping is that it only helps *reporting*. A genuinely fatal compile error still aborts the unit at that file, and later files aren't processed at all. Fixing that requires `:force` or resuming from the last successful file, which is ASDF's job (Exercise 20.6′s punchline, again).

20.5

```
(eval-when (:compile-toplevel :load-toplevel :execute)
  (defun count-forms (file)
    "Top-level forms in FILE, counted by reading."
    (with-open-file (in file)
      (loop for form = (read in nil nil)
            while form
            count form))))

(defmacro file-form-count ()
  "Expands to a constant: forms in the file being compiled."
  (count-forms *compile-file-truename*))
```

Use:

```
;; in game.lisp, alongside the other top-level forms
(defparameter *form-count* (file-form-count))
;; => loads as, e.g., 14 — a literal integer
```

Compile time, because the *answer must be in the compiled file*: a fasl contains no source to re-read, so the count has to be computed while the source is present — at `compile-file` time — and emitted as a constant. At load time the macro has already expanded: `*compile-file-truename*` is `nil` (it's only bound during compilation), and the expansion is a literal integer. What expands at load time is the integer; the whole reading step happened before the fasl existed.

The `eval-when` trio on the helper is Exercise 20.2's rule applied: `count-forms` is called at expansion time (compile time), so it must exist then — and the macro must also work evaluated directly (REPL) and after load (its expansion must be usable), hence all three situations.

One gotcha: `*compile-file-truename*` is `nil` when the macro is used in the REPL rather than in a file. `(count-forms nil)` signals a type error. A robust version either tests for the nil and returns a sentinel, or guards with `(when *compile-file-truename* ...)` in the expansion. The exercise's version is correct *for a file* — which is the point of a compile-time macro — and the REPL usage is a bug the reader should expect to see if they try it interactively.

20.6

```
(defun load-compile-if-stale ()
  "Recompile stale files and everything after them, then load all."
  (let ((seen-stale nil))
    (with-compilation-unit ()
      (dolist (src *game-files*)
        (when (or seen-stale (stale-p src))
          (setf seen-stale t)
          (compile-file src))
        (load (fasl-for src))))))
```

The rule: once any file is stale, every later file gets recompiled too — because later files might depend on the changed one, and we don't know which. It's conservative and correct for a serial chain.

The comment the exercise asks for:

This still gets several things wrong that ASDF gets right. First, a change to the *last* file in **game-files** forces a recompile of just that file, not the files before it — so a later-in-the-list file that game-files* doesn't list (a helper, a test) won't be rebuilt even though it depends on the changed thing. Second, `file-write-date` has one-second resolution on some filesystems, so two edits in the same second look identical. Third, files that *depend on each other out of order* (a macro in the last file, used by the first) break the "stale from here on" rule entirely — the recompile cascade has to go both directions, or the dependency graph has to be known. Fourth, fasl extensions are implementation-specific, and mixing two Lisps in one directory corrupts the cache; ASDF versions fasls per implementation. Fifth — and this is the big one — ASDF tracks *which files changed*, so a change to one file recompiles only its dependents, not "the file and everything after it in a list order that was chosen by the author, not the dependencies." The toy is honest about its lesson: a system definition is a dependency-aware make, and anything short of one will lie eventually. Chapter 16's

`defsystem` was already the right shape; the exercise is worth doing to see *why* the shape is right, not to build a replacement.

D.V. Chapter 21

21.1

```
(list (typep 3 '(integer 0 2))     ; => NIL — 3 is outside the bound
      (typep 3 '(or null integer)) ; => T — union types
      (typep :red '(member :red :green :blue)) ; => T
      (typep "abc" '(simple-array character (3))) ; => T
      (typep '(1 2) '(cons integer (cons integer null))) ; => T
      (subtypep 'null 'list)       ; => T, T — NIL is a list
      (subtypep 'null 'symbol))    ; => T, T — and a symbol too
```

The last two are the same door from two frames: `null` (the type of `NIL`) is a subtype of both `list` and `symbol`, which don't overlap with each other in any interesting way. Types in Lisp aren't a tree of disjoint boxes — they're sets, and a value sits in every set it belongs to. `(cons integer (cons integer null))` shows the recursive spine of list types: a cons whose `cdr` is another such cons or `nil` — exactly the two-element proper list `(1 2)`.

The `"abc"` check works because a string literal in source is a `simple-array character` of its length; a string built by `concatenate` or `make-string` may or may not have the same type, depending on the implementation. That's the difference between `typep` and "is the value a string": `typep "abc" 'string` is always true, but `(simple-array character (3))` is the specific type SBCL happens to give the literal.

21.2 **Wrong:** `declare` must precede all executable forms in the body — the `format` makes it a declaration after a form, and the compiler signals an error. Fix: move the `declare` to the top:

```
(defun f (x)
  (declare (type fixnum x))
  (format t "x is ~A~%" x)
  (* x 2))
```

Wrong: `declaim` declares, but `defparameter` *rebinds* unconstrained — the declaration applies to the variable named `*player-name*`, and the `defparameter` assigns a symbol to a declared-string variable: a type conflict SBCL warns about at the `defparameter`. The symbol `anon` is not a string. Fix by making the initial value match:

```
(declaim (type string *player-name*))
(defparameter *player-name* "anon")
```

The general lesson: a global declaration is a promise about *every* future assignment — including the ones your tests and REPL experiments make.

Wrong: at `(safety 0)` the compiler trusts `fixnum` and `1+ "oops"` runs unchecked — garbage, a memory fault, or silently wrong data, depending on the machine. With the default `(safety 1)` the same call signals a `type-error` before any arithmetic. The snippet is the chapter's warning in one line: safety 0 plus a wrong declaration is undefined behaviour; keep safety 1 until declarations are proven right, and this example never is.

21.3

```
(defun safe-elt (seq index)
  "Element of SEQ at INDEX, checked."
  (check-type index (integer 0 #.most-positive-fixnum))
  (elt seq index))

(defun fast-elt (seq index)
  "Unchecked: caller guarantees a valid fixnum index."
  (declare (optimize (speed 3) (safety 0))
           (type fixnum index))
  (elt seq index))
```

The comment:

safe-elt belongs in the game, because a bad index there is a bug to surface, not a cost to shave — the interactive type-error restart even lets you fix the variable in place. fast-elt is for measured hot paths only, and its docstring states the contract it no longer checks. "Almost everywhere" means everywhere you haven't profiled. The deeper point the pair teaches: check-type and declare are two ways to say the same thing — "this will be of this type" — one verified at run time as a service to the user, one trusted at compile time as a service to the compiler.

Check:

```
(list (safe-elt "abc" 1)
      (safe-elt '(a b c) 2)
      (fast-elt (vector 10 20 30) 1))
;; => (#\b C 20)
```

And the divergence, when both are given a bad index:

```
(ignore-errors (safe-elt "abc" -1))   ; => NIL — signal caught
(ignore-errors (fast-elt "abc" -1))   ; => implementation-dependent
```

At `safety 0`, `fast-elt`'s `(elt "abc" -1)` may return garbage, segfault, or (on some builds) still signal — the standard doesn't say. That's what "undefined behaviour" means concretely.

21.4

```
(defun type-chain-p (value types)
  "Is VALUE's type a subtype of every type in TYPES?"
  (let ((vtype (type-of value)))
    (every (lambda (tt) (subtypep vtype tt)) types)))

(type-chain-p 5 '(integer fixnum number))  ; => T on most machines
```

The return is `T`, given `type-of` returned a fixnum subtype — SBCL's `(integer 0 4611686018427387903)` is a subtype of `fixnum`, `integer` and `number`. What it tells you: types overlap — the one value `5` sits in all three sets simultaneously, because Lisp types are sets, not a disjoint class hierarchy. The caveat worth a comment:

type-of returns some type the value belongs to, not necessarily the most specific one that matters to you. The function answers about the implementation's chosen representation, not about the value's abstract type. For intent, test the value against the types directly with

typep: (type-chain-p 5 '(integer fixnum number)) is true, but (type-chain-p 5 '(unsigned-byte 3)) might or might not be, depending on whether type-of on this SBCL returned (integer 0 4611686018427387903) — which is not a subtype of (unsigned-byte 3), even though 5 is. The function is testing the implementation's answer, not 5's.

That's the sharp version of the lesson: `subtypep` on types is not the same question as `typep` on values, and `type-of` bridges them in a way that can hide what you meant.

21.5 A typical run (yours will differ — that's why the exercise demands reporting your own):

```
;; (a) no declarations:  ~8.0 ms
;; (b) (the double-float ...) + declared accumulator: ~5.0 ms
;; (c) + declared (simple-array double-float (*)) + speed 3: ~1.2 ms
```

With the code:

```
(defparameter *v*
  (make-array 1000000 :element-type 'twice-float
                      :initial-element 1.0d0))

;; (a)
(defun sum-a (v)
  (loop for x across v sum x))

;; (b)
(defun sum-b (v)
  (let ((total 0.0d0))
    (declare (type double-float total))
    (loop for x across v
          do (incf total (the double-float x)))
    total))

;; (c)
(defun sum-c (v)
  (declare (optimize (speed 3) (safety 1))
           (type (simple-array double-float (*)) v))
  (let ((total 0.0d0))
    (declare (type double-float total))
    (loop for x across v
          do (incf total (the double-float x)))
    total))

(time (sum-a *v*))
(time (sum-b *v*))
(time (sum-c *v*))
```

Which declaration bought the most: the array type — (b)→(c). Declaring the accumulator removed float boxing of the sum, but declaring the array removed the per-element type dispatch, the dominant cost. The order is the chapter's order in miniature: accumulate `the` declarations cheaply, but the data-structure declaration is where the compiler stops guessing.

One honest caveat: even sum-c is not what an optimising C compiler would produce for the same loop, because SBCL doesn't vectorise. To go faster you'd reach for `sb-simd` or a C

library via CFFI (Chapter 29). The chapter's point is that declarations bring you a factor of five or ten, not that they turn Lisp into Fortran.

21.6 The `declaim` and its requirement:

```
(declaim (type fixnum *moves*))
(defparameter *moves* 0)   ; a fixnum — the declaration demands it
```

The `play` check runs reliably because every assignment site (`incf *moves*` in the command loop) preserves fixnum-ness — and at `(safety 1)`, the moment some future edit assigns a string, the error surfaces loudly instead of corrupting.

```
(defun drift-check ()
  "Type-check the game's global state; return failing names."
  (let ((expected
         (list (cons '*moves* 'fixnum)
               (cons '*current-room* '(or null room))
               (cons '*item-locations* 'list)
               (cons '*room-list* 'list)
               (cons '*visits* '(simple-vector)))))
    (loop for (name . type) in expected
          unless (typep (symbol-value name) type)
            collect name)))
```

Check:

```
(setf *moves* "oops")     ; a drift
(drift-check)             ; => (*MOVES*)
(setf *moves* 3)          ; back to normal
(drift-check)             ; => NIL
```

Two notes on the type list. `(or null room)` for `*current-room*` permits the initial `nil` as well as a real room; the game's `play` sets `*current-room*` immediately, so the `nil` case never survives long, but the type spec is honest about the declared default. `(simple-vector)` for `*visits*` is the general vector type — `(vector fixnum)` would also work, but SBCL's actual type for a `fixnum`-filled array is `(simple-array fixnum (*))`, and `simple-vector` is the closest abstract name that covers it. When in doubt, `(typep x (type-of example))` confirms.

The kind of bug `drift-check` catches: a *latent* wrong-typed assignment — one that's been sitting in the image for an hour, poisoning state quietly, and will bite somewhere far from its cause. At `(safety 0)` a wrong `declaim` turns that same assignment into corruption with no error at all — the value lands, memory gets reinterpreted, and the symptom appears anywhere but there. `drift-check` is the safety-net counterpart: same type knowledge, checked *now*, on demand, reporting *which* variable drifted. Between them — checked types in development, declared types in profiled hot spots — lies the whole practice.

D.W. Chapter 22

22.1 Without the interposed `nil`:

```
(+ 2 3)          ; => 5      — * is now 5
(* * 4)          ; => 20     — * is 20, ** is 5
(list * ** ***)  ; => (20 5 <prior>)
```

The three elements of the last line: `20` (the value from `(* * 4)`), `5` (the value from `(+ 2 3)`), and whatever was `*` *before* `(+ 2 3)` — usually the return value of the previous REPL command, or a fresh-session error if there wasn't one.

With the interposed `nil`:

```
(+ 2 3)          ; => 5
(* * 4)          ; => 20
nil              ; => NIL    — * is NIL, ** is 20, *** is 5
(list * ** ***)  ; => (NIL 20 5)
```

The interposed form shifted everything back by one slot. This is the exercise's whole point: * doesn't mean "the answer I care about" — it means "the last value the REPL happened to print", and every evaluation silently redefines it. Any evaluation between two usages changes what the next usage sees. For anything you actually want to reason about, bind it with `defparameter`.

22.2 `CSS (describe #'append)` prints the signature — `append` takes any number of arguments, all of which must be lists except possibly the last — and its docstring. The point the exercise makes: you can answer "what arguments does this take" without reading any source.

`CSS (describe *commands*)` prints the alist's type and the list of `(name . function)` pairs currently in it. Read from that output alone: `*commands*` is a `LIST`, and each entry is a cons whose car is a symbol and whose cdr is a function or symbol. Even without knowing Lisp's alist convention, the shape is visible: each pair names a thing and points at a thing.

That's why `describe` is the first tool in unfamiliar code: both "what does this function want" and "what shape is this data" are one command away, and no source is needed for either.

22.3 The inspector shows a `type-error` has exactly two interesting slots: `datum`, holding `5`, and `expected-type`, holding `LIST`.

```
(defun datum-or-message (fn)
  (handler-case (funcall fn)
    (type-error (c)
      (format nil "Wanted ~A, got ~S."
              (type-error-expected-type c)
              (type-error-datum c)))))

(datum-or-message (lambda () (length 5)))
;; => "Wanted LIST, got 5."
```

Handlers read slots, never the printed message — Chapter 15's rule, now with the inspector as the way to *discover* what slots exist. The `~S` on `datum` is what keeps `5` (a number) distinct from `"5"` (a string) in the output; that's exactly the case where ~A would lie.

22.4 a) `trace` — it shows each call's arguments and result; when the shape is right, the inputs and outputs of the inner calls tell you which step corrupts. `step` also works but is slower for a function with many calls.

b) `apropos` — substring search over all symbol names; `(apropos "price")` finds it or proves it doesn't exist.

c) `time` — it reports milliseconds and allocation directly. `trace` would show calls but not cost; reasoning about "feels slow" without measuring is guessing.

d) `inspect` — one buffer, all slots and structure, navigable. `describe` prints a summary and stops; `print` shows a one-line view.

e) `step` — it walks subform by subform, which is exactly "watch each expression". `trace` only fires on function boundaries; a loop body isn't one.

f) `break` — it stops with the data in scope and resumes cleanly. `error` unwinds and loses the spot; `inspect` from outside needs you to reconstruct the situation first.

22.5

```
(defun safe-run (fn)
  "Run FN; return (:OK value) or (:ERROR message)."
  (handler-case (list :ok (funcall fn))
    (error (c) (list :error (princ-to-string c)))))
```

Check:

```
(list (safe-run #'+)
      (safe-run (lambda () (/ 1 0))))
```

Expected: `((:OK 0) (:ERROR "..."))` — the first result is `(:OK 0)` because `(+ )` with no arguments is 0, which is a reminder that `#'+` is a function taking any number of arguments, and `funcall` calls it with zero. The second result's message is implementation-flavoured — something like `"arithmetic error DIVISION-BY-ZERO signalled"` — which is exactly why the exercise had to say "message" and not "error type": a program that *acts* on the error reads the condition's type and slots; a program that *reports* the error uses `princ-to-string`, and the exact text doesn't matter.

22.6

```
(defun whereis (command)
  "Handle (WHEREIS ITEM): list rooms ITEM has appeared in."
  (let* ((name (second command))
         (present (assoc name *item-locations*))
         (seen nil))
    (dolist (snapshot *history*)
      (let ((loc (cdr (assoc name snapshot))))
        (when loc (pushnew loc seen))))
    (let ((current (and present (cdr present))))
      (when (and current (not (eq current 'player)))
        (pushnew current seen)))
    (cond ((null present)
           (format nil "No ~(~A~) here or in the log." name))
          ((null seen)
           (format nil "The ~(~A~) hasn't moved yet." name))
```

```
          (t
           (format nil "The ~(~A~) has been in: ~{~(~A~)~^, ~}."
                   name (reverse seen))))))
```

Add the command:

```
(push '(whereis . whereis) *commands*)
```

Check:

```
(setf *item-locations* '((key . kitchen) (lamp . cellar)))
(setf *history* nil)
(setf *current-room* (find-room 'kitchen))

(run-command '(take key))
(run-command '(go hallway))
(run-command '(drop key))
(run-command '(whereis key))
```

Expected: `"The key has been in: kitchen, hallway."` — the kitchen from the snapshot before `take`, the hallway from the snapshot before `drop`, and the current state (hallway) already in the list from the drop.

The condition question the exercise asks about: an unknown item is a `nil` lookup, not an error, so no `handler-case` is needed. The handling belongs in the command function, not in `run-command`, because `run-command` cannot know what "unknown" means for each command — for `whereis` it's a friendly message, for a future `examine` it might be a "you see no such thing" and for a `teleport-to-item` it might be a genuine error. Only the command function has the semantics.

The distinction is Chapter 15's flags-vs-conditions rule, at a finer grain: within a single command, some failures are flags (the player typed a non-item) and some are conditions (the item registry is corrupt). run-command's handler catches the second kind; whereis's own logic handles the first. The exercise's "where should this go" is really "which layer knows what 'unknown' means" — and the answer is always "the layer that has the domain knowledge".

22.7 Three bugs, three symptoms, one message per bug:

Bug 1 — break `item-names`.

```
(defun item-names (items)
  (declare (ignore items))
  nil)
```

Symptom: `inventory` prints `"You are carrying: ."` — the item list is empty in the format string, but there's no error. `(doctor)` reports "All clear", because the game's **data** is consistent — `*item-locations*` still has the right associations. A **function** is wrong. The lesson: `doctor` checks data, not behaviour.

Bug 2 — break *current-room*.*

```
(setf *current-room* 'ballroom)
```

Symptom: `look` fails with a type error on (room-name **current-room**) — a symbol isn't a room structure. But `inventory` still works, because it doesn't read `*current-room*`. And `doctor` reports "Current room is not in **rooms**" — the check you added in the project step catches the drift. `doctor` checks **data structure integrity**, not just data consistency.

Bug 3 — remove a room from *rooms* while items are in it.*

```
(setf *rooms*
      (remove (find-room 'kitchen) *rooms*))
```

Symptom: `go-to 'kitchen` from the hallway says "You can't go there from here" — the hallway's exit list still names kitchen, but `find-room` returns nil. And `items-here` on kitchen errors; `doctor` reports `"key is in unknown room KITCHEN"` (and any item that was in the kitchen), because the `find-room` check catches it.

The catalogue's pattern: data drift is loud (conditions, `doctor` catches it), function damage is silent (wrong values, no signal). `doctor` is how you give the silent kind a voice — and the redefinition loop is how you fix it, one `C-c C-c` per definition, no restart.

D.X. Chapter 23

23.1

```
(list (symbol-name 'kitchen)    ; => "KITCHEN"
      (symbol-package :kitchen) ; => #<PACKAGE "KEYWORD">
      (symbol-name :kitchen)    ; => "KITCHEN" — keywords' names, too
      (eq 'kitchen 'KITCHEN)    ; => T — the reader upcased both
      (eq (intern "KITCHEN") 'kitchen) ; => T — same table, same symbol
      (symbol-package (gensym))) ; => NIL — uninterned
```

The `intern` line is the reader's own machinery, used by hand: `intern` looks the string up in a package's table (default current package), creating the symbol if absent. Reading `kitchen` is interning `"KITCHEN"` — which is why the `eq` holds: there is only one symbol named `KITCHEN` home in a package, and both expressions reach it.

The last line's `NIL` is the tell for an uninterned symbol: no package holds it, so `symbol-package` has nothing to return. Every interned symbol has a non-`nil` package; every uninterned one has `nil`. There is no third case.

23.2 **Wrong:** `(get "kitchen" 'description)` signals an error — `get`'s first argument must be a symbol, because the property lives *on* it. Fix: use the symbol `(get 'kitchen 'description)`, or if the key arrives as a string, `(get (intern (string-upcase key)) 'description)`.

Surprising: the `remprop` line removes the property for the *whole image*. The later `(get 'kitchen 'exits)` returns `nil` — not because state was lost in a crash, but because something else deleted it. Property lists are global mutable state attached to shared symbols; the fix is not a different call but a different structure for anything that has versions, owners or a save file (an alist or hash table, as the chapter argued).

Surprising: `props` was captured *before* the `remprop`, but it still *changes*: `symbol-plist` returned the actual list the symbol points at, and `remprop` mutates that same list in place.

The captured variable doesn't hold a copy — it holds a live reference to the shared plist. If a snapshot was wanted, copy element-wise *after* the mutation you care about: `(copy-list (symbol-plist 'kitchen))`. Even then, the plist has no honest snapshot in general — properties come and go independently of any single structure.

Surprising: `(getf '(name "hall" name "cellar") :name)` returns `nil` — the plist's keys are the symbols `name` (not keywords `:name`), and `eq` never matches `name` against `:name`. A plist's keys must be compared with the same interned symbols you search with — the commonest plist bug, and the reason keyword keys are the convention: `:name` typed anywhere is the same object, so lookups never fail on case or internship accidents.

23.3

```
(defun plist-p (list)
  "Does LIST have an even number of elements?"
  (evenp (length list)))

(defun plist->alist (plist)
  "Convert (a 1 b 2) to ((A . 1) (B . 2))."
  (loop for (k v) on plist by #'cddr
        collect (cons k v)))

(defun alist->plist (alist)
  "Convert ((A . 1) (B . 2)) to (a 1 b 2)."
  (loop for (k . v) in alist
        append (list k v)))
```

Check:

```
(plist-p '(a 1 b 2))                    ; => T
(plist-p '(a 1 b))                      ; => NIL
(plist->alist '(a 1 b 2))               ; => ((A . 1) (B . 2))
(alist->plist (plist->alist '(a 1 b 2))) ; => (A 1 B 2)
```

The honest answer to "which direction loses information": for well-formed inputs, *neither*. Both structures hold the same objects, in the same order, paired the same way. The difference is representation, not information: an alist is a list of conses (each pair is one compound object); a plist is a flat list where keys and values are siblings.

What *changes* is the lookup contract. On either structure, a key whose value is `nil` is indistinguishable from an absent key when the lookup returns just the value:

```
(cdr (assoc 'b '((a . 1) (b . nil))))   ; => NIL — present with nil
(cdr (assoc 'c '((a . 1) (b . nil))))   ; => NIL — absent
(getf '(a 1 b nil) 'b)                  ; => NIL — present with nil
(getf '(a 1 b nil) 'c)                  ; => NIL — absent
```

Both structures need the found-flag protocol to disambiguate — `assoc`'s `nil` result is the flag (nil means absent); `getf` returns just the value, and `get-properties`'s first return is the flag. The "loss" isn't in conversion; it's that neither structure's plain lookup tells you the difference, and the converter can't fix that for you.

A second, smaller point: converting a plist with duplicate keys to an alist preserves both entries, and vice versa. Duplicates are a separate bug, not a loss.

23.4

```
(defun plist-keys (plist)
  "Distinct keys of a plist, in first-occurrence order."
  (let ((seen nil))
    (loop for (k v) on plist by #'cddr
          do (pushnew k seen)
          finally (return (nreverse seen)))))
```

`pushnew` keeps the accumulation distinct, using `eql` by default — the right equality for keyword/symbol keys. On a plist with a duplicated key, `plist-keys` returns that key once; the duplication itself is silently hidden, which is why `plist-well-formed-p` is a separate (and useful) check.

```
(defun plist-well-formed-p (plist)
  "Even length, no duplicate keys."
  (and (plist-p plist)
       (let ((keys (loop for (k v) on plist by #'cddr collect k)))
         (null (remove-duplicates keys :test #'eq :from-end t)))))
```

`remove-duplicates` returning a shorter list is the duplicate detector.

```
(defun plist-subset-p (sub super)
  "Does SUPER contain every key/value pair of SUB?"
  (loop for (k v) on sub by #'cddr
        always (multiple-value-bind (found-key found-value tail)
                   (get-properties super (list k))
                 (declare (ignore tail))
                 (and found-key (equal found-value v)))))
```

Check:

```
(plist-subset-p '(:a 1) '(:a 1 :b 2))    ; => T
(plist-subset-p '(:a 1 :c 3) '(:a 1))    ; => NIL
(plist-subset-p '(:a nil) '(:a nil))     ; => T — present with nil
```

The `get-properties` return is three values: the found key (nil if absent), its value, and the tail of the plist from that key onward. The first value is what `getf` alone can't give you — `getf` returns `nil` both for absent and for present-with-nil. `get-properties`'s found-key is the flag.

`equal` on the values (rather than `eq`) is what makes the function work for strings and lists, not just symbols.

23.5

```
(defmacro swap (a b)
  (let ((tmp (gensym "SWAP-")))
    `(let ((,tmp ,a))
       (setf ,a ,b
             ,b ,tmp))))
```

```
(macroexpand-1 '(swap x y))
;; => (LET ((#:SWAP-427 x)) (SETF X Y Y #:SWAP-427))
```

Check:

```
(let ((x 1) (y 2))
  (swap x y)
  (list x y))   ; => (2 1)
```

Now the naive version and its bug:

```
(defmacro swap-broken (a b)
  `(let ((tmp ,a))
     (setf ,a ,b ,b tmp)))

(macroexpand-1 '(swap-broken x tmp))
;; => (LET ((TMP X)) (SETF X TMP TMP TMP))
```

The expansion is the whole story. When the caller's second argument is literally the variable `tmp`, the macro's `(let ((tmp ...)) ...)` shadows it. `(setf tmp tmp)` writes the *macro's* `tmp` with the *macro's* `tmp` — a no-op — and the caller's `tmp` is never touched. Swap fails silently, and only for that one caller.

`gensym` manufactures a symbol no caller can have written — it's uninterned, so it can't appear in source text — closing the capture hole. This is the "variable capture" trap from Chapter 14 in its most classic form.

23.6

```
(defun describe-flavour (item)
  "Return the flavour text of ITEM, or a default."
  (or (get item 'flavour)
      (format nil "Just a plain ~(~A~)." item)))

(setf (get 'key 'flavour) "Brass, and warmer than it should be.")
(setf (get 'lamp 'flavour) "Its light doesn't quite reach the floor.")

(list (describe-flavour 'key)
      (describe-flavour 'lamp)
      (describe-flavour 'sword))
```

Expected:

```
("Brass, and warmer than it should be."
 "Its light doesn't quite reach the floor."
 "Just a plain sword.")
```

The comment the exercise asks for:

Flavour text is (a) an attribute of a named thing, consulted by name alone — the plist's natural shape; (b) fixed at definition time, so it never needs snapshotting, versioning, or save-game; (c) global by nature — the flavour of the key is the same every session, which is exactly the intent. Item **locations** are the opposite on all three: they change during play,

get saved and restored, and would be spread unownably across symbols on a plist. Chapter 7 gave them an alist for exactly that reason. The dividing line, in one sentence: plists for constants about names, owned structures for state about things.

D.Y. Chapter 24

24.1

```
(list (multiple-value-list (values))      ; => NIL — zero values
      (multiple-value-list (values 1))    ; => (1)
      (multiple-value-list (values 1 2 3))  ; => (1 2 3)
      (+ (values 1 2) (values 3 4))       ; => 4 — primaries only: 1 + 3
      (list (values 1 2) (values 3 4))    ; => (1 3)
      (values (values 1 2) (values 3 4))) ; => two values: 1 and 3
```

Expected: `(NIL (1) (1 2 3) 4 (1 3) 1)`.

```
(NIL (1) (1 2 3) 4 (1 3) 1)
```

Where each element came from:

- `(multiple-value-list (values))` → `NIL` (empty, because zero values)
- `(multiple-value-list (values 1))` → `(1)` (one value)
- `(multiple-value-list (values 1 2 3))` → `(1 2 3)`
- `(+ (values 1 2) (values 3 4))` → `4` (the primaries 1 and 3, summed)
- `(list (values 1 2) (values 3 4))` → `(1 3)` (the list's two argument positions each kept one value)
- The final `(values (values 1 2) (values 3 4))` → two values, `1` and `3` — but it sits in an argument position of `list`, which is a single-value context. `list` keeps only the primary, `1`. That is the last element of the result, and the `3` is discarded.

Two surprises worth naming. First, `(values)` produces *zero* values — and `multiple-value-list` faithfully gives the empty list `nil`, which is "no values collected". The distinction matters only in a multiple-value context; anywhere else, zero values and one nil value both read as nil. Second, `values` doesn't nest: each argument to a `values` call is evaluated in single-value context, so the outer `values` sees only the primaries of its arguments. Values are flat.

24.2 **Surprising:** `x` binds the primary value only — `:a` — and `:b` is gone. `(let ((x (two-things))) x)` prints `A`. Fix: `multiple-value-bind` or `multiple-value-list` at the call:

```
(multiple-value-bind (x y)
    (two-things)
  (list x y))   ; => (:A :B)
```

Surprising: `(:DONE 3)`, not `(:DONE 2)`. The snippet works — and the element that surprises people is the `3`, not the multi-value part. `loop` puts an implicit block named `nil` around its body, so `(return (values :done i))` is `(return-from nil (values :done i))`. `return-from` passes *every* value of its form to the block, and the loop's `finally` clause runs after the last iteration — with `i` already stepped to `3` — so the loop returns `:done` and `3`, and the outer `multiple-value-bind` catches both:

```
(:DONE 3)
```

The point the exercise makes is that `loop`'s return values are governed by `return`, which preserves them — same as `return-from`. What truncates in this snippet is not the loop but a `let` if you introduce one:

```
(let ((pair (loop for i below 3
                  finally (return (values :done i)))))
  pair)   ; => :DONE — pair binds one value
```

`let` truncates, `multiple-value-bind` doesn't. The trap is which binding form you use to catch the loop's result.

Surprising: `gethash` returns `nil` and `t`: the key *is* present, holding `nil`. The naive reading "nil means absent" is exactly what the second value refutes. This is the correct-behaviour example, not a bug — the fix is for code that ignored the flag:

```
(multiple-value-bind (val found-p) (gethash :k table)
  (if found-p val :absent))   ; => NIL (the stored one)
```

Surprising: (1), not~`(1 2)`. The function's body is the variable reference `form-values` — but the *caller's* argument position `(pass-through (values 1 2))` is a single-value context, so `pass-through` received 1 before any body ran. No function body can recover values its caller truncated. Fix: take a thunk and call it in a multiple-value context:

```
(defun pass-through (thunk)
  (funcall thunk))              ; body end: values preserved
(multiple-value-list (pass-through (lambda () (values 1 2))))
;; => (1 2)
```

The thunk receives no argument, so its result's values reach `multiple-value-list` intact. `(funcall thunk)` is the function body's last form, which is a multiple-value context — the values pass through unchanged. That's the entire difference between "function that takes a value" and "function that takes a computation".

24.3

```
(defun safe-divide (a b)
  "Two values: quotient, and :DIVISION-BY-ZERO or NIL."
  (if (zerop b)
      (values nil :division-by-zero)
      (values (/ a b) nil)))

(defun safe-sum (pairs)
  "Sum of quotients; two values: total, failed-p."
  (let ((total 0)
        (failed nil))
    (dolist (pair pairs (values total failed))
      (multiple-value-bind (q err)
          (safe-divide (first pair) (second pair))
        (if err
```

```
            (setf failed t)
            (incf total q))))))
```

Check:

```
(list (multiple-value-list (safe-sum '((7 2) (9 3))))
      (multiple-value-list (safe-sum '((7 0) (9 3))))
      (multiple-value-list (safe-sum nil)))
```

Expected: `((13/2 NIL) (3 T) (0 NIL))`.

The first case: (safe-divide 7 2) gives `7/2`, (safe-divide 9 3) gives `3`, sum `13/2`, no failure — (13/2 NIL).

The second: `(safe-divide 7 0)` gives `(nil :division-by-zero)` — failed, so the sum skips it and adds only the `9/3`'s `3`. `(3 T)`.

The third: empty input, `total` starts at `0`, `failed` stays `nil`. `(0 NIL)`.

The shape to notice: `safe-sum` propagates failure as a flag, not a condition, and callers who don't care about failure just use the total. It's `gethash`'s contract, rebuilt by hand. Compare:

- `gethash` returns value + found-flag. Callers ignoring the flag get the value or `nil`. Callers binding both get the distinction.
- `safe-sum` returns total + failed-flag. Same shape.
- `(handler-case ...)` is for bugs — errors the caller couldn't have anticipated. Flags are for predicted `failures` — errors the caller routinely branches on.

The rule of thumb: if the caller's natural response is "check the second value and maybe react", use a flag. If the caller's natural response is "let someone higher up deal with it", signal a condition.

24.4

```
(defun compose-values (f g)
  "Return a function that applies G to all of F's values."
  (lambda (&rest args)
    (multiple-value-call g (apply f args))))
```

Check:

```
(funcall (compose-values #'floor #'list) 7 2)
;; => (3 1)
```

`(apply f args)` calls `floor` with two arguments, returning two values. `multiple-value-call` then calls `g` with those two values as separate arguments — so `list` receives `3` and `1`, producing `(3 1)`. If the body had been the naive `(funcall g (apply f args))`, the intermediate `apply` would truncate to `3` before `g` was called, and the answer would be `(3)`.

The primitive that `compose-values` is a specialisation of: `multiple-value-call`. In fact:

```
(funcall (compose-values f g) args...)
= (multiple-value-call g (apply f args))
```

Every `multiple-value-call` expresses one composition — a call to `g` with all the values produced by the argument forms. `compose-values` packages that into a reusable function; the underlying primitive is the one you'd reach for directly in a one-off composition, without building the wrapper.

24.5

```
(defun first-or-default (thunk default)
  "THUNK's primary value, or DEFAULT if it returned none."
  (let ((vals (multiple-value-list (funcall thunk))))
    (if vals (first vals) default)))
```

Check:

```
(list (first-or-default (lambda () (values)) :none)
  (first-or-default (lambda () (values 1 2)) :none)
  (first-or-default (lambda () nil) :none))
```

Expected: `(:NONE 1 NIL)`.

The three cases:

`(values)` — zero values — `vals` is `nil`, so `default` is returned.

`(values 1 2)` — two values — `vals` is `(1 2)`, so `(first vals)` gives `1`.

`nil` — *one* value, which happens to be nil — `vals` is `(nil)`, so `(first vals)` gives `nil` (the value, not the default).

The `default`'s value is `:none` (a keyword) rather than `nil`, because `nil` is itself a legitimate *value*, and using `nil` as the default would conflate it with "the thunk returned nil". `:none` is unambiguous.

24.6

```
(defun save-game ()
  "Write the game state. Two values: success-p, path-or-message."
  (handler-case
    (progn
      (with-open-file (out save-file
          :direction :output
          :if-exists :supersede)
        (with-standard-io-syntax
          (format out "cave-of-noise v~D~%" save-format)
          (print (game-state) out)))
      (values t save-file))
    (file-error (c)
      (values nil (princ-to-string c)))))

(defun load-game ()
  "Restore the game state. Two values: state-or-nil, loaded-p."
  (handler-case
    (with-open-file (in save-file)
      (let ((header (read-line in)))
        (unless (search "cave-of-noise" header)
          (error "Not a Cave of Noise save file: ~S" header))
        (let ((state (with-standard-io-syntax (read in))))
```

```
          (values state t))))
    (file-error () (values nil nil))
    (end-of-file () (values nil nil))
    (error () (values nil nil))))
```

Note that `load-game` now returns the state rather than applying it — the caller decides whether to `restore-state` it. That's the flag pattern applied cleanly: the function's job is "get the data from disk"; whether the game adopts it is the game's decision.

Update the save and load commands:

```
(defun cmd-save ()
  "The SAVE command: report success or failure."
  (multiple-value-bind (ok path-or-err)
    (save-game)
    (if ok
      (format nil "Saved to ~A." path-or-err)
      (format nil "Could not save: ~A" path-or-err))))

(defun cmd-load ()
  "The LOAD command: load and adopt, or report failure."
  (multiple-value-bind (state loaded-p)
    (load-game)
    (if loaded-p
      (progn
        (restore-state state)
        (look))
      "No saved game found.")))
```

Register them:

```
(push '(save . cmd-save) commands)
(push '(load . cmd-load) commands)
```

Check:

```
(setf save-file "/tmp/test-roundtrip.lisp")
(setf current-room (find-room 'kitchen))
(setf item-locations '((key . kitchen)))
(setf history nil)

(run-command '(save))
(setf item-locations nil)
(run-command '(load))
item-locations ; => ((KEY . KITCHEN))
```

The comment the exercise asks for:

The string-matching approach has three problems that the two-value approach fixes. First, it's brittle: change the error message text, and the caller's match stops working. Second, it's not the information the caller wants — "there was no file" and "the file was corrupt" are both "no load happened", but a caller might want to distinguish them for logging or

for a different player message. Two values (state, loaded-p) say what happened; if the caller needs the reason, the function can return three values (state, loaded-p, reason), and callers who don't care ignore the third. Third, string matching is slow and requires allocation of a temp string; comparing against nil or t costs nothing.

The reason *this* function returns two values instead of signaling a condition: a missing save file is expected (first run of a fresh game), not a bug. Conditions are for bugs and emergencies. Flags are for expected outcomes the caller branches on. save-game and load-game are on the flag side of the line; the game's data structures on disk are on the condition side (see Exercise 18.6′s convert-save-v0 and the reader's handling of corrupt files).

The game's state at the end of this chapter: `run-command`, `play`, `save-game`, and `load-game` all use multiple values as their interface, `save` and `load` commands wrap the two plist-replacing commands from Chapter 16 with the two-value versions above, and the rest of the game is unchanged. The trickle-down effect of Chapter 24′s mechanism has been everywhere; the trickle-up effect, on the game's own code, has been to replace one string-match hack (`count-unknown`) and one string-return convention (save-game/load-game) with values.

D.Z. Chapter 25

25.1

```
(let ((fns (loop for i from 1 to 3
                 collect (lambda () i))))
  (mapcar #'funcall fns))
;; => (1 2 3)
```

`loop`'s `for i` binds a fresh `i` per iteration, so each closure owns its own. (In languages where a loop shares one variable, this prints `3 3 3`; Lisp's per-iteration binding is the reason the `make-bumpers` example works.) The standard guarantees it for `loop`; for `dotimes`, it's implementation-defined — SBCL does the right thing, most modern implementations do, but `loop` is the guaranteed form.

```
(multiple-value-bind (up down) (make-two)
  (list (funcall up) (funcall up) (funcall down) (funcall up)))
;; => (1 2 1 2)
```

Both closures share one `n`. Trace it: `up` takes n 0→1, returns 1; `up` takes n 1→2, returns 2; `down` takes n 2→1, returns 1; `up` takes n 1→2, returns 2. The list is `(1 2 1 2)`.

If your answer was `(1 2 -1 1)` — a guess that `down` returns the negative of something — re-read the closure: `down` is `(lambda () (decf n))`, and `decf` returns n's new value, not the amount decremented. Closure sharing of a single binding means increments and decrements on the same number, in order.

```
(let ((x 10))
  (let ((f (lambda () x)))
    (setf x 99)
    (funcall f)))
;; => 99
```

The closure captured the *binding*, not the value `10` at creation time; the later `setf` changed that binding, and `f` sees the change. Value-snapshot thinking predicts 10; binding thinking predicts 99; Lisp does 99.

25.2 **Wrong:** `total` has no binding — the `lambda`'s free variable names nothing, and calling the closure signals `UNBOUND-VARIABLE` (or, under some compilers, captures a dynamic `total` if one happens to exist — worse). Fix: the `let` was the whole design:

```
(defun make-accumulator ()
  (let ((total 0))
    (lambda (n) (setf total (+ total n)))))
```

Surprising, but it works — the surprise is the *shape* of the data. `(first *counters*)` is the pair `(a #<closure>)`, and `(second ...)` extracts the closure. `(funcall (second (first *counters*)))` correctly returns 1. The exercise's real trap is a different spelling that fails: `(funcall *counters*)`, which is `funcall` on the whole list, not on any closure. Use the pair's `.cdr` to extract the closure cleanly, which is also how you'd normally write it:

```
(defparameter *counters*
  (loop for name in '(a b c)
        collect (cons name (let ((count 0))
                             (lambda () (incf count))))))
(funcall (cdr (assoc 'a *counters*)))   ; => 1
```

— an alist of name to closure. The `funcall` form is the same as above; the extraction is `cdr (assoc ...)` instead of `second (first ...)`.

Surprising, but it works in SBCL and most modern implementations. `dotimes` binds `i` per iteration, so the closures capture 0, 1, and 2 respectively, and `(3 2 1)` comes out because `push` reverses. This is implementation-defined in the standard — the standards text is ambiguous. If you want the guarantee, use `loop for i below 3` instead; if you only care about SBCL, `dotimes` is fine.

The real trap the exercise warns about: when the closure captures an *outer function parameter* rather than the loop's own binding. That case always fails, in every implementation:

```
(defun make-wrong-fns (limit)
  (let ((fns nil))
    (dotimes (i limit (nreverse fns))
      (push (lambda () limit) fns))))  ; captures LIMIT not I
;; every closure returns the same LIMIT
```

The `lambda`'s free variable is `limit` — one binding, one value. Not `i`, which is per-iteration.

25.3

```
(defun make-average ()
  "Return a closure: running average, two values (average, count)."
  (let ((total 0)
        (count 0))
    (lambda (n)
      (incf count)
```

```
      (incf total n)
      (values (/ total count) count))))
```

Check:

```
(defparameter *avg* (make-average))
(multiple-value-list (funcall *avg* 4))   ; => (4 1)
(multiple-value-list (funcall *avg* 2))   ; => (3 2)
(multiple-value-list (funcall *avg* 9))   ; => (5 3)
```

Averages: 4/1, (4+2)/2=3, (4+2+9)/3=5. The two values (average, count) let callers check both the current result and how much data went into it, without a separate "how many samples?" function.

The `:count` extension to `make-registry`:

```
(defun make-registry ()
  (let ((table (make-hash-table)))
    (lambda (message &optional key value)
      (ecase message
        (:put  (setf (gethash key table) value))
        (:get  (gethash key table))
        (:keys (loop for k being the hash-keys of table
                     collect k))
        (:count (hash-table-count table))))))
```

Existing callers using `:put`, `:get`, `:keys` don't change; the closure gains a message. That's the payoff of the message-dispatch shape: *extension by addition*, no caller touched. Chapter 26's generic functions generalise exactly this to types.

25.4

```
(defun make-stack ()
  "Four closures over one list: push!, pop!, empty-p, contents."
  (let ((items nil))
    (values
     (lambda (x) (push x items))
     (lambda ()
       (if (null items)
           (error "Stack underflow.")
           (pop items)))
     (lambda () (null items))
     (lambda () (copy-list items)))))
```

Check:

```
(multiple-value-bind (push! pop! empty-p contents)
    (make-stack)
  (funcall push! 1)
  (funcall push! 2)
  (let ((snapshot (funcall contents)))
    (funcall pop!)
    (list snapshot (funcall contents) (funcall empty-p))))
```

Expected: `((2 1) (1) NIL)` — the snapshot taken before the pop still shows both elements; the second contents shows one; the stack isn't empty.

The comment the exercise asks for:

contents returns a copy, not the internal list. This is the difference between encapsulation and leaking. If contents returned items directly, a caller could do (setf (car (funcall contents)) :garbage) and corrupt the stack's internal state without using push!/pop! — bypassing the underflow check, the size discipline, everything the stack exists for. Returning a copy means: "here is the state as it currently stands," but the state itself is still private. This is a general principle of closure-based (and class-based) objects: accessors return values, not references to mutable internal state, unless the whole point is to share that state. The same choice appears in Chapter 7′s item-name (returns a fresh string) vs an accessor that returned the symbol-name directly (would be fine, since strings from symbol-name are fresh, but the habit matters). The trade-off: copying costs time and memory, so hot accessors sometimes cheat — as long as the caller's contract forbids mutation. The game's **item-locations** is a counter-example: it's returned directly because callers are expected to mutate it via set-location. Which is right depends on what the interface promises.

25.5

```
(defun make-key-matcher (key target)
  "A predicate: does (funcall KEY item) equal TARGET?"
  (lambda (item) (equal (funcall key item) target)))
```

Use with `find-if`:

```
(find-if (make-key-matcher #'room-name 'kitchen) *rooms*)
;; => #<ROOM kitchen>
(remove-if-not (make-key-matcher #'room-name 'kitchen) *rooms*)
;; => (#<ROOM kitchen>)
```

The comparison with `name-matcher`:

- `name-matcher` is `remove-if`'s predicate: "is this room the target?" — one argument, answers yes or no.
- `make-key-matcher` is the *generalised* version: the key function can be anything, and the target comparison uses `equal` so strings work too.

Which generalises, and why: `make-key-matcher`, because it makes the attribute-lookup part a parameter. `name-matcher` is the special case `(make-key-matcher #'room-name target)` with `eq` for the comparison — valid when the key returns symbols. The general version is what you'd use in library code; the specialised one is fine for a single use-site.

25.6

```
(defun make-cooldown (&optional (window 2))
  "Block commands seen in the last WINDOW calls."
  (let ((history nil))
    (lambda (message &optional cmd)
      (ecase message
        (:block
         (let ((blocked (member cmd history :test #'equal)))
           (push cmd history)
           (when (> (length history) window)
```

```
            (setf history (subseq history 0 window)))
          (not (null blocked))))
       (:reset
        (setf history nil)
        t)))))
```

Wire it into the game:

```
(defparameter *cooldown* (make-cooldown))

(defun run-command (command)
  "Execute COMMAND.  Two values: reply, known-p."
  (if (funcall *cooldown* :block command)
      (values "Not again." t)
      (restart-case
          (handler-case (call-command command)
            (program-error ()
              (values "Wrong number of arguments for that command." t)))
        (retry-with (fixed)
          :report "Run a corrected command instead."
          (run-command fixed)))))
```

Check:

```
(setf *current-room* (find-room 'kitchen))
(setf *item-locations* '((key . kitchen)))
(funcall *cooldown* :reset)

(list (run-command '(look))
      (run-command '(look))
      (run-command '(take key))
      (run-command '(look)))
```

Expected: the look description, `("Not again." T)`, the take message, then the look description again — because `look`, `take`, `look` doesn't repeat `look` within the last two commands; only consecutive-in-window repeats are blocked.

The comment the exercise asks for:

Cooldown is (a) session state, like history. It's about this play-through's interaction rhythm, not the world's state. If the player quits and reloads, a fresh cooldown is correct — punishing them for reconnecting would be bad UX. The rule: ask "would a loaded game have this state back where it was?" If yes, it's world state (structures, alists, saved). If no, it's session state (closures, not saved). Item locations are world state — the player expects the key to be where they left it. Cooldown is session state — the player expects a fresh start. The subtlety the exercise probes: could a designer argue the other way? Yes — "you have to wait 2 commands between looks" could be world state, surviving saves. That would be a game design choice, and it would require promoting the cooldown closure to a save-relevant structure (an alist or class). The default is session state because it serves the player; the alternative is available if the design needs it. The decision isn't mechanical — it's about *whose needs the state serves.*

D.AA. Chapter 26

26.1

```
(defclass lamp (thing)
  ((lit :initarg :lit
        :accessor lit-p
        :initform nil)))

(defparameter *my-lamp* (make-instance 'lamp :name 'lamp))
(lit-p *my-lamp*)            ; => NIL
(setf (lit-p *my-lamp*) t)
(lit-p *my-lamp*)            ; => T
```

The `:accessor` gives both directions; `:reader` would have made the `setf` a "no setf-expander" error. The `:initform nil` is redundant (all slots default to `nil` unless told otherwise), but writing it makes the intent explicit, which is worth the line.

26.2

```
(defclass a () ((x :reader x-of :initform 1)))
(defclass b (a) ((x :reader x-of :initform 2)))

(x-of (make-instance 'b))   ; => 2
```

The answer is 2 because `b` redefines the `x` slot: it does not add a second one. `make-instance` of `b` initialises `x` from `b`'s `:initform`, the most specific class in the precedence list, and `x-of` reads that slot. One slot per name, nearest definition wins — the precedence rule at work on slots rather than methods.

Contrast with a case where `b` *didn't* redefine `x`:

```
(defclass a () ((x :reader x-of :initform 1)))
(defclass b (a) ())
(x-of (make-instance 'b))   ; => 1 — inherited, unchanged
```

The `x` slot lives in `a`; `b` has no opinion about it; the inheritance chain supplies the value. Redefining a slot in a subclass doesn't extend it, it replaces it — a common source of confusion when refactoring a class hierarchy.

26.3

```
(defgeneric empty-thing-p (thing)
  (:documentation "Is THING empty?"))

(defmethod empty-thing-p ((t2 thing))
  nil)   ; the default: things are not empty

(defmethod empty-thing-p ((c container))
  (null (contents c)))
```

Check:

```
(list (empty-thing-p *key-thing*)   ; => NIL
      (empty-thing-p *chest*)       ; => NIL (it holds crown, gem)
      (empty-thing-p (make-instance 'container :name 'empty-box)))
;; => (NIL NIL T)
```

The comment the exercise asks for:

:before and :after compose — they all run — so they can't choose between answers; only primary methods select, and the question "is it empty" wants exactly one answer, the most specific one's. Whenever a method computes a *value* rather than performing a *side effect*, it belongs as a primary method. `:before` would run the primary anyway; `:after` would run after; either would give two answers for the same question, and callers wouldn't know which to trust. Primary methods answer; before/after methods prepare and clean up.

26.4 **Wrong:** `potion` never declares `:reader` or `:accessor` for `colour`, so `thing-colour` doesn't exist — and neither does `colour`, since no slot named `colour` is readable. And the name `thing-colour` is the wrong guess: it would be `potion-colour` or just `colour`, depending on how the slot was declared. Fix: declare `:reader colour`:

```
(defclass potion (thing)
  ((colour :initarg :colour :reader colour)))
```

Now `(colour *potion*)` works. `(thing-description *potion*)` also works — `potion` inherits `thing-description` — but only if the potion instance has a `:description`; the class inherits the `:initarg`, and the default `:initform` from `thing` covers it.

Wrong: the method names the class `thing` before `defclass` has run. Loading the file top to bottom, `defmethod` fails with "The class THING is not defined" — or worse, treats `thing` as an undefined variable in the specialiser. Order the file: classes, then methods — or let ASDF's `:serial` do it (Chapter 20's lesson, in a CLOS context).

Wrong: `(typep 'thing 'thing)` is `nil`: the first argument is the *symbol* `thing`, not an instance. (typep **key-thing** 'thing) is the question with an object in it, and gives `t`. The quote in `typep`'s second argument is type syntax, not object reference; the quote in `'thing` as the *first* argument is exactly what makes it a symbol rather than an instance.

26.5

```
(defmethod combine ((a symbol) (b symbol))
  (list a b))

(defmethod combine ((a string) (b number))
  (format nil "~A~D" a b))

(defmethod combine (a b)
  (error "Can't combine ~S and ~S." a b))
```

Check:

```
(list (combine 'a 'b)
      (combine "x" 5)
      (ignore-errors (combine 1 "x")))
```

Expected: `((A B) "x5" NIL)` — the third call signals, so `ignore-errors` returns nil; without it, the debugger opens.

Three more methods on top of the chapter's four. That's the point of the exercise: with dispatch on two arguments and several types each, the number of methods is the *product* of the cases you want — specific pairs — not the sum, and the catch-all on `t` keeps the

generic function total. A `typecase` version would have one ever-growing body; here each case is independent and can be added from a different file by a different author.

The catch-all method is the discipline: without it, calling `combine` on a pair with no matching method signals `no-applicable-method`, which is less helpful than a specific error. With it, the error names the arguments. That's the trade — a catch-all method gives you control over the error's shape, at the cost of one extra method.

26.6

```
(defmethod initialize-instance :after ((c container) &key)
  (when (contents c)
    (setf (contents c)
          (sort (copy-list (contents c)) #'string<
                :key #'symbol-name))))
```

Check:

```
(defparameter *box*
  (make-instance 'container
                 :name 'box
                 :contents '(zebra apple)))
(contents *box*)   ; => (APPLE ZEBRA)
```

The `:after` method runs after the slot is filled from `:initarg`, so it sees the given contents and repairs them. `copy-list` before `sort` is Chapter 10's rule about not sorting things you don't own — the list in the slot may be the caller's. The :key `#'symbol-name` makes `string<` compare the names of the symbols, alphabetically.

Sorting content is a repair, not a semantic transformation: the same set of things, put in a canonical order. That's the right shape for `initialize-instance :after` — it enforces an invariant without changing meaning. A `:after` that filtered duplicates would be more aggressive; it would be visible transformation, and better signalled by a separate method called by the caller, not by `make-instance`.

26.7

```
(defgeneric weight (thing)
  (:documentation "How much does THING weigh?"))

(defmethod weight ((t2 thing)) 1)

(defmethod weight ((c container))
  (+ 1 (length (contents c))))

(defun total-weight (symbols)
  "Sum the weights of the registered things named in SYMBOLS."
  (reduce #'+ (mapcar (lambda (s)
                        (weight (lookup-thing s)))
                      symbols)))
```

Check:

```
(total-weight '(key chest))
;; => 1 + 3 = 4
```

— with the chest holding `(crown gem)`, `(weight #strong[chest])` is `1 + 2 = 3`, and the key is `1`. Sum `4`.

What happens with an unregistered symbol: `(lookup-thing 'unregistered)` returns `nil`, and `(weight nil)` signals — `weight`'s methods both specialise on classes (`thing` and `container`) and `nil` is an instance of neither. So the chain of failures runs mapcar → weight → no-applicable-method → `debugger`. That suggests where a `handler-case` belongs — in `total-weight`, at the level that knows what "missing item" means for its own contract:

```
(defun total-weight (symbols)
  "Sum the weights of the registered things in SYMBOLS."
  (handler-case
      (reduce #'+ (mapcar (lambda (s) (weight (lookup-thing s)))
                          symbols))
    (error () nil)))   ; or signal a specific condition
```

The generic version of the exercise's question — "where does condition handling belong" — is Chapter 19's flags-vs- conditions rule, one more time. `total-weight`'s caller wants a total or a signal of "some symbols don't resolve", not a raw `no-applicable-method` error about `nil`. The handler is the translation layer.

26.8

```
(defun list-things (command)
  "Handle (LIST-THINGS): list every registered thing, alphabetically."
  (declare (ignore command))
  (let ((lines nil))
    (maphash (lambda (sym instance)
               (push (format nil "~(~A~): ~A"
                             sym (thing-description instance))
                     lines))
             *thing-registry*)
    (format t "~{~A~%~}" (sort lines #'string<))))
```

Register it:

```
(push '(list-things . list-things) *commands*)
```

Check:

```
(list-things '(list-things))
```

Expected output (sorted by the string, so the symbol's name sort matters — `chest`, `crown`, `key`, `lamp`, `vase`):

```
chest: a wooden chest.
crown: a tarnished crown.
key: a small brass key.
lamp: an oil lamp, dusty but intact.
vase: a thin porcelain vase.
```

The comment the exercise asks about — what does the class hierarchy buy over a plist:

Storing the description on the symbol's plist would work for the description itself: (setf (get 'key 'description) "…") puts it where get can find it. Three things the class hierarchy buys beyond that:

1. Structured data. A thing can have several slots — name, description, weight, value, contents — that belong together as one thing, distinguishable by which instance carries which. A plist on a symbol mixes the description, the weight, and everything else into one flat list; keeping the fields associated per-instance is a discipline the plist doesn't enforce.
2. Behaviour that varies by kind. describe-thing dispatches on class; there is no way to make (get 'key 'description) vary by kind without writing if-typecase logic. Adding a new kind (fragile, say) means one new class and one new method; adding it to the plist world means editing every function that reads the plist.
3. State safety. A plist is global mutable state attached to a symbol — remprop, setf get, any code anywhere can corrupt it (Chapter 23's warnings). An instance's slot is protected by the class's accessor contract, and two instances of the same class never share state accidentally.

The plist's one advantage: symbolic stability. A symbol's plist is the same wherever the symbol appears; a class instance must be passed. For constants attached to a name, plists are fine. For a *thing*, with several slots and behaviour, classes are the right shape.

D.AB. Chapter 27

27.1

```
(defclass a () ())
(defclass b (a) ())
(defclass c (a) ())
(defclass d (b c) ())

(mapcar #'class-name
        (sb-mop:class-precedence-list (find-class 'd)))
```

Expected: `(D B C A STANDARD-OBJECT T)`.

The rule: the class itself first, then its direct superclasses *left to right*, then their superclasses, deduplicated. `b` and `c` share `a`, so `a` comes once (after both its children — it's the superclass of both). Then `standard-object` (the default CLOS parent), then `t` (everything).

`b`'s methods win over `c`'s: `b` is leftmost in `d`'s superclass list, so it appears first in the precedence list, so a method specialised on `b` is more specific than one specialised on `c` for a `d` instance. If `d`'s superclass list were `(c b)` instead, `c`'s methods would win.

That leftmost-wins rule is why the order in the superclass list is meaningful, and why "put the primary nature first" is the advice. If you reorder the superclass list as a "refactor", you may change method dispatch throughout the hierarchy.

27.2 **Wrong:** the `:after` method computes a string that is then *discarded* — auxiliary method results don't contribute to a generic function's return value, only side effects matter. The description never appears anywhere. Fix: store the result or make it a primary method:

```
(defmethod describe-widget ((w widget))
  (format nil "x is ~A" (widget-x w)))
```

A primary method's value *is* the generic function's result when it's the only one or the most-specific one. That's the shape you want for "return a string describing this thing".

Surprising: the `:after` on `sub` runs after `sub`'s primary method for the accessor `n`. That means every read of `n` on a `sub` instance increments the slot by 10 — a read that mutates. Ten reads in a row, the slot has grown by 100. That's not "add 10 to the result"; that's "the act of reading changes the state". It looks like a bug in the accessor, and it is. Fix: don't mutate in an accessor-adjacent method. If `sub` needs a derived value, give it its own accessor and compute on demand:

```
(defclass sub (base)
  ((bonus :initform 10 :accessor bonus)))

(defun n-total (s) (+ (n s) (bonus s)))
```

Then `n` stays pure (reads the slot, returns the value); `n-total` is the derived value, computed fresh each call.

Wrong: `(defmethod area ((s shape)) 0)` specialises on `shape`, which is not defined — the `defmethod` signals "The class SHAPE is undefined" unless a `shape` class exists. The fix is to define the class first (or specialise on `t` for the catch-all):

```
(defclass shape () ())
(defgeneric area (shape))
(defmethod area ((s shape)) 0)
```

Also note the `defgeneric` placement: declaring the generic function *before* its methods is the readable order, and catch-all `t` specialisers read poorly — prefer a real base class.

27.3

```
(defparameter *method-notes* nil)
;; entries: (class-name kind function-or-description)

(defun note-method (class-name kind what)
  (push (list class-name kind what) *method-notes*))

(defun describe-chain (instance)
  "Print the precedence list with known methods per class."
  (dolist (class (sb-mop:class-precedence-list (class-of instance)))
    (let ((name (class-name class))
          (notes (remove-if-not
                  (lambda (n) (eq (first n) name))
                  *method-notes*)))
      (format t "~&~(~A~):~%" name)
      (dolist (n notes)
        (format t "  ~A — ~A~%" (second n) (third n))))))

;; demo data:
(note-method 'thing :primary "price: base 1")
```

```
(note-method 'treasure :primary "price: (value tr)")
(note-method 'antique :primary "price: 50")
(note-method 'container :primary "price: 2")
(describe-chain (make-instance 'antique
                  :name 'mirror
                  :value 200))
```

Expected output:

```
ANTIQUE:
  PRIMARY — price: 50
TREASURE:
  PRIMARY — price: (value tr)
THING:
  PRIMARY — price: base 1
STANDARD-OBJECT:
T:
```

The output teaches the chapter's central fact: method order surprises are *precedence surprises* — the methods aren't mysterious; they're listed class by class, and what felt like random dispatch is the precedence list being walked in order. When dispatch surprises you, print the chain before guessing.

The real MOP versions of this — `compute-applicable-methods` returning the actual applicable methods for a given call, `find-method` looking up a method by specialisers — exist and are worth knowing by name. But `compute-applicable-methods` is on `standard-generic-function`, and its results are implementation-defined in some respects. The manual registry above is honest: it says "here's what I know about my own code's methods", which for application code is what you can actually rely on.

27.4

```
(defgeneric participants ()
  (:method-combination list))

(defclass round-table () ())
(defclass round-table-plus (round-table) ())

(defmethod participants list ((r round-table)) 'alice)
(defmethod participants list ((r round-table-plus)) 'bob)

(participants (make-instance 'round-table-plus))
;; => (BOB ALICE) — most specific first
```

The timing around-method:

```
(defmethod participants :around ((r round-table))
  (let ((start (get-internal-run-time))
        (result (call-next-method)))
    (format *trace-output* "participants: ~D ticks~%"
            (- (get-internal-run-time) start))
    result))
```

```
(participants (make-instance 'round-table-plus))
;; => (BOB ALICE), and a trace line
```

The `:around` must specialise on the most general class the generic function `serves` — `round-table` here, or `t`, but never on `round-table-plus` — so it wraps every call. If it specialised on `round-table-plus`, timing would record only calls on plus instances, and a caller timing `(make-instance 'round-table)` would see no trace. The `:around`'s specialiser controls *which calls it sees*, and you want it to see all of them.

One more subtlety: `call-next-method` in an `:around` invokes the *rest of the combination* — all remaining methods, combined — not merely "the next primary". For a `list` combination, that means all remaining primary methods contribute to the returned list. The `:around` sees them as a single combined result.

27.5

```
(define-condition class-change-drops-slots (error)
  ((slots :initarg :slots :reader dropped-slots))
  (:report (lambda (c s)
             (format s "change-class would drop: ~{~A~^, ~}"
                     (dropped-slots c)))))

(defun slot-names (class)
  "Names of CLASS's slots, including inherited."
  (let ((names nil))
    (labels ((walk (c)
               (dolist (s (sb-mop:class-direct-slots c))
                 (pushnew (sb-mop:slot-definition-name s) names))
               (dolist (super (sb-mop:class-direct-superclasses c))
                 (walk super))))
      (walk class)
      names)))

(defun safe-change-class (instance new-class-name)
  "Change INSTANCE's class, refusing to drop slots."
  (let* ((old-class (class-of instance))
         (new-class (find-class new-class-name))
         (old-slots (slot-names old-class))
         (new-slots (slot-names new-class))
         (dropped (set-difference old-slots new-slots)))
    (when dropped
      (error 'class-change-drops-slots :slots dropped))
    (change-class instance new-class)))
```

Check:

```
(defclass small-thing (thing) ())
(defclass large-thing (thing) ())
(defparameter *item* (make-instance 'small-thing :name 'coin))
(safe-change-class *item* 'large-thing)   ; works; no slot dropped

(defclass bigger-thing (thing)
  ((mass :initform 1)))
```

```
(safe-change-class *item* 'bigger-thing)  ; works too
;; (round-trip to a smaller class would refuse if slots dropped)
```

What the refusal teaches: `change-class`'s slot policy is *name-based* — slots with the same name carry over, new slots are added with their `:initforms`, and slots the new class *lacks* are simply *gone*, silently, no condition, no warning. That's generous for interactive work (a quick experiment doesn't get blocked by slot mismatches) and dangerous for state you care about (the game's inventory could lose an item's worth slot by an accidental class change).

The MOP functions `slot-definition-name`, `class-direct-slots` and `class-direct-superclasses` are technically beyond the ANSI standard — they're the Metaobject Protocol, and SBCL ships them (mostly via `closer-mop` for the portable name). The `slot-names` above is portable-ish but not guaranteed; a hand-maintained slot table for the classes you care about would work too. The check is the point; the mechanism is a detail.

27.6

```
(defclass room ()
  ((name :initarg :name :reader room-name)
   (description :initarg :description :reader room-description)
   (exits :initarg :exits :initform nil :accessor room-exits)))

(defclass dark-room (room) ())
(defclass visited-room (room) ())

(defgeneric weight (room)
  (:method-combination +))

(defmethod weight + ((r room))
  (max 1 (length (room-exits r))))

(defmethod weight + ((r dark-room))
  5)

(defmethod weight + ((r visited-room))
  -2)

(defparameter *dungeon*
  (list (make-instance 'room :name 'hall :exits '(a b c))
        (make-instance 'dark-room :name 'crypt :exits '(a))
        (make-instance 'visited-room :name 'foyer :exits '(b))))

(defun find-room-by-name (name)
  (find name *dungeon* :key #'room-name))

(defun safest-exit (here)
  "The exit destination with the lowest weight."
  (first
   (sort (copy-list (room-exits here))
         #'<
         :key (lambda (dest-name)
                (weight (find-room-by-name dest-name))))))
```

```
(safest-exit (find-room-by-name 'hall))
;; => error: no applicable method for NIL
```

The demo data is wrong, and the error is worth reading: exits are room *names* (symbols), but `hall`'s exits are `(a b c)` and none of those rooms exist in `*dungeon*`. Then `find-room-by-name 'a` is `nil`, and `(weight nil)` has no applicable method. Fix the data:

```
(defparameter *dungeon*
  (list (make-instance 'room :name 'hall :exits '(crypt foyer))
        (make-instance 'dark-room :name 'crypt :exits '(hall))
        (make-instance 'visited-room :name 'foyer :exits '(hall))))

(safest-exit (find-room-by-name 'hall))
;; => FOYER
```

Now `crypt` is a `dark-room` (weight 5 from dark + 1 from room = 6); `foyer` is a `visited-room` (weight 1 from room + −2 from visited = −1). Safest exit is the one with the lowest weight, `foyer` at −1.

The comment the exercise asks for:

`+` is right because room difficulty *adds up* — base cost from exit count, plus darkness penalty, minus visit discount is an arithmetic sum of independent contributions, and each class owns exactly its contribution. `max` would be wrong because it picks *one* contribution and discards the others: a dark, visited room would weigh 5 (the darkness penalty) instead of 1+5-2=4, and the whole point of the visit discount would be lost. Adding a new factor (say, a `flooded-room` penalty class) with `max` means rewriting the combination logic, where `+` just gains another method. The rule: `+` when contributions are independent quantities that sum; `max` when the answer is "the single worst (or best) factor" — and mixing the two readings in one generic function is the design smell to fix by splitting it in two.

There's a subtlety worth stating: negative contributions. −2 from `visited-room` means a visited room is *safer* than an unvisited one. With `max` combination, negative contributions would vanish; with `+`, they accumulate. If your game design wanted "visited rooms always score at least as safe as unvisited ones, never less than 0", you'd need a floor: max 0 `(weight ...)` applied after the sum. The combination type controls what "score" means; the game design controls where the floor is.

D.AC. Chapter 28

28.1

```
(list (find-symbol "LIST" :cl)   ; => LIST, :EXTERNAL
      (find-symbol "list" :cl)   ; => NIL, NIL — tables are case-sensitive
      (eq 'cl:car (intern "CAR" :cl))  ; => T — same symbol
      (symbol-package 'cl:car)   ; => #<PACKAGE "COMMON-LISP">
      (symbol-package 'car)      ; => #<PACKAGE "COMMON-LISP">
      (multiple-value-list (find-symbol "RUN-COMMAND" :con)))
;; => (RUN-COMMAND :EXTERNAL) if exported, (RUN-COMMAND :INTERNAL)
;;    if it's interned but not exported
```

The `symbol-package 'car` line is the inheritance lesson: `symbol-package` answers "where is this symbol *home*", not "how did I reach it". In `cl-user`, `car` resolves to the `CL:CAR` symbol by inheritance — the `cl-user` table doesn't hold a `CAR` of its own — and so `symbol-package` correctly reports `:common-lisp`. Inheritance is a lookup fact, not a property of the symbol.

The last line is about the game's own package: `:con` is the nickname for `:cave-of-noise`, and `run-command` should be exported (it's in the API surface from Chapter 28′s project step), so its status is `:external`. If you get `:internal`, the export list is missing it; if `NIL NIL`, the package isn't loaded.

28.2 **Surprising, but it works:** `(use-package :p1)` after `:cl` means `cl-user` now sees both `CL`'s exports and `P1`′s exports. `run` resolves to `p1:run` by inheritance, and evaluating `run` calls `p1:run` (returns `:p1s`). No conflict, because `cl-user` didn't have its own `run` before the `use-package`.

The trap that *would* fail:

```
(defpackage :p1 (:use :cl) (:export :run))
(defun run () :cl-users-own)  ; interns RUN in cl-user
(use-package :p1)              ; conflict! cl-user's RUN vs p1's RUN
```

The conflict fires at `use-package` time, because `run` is now findable two ways in `cl-user`. The resolution restarts offer "keep the local one" (shadowing-import) or "keep the imported one" (unintern the local). In the exercise's original snippet, the order (`p1`′s `run` interned first, `cl-user`'s `run` never interned) means no conflict — but that's a fragile situation where the code's correctness depends on what's been evaluated before.

Surprising: `(defpackage :p2 (:use :cl :p1))` is fine at definition time; the conflict fires at `(defun run () :p2s)`, because `p2` now inherits `run` from `p1` and tries to intern its own `run`. SBCL signals a correctable error: which `run` does `p2` mean? The fix — as the answer above shows — is to explicitly shadow the name:

```
(defpackage :p2
  (:use :cl)
  (:import-from :p1 :other-name)   ; only what's needed
  (:shadow :run))
```

Better still, don't `:use` a package you're going to redefine names from — that's what `:import-from` is for. Only import the specific names you want; the ones you redefine stay local, with no conflict.

Surprising: `(unintern 'temp :p1)` doesn't do what it looks like. `'temp` reads as `cl-user::temp` — the symbol named `TEMP` home in `CL-USER`, not the one interned in `:p1` by `(intern "TEMP" :p1)` above. `unintern` asks: is *this symbol* (the `cl-user` one) present in `:p1`′s table? It isn't — different symbol, same name. So `unintern` does nothing, and the `p1:temp` symbol from the first line stays interned.

Surprising: `p1::temp` reads as "look up `TEMP` in `:p1`, creating a new symbol if absent". The old one is still there (the `unintern` above missed), so `p1::temp` returns it. The symbol is unbound (never used as a variable), so evaluating `p1::temp` as a variable signals `UNBOUND-VARIABLE`.

The fix, if the intent was to actually unintern:

```
(unintern (find-symbol "TEMP" :p1) :p1)
;; or more directly:
(unintern (intern "TEMP" :p1) :p1)   ; two intern calls, same symbol
```

`find-symbol` is safer — it doesn't accidentally intern if the symbol is missing. `(unintern (find-symbol "TEMP" :p1) :p1)` returns non-nil if the unintern succeeded, nil if `find-symbol` returned nil (symbol wasn't there to begin with).

28.3

```
(defun where-does-name-live (name &optional (package *package*))
  "Where NAME is findable in PACKAGE, as a keyword or :absent."
  (multiple-value-bind (sym status)
      (find-symbol (string name) package)
    (declare (ignore sym))
    (or status :absent)))

(defun find-anywhere (name package)
  "The first package, in precedence order, where NAME is findable."
  (let ((status (where-does-name-live name package)))
    (cond ((eq status :external) package)
          ((eq status :internal) package)
          (t
           (some (lambda (used)
                   (when (member (where-does-name-live name used)
                                 '(:external :internal))
                     used))
                 (package-use-list package))))))
```

Check:

```
(where-does-name-live "CAR" :cl-user)     ; => :INHERITED
(where-does-name-live "PLAY" :con)        ; => :EXTERNAL
(where-does-name-live "NONEXISTENT" :cl)  ; => :ABSENT
(find-anywhere "CAR" :cl-user)            ; => #<PACKAGE "COMMON-LISP">
```

The precedence lesson: the reader resolves an unqualified name first in the current package's own table (internal and external alike), then walks the use-list in *order*. Local beats inherited — which is what `shadow` exploits, and what makes definition order in your own package irrelevant to whether your names beat `cl`'s.

The `:INHERITED` status is what `where-does-name-live` gives when `:cl-user` doesn't hold the symbol itself but can find it via a used package. `:EXTERNAL` and `:INTERNAL` are the package's *own* symbols. `:ABSENT` means neither the package nor any of its used packages has it.

The reader's algorithm, stated precisely:

1. Look up the name in the current package's own table. If found, use that symbol (regardless of whether it's exported).
2. Otherwise, walk the use-list, in the order the packages appear in it. The first package with the name (external only — internal names aren't inherited) supplies the symbol.
3. Otherwise, intern a fresh symbol in the current package.

Step 3 is why evaluating `nonexistent` at the REPL creates a new symbol in `cl-user` rather than erroring. The reader's job is to produce *a symbol*, and if it can't find one, it makes one.

28.4

```
(defun package-externals (package)
  "The external symbols of PACKAGE, as a list."
  (let ((names nil))
    (do-external-symbols (s package names)
      (push s names))))

(defun package-diff (p1 p2)
  "Three values: shared externals, only-p1, only-p2."
  (let ((e1 (package-externals p1))
        (e2 (package-externals p2)))
    (values (intersection e1 e2)
            (set-difference e1 e2)
            (set-difference e2 e1))))
```

Check (on SBCL — other implementations differ):

```
(multiple-value-bind (both only-cl only-sb)
    (package-diff :cl :sb-ext)
  (list (length both) (length only-cl) (length only-sb)))
;; => (0 978 42) — typical SBCL
```

The near-zero intersection is the point: implementation packages are *additions*, never redefinitions of `:cl`. `:sb-ext` exports things like `save-lisp-and-die`, `exit`, `*posix-argv*` — extension symbols not present in the standard. `:cl` exports the language. They don't overlap because the implementation isn't allowed to redefine standard symbols; it can only add.

`package-diff` is the audit that proves it for any pair of packages you're about to `:use` together. If the intersection is non-empty, you'll get conflicts at `use-package` time; if it's empty, the two packages compose cleanly. That's why `:use :cl` is safe and `:use` of two foreign libraries is risky.

One caveat the exercise's counts hide: `set-difference` on `e1` and `e2` returns symbols, not names. Two packages can export symbols with the same *name* that are distinct *symbols* — `(set-difference e1 e2)` won't find those as intersections. For a real conflict-audit, compare `(symbol-name s)` across the sets, not `s` itself.

28.5

```
(defmacro with-temp-package (&body body)
  "Evaluate BODY with *PACKAGE* bound to a fresh package."
  (let ((pkg-var (gensym "PKG-"))
        (name-var (gensym "NAME-")))
    `(let* ((,name-var (symbol-name (gensym "TEMP-PKG-")))
            (,pkg-var (make-package ,name-var :use '(:cl)))
            (*package* ,pkg-var))
       (unwind-protect
            (progn ,@body)
         (when (find-package ,name-var)
           (delete-package ,name-var))))))
```

Check:

```
(with-temp-package
  (intern "FRESH")
  (find-symbol "FRESH"))
;; => FRESH, :INTERNAL — created in the temp package

(with-temp-package
  (defun probe () :inside)
  (probe))
;; => :INSIDE, but note where PROBE was interned
```

Two things to notice in that second check:

First, `(defun probe ...)` *was read* before the macro ran — the reader read `probe` as a symbol in whatever package was current *at the call site*, which is `cl-user`. The `defun` form evaluates (installing the function), but the symbol `probe` was already interned in `cl-user`. The `*package*` binding inside the temp package doesn't change where already- read symbols are homed.

Second, `(probe)` inside the body is a symbol reference `probe` — read in `cl-user` — that the reader resolved before the temp package existed. So it calls the `cl-user::probe` function.

The macro's `*package*` binding affects what *runtime interning* does, not what the reader does during source reading. That matters for `(intern "X")` and `(read ...)` calls inside the body, not for `defun`/`defparameter`/etc., whose symbols were read as tokens before the macro ran.

What the exercise assumes about `delete-package`'s requirements: `delete-package` requires no other package to *use* the package. Since the temp package is freshly created with no consumers, this holds — the deletion succeeds. If some body form did `(use-package temp-name ...)` from another package (hard to arrange in practice, since the temp package has no exports and no code could reference it by name — `find-package` on the gensym's name string works, but the name is computed at macro-expansion time and not accessible to the body), deletion would fail with a condition, and the `when (find-package ...)` guard wouldn't help.

The exercise's honest limit: the macro is a demonstration of `*package*` binding and `delete-package`, not a general tool for package isolation. A real "temp package" macro in production would need to also handle `use-package` by others, and would need to expose the package name for testing if anything outside wanted to reach in. For a session-local experiment, the version above is enough.

28.6 The split, at file level:

```
;; data-package.lisp
(defpackage :cave-of-noise.data
  (:use :cl)
  (:export
   ;; state
   :*rooms* :*current-room* :*item-locations*
   :*thing-registry* :*history*
   ;; data accessors
```

```
    :find-room :lookup-thing :register-thing
    :room-name :room-description :room-exits :room-locked
    :items-here :item-location :set-location
    ;; persistence
    :save-game :load-game :*save-file* :*save-format*))
```

```
;; main-package.lisp
(defpackage :cave-of-noise
  (:use :cl)
  (:import-from :cave-of-noise.data
                :*current-room*
                :*item-locations*
                :*rooms*
                :*history*
                :find-room
                :items-here
                :item-location
                :set-location
                :lookup-thing
                :save-game :load-game)
  (:export :play :run-command :look :take :drop :inventory
           :examine :help))
```

The names the main package actually needed — from counting actual uses in the game's source:

- `*current-room*` — read by `look`, `go-to`, `take`, `drop`, `describe-room-text`, `doctor`.
- `*item-locations*` — read/written by `take`, `drop`, `inventory`, `loot`, `snapshot`, `undo`, `doctor`.
- `*rooms*` — read by `find-room`, `describe-all`, `locked-rooms`, `doctor`.
- `*history*` — read by `snapshot` and `undo`.
- `find-room` — used by `go-to`, `doctor`, `play-to-string`.
- `items-here` — used by `look`, `inventory`, `loot`, `describe-room-text`, `describe-all`.
- `item-location` — used by `take`, `drop`.
- `set-location` — used by `take`, `drop`.
- `lookup-thing` — used by `describe-thing`, `examine`, `inventory-score`.
- `save-game`/`load-game` — used by `cmd-save` and `cmd-load`.

That's ten imports, for a game whose total API surface is about nine exports. The main package leans almost entirely on the data package — nearly every command reads or writes state.

What the import list tells you: *the split is premature for this game*. A clean split would put most commands in the data package too, leaving the main package almost empty. Or the split would be by a different axis — UI (command loop, message formatting) vs model (state, accessors) — with `play`, `transcript`, `help-command` in the UI package and everything else in the model package.

The honest conclusion the exercise points at: packages are for *coupling at boundaries*, not for *tidiness*. If two halves of a program pass state back and forth constantly, splitting them into separate packages adds `:import-from` lines without adding isolation. The game's natural

seam is between the session loop (`play`, `transcript`) and the model (everything else); that's a smaller, cleaner split than the one the exercise sketched. That's the audit the exercise wanted.

D.AD. Chapter 29

29.1

```
(list (uiop:run-program '("echo" "hello world") :output :string)
      (uiop:run-program '("echo" "a;b|c") :output :string))
```

Expected: `("hello world\n" "a;b|c\n")` — with the trailing newline that `echo` emitted, and with the semicolon and pipe arriving *verbatim*. The second line is the security argument in one example: those characters are shell metacharacters, and they came through untouched because `uiop:run-program` handed the list to `execve` directly. No shell parsed them, no injection was possible.

```
(list (cl-json:decode-json-from-string "[1, 2, [3]]")
      (cl-json:decode-json-from-string "{\"a\": null, \"b\": 0}"))
```

Expected: `((1 2 (3)) ((:A) (:B . 0)))`.

The first result is straightforward — a JSON array decodes to a Lisp list, nested arrays to nested lists.

The second is the one to look at. `(:B . 0)` is a cons whose car is the keyword `:B` and whose cdr is the integer `0` — the normal alist pair. `(:A)` is a cons whose car is `:A` and whose cdr is `nil` — a JSON `null` decoded as a pair with nil value. It prints as `(:A)` (no dot) because nil is the empty list, but it's still a cons: `(consp (first result))` is `t`.

The ambiguity is already here, in the output shape: `(cdr (assoc :a result))` is `nil` — indistinguishable from a missing key. The chapter's warning about `null`-vs-absence is visible in the raw decode. Callers that care about the distinction need to test `(assoc :a result)`, not~`(cdr (assoc :a result))`.

29.2 **Wrong:** the `format nil` string goes through a shell — injection (a `dir` of `"; rm -rf ~"` executes), quoting bugs, platform differences. Fix: the list form:

```
(uiop:run-program (list "ls" dir) :output :string)
```

Surprising: `(assoc "key" decoded)` returns `nil` — the decoded alist's keys are keywords `:key`, and `assoc` defaults to `eql`, which never matches a string against a keyword. Fix: `(assoc :key decoded)`, or pass :test `#'equal` if the keys were strings by design (some JSON libraries decode with string keys, and this test separates the cases).

Wrong: `gzip` on a missing file exits non-zero, and `run-program` signals a `subprocess-error` — uncaught, it interrupts the caller. Fix depends on intent:

- `:ignore-error-status t` — return the exit code as the third value; caller checks.
- `handler-case` around the call — translate to a condition the caller understands.

The choice is Chapter 24′s flags-vs-conditions rule: if the caller routinely branches on "did the command succeed?", take the flag. If the command should always succeed and a failure is a bug, let the condition fly.

Wrong: the decode delivers `:room` as a string `"kitchen"`, and `find-room` needs a symbol. `(find-room "kitchen")` — find-room calls `(find name #strong[rooms] :key #'room-name)`, which uses `eql` by default, and no symbol is `eql` to a string. It returns `nil`, the state gets set to `nil`, and the game is quietly broken. Fix:

```
(defun json->state (json)
  (let ((room-str (cdr (assoc :room json))))
    (unless (stringp room-str)
      (error "Bad import: no :room field"))
    (setf *current-room*
          (find-room (intern (string-upcase room-str))))))
```

The intern `(string-upcase ...)` pair is the boundary's string-to-symbol conversion. It works because the room names are interned in the current package by the game's own source, and `intern` finds the existing symbol rather than creating a new one. If the game moved its packages (Chapter 28′s split), this would need the explicit package argument — `(intern (string-upcase room-str) :cave-of-noise)` — which is exactly the "the seam knows the package" discipline the chapter's project step keeps.

29.3

```
(defun csv->json (path)
  "CSV with headers to a JSON array of objects."
  (let* ((rows (read-csv path))
         (headers (mapcar (lambda (h) (intern (string-upcase h)
                                              :keyword))
                          (first rows)))
         (data (rest rows)))
    (cl-json:encode-json-to-string
     (mapcar (lambda (row)
               (mapcar (lambda (h val)
                         (cons h (maybe-number val)))
                       headers row))
             data))))

(defun maybe-number (s)
  "S as an integer, if it looks like one; otherwise the string."
  (or (ignore-errors (parse-integer s)) s))
```

Check against the file from the chapter's project step:

```
(csv->json "/tmp/visits.csv")
;; => "[{\"ROOM\":\"kitchen\",\"VISITS\":3},...]"
```

— a JSON array with one object per data row, keys matching the headers, numeric fields as numbers.

The comment the exercise asks for:

The naive number test is `ignore-errors parse-integer` — it accepts "42", "−7", and "+3" (parse-integer handles the sign); it rejects "3.5" (which JSON would call a number, so this version keeps it a string — a deliberate under-parsing). Three things it misses that a real schema would care about:

1. It interns a keyword from header text. A hostile or mistaken header row floods the keyword table (Chapter 23's warning). Production code should validate against a known header set, not trust the file.
2. `parse-integer=` on " 3 " returns 3 (it skips leading whitespace), but on "3.0" returns 3 with a second value of 2 (where it stopped). The naive `or=` keeps only the integer, silently dropping the ".0" — a value mismatch an upstream consumer might not notice.
3. Empty fields parse as nil from parse-integer's error, so `maybe-number ""` returns "", not nil — inconsistent with how JSON treats missing fields. The seam's schema should decide: is "" the empty string, a null, or absent?

When cl-csv becomes necessary: as soon as any field contains a quoted comma, an embedded newline, or a double-quote. `uiop:split-string` on commas silently breaks those cases (producing extra fields), and the breakage is invisible until downstream arithmetic fails. For data you produce, the naive reader is fine. For data you consume, always cl-csv.

29.4

```
(cffi:defcfun ("abs" c-abs) :int (n :int))
(cffi:defcfun ("getenv" c-getenv) :string (name :string))

(c-abs -7)                  ; => 7
(c-getenv "HOME")           ; => "/home/you", or NIL below
(cffi:null-pointer-p
 (cffi:foreign-funcall "getenv" :string "NOPE" :pointer))
;; => T — signals the missing variable
```

The `:string` return type on `getenv` is the trap. When the variable is absent, `getenv` returns a C `NULL` pointer, and converting `NULL` to a Lisp string is implementation- dependent — SBCL gives `nil`, most implementations give `nil`, but the standard doesn't say. The portable version declares `:pointer` return, checks `null-pointer-p`, and converts only when non-NULL:

```
(cffi:defcfun ("getenv" c-getenv-ptr) :pointer (name :string))

(let ((p (c-getenv-ptr "NOPE")))
  (if (cffi:null-pointer-p p)
      nil
      (cffi:foreign-string-to-lisp p)))
;; => NIL, reliably
```

Why `abs` needs no memory management: it takes and returns integers by `value`. The number `-7` is copied from Lisp's integer representation to a C `int` (in a CPU register, on typical ABIs), the function computes `7`, and the result is copied back. Nothing is allocated across the boundary that anyone needs to free.

A hypothetical `malloc` binding is the opposite:

```
(cffi:defcfun ("malloc" c-malloc) :pointer (size :size))
(cffi:defcfun ("free" c-free) :void (ptr :pointer))
```

`malloc` returns a pointer to memory *C owns until freed*. The Lisp side receives the pointer, uses it (probably by copying through `cffi:mem-ref` or similar), and then must call `c-free` to

release it. `with-foreign-object` is the automatic version of this discipline (allocate, use, free on exit); `foreign-alloc` / `foreign-free` is the manual version (allocate, use, free explicitly). Every C resource follows the same pattern: Lisp-side lifetime management, because C doesn't know Lisp's GC exists and Lisp doesn't know C's allocations exist. Getting this wrong leaks memory through a boundary the GC can't see.

The exercise's deeper lesson: *the by-value / by-pointer distinction determines the interop discipline*. `abs` (by value) is a one-line call. `malloc` (by pointer) requires lifetime management on both sides. `getenv` (returning a pointer to *C-owned, read-only* memory) is the awkward middle case — no free needed, but the pointer is only valid until the environment changes, and copying to a Lisp string is the right thing to do at the seam.

29.5

```
(defun run-and-parse (cmd)
  "Run CMD; decode each output line as a JSON object."
  (let ((out (uiop:run-program cmd :output :string)))
    (loop for line in (uiop:split-string out :separator
        (string #\Newline))
      for decoded = (ignore-errors
        (cl-json:decode-json-from-string line))
      when decoded collect decoded)))
```

Simulate a producer:

```
(with-open-file (o "/tmp/events.jsonl"
    :direction :output
    :if-exists :supersede)
  (format o "{"event":"look"}~%")
  (format o "{"event":"take"}~%"))

(run-and-parse '("cat" "/tmp/events.jsonl"))
;; => ((:EVENT . "look")) and ((:EVENT . "take"))
```

The `ignore-errors` skips any line that fails to decode as JSON — a defensive choice, since a producer's output might include headers, blank lines, or log messages mixed in.

What line-at-a-time buys: constant memory per line (the giant array case allocates all decoded objects at once before returning), incremental processing (the pipeline from Chapter 30 can consume while the producer is still writing), and resilience (one bad line doesn't lose the rest).

When it lies: a "JSON object per line" contract (JSONL) is exactly what the format promises, but JSON itself permits objects to span lines. A producer that emits one pretty-printed object over several lines breaks this design — the parser sees partial objects, fails on each, and returns nothing. The contract is JSONL's, not JSON's, and the chapter's warning is about trusting JSONL output from a producer that doesn't actually follow JSONL.

A second caveat: `uiop:run-program` with :output `:string` collects the *entire output before this code sees it — so the "constant memory" property is only per-line during processing*, not during the program's execution. For truly streaming use, `run-program` needs the streaming-output form (the one whose exact keyword has drifted across uiop versions — the chapter's pointer to `M-.` on `uiop:run-program` is the actual guidance here).

29.6

```
(define-condition bad-import (error)
  ((field :initarg :field :reader bad-import-field))
  (:report (lambda (c s)
      (format s "Bad import: invalid or missing ~A"
        (bad-import-field c)))))

(defun import-state-json (json-string)
  "Restore state from a JSON exchange. Signals BAD-IMPORT."
  (let* ((alist (cl-json:decode-json-from-string json-string))
      (room-name (cdr (assoc :room alist)))
      (items (cdr (assoc :items alist)))
      (carrying (cdr (assoc :carrying alist))))
    ;; validation phase — no state touched
    (unless (and room-name (stringp room-name))
      (error 'bad-import :field :room))
    (let ((room (find-room (intern (string-upcase room-name)))))
      (unless room
        (error 'bad-import :field :room))
      (dolist (name items)
        (unless (and (stringp name)
            (lookup-thing (intern (string-upcase name))))
          (error 'bad-import :field :items))
        (dolist (name carrying)
          (unless (and (stringp name)
              (lookup-thing (intern (string-upcase name))))
            (error 'bad-import :field :carrying))
          ;; mutation phase — validation complete
          (setf current-room room)
          (dolist (name items)
            (setf item-locations
              (set-location item-locations
                (intern (string-upcase name))
                (room-name room))))
          (dolist (name carrying)
            (setf item-locations
              (set-location item-locations
                (intern (string-upcase name))
                'player)))
          t)))
```

The comment the exercise asks for:

Validate-then-apply is the only safe order because the game's state changes are not transactional. A partially-applied import — room set, items partially moved, then a validation error — leaves the game in a state no handler-case can recover from cleanly. The handler can catch the error and report "import failed", but the game is now half-imported. Validate-then- apply makes the mutation phase atomic in practice: every error fires before the first setf, so catching the condition leaves state exactly as it was.

This is a form of the general principle "make invalid states unrepresentable" — here, "make half-applied states unreachable". True atomicity (rollback on failure) would need a transaction log or shadow copy; for single-threaded REPL use, ordering suffices.

The bad-import condition is not a return value; it's a condition. Chapter 24's flags-vs-conditions rule applies: a corrupt import is a bug-shaped event the caller isn't expected to branch on routinely. Contrast with save-game's two-value return — "no save file" is a normal outcome (first run), so it's a flag. "Save file has the wrong shape" is a bug, so it's a condition.

The exercise asks about "first invalid field" — the condition carries it as a slot (bad-import-field), not as a return value. The caller reads (bad-import-field c) from the caught condition. That's the same "carry the facts in slots" pattern Chapter 15's `room-not-found` showed. A return value couldn't do this — by the time the caller sees it, the error has been abstracted to "failed", and the which `field` detail is lost unless it's in the condition object.

Check the round trip:

```
(setf current-room (find-room 'kitchen))
(setf item-locations '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))
(register-thing 'key (make-instance 'thing :name 'key))
(register-thing 'lamp (make-instance 'thing :name 'lamp))
(register-thing 'letter (make-instance 'thing :name 'letter))

(let ((json (game-state-json)))
  (setf current-room (find-room 'bedroom))
  (setf item-locations nil)
  (import-state-json json)
  (list (room-name current-room)
    (item-location 'key)))
```

Expected: `(KITCHEN KITCHEN)` — the room and the item location back where the JSON said they were.

And a validation failure:

```
(handler-case (import-state-json "{"room": "nowhere"}")
  (bad-import (c) (format nil "caught: ~A" c)))
;; => "caught: Bad import: invalid or missing ROOM"
```

The room was "nowhere", `find-room` returned nil, the condition fired *before* any `setf` on the game's state. The game is untouched.

D.AE. Chapter 30

30.1

```
(defparameter *g* (make-list-gen '(a b)))

(multiple-value-list (funcall *g*))   ; => (A T)
(multiple-value-list (funcall *g*))   ; => (B T)
(multiple-value-list (funcall *g*))   ; => (NIL NIL) — exhausted
(multiple-value-list (funcall *g*))   ; => (NIL NIL) — stays
```

The last two lines are the protocol's stickiness: a generator that has produced all its items keeps reporting exhausted. That matters for the consumers — `for-each` exits the loop on

the first `(nil nil)`, and calling it again returns immediately, which is what you want if a consumer is called twice.

```
(collect-gen (take-gen 3 (map-gen #'* (make-counter-gen 1))))
;; => (2 3 4)
```

Trace it. `make-counter-gen 1` produces 2, 3, 4, … (its `incf` runs before the first return), and `map-gen #'*` applies `#'*` to each value. With *one* argument, `*` returns that argument, so the mapping is the identity:

```
(list (funcall #'* 2) (funcall #'* 5))   ; => (2 5)
```

So the result is the first three counter values, `(2 3 4)`, not the `(2 6 12)` a quick reading suggests. If the intent was "double each counter value", the function argument has to be `(lambda (n) (* 2 n))`; if it was "square each", then `(lambda (n) (* n n))`. That is the trap the exercise is built on: `#'*` alone is not "multiply by something".

```
(collect-gen (filter-gen (lambda (x) (> x 3))
    (take-gen 2 (make-counter-gen 0))))
;; => NIL
```

Surprising until you read the composition inside-out: take-gen `2` yields 1 and 2 (the counter starts at 0, and `incf` makes the first value 1), neither exceeds 3, so the filter yields nothing and reports exhausted. Lazy pipelines are read inside-out — source first — the same discipline as Chapter 10's nested sequence calls, with the order of execution added: left to right per item, not stage by stage.

30.2 Fix: two generators.

```
(list (collect-gen (make-list-gen '(1 2 3 4)))
  (collect-gen (make-list-gen '(1 2 3 4))))
;; => ((1 2 3 4) (1 2 3 4))
```

If the intent really was "two independent views of the *same* data", each consumer gets its own generator. If the intent was "share one traversal between two consumers", that's a different design — a `tee` that buffers — and it's a real design task, not a freebie.

Surprising: `lazy-all-p` *consumes* the generator as it checks — even when it returns `t`, the source is drained, and the caller's "keep checking afterwards" expectation fails. Every later read on that generator reports exhausted.

The fix is context-dependent. If the caller needs to check other things on the same data, pass a list and let the function rebuild a generator internally:

```
(defun lazy-all-p (pred items)
  (let ((g (if (functionp items) items (make-list-gen items))))
    (loop
      (multiple-value-bind (item ok) (funcall g)
        (if (not ok) (return t)
          (unless (funcall pred item) (return nil)))))))
```

If the caller really does want to share one traversal, the "peek without consuming" operation has to be part of the generator protocol — a one-item buffer inside the generator — and that's a protocol extension, not a free feature.

The general statement the exercise teaches: any consumer that walks a generator consumes it. There's no `peek` in the two-value protocol; if you need one, either the generator provides it or the caller keeps a separate source.

```
(let ((g (make-line-gen "/tmp/big.txt")))
  (unwind-protect
    (multiple-value-list (funcall g :next))
    (funcall g :close)))
```

Or, simpler, use `with-open-file` around the whole consumption and don't use a generator at all — the generator pattern is for cases where the consumption spans multiple functions, and the resource discipline should match.

30.3

```
(defun gen-take-while (pred gen)
  "Items of GEN while PRED holds; exhausted at first failure."
  (lambda ()
    (multiple-value-bind (item ok)
      (funcall gen)
      (if (and ok (funcall pred item))
        (values item t)
        (values nil nil)))))
```

```
(defun gen-append (gen1 gen2)
  "All of GEN1, then all of GEN2."
  (let ((current gen1))
    (lambda ()
      (multiple-value-bind (item ok)
        (funcall current)
        (if ok
          (values item t)
          (progn (setf current gen2)
            (funcall current)))))))
```

Check:

```
(collect-gen (gen-take-while #'plusp
    (make-list-gen '(1 3 -1 2))))
;; => (1 3) — stops at -1, never yields 2

(collect-gen (gen-append (make-list-gen '(a b))
    (make-list-gen '(c d))))
;; => (A B C D)
```

The `gen-take-while` is the lazy cousin of Chapter 10's `take-while` — the same "stop at the first failure" contract, delivered as a generator instead of a list. The difference is that the source could be infinite; a take-while over an infinite source produces a finite prefix of it.

30.4

```
(defun zip-gens (g1 g2)
  "Two values per step: next item of each, plus ok flag."
  (lambda ()
    (multiple-value-bind (i1 ok1) (funcall g1)
      (if (not ok1)
        (values nil nil nil)
        (multiple-value-bind (i2 ok2) (funcall g2)
          (if (not ok2)
            (values nil nil nil)
            (values i1 i2 t)))))))
```

The protocol stretch: one extra value. `zip-gens` yields three — the item from `g1`, the item from `g2`, and the ok flag. When either source is exhausted, it yields `(nil nil nil)` and both consumer-side pipelines terminate.

The critical sequencing detail: the `g2` call happens *after* `g1` has produced a value, and only then. If `g1` is exhausted first, `g2` is *not* called — no item is consumed from `g2` that won't be delivered. That's the correct behaviour, and the mirror case (checking `ok1` before calling `g2`) is what makes it correct. Get the order wrong and you lose an item from `g2` every time `g1` runs out early.

Use it to generate the multiplication table's diagonal — the squares:

```
(defparameter squares
  (zip-gens (make-counter-gen 0) (make-counter-gen 0)))

(loop repeat 5
  collect (multiple-value-bind (i j ok) (funcall squares)
    (if ok (* i j) nil)))
;; => (1 4 9 16 25)
```

Why the first value is 1, not 0: make-counter-gen `0` starts its `incf` at 0, so the first call returns 1. The counter generator is "successive integers starting with one more than start" — its name is a little off, but its behaviour is consistent.

The result `(1 4 9 16 25)` is the diagonal of the multiplication table: 1×1, 2×2, 3×3, 4×4, 5×5. The zip is the diagonal; the multiplication is the "square" that falls out of it. That's the exercise's quiet joke — the multiplication table's diagonal *is* the squares.

30.5

```
(defun make-fib-gen ()
  "Infinite Fibonacci generator: 1 1 2 3 5 8 #| ... |#"
  (let ((a 1) (b 1))
    (lambda ()
      (let ((result a))
        (setf a b
          b (+ result b))
        (values result t)))))
```

Two bindings, one shift per call. Trace it:

call 1: result = 1, a = 1 (old b), b = 1+1 = 2. returns 1.

call 2: result = 1 (old a), a = 2 (old b), b = 1+2 = 3. returns 1.

call 3: result = 2 (old a), a = 3 (old b), b = 2+3 = 5. returns 2.

call 4: result = 3, a = 5, b = 3+5 = 8. returns 3.

The first two results are 1 and 1 — the exercise's expected sequence.

The sum-below consumer needs *early stop* — `for-each` runs to exhaustion, which never comes for an infinite source. The stopping belongs to the consumer side of the protocol:

```
(defun fib-sum-below (limit)
  (let ((g (make-fib-gen))
        (total 0))
    (loop
      (multiple-value-bind (n ok) (funcall g)
        (if (or (not ok) (>= n limit))
          (return total)
          (incf total n))))))

(fib-sum-below 100)
;; => 232
```

The expected value: 1 + 1 + 2 + 3 + 5 + 8 + 13 + 21 + 34 + 55

89 = 232. (89 + 55 = 144, which is > 100, so the sum stops before adding it. The check < n `limit` returns before `incf`, so 144 never enters the total.)

The answer to the exercise's question — "which side of the protocol does the stopping belong to?" — is *the consumer*. The generator is passive; it produces only when called, and calling is the consumer's act. An infinite source is safe *because* the consumer can stop calling. If the generator tried to be "the thing that decides when to stop", it would need to know things it can't know (what limit is the current consumer using?), and the two-value protocol is designed to keep that decision on the consumer side.

30.6

```
(defun make-room-viz-gen (&optional (start current-room))
  "Depth-first generator of reachable rooms, no repeats."
  (let ((visited (list (room-name start)))
        (queue (list start)))
    (lambda ()
      (if (null queue)
        (values nil nil)
        (let ((room (pop queue)))
          (dolist (dest (room-exits room))
            (unless (member dest visited)
              (push dest visited)
              (let ((r (find-room dest)))
                (when r (push r queue)))))
          (values room t))))))
```

For BFS, the discoveries need to be appended, not pushed:

```
(defun make-room-viz-gen-bfs (&optional (start current-room))
  "Breadth-first generator of reachable rooms, no repeats."
  (let ((visited (list (room-name start)))
```

```
      (queue (list start)))
    (lambda ()
      (if (null queue)
        (values nil nil)
        (let ((room (pop queue)))
          (dolist (dest (room-exits room))
            (unless (member dest visited)
              (push dest visited)
              (let ((r (find-room dest)))
                (when r (setf queue (append queue (list r)))))))
          (values room t))))))
```

Use it:

```
(defun print-map (&optional (limit 5))
  "Print a partial map of reachable rooms."
  (for-each (lambda (room)
      (format t "~&~(~A~) — exits: ~{~(~A~)~^, ~}~%"
        (room-name room)
        (room-exits room)))
    (take-gen limit (make-room-viz-gen))))
```

Check (with the game's five rooms, kitchen as current):

```
(setf current-room (find-room 'kitchen))
(print-map 5)
```

Expected (DFS order, which may vary depending on exit list order):

```
kitchen — exits: hallway, garden, cellar
cellar — exits: kitchen
garden — exits: kitchen
hallway — exits: bedroom, kitchen
bedroom — exits: hallway
```

The comment the exercise asks for:

Over the recursive traversal of Chapter 8 (items-in, walking a flat alist), the generator version buys *stoppability* — a map of five rooms costs exactly five room visits, and an abort mid-walk wastes nothing. Chapter 8′s recursive walk had to complete before its result could be used; the generator produces incrementally. For a small map that doesn't matter; for a real exploration of a graph where each room visit might do work, it matters.

What closure-private visited makes hard: the caller can't ask "how many rooms did you reach?" or "did you reach the cellar?" — the walk's knowledge dies with the generator's exhaustion, or with the generator itself if abandoned. The fix is the familiar one: the constructor returns *two* closures (Chapter 25′s multiple doors on one state) — the stepper, and a :visited reporter. Or the state graduates to a defclass. When callers start wanting to interrogate the walk rather than only drain it, the generator has outgrown one lambda.

D.AF. Chapter 31

31.1 The answers vary by library; the shape of a full-credit answer, using split-sequence as the worked specimen:

```
;; Files: 2 source files (package.lisp, split-sequence.lisp).
;; Order: the .asd's :components list — package first, then
;; split-sequence, which is Chapter 25's rule expressed as
;; build order: the package must exist before code that
;; uses in-package.
;; Promise: "Given a delimiter and a sequence, split into a
;; list of subsequences, with :start/:end/:test/:key
;; options, plus -if/-if-not predicate variants."
```

The three sentences are the API contract, written from `package.lisp` (the export list) and `README.md` (the example) alone. If three sentences won't cover the export list, the surface is bigger than the documentation — the checklist's caution sign.

The `:components` order is worth a second look. It isn't "alphabetical" or "by size"; it's "what must exist before what". That's the file structure of a Lisp system, and it's readable at a glance once you know to look for it.

31.2

31.3

```
(defun system-report (name)
  "An audit alist for an ASDF system."
  (let* ((system (asdf:find-system name))
         (dir (asdf:system-source-directory system)))
    (list (cons :directory dir)
      (cons :files
        (mapcar (lambda (component)
            (if (typep component 'asdf:source-file)
              (pathname-name
                (asdf:component-pathname component))
              component))
          (asdf:component-children system)))
      (cons :depends-on (asdf:system-depends-on system))
      (cons :license (ignore-errors
          (asdf:system-license system))))))
```

Check on a real system:

```
(system-report :split-sequence)
;; => ((:DIRECTORY #P"/home/you/quicklisp/.../split-sequence/")
;; (:FILES ("package" "split-sequence"))
;; (:DEPENDS-ON NIL)
;; (:LICENSE ...))
```

The distinctions the question probes:

`asdf:find-system` searches, loads the `.asd` file if needed, and is the step that registers. So `system-source-directory` on an unregistered system errors, but `find-system` registers then returns it.

`:license` is present only if the `defsystem` declared it — an audit point in itself: a system with no `:license` line sends you to the LICENSE file, checklist step 1.

The `component-children` accessor is ASDF's way of getting the components of a system (newer than the older `components` name); `M-.` on it opens the ASDF source, which is another application of this chapter's reading skill.

31.4 The rating rubric, applied to split-sequence's `split-sequence` as the worked line:

```
;; split-sequence: example in README — yes; argument roles
;; named in docstring — yes ("delimiter", "seq"); failure
;; mode stated — partially (doesn't say what happens with
;; an empty seq, or how adjacent delimiters behave: read
;; the source, checklist step 7).
```

And the self-comparison `run-command` deserves:

```
;; "Execute CMD (a list whose first element names a command
;; and whose rest are its arguments). Returns two values:
;; the reply string, and a boolean, T when the command was
;; known. Unknown commands produce a reply and NIL."
```

— two values stated (Chapter 24′s contract), argument shape stated, failure mode stated. If your actual docstring lacks any of the three, that's the exercise's finding.

The meta-lesson: the docstring a function should have is a question the docstring itself can't answer — you discover it by using the function, finding the places where its behaviour surprises you, and adding sentences to the docstring for each. Every `run-command` caller who's had to check "did the command succeed?" is a sentence the docstring needs.

31.5 The shape of the exercise, worked with a plausible unknown: a library's `(with-collected (items) ...)`.

```
(macroexpand-1
'(with-collected (out) (push 1 out) (push 2 out)))
;; => (LET ((OUT (MAKE-ARRAY 0 :ADJUSTABLE T :FILL-POINTER 0)))
;; (PUSH 1 OUT) (PUSH 2 OUT) OUT)
```

The expansion is the whole story: a `let` binding `out` to an adjustable vector with a fill pointer, the body spliced into the `let`, and the vector returned as the result. The "collected" value is the vector, not a list. That's the interesting choice — the macro's contract, if the docstring doesn't say, is revealed by the expansion.

Three sentences: "Binds `out` to an adjustable vector. Body forms run with `out` in scope. Returns the vector." Whether the caller wanted a list and got a vector is a design choice to check.

```
(macroexpand-1
  '(with-collected (out) (vector-push-extend 1 out)))
;; => (LET ((OUT (MAKE-ARRAY 0 :ADJUSTABLE T :FILL-POINTER 0)))
;; (VECTOR-PUSH-EXTEND 1 OUT) OUT)
```

If the library really does override `push`, that's a warning sign: it's nonstandard, and any code reading `push` in that library needs to know the redefinition is in effect.

The gensym question: no `gensym` needed, because the binding name `out` is provided by the `macro` (the caller names it in the macro call (with-collected (out) …)). The caller chose the name; the macro binds it in the caller's scope. Capture risk only exists where caller code and macro temporaries share a namespace — here, `out` is exactly the name the caller asked for.

31.6

```
(defun undocumented-exports (&optional (package :cave-of-noise))
  "Two values: exported non-function symbols with no docs,
and exported function names with no function docstring."
  (let ((no-docs nil)
        (fns-missing-docs nil))
    (do-external-symbols (s package)
      (cond ((fboundp s)
             (unless (documentation s 'function)
               (push s fns-missing-docs)))
            ((boundp s)
             (unless (documentation s 'variable)
               (push s fns-missing-docs)))
            (t
             (push s no-docs))))
    (values (nreverse no-docs)
      (nreverse fns-missing-docs))))
```

Check:

```
(undocumented-exports :cave-of-noise)
;; => two lists of symbols — rename or document each
```

The audit distinguishes two failures:

A third, rarer failure the version above doesn't catch: exported, documented, but the docstring is empty (`==""`) rather than nil. `(documentation s 'function)` returns "", which is truthy, so the symbol passes. The `string` check:

```
(let ((s (find-symbol "LOOK" :cave-of-noise)))
  (unless (and (stringp (documentation s 'function))
               (plusp (length (documentation s 'function))))
    #| ... |#))
```

The chapter closes the loop it opened: the query tools from Chapter 28 (`do-external-symbols`), the documentation accessor from Chapter 22, the package contract from Chapter 16 — the audit of a stranger's system and of your own are the same two queries, run from different chairs. Eight months from now, the stranger will be you.

D.AG. Chapter 32

32.1

```
;; The smallest possible executable:
(defun hello-main ()
  (format t "hello from a saved image~%")
```

```
  (sb-ext:exit :code 0))

(sb-ext:save-lisp-and-die "hello"
                          :executable t
                          :toplevel #'hello-main)
```

From a shell: `./hello` prints the line and exits 0. Its size on a typical SBCL — around 45 MB uncompressed, around 25 MB with `:compression t`. That's the floor: the runtime dominates, and your code is a rounding error.

Now the payload check:

```
(defparameter *payload* (make-array 1000000 :initial-element 0))

(defun hello-main ()
  (format t "hello from a saved image (payload: ~D)~%"
          (length *payload*))
  (sb-ext:exit :code 0))

(sb-ext:save-lisp-and-die "hello-with-payload"
                          :executable t
                          :toplevel #'hello-main)
```

The size barely changes: a million 64-bit integers is 8 MB in memory, but a zero-initialised array compresses to a few hundred KB in the image, and SBCL's `save-lisp-and-die` already compresses constant structures by default. The exact difference varies — check yours with `ls -l` — but the lesson holds: the runtime is the cost. Your data matters only when it's very large or very incompressible.

The meta-point: an SBCL executable is *not* a thin wrapper around your code. It's your code plus a complete Lisp system plus Quicklisp if you loaded it. That's why Lisp executables are 40 MB when a Go equivalent is 5 MB. The trade is what you get: an interactive image that's still a live REPL if you save it as a core rather than an executable, a program you can inspect, redefine, and debug while it runs.

32.2

```
(defun main ()
  "Entry point with argument handling."
  (let ((args (rest sb-ext:*posix-argv*)))
    (cond
      ((member "--help" args :test #'string=)
       (format t "Usage: game [--help] [--version] [--room NAME]~%")
       (sb-ext:exit :code 0))
      ((member "--version" args :test #'string=)
       (format t "The Cave of Noise, version ~A~%" *version*)
       (sb-ext:exit :code 0))
      ((member "--room" args :test #'string=)
       (let* ((room-str (second (member "--room" args
                                        :test #'string=)))
              (room-sym (and room-str
                             (intern (string-upcase room-str)))))
         (if (find-room room-sym)
```

```
            (setf *current-room* (find-room room-sym))
            (progn
              (format *error-output*
                      "No such room: ~A~%" room-str)
              (sb-ext:exit :code 2)))))
      ((member "--save-file" args :test #'string=)
       (let ((path (second (member "--save-file" args
                                   :test #'string=))))
         (when path (setf *save-file* path)))))
    ;; fall through to the game:
    (run-game)))
```

Where `run-game` is the play/save/exit wrapper:

```
(defun run-game ()
  "Play the game, save on exit, return the exit code."
  (handler-case
      (progn
        (multiple-value-bind (state loaded-p)
            (load-game)
          (if loaded-p
              (restore-state state)
              (progn
                (setf *current-room* (find-room 'kitchen))
                (setf *item-locations* nil)
                (setf *history* (make-history)))))
        (play)
        (save-game)
        (sb-ext:exit :code 0))
    (error (c)
      (format *error-output* "cave: ~A~%" c)
      (sb-ext:exit :code 1))))
```

Test all three from a shell:

```
./game --help
# => Usage: game [--help] [--version] [--room NAME]
# exit code 0

./game --version
# => The Cave of Noise, version 1.0
# exit code 0

./game --room cellar
# starts in the cellar instead of the kitchen
# exit code 0 (on quit)
```

Now the question the exercise asks: what does `--room attic` do when `attic` doesn't exist?

The version above answers: it checks `(find-room room-sym)` and, when that's nil, prints an error to stderr and exits with code 2. That's a deliberate choice — a caller who asked for a specific room and didn't get it should know the command failed, and exit 2 distinguishes

"bad argument" from exit 1's "unexpected failure". Many Unix tools use 2 for usage errors; it's a convention worth following.

Which chapter's tool handles it: Chapter 24's `loaded-p` style flag pattern for the check, and Chapter 19's conditions for anything more elaborate. The `(find-room room-sym)` here is simple enough — one lookup, one branch — that a flag is the right weight. If the game grew a dozen room-related flags and room lookups could fail for several reasons (missing, locked, destroyed), a `bad-room-argument` condition would be cleaner.

32.3 **Wrong:** `:executable t` is missing. Without it, `save-lisp-and-die` writes a core `file` — loadable with sbcl –core `game`, but not a standalone executable. The user has to have SBCL installed and know the flag. Fix:

```
(sb-ext:save-lisp-and-die "game"
                          :executable t
                          :toplevel #'main)
```

Wrong: the `main` prints usage and returns, without `sb-ext:exit`. After `main` returns, SBCL falls through into its default REPL. The user sees the usage message, then a bare * prompt from a Lisp they didn't know was running, waiting for input that will never come. Fix:

```
(defun main ()
  "Entry point; --help prints usage and exits."
  (let ((args (rest sb-ext:*posix-argv*)))
    (cond ((member "--help" args :test #'string=)
           (format t "Usage: game [--help]~%")
           (sb-ext:exit :code 0))
          (t (run-game)))))
```

Wrong: `(quit)` — in modern SBCL, `quit` is deprecated in favour of `sb-ext:exit`, and the exact behaviour across versions varies (older `quit` had no portable way to specify the exit code). More importantly: placed *after* `save-lisp-and-die`, it never runs — `save-lisp-and-die` terminates the process. The `:toplevel` function is the after-save continuation. Remove the `(quit)` line entirely.

The general pattern: `save-lisp-and-die` is the *last thing a build script runs*, and the executable it produces has `:toplevel` as its own first thing. Two different processes, two different "beginnings".

32.4 The build script with tests:

```
#!/bin/sh
# build.sh — test, then build
set -e                          # stop on first error

SKIP_TESTS=no
for arg in "$@"; do
  if [ "$arg" = "--skip-tests" ]; then
    SKIP_TESTS=yes
  fi
done
```

```
if [ "$SKIP_TESTS" = "no" ]; then
  echo "Running tests..."
  sbcl --noinform --non-interactive \
       --eval "(asdf:load-system \"cave-of-noise\")" \
       --eval "(asdf:load-system \"cave-of-noise/tests\")" \
       --eval "(let ((results (cave-of-noise/tests::run-tests))) \
                 (when (some (lambda (pair) (not (cdr pair))) results) \
                   (format t \"FAILED: ~S~%\" results) \
                   (sb-ext:exit :code 1)))"
fi

echo "Building executable..."
sbcl --noinform --non-interactive \
     --eval "(asdf:load-system \"cave-of-noise\")" \
     --eval "(sb-ext:save-lisp-and-die \"game\" \
               :executable t :compression t :toplevel #'cave-of-noise:main)"

echo "Built: ./game"
```

No `main` changes are needed — the flag handling is in the shell script, not in Lisp. That's the right division of labour: shell arguments (`$@`) and file checks are shell work; loading systems and running tests are Lisp work.

Test it:

```
sh build.sh
# tests run, then build

sh build.sh --skip-tests
# build only
```

The `some` in the test check is worth understanding: `run-tests` returns a list of `(name . ok)` pairs; the `(lambda (pair) (not (cdr pair)))` is "is any pair's ok nil". If so, print the failing results and exit 1. If not, the `sb-ext:exit` inside is never reached, and SBCL continues to the build step. That's the shell-level test-gate: the script's `set -e` plus the `sb-ext:exit :code 1` inside the `--eval` makes a failing test stop the build.

A small refinement: when the test step exits 1, the shell prints nothing about why — that's why the `(format t "FAILED: ~S~%" results)` is there. The user needs to know which test failed; a generic exit code isn't enough.

32.5

```
(defun main ()
  "Entry point with --replay handling."
  (let ((args (rest sb-ext:*posix-argv*)))
    (cond
      ((member "--help" args :test #'string=)
       (format t "Usage: game [--help] [--version] \
[--room NAME] [--replay FILE]~%")
       (sb-ext:exit :code 0))
      ((member "--version" args :test #'string=)
```

```
       (format t "The Cave of Noise, version ~A~%" *version*)
       (sb-ext:exit :code 0))
      ((member "--replay" args :test #'string=)
       (let ((path (second (member "--replay" args
                                   :test #'string=))))
         (if path
             (handler-case
                 (progn
                   (replay-streamed path)
                   (sb-ext:exit :code 0))
               (error (c)
                 (format *error-output* "replay failed: ~A~%" c)
                 (sb-ext:exit :code 1)))
             (progn
               (format *error-output* "--replay needs a FILE~%")
               (sb-ext:exit :code 2)))))
      (t (run-game)))))
```

The flag belongs in `main`, not `play`, because it changes *what play does* — whether to enter interactive mode at all. `play` is "the game loop"; a `--replay` run doesn't want a loop, it wants to execute a transcript and exit. Putting the flag in `play` would mean `play` had a non-loop mode, which dilutes its single purpose. `main` is the dispatch layer: given arguments, decide what to do. `play` and `replay-streamed` are both things `main` can call; `main` picks.

The general principle: *argument parsing belongs at the top of the program, and its consequences belong at the bottom.* The top knows the world the program was invoked in (flags, environment); the bottom knows the program's own logic. The middle should be reachable both ways (interactive, replay) without knowing which way it's being reached.

The exercise also tests a shell idiom the game will grow into: `--replay FILE` is one of the two non-interactive modes a Lisp program commonly grows (the other is `--batch`-style script execution). Both exist because programs get used in scripts.

32.6

```
(defun build-executable (path toplevel &key (compress t))
  "Save the current image as an executable at PATH."
  #+sbcl (sb-ext:save-lisp-and-die
          path
          :executable t
          :compression compress
          :toplevel toplevel)
  #+ccl  (ccl:save-application
          path
          :toplevel-function toplevel
          :prepend-kernel t)
  #-(or sbcl ccl)
  (error "build-executable: no implementation branch for ~A"
         (lisp-implementation-type)))
```

Called from a fresh SBCL:

```
sbcl --noinform --non-interactive \
     --eval "(asdf:load-system \"cave-of-noise\")" \
     --eval "(cave-of-noise::build-executable \
              \"game\" #'cave-of-noise:main)"
```

The `::` is the one place it's defensible: the function is the build script's tool, not part of the game's public API. Exporting `build-executable` would be export-creep; the `cave-of-noise/tests` package's use of `::` is the same shape — build-time access to internals.

Now build.sh becomes:

```
#!/bin/sh
set -e
sbcl --noinform --non-interactive \
     --eval "(asdf:load-system \"cave-of-noise\")" \
     --eval "(cave-of-noise::build-executable \
              \"game\" #'cave-of-noise:main)"
```

What the wrapper buys:

- *Portability.* A single call site for the implementation-specific code. Chapter 32′s later section showed the `#+` branches; wrapping them means the build script doesn't grow when you add CCL support, and the branching logic is in one tested function, not scattered through every build invocation.
- *Parameter control.* `:compress` can be flipped from the command line (`--skip-compress` in build.sh reads it, passes it to `--eval`), without editing the script. Adding options later — `:strip`, `:output-name` — is one keyword each.
- *Testability.* `build-executable` is a function; you can call it from the REPL, from a test, from a future make `install` target, without shelling out.

What it costs:

- *One more layer to read.* The direct `save-lisp-and-die` call in the script is self-explanatory; the wrapper requires knowing what `build-executable` does. For a single-implementation project, the wrapper is over-engineering.
- *A function that terminates the process.* `build-executable` calls `sb-ext:save-lisp-and-die`, which exits. That's not a function you can usefully compose with — you can't, say, build two executables in one SBCL session. (SBCL supports it, in principle — but the first call exits.) The wrapper hides this; the direct call makes it obvious.

The trade-off is the standard one: wrap when the branches matter, call directly when they don't. For the game's single-implementation state, direct is honest; the wrapper is the shape it grows into.

D.AH. Chapters 33 and 34

The exercises in the last two chapters don't take a single answer. They ask you to add checks to your own suite, to break your own code on purpose, to read a library you have never seen, and to write the README a stranger needs. Each one states its own success condition, and in most

cases that condition is something that runs. Where an exercise names a shape — the `run-tests` of Chapter 33, the `fiveam` conversion, the `--version` flag — the shape is the answer.

APPENDIX E.

The Game's Redefinitions

The game has evolved across the book. This table is the map of every function, variable, structure, or class that was defined more than once — what changed, in which chapter, and which version is current. A reader working straight through will hit "already defined" moments; this table tells you which definitions to keep.

Name	Chapters	What changed, and current version
`find-room`	7 → 13	Ch 7: alist scan with `assoc`. Ch 13: `find` with `:key #'room-name` after `*rooms*` became a list of structures. *Current:* Ch 13.
`take`	7 → 8 → 14 → 24	Ch 7: `push` pair on front (buggy — hid old pairs). Ch 8: `set-location` rebuild. Ch 14: wrapped in `with-undo`. Ch 24: two-value return. *Current:* Ch 24 with `with-undo` and `set-location`.
`drop`	7 → 8 → 14 → 24	Ch 7: `push` pair (buggy). Ch 8: `set-location` rebuild. Ch 14: registered in `*commands*`. Ch 24: two-value return. *Current:* Ch 24.
`set-location`	8 → 10	Ch 8: recursive three-clause walk. Ch 10: `mapcar` with `lambda`. *Current:* Ch 10.
`items-in`	8 → 10	Ch 8: recursive walk. Ch 10: `remove-if-not` + `mapcar`. *Current:* Ch 10.
`item-names`	8 → 12	Ch 8: recursive with "and". Ch 12: kept Ch 8′s version (item-list-string is a separate, comma-only helper). *Current:* Ch 8′s (see item-list-string).
`look`	5 → 8 → 13	Ch 5: `describe-room`. Ch 8: added items. Ch 13: uses structure accessors (same names). *Current:* Ch 8′s body, working with structures.
`describe-room`	5 → 13	Ch 5: on alist. Ch 13: structure accessors replace the alist's `first/second/third`. Same function name, same result. *Current:* Ch 13.
`go-to`	6 → 7 → 13 → 19 → 24	Ch 6: `cond` with hard-coded garden lock. Ch 7: `find-room` now uses `*rooms*`. Ch 13: checks `room-locked` slot. Ch 19: adds drift warning on unknown exits. Ch 24: two-value return from `call-command`. *Current:* Ch 19 body; two-valued via `call-command` in Ch 24.
`find-room` (Ex-	6 → 7	The exercise's version; superseded by Ch 7′s `assoc` version.

Name	Chapters	What changed, and current version
ercise 6.10)		
`*rooms*`	7 → 13	Ch 7: alist. Ch 13: list of structures. *Current:* Ch 13′s.
`*history*`	14 → 25	Ch 14: a plain list of alist snapshots. Ch 25: a closure object from `make-history`. *Current:* Ch 25′s closure; `snapshot` and `undo` are the public interface and unchanged.
`*commands*`	9 → 14 → 23 → 33	Ch 9: alist of symbol→function. Ch 14: added `undo`. Ch 23: `define-command` registers metadata on plists. Ch 33: test references. *Current:* Ch 23′s registry; `run-command` unchanged.
`run-command`	9 → 15 → 24 → 30	Ch 9: `apply` via assoc. Ch 15: `restart-case` + `program-error` handler. Ch 24: two-value return (`reply`, `known-p`). Ch 30: cooldown check (Exercise 30.6). *Current:* Ch 24′s body; Ch 30′s cooldown is optional.
`call-command`	15 → 24	Ch 15: returns reply (one value). Ch 24: returns `(values reply known-p)`. *Current:* Ch 24′s.
`save-game`	16 → 18 → 24 → 33	Ch 16: `with-open-file` + `print`. Ch 18: `with-standard-io-syntax` + header line. Ch 24: two-value return (`success-p`, path-or-error). *Current:* Ch 24′s.
`load-game`	16 → 18 → 24 → 33	Ch 16: `handler-case` on `file-error`. Ch 18: reads header, converts v0. Ch 24: two-value return (`state`, `loaded-p`). *Current:* Ch 24′s.
`play`	9 → 24	Ch 9: `mapcar` over commands. Ch 24: two-value return (`replies`, `unknowns`). *Current:* Ch 24′s.
`transcript`	11 → 32	Ch 11: prints with header/footer. Ch 32: `play-streamed` and `replay-streamed` add streaming variants. *Current:* Ch 11′s for eager, Ch 30′s for streaming.

Name	Chapters	What changed, and current version
`count-unknown`	11 → 24	Ch 11: string-matching against unknown message. Ch 24: uses the two-valued `run-command`. *Current:* Ch 24′s.
`item-report`	8 → 11 → 12	Ch 8: recursive. Ch 11: `loop` with destructuring. Ch 12: `format` with `~@(~A~)`. *Current:* Ch 12′s.
`*item-locations*`	7 → 33	Ch 7: initial state. Ch 33: reset in tests. The variable itself is the same; only its tests change.
`*save-file*`	16 → 18	Ch 16: default `"/tmp/game-save.lisp"`. Ch 18: same. Bound via `let` in tests.
`price` generic	27 → 33	Ch 27: `+` combination with methods on `thing`, `container`, `treasure`. Ch 33′s test notes the `thing` method contributes 1 to every price. *Current:* Ch 27′s, with that note.

The general rule for "already defined" warnings: in the REPL, they're expected and harmless (Chapter 22′s redefinition loop). In a file being loaded via ASDF, they mean two chapters' definitions ended up in the same file — usually a sign that one of them should be removed.

APPENDIX F.

SBCL Error Messages

The error messages you'll meet in this book, alphabetically, with what they mean and where to look. The list isn't exhaustive; it's the ones whose phrasing is less than self-explanatory, or whose fix isn't where the message points.

Message	Meaning / Where to look
`The function COMMON-LISP-USER::FOO is undefined.`	Calling a function that hasn't been defined. Check the spelling, or whether you're in the right package (Ch 16, Ch 28).
`The variable FOO is unbound.`	Reading a name with no value. Ch 4's `let`, `defparameter`, `defvar` — which one did you mean?
`The value 5 is not of type LIST.`	A function that takes a list got something else. Common: `length`, `first`, `cdr` with a non-list. Ch 7, Ch 10.
`The value (A B C) is not of type VECTOR.`	`aref` got a list. Use `nth` for lists, `aref` for arrays (Ch 13).
`The value 2 is not of type (UNSIGNED-BYTE 62).`	An array index went out of bounds, or negative. Ch 13's `aref`.
`The value NIL is not of type NUMBER.`	Arithmetic on `nil`. Usually a base case returning the wrong type (Ch 8).
`Invalid number of arguments: 1.`	Calling a function with too few (or many) arguments. Check the signature with C-c C-d `d` or `describe` (Ch 22).
`Odd number of &KEY arguments.`	Calling a `&key` function with a keyword and no value, or a positional arg without its keyword (Ch 5).
`Control stack exhausted (no more space for function call frames).`	Infinite recursion, or recursion too deep for the frames available. Check the recursive call's argument (Ch 8).
`The value NIL is not of type (OR FUNCTION SYMBOL).`	`funcall` or `apply` on a non-function. Common: quoting a `lambda` (Ch 9).
`Lock on package COMMON-LISP violated when binding FOO.`	Trying to redefine or rebind a standard symbol (Ch 5). Pick another name.
`Cannot find the external symbol FOO in #<PACKAGE "BAR">.`	Referencing `bar:foo` where `foo` isn't exported. Double-colon to reach internals (Ch 28).
`Package BAR not found.`	Using `bar:...` where `bar` isn't a loaded package (Ch 16).
`Reader error on #<...>`	Syntax error in source: an unbalanced parenthesis, a stray comma, or something the reader couldn't parse (Ch 18).
`end of file`	Usually an unbalanced parenthesis — the reader kept reading past the last). Ch 1, Ch 18.
`The value (PRINT X) found where a LOOP keyword was expected.`	A plain Lisp form inside `loop` without `do` (Ch 11).
`COMPILED-FUNCTION is not of type FUNCTION.`	Trying to call something that isn't callable. Usually a typo in a variable that holds a function.

Message	Meaning / Where to look
`QUIT is deprecated.`	`quit` was replaced by `sb-ext:exit`. Use the latter (Ch 32).
`There is no class named FOO.`	A `defmethod` or `typep` referencing a class that hasn't been defined, or has the wrong name (Ch 26).
`No applicable method for FOO when called with arguments (...).`	Dispatch found no matching method. Check the specialisers, or add a catch-all (Ch 26, Ch 27).
`The class FOO is not defined.`	`defclass` or method specialiser on a class that doesn't exist yet. Load order (Ch 26, Ch 20).
`The assertion (= X Y) failed.`	A `check-type`, `the`, or `assert` failed (Ch 21).
:test (OR …) is not a valid hash table test.	make-hash-table `:test` was given something other than `eq`, `eql`, `equal`, or `equalp` (Ch 13).
`Reader error: comma not inside a backquote.`	A , outside a template (Ch 14).
`Unmatched }` (or other reader macro).	A custom reader macro saw syntax it didn't expect (Ch 18).
`Cannot coerce #<...> to the requested type.`	`coerce` couldn't produce the target type. Common: `(coerce 'a 'string)`.

APPENDIX G.

The Complete Game

The full source of the game as it stands after Chapter 34. This is the reference version; the reader's own game may differ in details (item names, room descriptions, tests) but the structure is the one the book taught. Six files.

G.A. `game.asd`

```
;;;; game.asd
(asdf:defsystem "cave-of-noise"
  :description "A small text adventure"
  :version "1.0"
  :author "You"
  :license "MIT"
  :serial t
  :depends-on ()
  :components ((:file "packages")
               (:file "classes")
               (:file "data")
               (:file "persistence")
               (:file "commands")
               (:file "main")))

(asdf:defsystem "cave-of-noise/tests"
  :depends-on ("cave-of-noise")
  :components ((:file "tests"))
  :perform (asdf:test-op (op system)
             (declare (ignore op system))
             (let ((results (cave-of-noise/tests::run-tests)))
               (unless (every #'cdr results)
                 (error "Test suite failed: ~{~A~^, ~}"
                        (remove-if #'cdr results))))))
```

G.B. `packages.lisp`

```
;;;; packages.lisp
(eval-when (:compile-toplevel :load-toplevel :execute)
  (unless (find-package :cave-of-noise)
    (defpackage :cave-of-noise
      (:use :cl)
      (:shadow #:room)
      (:nicknames #:con)
      (:export :play
               :run-command
               :look :take :drop :inventory :examine
               :save-game :load-game
               :*version*))))
```

```
(in-package :cave-of-noise)
```

G.C. classes.lisp

```
;;;; classes.lisp
(in-package :cave-of-noise)

(defstruct (room (:print-function print-room))
  name
  description
  exits
  (locked nil)
  (items nil))

(defun print-room (room stream depth)
  (declare (ignore depth))
  (format stream "<room ~(~A~)>" (room-name room)))

(defclass thing ()
  ((name :initarg :name :reader thing-name)
   (description :initarg :description
                :reader thing-description
                :initform "an ordinary thing"))
  (:documentation "Something the player can carry or examine."))

(defclass container (thing)
  ((contents :initarg :contents
             :accessor contents
             :initform nil)))

(defclass treasure (thing)
  ((value :initarg :value
          :reader value
          :initform 1)))

(defgeneric describe-thing (thing)
  (:documentation "Return a sentence describing THING."))

(defmethod describe-thing ((t2 thing))
  (format nil "You see ~(~A~): ~A."
          (thing-name t2) (thing-description t2)))

(defmethod describe-thing ((c container))
  (format nil "You see ~(~A~): ~A It holds ~{~(~A~)~^, ~}."
          (thing-name c) (thing-description c) (contents c)))

(defmethod describe-thing ((tr treasure))
  (format nil "You see ~(~A~): ~A It's worth ~D coins."
          (thing-name tr) (thing-description tr) (value tr)))
```

```
(defmethod initialize-instance :after ((c container) &key)
  (unless (listp (contents c))
    (setf (contents c) (list (contents c)))))

(defgeneric price (thing)
  (:method-combination +)
  (:documentation "How valuable is THING, summing contributions?"))

(defmethod price + ((t2 thing)) 1)
(defmethod price + ((c container)) 2)
(defmethod price + ((tr treasure)) (value tr))
```

G.D. data.lisp

```
;;;; data.lisp
(in-package :cave-of-noise)

(defparameter *version* "1.0")

(defparameter *rooms*
  (list
   (make-room :name 'bedroom
              :description "A small bedroom with an unmade bed."
              :exits '(hallway))
   (make-room :name 'hallway
              :description "A narrow hallway with creaky floorboards."
              :exits '(bedroom kitchen))
   (make-room :name 'kitchen
              :description "A warm kitchen that smells of bread."
              :exits '(hallway garden cellar))
   (make-room :name 'garden
              :description "An overgrown garden."
              :exits '(kitchen)
              :locked t)
   (make-room :name 'cellar
              :description "A damp cellar. Something drips in the dark."
              :exits '(kitchen))))

(defparameter *current-room* nil)

(defparameter *item-locations*
  '((lamp . cellar)
    (key . kitchen)
    (letter . bedroom)))

(defparameter *thing-registry* (make-hash-table :test #'eq))

(defun find-room (name)
  (find name *rooms* :key #'room-name))

(defun lookup-thing (sym)
```

```
  (gethash sym *thing-registry*))

(defun register-thing (sym instance)
  (setf (gethash sym *thing-registry*) instance))

(register-thing 'key (make-instance 'thing
                                    :name 'key
                                    :description "a small brass key"))
(register-thing 'lamp (make-instance 'thing
                                     :name 'lamp
                                     :description "an oil lamp"))
(register-thing 'letter (make-instance 'thing
                                       :name 'letter
                                       :description "a folded letter"))
(register-thing 'chest (make-instance 'container
                                     :name 'chest
                                     :description "a wooden chest."
                                     :contents '(crown gem)))
(register-thing 'crown (make-instance 'treasure
                                     :name 'crown
                                     :description "a tarnished crown."
                                     :value 100))

(defun make-history ()
  "Bounded undo history."
  (let ((snapshots nil)
        (limit 20))
    (lambda (message &optional arg)
      (ecase message
        (:add (push arg snapshots)
              (when (> (length snapshots) limit)
                (setf snapshots (subseq snapshots 0 limit)))
              t)
        (:recent (subseq snapshots 0 (min (or arg limit)
                                          (length snapshots))))
        (:undo-to (when snapshots
                    (let ((s (first snapshots)))
                      (pop snapshots) s)))
        (:clear (setf snapshots nil) t)))))

(defparameter *history* (make-history))

(defun snapshot ()
  (funcall *history* :add (copy-alist *item-locations*)))

(defun undo ()
  (let ((s (funcall *history* :undo-to)))
    (if s
        (progn (setf *item-locations* s) "Undone.")
        "Nothing to undo.")))

(defun set-location (pairs item place)
```

```
  (mapcar (lambda (pair)
            (if (eq (car pair) item)
                (cons item place)
                pair))
          pairs))

(defun item-location (item)
  (cdr (assoc item *item-locations*)))

(defun item-name (item)
  (string-downcase (symbol-name item)))

(defun items-here (place)
  (mapcar #'car
          (remove-if-not (lambda (p) (eq (cdr p) place))
                         *item-locations*)))

(defun item-names (items)
  (cond ((null items) "")
        ((null (cdr items)) (item-name (first items)))
        ((null (cddr items))
         (format nil "~A and ~A"
                 (item-name (first items))
                 (item-name (second items))))
        (t (format nil "~A, ~A"
                   (item-name (first items))
                   (item-names (rest items))))))

(defun item-list-string (items)
  (format nil "~{~(~A~)~^, ~}" (mapcar #'symbol-name items)))

(defparameter *game-output* t)

(defun say (control &rest args)
  (apply #'format *game-output* control args))

(defun sayln (control &rest args)
  (apply #'format *game-output*
         (concatenate 'string control "~%") args))
```

G.E. commands.lisp

```
;;;; commands.lisp
(in-package :cave-of-noise)

(defun describe-room-text (room)
  (let ((here (items-here (room-name room))))
    (if (null here)
        (format nil "You are in the ~(~A~). ~A"
                (room-name room) (room-description room))
        (format nil "You are in the ~(~A~). ~A You can see: ~A."
```

```
                (room-name room) (room-description room)
                (item-names here)))))

(defun look (&optional (place *current-room*))
  (describe-room-text place))

(defun take (item)
  (cond ((eq (item-location item) 'player)
         (format nil "You already have the ~(~A~)." item))
        ((eq (item-location item) (room-name *current-room*))
         (snapshot)
         (setf *item-locations*
               (set-location *item-locations* item 'player))
         (format nil "You take the ~(~A~)." item))
        (t "You don't see that here.")))

(defun drop (item)
  (if (eq (item-location item) 'player)
      (progn
        (snapshot)
        (setf *item-locations*
              (set-location *item-locations* item
                            (room-name *current-room*)))
        (format nil "You drop the ~(~A~)." item))
      "You don't have that."))

(defun inventory ()
  (let ((carried (sort (items-here 'player)
                       #'string< :key #'symbol-name)))
    (if (null carried)
        "You are carrying nothing."
        (format nil "You are carrying: ~A." (item-names carried)))))

(defun loot ()
  (let ((found (items-here (room-name *current-room*))))
    (if (null found)
        "There is nothing here to take."
        (progn
          (dolist (item found)
            (take item))
          (format nil "You take ~A." (item-names found))))))

(defun examine (sym)
  (let ((instance (lookup-thing sym)))
    (if instance
        (describe-thing instance)
        (format nil "You see nothing special about the ~(~A~)."
                sym))))

(defun go-to (name)
  (cond ((eq name (room-name *current-room*))
         "You are already there.")
```

```
        ((not (member name (room-exits *current-room*)))
         "You can't go there from here.")
        ((null (find-room name))
         (warn 'simple-warning
               :format-control "Exit to nonexistent room ~(~A~)"
               :format-arguments (list name))
         "You can't go there from here.")
        ((room-locked (find-room name))
         (format nil "The ~(~A~) is locked." name))
        (t (setf *current-room* (find-room name))
           (look))))

(defparameter *commands*
  (list (cons 'look #'look)
        (cons 'take #'take)
        (cons 'drop #'drop)
        (cons 'inventory #'inventory)
        (cons 'loot #'loot)
        (cons 'examine #'examine)
        (cons 'go #'go-to)
        (cons 'undo #'undo)
        (cons 'save #'cmd-save)
        (cons 'load #'cmd-load))
  "Command table.")

(defun call-command (command)
  (let ((entry (assoc (first command) *commands*)))
    (if (null entry)
        (values (format nil "No such command: ~(~A~)."
                        (first command))
                nil)
        (values (apply (cdr entry) (rest command))
                t))))

(defun run-command (command)
  (restart-case
      (handler-case (call-command command)
        (program-error ()
          (values "Wrong number of arguments for that command." t)))
    (retry-with (fixed)
      :report "Run a corrected command instead."
      (run-command fixed))))

(defun play ()
  "Start the game.  Reads s-expression commands from *standard-input*."
  (unless *current-room*
    (setf *current-room* (find-room 'kitchen)))
  (loop
    (format t "~&> ")
    (finish-output)
    (let ((input (read)))
      (when (eq (first input) 'quit)
```

```
        (format t "Goodbye.~%")
        (return))
      (multiple-value-bind (reply known-p)
          (run-command input)
        (declare (ignore known-p))
        (format t "~A~%" reply)))))
```

G.F. persistence.lisp

```
;;;; persistence.lisp
(in-package :cave-of-noise)

(defparameter *save-file* "/tmp/game-save.lisp")
(defparameter *save-format* 1)

(defun game-state ()
  (list *item-locations*
        (room-name *current-room*)
        (funcall *history* :recent)))

(defun restore-state (state)
  (setf *item-locations* (first state))
  (setf *current-room* (find-room (second state)))
  (let ((h *history*))
    (funcall h :clear)
    (dolist (snap (nreverse (third state)))
      (funcall h :add snap))))

(defun save-game ()
  (handler-case
      (progn
        (with-open-file (out *save-file*
                             :direction :output
                             :if-exists :supersede)
          (with-standard-io-syntax
            (format out "cave-of-noise v~D~%" *save-format*)
            (print (game-state) out)))
        (values t *save-file*))
    (file-error (c) (values nil (princ-to-string c)))))

(defun load-game ()
  (handler-case
      (with-open-file (in *save-file*)
        (let ((header (read-line in)))
          (unless (search "cave-of-noise" header)
            (error "Not a save file: ~S" header))
          (values (with-standard-io-syntax (read in)) t)))
    (file-error () (values nil nil))
    (end-of-file () (values nil nil))
    (error () (values nil nil))))
```

```
(defun cmd-save ()
  (multiple-value-bind (ok path-or-err) (save-game)
    (if ok
        (format nil "Saved to ~A." path-or-err)
        (format nil "Could not save: ~A" path-or-err))))

(defun cmd-load ()
  (multiple-value-bind (state loaded-p) (load-game)
    (if loaded-p
        (progn (restore-state state) (look))
        "No saved game found.")))
```

G.G. main.lisp

```
;;;; main.lisp
(in-package :cave-of-noise)

(defun main ()
  "Entry point for the deployed executable."
  (handler-case
      (let ((args (rest sb-ext:*posix-argv*)))
        (cond ((member "--help" args :test #'string=)
               (format t "Usage: game [--help] [--version]~%")
               (sb-ext:exit :code 0))
              ((member "--version" args :test #'string=)
               (format t "Cave of Noise, version ~A~%" *version*)
               (sb-ext:exit :code 0))
              (t nil)))
      (multiple-value-bind (state loaded-p) (load-game)
        (if loaded-p
            (restore-state state)
            (progn
              (setf *current-room* (find-room 'kitchen))
              (setf *item-locations*
                    '((lamp . cellar) (key . kitchen) (letter . bedroom)))
              (setf *history* (make-history)))))
      (play)
      (save-game)
      (sb-ext:exit :code 0)
    (error (c)
      (format *error-output* "cave: ~A~%" c)
      (sb-ext:exit :code 1))))
```

G.H. tests.lisp

```
;;;; tests.lisp
(defpackage :cave-of-noise/tests
  (:use :cl :cave-of-noise)
  (:import-from :cave-of-noise
                ;; state the tests read and rebind
```

```
                #:*item-locations* #:*current-room* #:*history*
                #:*save-file* #:*game-output*
                ;; functions and classes they call
                #:find-room #:item-location #:register-thing
                #:describe-thing #:make-history #:save-game
                #:load-game #:restore-state #:price #:treasure #:thing
                ;; command and data symbols: identity matters
                #:undo #:key #:kitchen #:player)
  (:export :run-tests))

(in-package :cave-of-noise/tests)

(defun run-tests ()
  (let ((results nil))
    (flet ((check (name got want)
             (push (cons name (equal got want)) results)))
      (check "take-moves-item"
             (progn
               (setf *item-locations* '((key . kitchen)))
               (setf *current-room* (find-room 'kitchen))
               (cave-of-noise::take 'key)
               (cave-of-noise::item-location 'key))
             'player)
      (check "inventory-alphabetical"
             (progn
               (setf *item-locations*
                     '((zebra . player) (apple . player)))
               (not (null (search "apple and zebra"
                                  (string-downcase
                                   (inventory))))))
             t))
    (nreverse results)))
```

That's the reference game. The chapters' individual project steps build it piece by piece; this is what they add up to. The reader's version may differ in room descriptions, item list, test set, and history limits — that's expected. The structure is what matters.

APPENDIX H.

Glossary

Terms the standard itself defines, plus the ones this guide introduced. Alphabetical.

Term	Definition
atom	Any value that isn't a cons — numbers, symbols, strings, etc.
binding	The association between a name and a value in a scope
cdr	The second part of a cons; the tail of a list
car	The first part of a cons; the first element of a list
class	A CLOS type; the "template" for its instances
closure	A function that captures bindings from an enclosing scope
condition	An object that represents a situation (error, warning, etc.)
cons	The fundamental pair structure; a cell with two parts
destructive	A function that modifies its argument; "n"-prefixed by convention
dynamic extent	The time during which a form's bindings are in force
form	A Lisp expression; the unit of code
function	A callable value; the function slot of a symbol
generic function	A named dispatch target; collects methods
instance	An object built from a class
keyword	A self-evaluating symbol whose name begins with `:`
lexical scope	Scope determined by position in the source text
list	A chain of conses ending in nil
macro	A function from forms to forms, run at compile time
method	One implementation of a generic function's behaviour
multiple values	Several results returned together, distinguishable by binding
package	A namespace for symbols
plist	A flat alternating key/value list; on symbols, the property list
predicate	A function returning true/false (conventionally `-p` suffixed)
proper list	A list ending in nil
reader	The parser that turns text into Lisp objects
restart	A recovery choice offered at a signal site
sequence	An ordered collection (list, string, vector)
sexp	A single Lisp form (the reader's output)
shadow	A package's own symbol for a name it would otherwise inherit
special	A dynamic variable; its binding follows the call stack
symbol	A named object with slots for value, function, and plist
tail position	The last expression a function evaluates before returning
uninterned	A symbol not in any package's table (e.g., from `gensym`)
value	What a form evaluates to

APPENDIX I.

Concept Index

A guided index of the concepts each chapter covered, in chapter order, so it doubles as a table of contents by idea. It is not a name index: it is for looking up *ideas*.

I.A. Part I: First Steps

- Ch 1: REPL loop; prefix notation; `C-c C-c` workflow
- Ch 2: evaluation rule; `quote`; atoms vs lists; `nil` truth
- Ch 3: exact numbers (ratios); characters vs strings vs symbols; upper-casing
- Ch 4: `defparameter` vs `defvar`; `let` vs `let*`; earmuffs
- Ch 5: `defun`; `&optional`/`&key`/`&rest`; docstrings; the return-value rule
- Ch 6: `if`/`when`/`unless`/`cond`/`case`; truthiness (only nil is false); `and`/`or` short-circuit

I.B. Part II: Thinking in Lists

- Ch 7: cons cells; alists; `car`/`cdr`; sharing
- Ch 8: recursion; base case; accumulator; `trace`
- Ch 9: functions as values; `#'`; closures; higher-order functions
- Ch 10: sequence functions; the four equalities; `sort` is destructive

I.C. Part III: Core Tools

- Ch 11: `dolist`/`dotimes`/`loop`; `return` vs `return-from`
- Ch 12: `format` directives; `~{~}` iteration; conditional directives
- Ch 13: structures; hash tables; arrays; multiple values (first contact)
- Ch 14: `defmacro`; backquote; `gensym`; macro hygiene
- Ch 15: signalling; `handler-case`; `restart-case`; interactive restarts
- Ch 16: packages; files; ASDF; the `:serial` rule

I.D. Part IV: The Lisp Way

- Ch 17: lexical vs dynamic scope; earmuffs as discipline; `*game-output*`
- Ch 18: the reader; `*readtable*`; `#=` dispatch; `with-standard-io-syntax`
- Ch 19: `handler-bind`; `muffle-warning`; the condition hierarchy
- Ch 20: `compile-file`/`load`; three times; `eval-when`
- Ch 21: types as value-properties; `declare`/`declaim`; `optimize` qualities
- Ch 22: the REPL as workbench; `inspect`/`describe`/`break`; reading backtraces
- Ch 23: symbols as five-part records; plists; `get`/`getf` distinction
- Ch 24: multiple values; `multiple-value-bind`; the single-value context table
- Ch 25: closures as objects; shared environments; closure vs defparameter state
- Ch 26: CLOS; `defclass`/`defmethod`; `initialize-instance`
- Ch 27: method combination (`+`, `max`); `change-class`; `shared-initialize`
- Ch 28: package internals; `shadow`; the `:use`/`:import-from` distinction
- Ch 29: JSON/CSV interop; `uiop:run-program`; CFFI
- Ch 30: generators; the two-value protocol; lazy vs eager

- Ch 31: the reading habit; the library audit checklist
- Ch 32: `save-lisp-and-die`; `:toplevel`; the build script; exit codes
- Ch 33: test discipline; test the behaviour not the internals; one test per bug
- Ch 34: the ecosystem; the Hyperspec habit; “thinking in Lisp”

Colophon: How the Guide Grew

This book was written as one continuous conversation, part by part, against a running project that never used an idea before the chapter that taught it. The core — Parts I through III and the game's first rooms — came first; the later chapters were added in response to an audit of what a working Lisp programmer meets that the early chapters had left implicit: the reader and the printer's knobs, multiple values, closures as objects, CLOS beyond the basics, package internals, generators, the disciplines of declarations, of reading strangers' code, of interoperating, of shipping.

The running game — the Cave of Noise — carried every idea in turn: alists for rooms, closures for history, conditions for failsafes, CLOS for items, packages for the split, generators for the streamed replay, and finally `save-lisp-and-die` for the shipped executable. If you worked the project steps, you built a program; if you worked the exercises, you built the reflexes. The two together were the promise: not a reference that tells, but a guide that holds your hand until the hand lets go.

The tools that made it: Emacs, Org mode, SBCL. The tools you now know better than the person who configured your init file did.